

Programiranje za UNIX

Damir Krstinić, Maja Braović

Lipanj 2026.

Sadržaj

Predgovor	i
1 Osnove UNIX-a	1
1.1 Arhitektura UNIX operacijskog sustava	3
1.2 Sustav datoteka	5
1.2.1 Tipovi datoteka	5
1.2.2 Struktura direktorija	5
1.2.3 Razlike u odnosu na Windows datotečni sustav	6
1.2.4 Prava pristupa	6
1.3 Naredbena ljuska	8
1.3.1 Osnovne naredbe	8
1.3.2 Pokretanje programa u pozadini	9
1.3.3 Priručnik — naredba man	9
1.4 Preusmjerenje ulaza i izlaza	10
1.5 Ulančavanje naredbi (pipes)	10
1.6 UNIX procesi	11
1.7 Shell skripte	12
1.7.1 Najjednostavnija skripta	13
1.7.2 Petlje, varijable i argumenti	14
1.7.3 Praktičan primjer — backup direktorija	15
1.7.4 Pretraga datoteka — find, grep, brojanje	16
1.7.5 Provjera ispravnosti argumenata i izlazni status	17
1.7.6 Obrada svih datoteka u direktoriju	17
1.8 Što dalje?	18
1.9 Bibliografija	18
2 Osnove programiranja	21
2.1 Prevođenje i povezivanje izvornog koda	21
2.2 GCC — GNU projekt C i C++ prevodilac	23
2.2.1 Sintaksa i osnovne opcije	23
2.2.2 Ulazni tipovi datoteka	24
2.2.3 Primjeri korištenja	24
2.3 Najjednostavniji program	24
2.3.1 Prevođenje u jednom koraku	24
2.3.2 Prevođenje u dva koraka	25
2.4 Program u više datoteka	25
2.4.1 Prevođenje programa iz više datoteka	26
2.5 Korištenje alata make	26
2.5.1 Struktura pravila	26
2.5.2 Gradnja Makefilea korak po korak	27

2.6	Arhive objektnih datoteka — libovi	30
2.6.1	Korišćenje tuđih libova	31
2.6.2	Stvaranje vlastite arhive — alat ar	31
2.6.3	Primjer	32
2.6.4	Makefile za sva tri načina	34
2.7	Pokretanje	34
2.8	Bibliografija	34
3	Ulazno/izlazne operacije	37
3.1	Deskriptori datoteka	38
3.2	Sistemske pozivi za rad s datotekama	39
3.2.1	Funkcija open()	39
3.2.2	Funkcija creat()	40
3.2.3	Funkcija close()	41
3.2.4	Funkcija read()	41
3.2.5	Funkcija write()	42
3.3	Primjeri	42
3.3.1	Otvaranje, čitanje i pisanje	42
3.3.2	Pozicioniranje unutar datoteke (lseek)	44
3.3.3	Prava pristupa i maska (umask)	46
3.3.4	Argumenti naredbenog retka	47
3.4	Ulazno-izlazne strukture podataka	50
3.4.1	Tablica procesa, tablica datoteka i v-node tablica	50
3.5	Dijeljenje datoteka između procesa	54
3.5.1	Dijeljenje na razini v-node tablice	54
3.5.2	Race condition	55
3.5.3	Atomske operacije	56
3.5.4	Dijeljenje na razini tablice datoteka	57
3.5.5	Dupliciranje deskriptora — dup() i dup2()	58
3.6	Sistemske pozivi i funkcije C biblioteke	61
3.7	Prevođenje	62
3.8	Bibliografija	62
4	Upravljanje datotekama	63
4.1	Tipovi datoteka	63
4.2	Svojstva datoteka	64
4.2.1	Struktura struct stat	64
4.2.2	Otkrivanje tipa datoteke	66
4.2.3	Ispis atributa datoteke	66
4.2.4	Razlika stat i lstat	68
4.3	Korisnici, grupe i prava pristupa	69
4.3.1	Procesi i njihovo vlasništvo	69
4.3.2	Provjera prava pristupa — access()	71
4.3.3	Promjena prava pristupa — chmod() i fchmod()	72
4.3.4	Promjena vlasništva — chown(), fchown(), lchown()	73
4.4	Linkovi	74
4.4.1	Čvrsti linkovi	74
4.4.2	Simbolički linkovi	76
4.5	Vremena pristupa	79
4.6	Rad s direktorijima	80
4.6.1	Čitanje sadržaja direktorija	82
4.7	Što smo zapravo radili	86

4.8	Prevođenje	86
4.9	Bibliografija	87
5	Okruženje procesa	89
5.0.1	Argumenti naredbenog retka	91
5.0.2	Varijable okruženja	94
5.0.3	Životni ciklus procesa	98
5.0.4	Pokretanje novog programa	103
5.0.5	Ograničavanje resursa (setrlimit)	109
5.0.6	Osirotjeli procesi (engl. <i>orphan processes</i>)	110
5.1	Prevođenje	111
5.2	Bibliografija	111
6	Signali	113
6.1	Najčešći signali	113
6.1.1	Hvatanje signala	116
6.1.2	Vlastiti alarm — SIGALRM i alarm()	118
6.1.3	Korištenje signala za komunikaciju između procesa	121
6.2	Sistemske pozive sigaction	124
6.3	Blokiranje i ignoriranje signala	126
6.3.1	Sistemske pozive sigprocmask	126
6.4	Pokupljanje djece — SIGCHLD	129
6.5	Prevođenje	131
6.6	Bibliografija	132
7	Komunikacija između procesa	133
7.1	Pregled mehanizama IPC-a	133
7.1.1	Signali kao oblik IPC-a	134
7.2	Anonimni cjevovodi	134
7.2.1	Sistemske pozive pipe()	135
7.2.2	Cjevovod između roditelja i djeteta	136
7.2.3	Preusmjerenje i cjevovodi	137
7.3	Imenovani cjevovodi (FIFO)	140
7.3.1	Primjer: razmjena poruke kroz FIFO	141
7.4	Dijeljena memorija	143
7.4.1	Primjer: dijeljenje memorije između nezavisnih procesa	146
7.5	Semafori: upravljanje pristupom dijeljenim resursima	148
7.5.1	Semafor kao sinkronizacijski mehanizam	150
7.6	POSIX redovi poruka	153
7.6.1	Primjer: razmjena poruka kroz red	154
7.7	Mapiranje datoteka u memoriju	156
7.8	System V IPC — kratko upoznavanje	158
7.8.1	Primjer: System V redovi poruka	158
7.9	Što smo zapravo radili	161
7.10	Prevođenje	162
7.11	Bibliografija	162
8	Višenitno programiranje	163
8.1	Niti i procesi	163
8.2	Raspoređivač (scheduler) i niti	164
8.3	POSIX niti — pthreads	165
8.4	Stvaranje i terminiranje niti	165

8.4.1	Stvaranje niti — <code>pthread_create</code>	166
8.4.2	Terminiranje i prikupljanje rezultata niti — <code>pthread_exit</code> i <code>pthread_join</code>	166
8.4.3	Otkazivanje niti — <code>pthread_cancel</code>	167
8.4.4	Primjer: pozdrav1	167
8.4.5	Primjer: pozdrav2	168
8.4.6	Primjer: pozdrav3	169
8.4.7	Primjer: kvadrat — prijenos podataka u nit i natrag	170
8.4.8	Primjer: argumenti — više niti s različitim argumentima	172
8.5	Joinable i detached niti	174
8.6	Race condition	175
8.7	Mutex	177
8.7.1	Primjer: broji2	178
8.7.2	Deadlock	179
8.8	Kondicijske varijable	180
8.8.1	Više proizvođača i potrošača	183
8.9	Signali i niti	185
8.9.1	Standardni obrazac: jedna nit obrađuje signale	185
8.9.2	Slanje signala specifičnoj niti	187
8.10	Thread-safe i reentrant funkcije	187
8.11	Što smo zapravo radili	188
8.12	Prevođenje	189
8.13	Bibliografija	189
9	Socketi	191
9.1	Socket kao deskriptor	191
9.2	UNIX domain socketi	194
9.2.1	Primjer: <code>uds_server</code> i <code>uds_klijent</code>	194
9.3	Mrežni socketi (Network domain Sockets) – TCP/IP	197
9.3.1	Network byte order	198
9.3.2	Primjer: <code>tcp_server</code> i <code>tcp_klijent</code>	199
9.4	Više klijenata istovremeno	201
9.4.1	Posluživanje klijenta u novom procesu	201
9.4.2	Alternativni pristupi	204
9.5	UDP socketi	204
9.5.1	Primjer: <code>udp_server</code> i <code>udp_klijent</code>	205
9.6	Prevođenje	206
9.7	Što smo zapravo radili	207
9.8	Bibliografija	207

Predgovor

Skripta je prvenstveno namijenjena studentima koji slušaju kolegij *Programiranje za UNIX* na Fakultetu elektrotehnike, strojarstva i brodogradnje Sveučilišta u Splitu. Djelomično se obrađuje i gradivo kolegija *Paralelno računanje* koji se predaje na istom fakultetu. Pored toga, skripta je namijenjena i drugim studentima tehničkih i prirodnih znanosti koje zanima programiranje i operacijski sustavi.

Cjelokupan tekst skripte, uključujući i sve primjere, dostupan je online na adresi https://github.com/dkrst/Programiranje_za_UNIX.

Skripta je organizirana po poglavljima koja postupno uvode čitatelja u ključne koncepte UNIX sustava — od osnova rada u ljusci, preko C programiranja i sistemskih poziva, do procesa, signala, niti i mrežnog programiranja. Svaki primjer ilustrira specifičnu funkcionalnost (ljuska, sistemski pozivi, rad s procesima, datotečni sustav, signali, niti, socketi) i predviđen je za prevođenje standardnim gcc-om na proizvoljnom POSIX-kompatibilnom sustavu. Svaki direktorij ima vlastiti README s detaljnim opisom primjera i teorijskim uvodom u temu poglavlja. Na kraju svakog poglavlja dana je bibliografija s referencama na dodatnu literaturu — knjige, sveučilišne udžbenike i online vodiče, kako na engleskom tako i na hrvatskom jeziku. Iako se uglavnom koristi istih nekoliko naslova, bibliografija je ciljano dana zasebno za svaku cjelinu kako bi je čitatelj lakše pratio uz odgovarajuću temu.

Preduvjeti

- POSIX-kompatibilan sustav (Linux, macOS, WSL na Windowsu, BSD varijante)
- gcc (ili drugi C prevodilac koji razumije GNU sintaksu Makefilea)
- make
- bash (za primjere shell skripti u P01)

Struktura

Svaki direktorij odgovara jednom poglavlju skripte:

P01-Osnove_UNIXa/

Uvodno poglavlje koje upoznaje čitatelja s arhitekturom UNIX operacijskog sustava, organizacijom datotečnog sustava, pravima pristupa, naredbenom ljuskom, preusmjeravanjem ulaza i izlaza, ulančavanjem naredbi te osnovama pisanja shell skripti. Sadrži četiri **bash skripte** koje ilustriraju ključne koncepte: `pozdrav.sh`, `brojac.sh`, `backup.sh`, `trazi.sh`.

P02-Osnove_programiranja/

Uvod u proces prevođenja i povezivanja C programa s primjerima. Demonstrira osnovnu strukturu C programa, uporabu zaglavlja i razlaganje koda na više prevodbenih jedinica. Detaljno se obrađuje korištenje alata make — od ručnog prevođenja preko jednostavnih pravila do potpunog Makefilea s varijablama, implicitnim pravilima i konvencionalnim pseudo-pravilima default, all i clean.

P03-Ulazno_izlazne_operacije/

Primjeri koji ilustriraju UNIX sistemske pozive za rad s datotekama (open, creat, close, read, write, lseek, umask, dup, dup2) i temeljni UNIX koncept “*sve je datoteka*” — datoteke, uređaji, terminali, cijevi i mrežni socketi koriste se kroz isto sučelje deskriptora datoteka. Obrađuje se i dijeljenje datoteka među procesima i unutar istog procesa, s mehanizmom preusmjerenja standardnih tokova.

P04-Upravljanje_datotekama/

Upravljanje datotekama na razini datotečnog sustava: rad s metapodacima, prava pristupa, vlasništvo, vremenski žigovi, simboličke i tvrde veze, te navigacija direktorijima.

P05-Okruženje_procesa/

Primjeri koji obrađuju životni ciklus UNIX procesa: argumente naredbenog retka, varijable okruženja, stvaranje novih procesa pozivom fork(), pokretanje programa funkcijama iz exec obitelji, čekanje završetka djece pomoću wait(), ograničavanje resursa kroz setrlimit(), te problematiku zombi i osirotjelih procesa.

P06-Signali/

Primjeri koji obrađuju signale — UNIX-ov primarni mehanizam za asinkronu komunikaciju s procesom. Pokrivaju hvatanje signala (signal, sigaction), korištenje SIGALRM za vlastite alarme, signalnu komunikaciju između procesa, blokiranje i ignoriranje signala, te pokupljanje djece preko SIGCHLD rukovatelja.

P07-Komunikacija_izmedju_procesa/

Mehanizmi međuprocesne komunikacije (IPC): anonimni i imenovani cjevovodi (pipe, FIFO), POSIX dijeljena memorija (shm_open, mmap), sinkronizacija pomoću semafora, POSIX redovi poruka, mapiranje datoteka u memoriju, te kratki uvod u System V IPC. Poglavlje uvodi i koncepte race condition-a i atomskih operacija u kontekstu paralelnog rada više procesa.

P08-Visenitno_programiranje/

Uvod u višenitno programiranje s POSIX nitima (pthreads). Pokriva odnos niti i procesa, raspoređivanje niti, stvaranje i terminiranje niti, joinable i detached niti, race condition u kontekstu niti, mutex, kondicijske varijable s klasičnim primjerom proizvođač-potrošač, te specifičnosti rada sa signalima u višenitnom programu.

P09-Socketi/

Mrežno i lokalno komuniciranje između procesa kroz socket sučelje. Pokriva UNIX domain sockete (komunikacija na istom računalu kroz putanju u datotečnom sustavu) i mrežne TCP/IP sockete (preko mreže ili lokalno kroz 127.0.0.1), tijekom socket/bind/listen/accept na serverskoj strani i socket/connect na klijentskoj, network byte order, te obrazac opsluživanja više klijenata istovremeno kroz fork.

Prevođenje i pokretanje

Svaki direktorij od P02 nadalje sadrži vlastiti Makefile s istim konvencijama (varijable CC, CFLAGS, LDFLAGS, TARGETS; implicitno pravilo .c.o; pseudo-pravila default, all, clean). Pozicionirajte se u željeni direktorij i pokrenite:

```
make all      # prevede sve primjere iz direktorija
make clean    # briše generirane izvršne i objektne datoteke
```

Za prevođenje pojedinačnog primjera dovoljno je:

```
make <ime_primjera>
```

Konkretnu uputu za pokretanje pojedinačnih primjera (uključujući očekivane argumente i izlaz) nalaze se u README datoteci svakog poglavlja.

P01 ne sadrži C kod nego shell skripte; one se pokreću izravno nakon dodjeljivanja prava izvršavanja:

```
chmod +x *.sh
./pozdrav.sh
```

Licenca

Kod je objavljen pod licencom **GNU General Public License v3.0**. Detalji su u datoteci LICENSE.

Poglavlje 1

Osnove UNIX-a

Razvoj operacijskog sustava UNIX započinje krajem šezdesetih godina prošlog stoljeća u tvrtki Bell Labs, s primarnim ciljem omogućavanja istovremenog rada više korisnika na tadašnjim mainframe računalima. Prva službena verzija UNIX-a objavljena je 1971. godine pod autorstvom Dennisa Ritchieja i Kena Thompsona [1], a distribuirana je unutar akademskih i istraživačkih institucija. Unatoč tome što je kroz desetljeća značajno evoluirao i prilagođavao se novim tehnologijama i trendovima, temeljne značajke i ideje na kojima je UNIX građen — kao što su višekorisništvo, višezadaćnost te modularna arhitektura — zadržale su se do danas. Štoviše, može se reći da je UNIX postavio temelje za brojne moderne operacijske sustave koji se danas koriste.

Ključne osobine UNIX operacijskog sustava uključuju:

- **Višekorisnički i višezadaćni:** UNIX je od samog početka dizajniran za istovremeni rad više korisnika, od kojih svaki može pokretati više zadataka istovremeno. Operacijski sustav sam brine o pravednoj raspodjeli procesorskog vremena i ostalih resursa računala.
- **Modularna arhitektura:** Osnovne funkcionalnosti nalaze se u malim, relativno jednostavnim i specijaliziranim programima. Ove funkcionalnosti mogu se kombinirati u veće i složenije cjeline iz aplikativnih programa, ili izravno od strane korisnika korištenjem UNIX ljuske. Ovo načelo poznato je kao UNIX filozofija: svaki program radi jednu stvar, ali je radi dobro [2].
- **UNIX ljuska:** UNIX ljuska (engl. *shell*) nudi naredbeni redak putem kojeg korisnik može zadavati naredbe operacijskom sustavu. Naredbe se interpretiraju te se izvršavaju određene operacije OS-a ili pokreću drugi programi. Rezultat izvođenja naredbe ispisuje se korisniku u naredbeni redak (terminal) putem kojeg je naredba zadana. Osim toga, ljuska se može koristiti i za pisanje skripti — nizova UNIX naredbi zapisanih u tekstualnoj datoteci — što se često koristi za automatizaciju zadataka.
- **Kontrola pristupa i sigurnost:** UNIX ima detaljno razrađene mehanizme vlasništva i kontrole pristupa resursima. Ovi mehanizmi obuhvaćaju podatke u obliku datoteka, ali i sve druge resurse, kao što su procesi koji se izvršavaju, pristup memoriji i računalno sklopovlje. Korisnici imaju različite razine privilegija, što doprinosi sigurnosti sustava.
- **Prenosivost:** UNIX je prenosiv na razini izvornog koda. To znači da izvorni kod programa možemo prevesti na različitim varijantama UNIX operacijskog

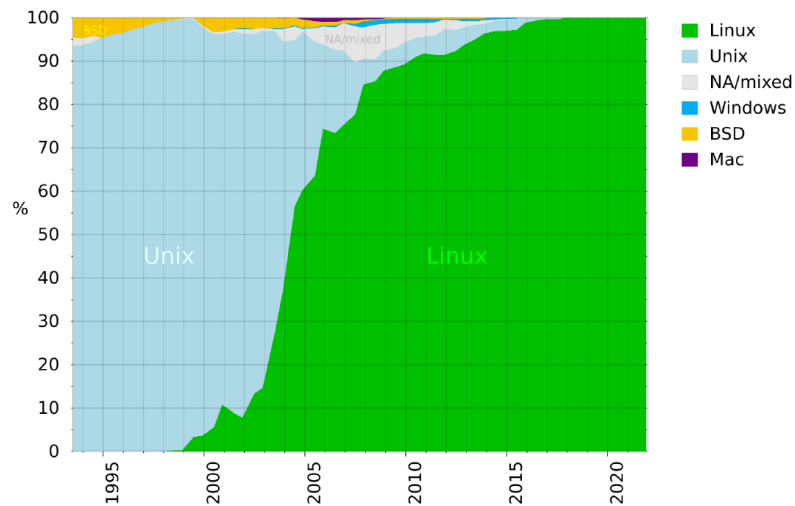
sustava i na različitim računalnim platformama (različitom sklopovlju). Ova osobina, ostvarena ponajprije pisanjem UNIX-a u programskom jeziku C, bila je revolucionarna u doba kada su operacijski sustavi gotovo bez iznimke bili pisani u sklopovlju ovisnom asemblerskom jeziku.

Važno je naglasiti da danas postoji više varijanti UNIX-a, kao što su BSD (Berkeley Software Distribution), Solaris, IBM AIX te HP-UX (Hewlett-Packard UNIX). Danas najpopularnija i najkorištenija varijanta UNIX-a je Linux, koji je kao studentski projekt razvio Linus Torvalds početkom devedesetih godina prošlog stoljeća, a čije je temeljno obilježje otvoreni kôd (Open Source). UNIX i njegove varijante danas dominiraju na serverskim sustavima, u akademskim krugovima, u industriji te u svim okruženjima koja zahtijevaju robusnost, skalabilnost i fleksibilnost. Otvoreni kôd Linuxa, pak, omogućio je razvoj brojnih operacijskih sustava koje nalazimo na raznim ugradbenim računalima kao i na mobilnim platformama.

Kratka povijest UNIX-a

- **1969-1971:** Dennis Ritchie, Ken Thompson i drugi inženjeri razvijaju prvu verziju UNIX-a u tvrtki Bell Labs. Neke od ideja UNIX-a preuzete su iz ranijeg Berkeley Timesharing System razvijenog na sveučilištu Berkeley i MULTICS-a (Multiplexed Information and Computer Services), razvijenog u suradnji Bell Labs i sveučilišta MIT. Prva verzija UNIX-a objavljena je 1971. godine. Tijekom sedamdesetih, UNIX se širi na američkim sveučilištima, što daje dodatni poticaj razvoju novih značajki.
- **BSD (1977):** Berkeley Software Distribution, razvijen na sveučilištu Berkeley, donosi brojna poboljšanja, uključujući virtualnu memoriju, naprednije upravljanje procesima i bolji datotečni sustav. Također, BSD promiče otvoreni kôd (Open Source), što je omogućilo uključivanje šire zajednice programera i entuzijasta te dodatnu popularizaciju UNIX-a.
- **1980-e:** Prihvaćanje u industriji i pojava komercijalnih verzija, kao što su AT&T UNIX, SunOS, IBM AIX, SGI IRIX, HP-UX i druge.
- **POSIX (1988):** (Portable Operating System Interface) — standard uspostavljen s ciljem ujednačavanja različitih verzija UNIX-a i osiguranja prenosivosti programa na razini izvornog koda [3].
- **Linux (1991):** Linus Torvalds objavljuje Linux, verziju UNIX-a koja je slobodna i otvorenog koda. Uskoro se pojavljuju različite distribucije Linuxa te kompanije koje nude komercijalnu podršku.
- **Android (2008):** Google objavljuje mobilni operacijski sustav za pametne telefone baziran na Linuxu.
- **iOS (2007):** Apple objavljuje mobilni operacijski sustav za svoje telefone temeljen na BSD UNIX-u.
- **TOP500:** UNIX dominira na rang-listi 500 najsnažnijih računala na svijetu. Od 2017. godine sva računala na listi TOP500 koriste operacijske sustave bazirane na Linuxu [4]. Zastupljenost pojedinih sustava kroz godine prikazana je na slici 1.1.

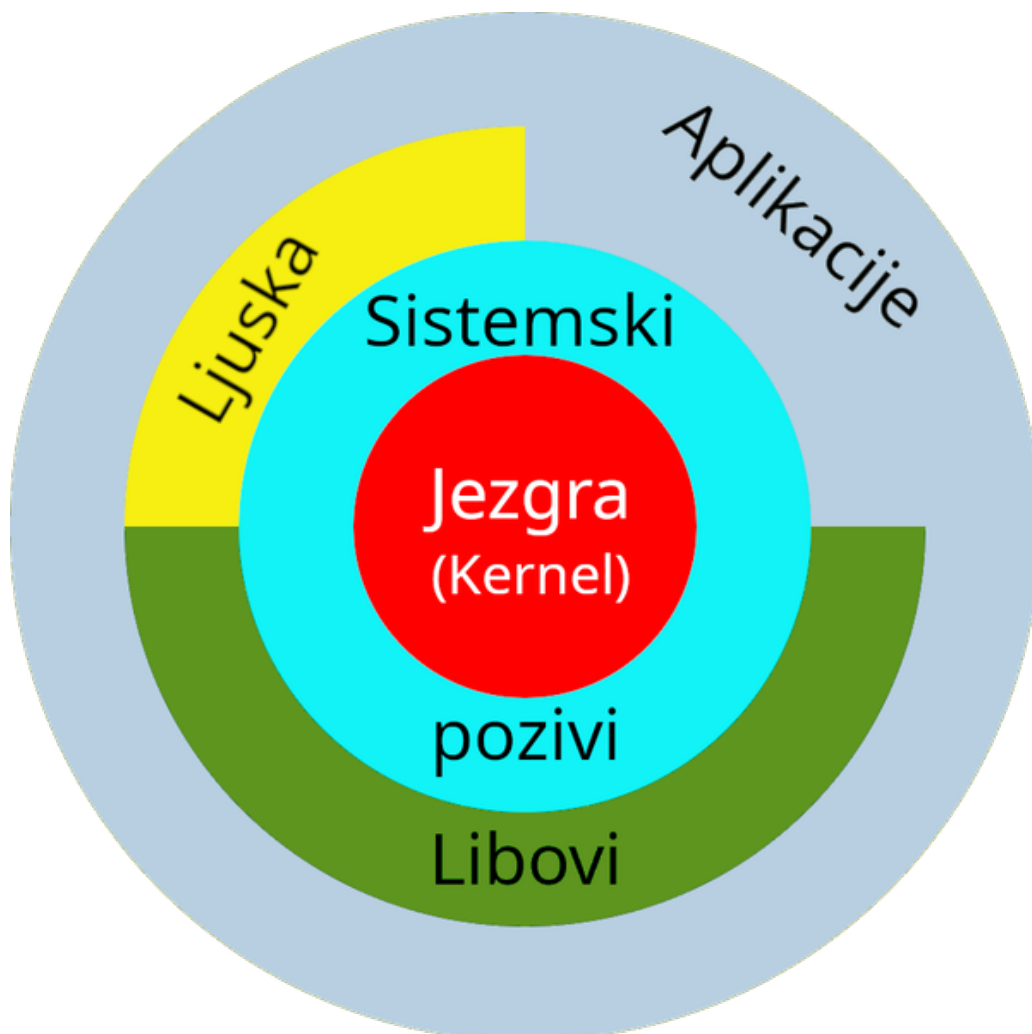
Čitatelji koji žele saznati više o povijesti UNIX-a mogu posegnuti za knjigom *Kratka povijest UNIX-a: Od UNICS-a do FreeBSD-a i Linuxa* [5].



Slika 1.1: Zastupljenost operacijskih sustava na TOP500 listi superračunala

1.1 Arhitektura UNIX operacijskog sustava

UNIX je organiziran u nekoliko slojeva apstrakcije, prikazanih na slici 1.2:



Slika 1.2: Slojevita arhitektura UNIX operacijskog sustava

U središtu se nalazi **jezgra** (engl. *kernel*) — najniži sloj softvera koji izravno upravlja sklopovljem računala (procesorom, memorijom, diskovima, mrežnim sučeljima, ulazno-izlaznim uređajima). Jezgra je jedini dio sustava koji ima privilegirani pristup hardveru; sve ostale komponente — uključujući vlastite uvodne programe poput ljuske — moraju s hardverom komunicirati posredno, kroz nju.

Problem nepostojanja standardiziranog skupa komponenti i brojnih nezavisnih proizvođača računalnog sklopovlja operacijski sustavi rješavaju korištenjem **modula** — zasebnih dijelova jezgre koji se po potrebi mogu dinamički učitati bez ponovnog pokretanja OS-a. Moduli su upravljački programi za pojedine uređaje i komponente, a mogu se dodavati u bilo kojem trenutku. Često ih izrađuju i distribuiraju sami proizvođači komponenti. U slučaju OS-a otvorenog koda, kao što je Linux, modul može izraditi i šira zajednica programera. U Windows terminologiji ekvivalent modulu je **driver** (engl. *device driver*).

Komunikacija s jezgrom odvija se putem **sistemskih poziva** (engl. *system calls*) — strogo definiranog skupa funkcija koje korisnički programi pozivaju kad žele zatražiti uslugu od jezgre. Sistemski pozivi su jedino sučelje između korisničkih programa i jezgre. Na taj način programer ne mora poznavati detalje implementacije sklopovlja, nego se oslanja na unificirano i stabilno sučelje.

Iznad sistemskih poziva nalaze se **biblioteke funkcija** (engl. *libraries*, *libovi*) — zbirke funkcija više razine koje predstavljaju dodatnu razinu apstrakcije. Najpoznatiji primjer je standardna C biblioteka (*libc*) koja sadrži funkcije poput *printf*, *fopen*, *malloc* i mnoge druge. Funkcije iz biblioteka u konačnici se svode na jedan ili više sistemskih poziva, ali programeru pružaju znatno jednostavnije sučelje.

Naredbena ljuska (engl. *shell*) je interpreter naredbenog retka koji korisniku omogućuje interaktivan pristup svim funkcijama sustava. Bitno je naglasiti da ljuska **nije dio jezgre** — ona je obični korisnički program koji koristi sistemske pozive i biblioteke kao i svaki drugi program. UNIX ljuska istovremeno je i kompletan programski jezik: korisnik može pisati skripte koje uključuju varijable, uvjetno grananje, petlje i funkcije, sve koristeći isključivo ljusku. Jedna od temeljnih prednosti je **filozofija ulančavanja** — izlaz jedne naredbe može postati ulaz druge, što omogućuje izgradnju složenih lanaca obrade podataka od jednostavnih, specijaliziranih programa. Ovo ujedno predstavlja i jedno od temeljnih načela UNIX filozofije: *mali programi koji rade jednu stvar dobro mogu se kombinirati u moćne lance obrade*.

Najviši sloj čine **aplikacije** — svi korisnički programi: tekstualni editori, web preglednici, baze podataka, inženjerski alati. Sve operacije ostvaruju pozivanjem funkcija iz biblioteka ili izravnim sistemskim pozivima.

Iako je na slici 1.2 prikazan kao zaseban sloj, **datotečni sustav** zapravo prožima cijelu arhitekturu. Jezgra ga implementira i izlaže putem sistemskih poziva. UNIX-ova filozofija *“sve je datoteka”* znači da se i uređaji, međuprocena komunikacija i mnogi drugi resursi sustava prikazuju kao datoteke — što znatno pojednostavljuje programiranje jer se sve resursi koriste kroz isto sučelje.

1.2 Sustav datoteka

1.2.1 Tipovi datoteka

UNIX razlikuje nekoliko tipova datoteka. Tip se može vidjeti prvim znakom u ispisu naredbe `ls -l`:

Oznaka	Tip	Opis
-	obična datoteka	bilo koji podaci na disku — tekst, binarni programi, slike, archive
d	direktorij	popis zapisa o datotekama i poddirektorijima
l	simbolički link	datoteka koja sadrži putanju na drugu datoteku ("prečac")
c	character special	uređaj kojemu pristupamo znak po znak (/dev/tty)
b	block special	uređaj kojem pristupamo blok po blok (/dev/sda)
s	socket	krajnja točka mrežne ili lokalne komunikacije
p	imenovani cjevovod (FIFO)	jednosmjerni komunikacijski kanal između procesa

Ključna posljedica filozofije "*sve je datoteka*": svi ovi tipovi resursa koriste se istim malim skupom sistemskih poziva — `open()`, `read()`, `write()`, `close()`. Program ne treba znati radi li o običnoj datoteci na disku, terminalu, mrežnom socketu ili imenovanom cjevovodu — sa svima radi na isti način.

1.2.2 Struktura direktorija

UNIX datotečni sustav organiziran je kao stablo: jedan korijenski direktorij označen znakom `/` na vrhu, ispod njega proizvoljno mnogo razina poddirektorija. Svaki direktorij u trenutku stvaranja automatski dobiva dva posebna unosa koji se ne mogu izbrisati:

- `.` — pokazivač na **trenutni** direktorij,
- `..` — pokazivač na **roditeljski** direktorij (jednu razinu više). Iznimka je korijenski direktorij `/`, gdje `.` i `..` pokazuju na sam `/`.

Pri navigaciji datotečnim sustavom razlikujemo apsolutnu i relativnu putanju.

Apsolutna putanja uvijek počinje znakom `/` i jednoznačno određuje položaj datoteke neovisno o tome gdje se trenutno nalazimo. Primjer:

```
/home/dkrst/nastava/unix
```

Ova putanja označava isti direktorij neovisno o korisnikovoj trenutnoj lokaciji. Možemo je zamisliti kao kompletnu kućnu adresu — bez obzira odakle u svijetu napišemo adresu, razglednica će stići pravoj osobi.

Relativna putanja polazi od trenutnog radnog direktorija i ne počinje s `/`. Možemo je zamisliti kao uputu kako stići do određenog mjesta u gradu: "*na prvom križanju lijevo, pa dva ravno...*" — vrijedi samo ako polazimo s točno određene lokacije. Tako, ako se nalazimo u `/home/dkrst`, putanja `nastava/unix` označava `/home/dkrst/nastava/unix`. Ako se nalazimo u `/home/dkrst/vjezbe`, do istog cilja stizemo putanjom `../nastava/unix` — `..` nas vraća jedan korak gore.

Trenutni radni direktorij ispisujemo naredbom `pwd` (*print working directory*), a mijenjamo naredbom `cd` (*change directory*).

1.2.3 Razlike u odnosu na Windows datotečni sustav

- UNIX **ne dijeli datotečni sustav prema fizičkoj ili logičkoj podjeli diska**. U Windowsima svaka particija ili disk dobiva slovo (C:, D:, E:, ...), dok UNIX sve montira pod jedinstvenim stablom s korijenom /. Različite fizičke ili logičke jedinice za pohranu podataka montiraju se (engl. *mount*) na bilo koje mjesto u stablu — ne nužno na prvoj razini. Ovo je za korisnika transparentno: kad se prijeđe s jednog uređaja ili particije na drugu, navigacija ostaje jednaka.
- UNIX datoteke **nemaju ekstenziju u operacijsko-sustavnom smislu**. Ovo je važno točno razumjeti:
 - Datoteka u svom imenu može sadržavati točku i nastavak, primjerice `program.c` ili `dat1.txt`. Taj nastavak za UNIX je samo dio imena datoteke — ništa više.
 - Operacijski sustav ne gleda nastavak kada odlučuje o tipu datoteke niti kada odlučuje može li se datoteka izvršiti.
 - Hoće li se datoteka moći pročitati, izmijeniti ili pokrenuti određuje isključivo sustav prava pristupa (*permissions*).
 - Zbog toga datoteka koja se zove `dat1.txt` može biti tekstualna datoteka, izvršni binarni program, skripta ili čak direktorij — bez ikakvog ograničenja. Praktična posljedica je da uzorak `*.*` koji bi u Windowsima označavao “sve datoteke” u UNIX-u označava samo datoteke koje u imenu sadrže barem jednu točku. Za “sve datoteke” koristi se jednostavno `*`.

1.2.4 Prava pristupa

UNIX je višekorisnički sustav, što znači da na istom računalu istovremeno može raditi više korisnika. Da različiti korisnici ne bi međusobno smetali — slučajno ili namjerno mijenjajući tuđe datoteke — uveden je **sustav prava pristupa**.

Prava se dodjeljuju u tri razine, prema skupini korisnika:

- **user** — vlasnik datoteke (obično onaj tko ju je stvorio),
- **group** — grupa kojoj datoteka pripada (npr. zaposlenici istog odjela),
- **others** — svi ostali korisnici sustava.

Za svaku skupinu definirana su tri prava:

Oznaka	Naziv	Značenje za datoteke
r	čitanje (<i>read</i>)	dopušta otvaranje i čitanje sadržaja
w	pisanje (<i>write</i>)	dopušta izmjenu sadržaja datoteke
x	izvršavanje (<i>execute</i>)	dopušta pokretanje datoteke kao programa

Dakle, svaka datoteka ima ukupno **devet bitova** prava — tri prava puta tri skupine. Ovih devet bitova pri ispisu naredbom `ls -l` izgleda kako je prikazano na slici 1.3:

	r	W	X	r	W	X	r	W	X
—	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1
	4	2	1	4	2	1	4	2	1
	<i>user</i>			<i>group</i>			<i>others</i>		

Slika 1.3: Struktura prava pristupa (rwx) za vlasnika, grupu i ostale

Primjer: prava `rw-r--r--` znače da vlasnik može čitati i pisati, a grupa i ostali samo čitati. Prava `rw-r-x---` znače da vlasnik ima sva tri prava, grupa može čitati i izvršavati, a ostali nemaju nikakva prava.

Posebnu pažnju zaslužuje pravo **izvršavanja (x)**. Korisnicima koji dolaze s Windowsa ovaj koncept može biti neobičan: na Windowsima sustav prepoznaje izvršne datoteke po **ekstenziji** (.exe, .bat, .com). Na UNIX-u taj mehanizam ne postoji — ekstenzija je samo dio imena i ne nosi nikakvo posebno značenje za sustav. Umjesto toga, UNIX koristi bit **x** kao jedini i dovoljan kriterij: ako je postavljen, datoteka se može pokrenuti. Kao posljedica:

- datoteka `program.txt` može biti potpuno valjani izvršni program (binarni ili shell skripta) ako joj je postavljen bit **x**,
- datoteka `program.exe` na UNIX-u nije ništa posebno — izvršit će se samo ako joj je postavljen bit **x**.

Naravno, davanje prava izvršavanja ne čini bilo koju datoteku stvarno izvršnom: ako sadržaj nije ni binarni kod ni interpretirajuća skripta, pri pokušaju izvršavanja dobit ćemo grešku.

Za **direktorije**, prava **r**, **w**, **x** imaju nešto izmijenjeno značenje:

- **r** — pravo pregleda popisa datoteka (`ls`),
- **w** — pravo izmjene popisa: stvaranja, brisanja, preimenovanja datoteka u direktoriju,
- **x** — pravo “ulaska” u direktorij, tj. pristupa datotekama u njemu (uključujući mogućnost da direktorij bude radni direktorij).

Bitno je razlikovati pravo brisanja datoteke od prava pisanja na samu datoteku: brisanje datoteke kontrolira se pravom **w** na **direktoriju** u kojem se ona nalazi, ne na samoj datoteci.

1.2.4.1 Promjena prava pristupa

Prava pristupa mijenjamo naredbom `chmod`. Postoje dva načina zadavanja:

Numerički (apsolutni) način — svako pravo predstavlja jedan bit s težinom: **r**=4, **w**=2, **x**=1. Zbrajanjem dobivamo brojku za svaku skupinu, a tri brojke (vlasnik, grupa, ostali) zapisujemo zajedno:

```
chmod 644 dat1.txt
```

Postavlja `rw-` za vlasnika ($6 = 4+2$), `r--` za grupu (4) i `r--` za ostale (4) — što odgovara zapisu `rw-r--r--`. Apsolutni način uvijek **u potpunosti** zamjenjuje prethodna prava.

Simbolički način — koristi slova za skupine (**u**, **g**, **o**, ili **a** za sve) i operatore **+** (dodaj), **-** (oduzmi):

```

chmod u+x skripta.sh      # daj vlasniku pravo izvršavanja
chmod go+rx skripta.sh    # daj grupi i ostalima čitanje i izvršavanje
chmod a-x dat1.txt        # oduzmi izvršavanje svima

```

Važno je uočiti razliku između simboličkog i apsolutnog načina zadavanja prava. Kod simboličkog načina mijenjaju se samo navedena prava za navedenu skupinu, a sva ostala prava ostaju nepromijenjena. Naredba neće imati nikakav učinak ako dotična skupina već ima pravo koje želimo dodati, ili nema pravo koje na datoteci želimo oduzeti.

Kod apsolutnog načina uvijek je potrebno navesti sva prava za vlasnika, grupu i ostale. Rezultat naredbe je točno navedeni skup prava, bez obzira na prethodne postavke.

Pored običnih korisnika postoji i poseban korisnik s neograničenim ovlastima — **administrator** ili **superuser**, u UNIX terminologiji nazvan **root**. Za razliku od običnih korisnika, root može čitati, mijenjati i brisati bilo koju datoteku te pokretati i zaustavljati bilo koji proces. Upravo zbog toga administratorski račun treba koristiti s velikim oprezom — greška izvršena s root ovlastima može nepovratno oštetiti sustav.

Riječ “root” u UNIX kontekstu ima dva potpuno različita značenja: **administratorski korisnički račun** (s matičnim direktorijem /root) i **korijenski direktorij** datotečnog stabla (/). Ova dva pojma nemaju nikakve veze osim što dijele isti engleski naziv — početnicima ova podudarnost zna stvarati zabunu, ali iskusni korisnici iz konteksta uvijek znaju o čemu se radi.

1.3 Naredbena ljuska

Ljuska je program koji čita naredbe sa standardnog ulaza (stdin), interpretira ih, izvršava i ispisuje rezultat na standardni izlaz (stdout), a poruke o greškama na standardni izlaz za greške (stderr).

1.3.1 Osnovne naredbe

Naredba	Opis	Primjer
pwd	ispisuje apsolutnu putanju trenutnog direktorija	pwd
cd	mijenja trenutni direktorij; bez argumenta vraća u matični (\$HOME)	cd /home/dkrst, cd ..., cd
ls	ispisuje sadržaj direktorija (-l dugi format, -a skrivene, -t po vremenu)	ls -la
mkdir	stvara novi direktorij (-p stvara i sve roditelje)	mkdir projekti, mkdir -p a/b/c
cp	kopira datoteke (-r rekurzivno)	cp dat1.txt dat2.txt, cp -r dir1/ dir2/
mv	premješta ili preimenovava	mv dat1.txt arhiva/, mv staro.txt novo.txt
rm	brise datoteke (-r rekurzivno, -f prisilno bez upita)	rm dat1.txt, rm -rf stari_dir/
cat	ispisuje sadržaj jedne ili više datoteka na standardni izlaz	cat dat1.txt, cat *.log

man

otvara priručnik za zadanu
naredbu ili sistemski pozivman ls

Upozorenje za rm: UNIX nema “koš za smeće” — brisanje je trajno. Posebno opasna kombinacija je `rm -rf` koja briše rekurzivno bez ikakvog upita. Naredba `rm -rf /` (ili neka njezina slučajna varijacija s razmacima) može obrisati cijeli sustav. Uvijek dvaput provjerite što ste utipkali prije nego pritisnete Enter.

Zadavanjem naredbi iz ljuske možemo pokretati korisničke programe, ali i koristiti naredbe ljuske. Osnovna razlika leži u činjenici da svakom korisničkom programu uvijek odgovara izvršna datoteka — program, koja se pokretanjem učitava u memoriju i izvršava. Nasuprot tome, pokretanjem ugrađenih naredbi ljuske (engl. *Shell Built-in Commands*) aktiviramo funkcije implementirane u samoj ljusci.

S aspekta prosječnog korisnika ova razlika nema veliko značenje i nije nužno uočljiva kada ljusci zadamo neku naredbu. Ipak, ugrađene naredbe ovise o izboru ljuske i ne stvaraju novi proces pri izvršavanju. Naredba `cd` je primjer ugrađene naredbe (nema odgovarajuću izvršnu datoteku), a pokretanjem ove naredbe mijenja se radni direktorij ljuske. Pokretanje vanjskog programa uvijek znači i stvaranje novog procesa, dok pokretanje ugrađene naredbe ljuske ne stvara novi proces već se poziva određena funkcionalnost same ljuske — procesa koji se već aktivno izvodi na sklopovlju računala.

1.3.2 Pokretanje programa u pozadini

Po zadanom, kad pokrenemo neki program iz ljuske, ona čeka da on završi prije nego nam vrati kontrolu. To je u redu za kratke naredbe, ali za one koje traju dugo (kompresiranje velikog direktorija, kompilacija velikog projekta, mrežna preuzimanja) često želimo nastaviti raditi paralelno. UNIX to omogućuje znakom `&` na kraju naredbe — program se pokreće “u pozadini”, a ljuska odmah vraća kontrolu:

```
$ tar -czf backup.tar.gz /home/dkrst &  
[2] 3963  
$
```

Ljuska ispisuje **redni broj posla** u uglatim zagradama ([2]) i **PID** (Process ID) procesa (3963), te odmah pokazuje promptom da je spremna za novu naredbu. Status pokrenutih poslova provjeravamo naredbom `jobs`, premještamo ih iz pozadine u prvi plan s `fg`, a iz prvog plana u pozadinu s `bg` (uz prethodno suspendiranje pomoću `Ctrl+Z`).

1.3.3 Priručnik — naredba man

`man` je tradicionalni UNIX alat za dokumentaciju, dostupan izravno u terminalu bez potrebe za internetskom vezom ili pretraživačem. Iako većina dokumentacije danas postoji i online, `man` stranice ostaju nezamjenjive na ugrađenim sustavima, serverskim računalima i udaljenim sustavima koji nemaju grafičko sučelje, a ponekad ni pristup internetu — a to su upravo okruženja u kojima UNIX pokazuje svoje najveće prednosti. Uz to, `man` stranice su uvijek usklađene s točnom verzijom alata instaliranog na konkretnom sustavu, što online izvori ne mogu jamčiti.

Pretraživanje stranica vrši se naredbom `man <naredba>`, navigacija strelicama ili Space tipkom, a izlazi pritiskom na q. Stranice su organizirane u sekcije: - sekcija 1 — korisničke naredbe (`man 1 ls`), - sekcija 2 — sistemski pozivi (`man 2 open`), - sekcija 3 — bibliotečke funkcije (`man 3 printf`).

1.4 Preusmjeravanje ulaza i izlaza

Svaki program u UNIX-u standardno koristi tri toka podataka: standardni ulaz (`stdin`), standardni izlaz (`stdout`) i standardni izlaz za greške (`stderr`). Ljuska pruža mehanizam **preusmjeravanja** kojim te tokove možemo prije pokretanja programa povezati s datotekama. Sam program time se ne mijenja — i dalje misli da čita sa `stdin` i piše na `stdout` — ali ljuska tiho preusmjeri te tokove na zadane datoteke.

Najčešći operatori preusmjeravanja u **bash** ljusci:

Operator	Opis
<code>></code>	preusmjeri <code>stdout</code> u datoteku (briše postojeći sadržaj)
<code>>></code>	dodaj <code>stdout</code> na kraj datoteke
<code><</code>	čitaj <code>stdin</code> iz datoteke
<code>2></code>	preusmjeri <code>stderr</code> u datoteku
<code>2>></code>	dodaj <code>stderr</code> na kraj datoteke
<code>&></code>	preusmjeri <code>stdout</code> i <code>stderr</code> zajedno
<code>&>></code>	dodaj <code>stdout</code> i <code>stderr</code> na kraj
<code>2>&1</code>	spoji <code>stderr</code> na <code>stdout</code> (vrlo čest "trik")

Primjer — preusmjerimo izlaz `ls -la` u datoteku, pa pogledajmo sadržaj:

```
$ ls -la > popis.txt
$ cat popis.txt
total 8
drwxr-xr-x 4 dkrst users 168 2026-03-11 18:04 .
drwxr-xr-x 11 dkrst users 352 2026-03-11 12:38 ..
-rw-r--r-- 1 dkrst users 33 2026-03-11 12:58 dat1.txt
-rw-r--r-- 1 dkrst users 33 2026-03-11 16:12 dat2.txt
-rw-r--r-- 1 dkrst users 0 2026-03-11 18:04 popis.txt
```

Zasebno preusmjeravanje `stdout` i `stderr`:

```
$ ls /home /nepostoji > rezultati.txt 2> greske.txt
$ cat rezultati.txt
/home:
marko ana petar
$ cat greske.txt
ls: cannot access '/nepostoji': No such file or directory
```

Posebna datoteka **/dev/null** je "crna rupa" jezgre — sve što se u nju upiše tiho nestaje. Korisna je kad želimo potpuno odbaciti neki izlaz:

```
$ neka_naredba 2> /dev/null          # zanemari greške, prikaži samo stdout
$ neka_naredba > /dev/null 2>&1      # zanemari sve, samo izvrši
```

1.5 Ulančavanje naredbi (pipes)

Pored preusmjeravanja u datoteku, izlaz jedne naredbe možemo izravno spojiti na ulaz druge — operatorom `|` (vertikalna crta, *pipe*). Ovo je realizacija temeljne UNIX filozofije, koju vrijedi još jednom spomenuti: *mali programi koji rade jednu stvar dobro mogu se kombinirati u moćne lance obrade*.

Klasičan primjer — koliko redaka u log datoteci sadrži riječ ERROR?

```
$ cat program.log | grep "ERROR" | wc -l
42
```

Lanac radi ovako: cat čita datoteku i šalje sadržaj na stdout; taj stdout ulančan je s stdin-om naredbe grep koja propušta samo retke s riječi ERROR; izlaz grep-a ulančan je s wc -l koji broji retke. Konačni rezultat — broj redaka s greškom.

Drugi primjer — prikaži samo .txt datoteke u direktoriju:

```
$ ls -al | grep "\.txt"
-rw-r--r-- 1 dkrst users 33 2026-03-11 dat1.txt
-rw-r--r-- 1 dkrst users 45 2026-03-11 biljeske.txt
```

Lanci mogu biti dugi koliko god je potrebno. Klasičan primjer iz administracije — ispiši **različita** korisnička imena koja su u auth.log izazvala grešku:

```
$ grep "ERROR" auth.log | awk '{print $5}' | sort | uniq
admin
ana
marko
```

Ovdje grep filtrira retke s greškom, awk izvlači peto polje (korisničko ime), sort ih poreda, a uniq uklanja duplikate (uniq traži susjedne duplikate, pa zato sortiramo prije).

Anonimni cjevovod stvoren operatorom | postoji samo dok se naredbe izvršavaju. Pored njega postoji i **imenovani cjevovod** (FIFO), koji je trajan objekt u datotečnom sustavu — može ga otvoriti bilo koji proces koji zna njegovu putanju, čak i nakon završetka procesa koji ga je stvorio. Stvara se naredbom mkfifo, a u ispisu ls -l prikazuje se oznakom p.

1.6 UNIX procesi

U trenutku kada se program s diska učitava u memoriju, nastaje **proces** koji odgovara pokrenutom programu. Procesu se dodjeljuju resursi, stvara se njegovo radno okruženje (*environment*), definiraju se ovlasti i prava pristupa, te se dodjeljuju identifikacijske oznake. Najvažnije od njih:

- **PID** (*Process ID*) — jedinstveni broj kojim sustav identificira proces. Jezgra svakom procesu dodjeljuje vlastiti PID.
- **PPID** (*Parent Process ID*) — PID roditeljskog procesa, odnosno onog koji je ovaj proces pokrenuo. Ako roditelj završi prije djeteta, dijete nasljeđuje proces init (PID 1) kao novog roditelja.
- **UID** (*User ID*) — broj korisnika koji je vlasnik procesa. Određuje ovlasti procesa nad resursima sustava.
- **GID** (*Group ID*) — broj grupe vlasnika procesa.
- **EUID, EGID** — *Effective UID/GID*; koriste se za privremeno podizanje ovlasti, npr. pri pokretanju programa s postavljenim *setuid* bitom (klasičan primjer: passwd — alat za promjenu lozinke koji mora pisati u /etc/shadow, datoteku kojoj obični korisnici nemaju pristup).

Procese pregledavamo naredbom ps. ps -ef prikazuje sve procese u sustavu s ključnim atributima:

```
$ ps -ef | head -5
UID      PID  PPID  C  STIME TTY          TIME CMD
root         1      0  0  Mar01 ?        00:00:42 /sbin/init
root         2      0  0  Mar01 ?        00:00:00 [kthreadd]
```

```
dkrst    14567 14566  0 09:15 pts/0    00:00:00 -bash
dkrst    25016 14567  0 12:30 pts/0    00:00:00 ./program
```

Bitna napomena: novi proces u UNIX-u uvijek nastaje iz **postojećeg** procesa — sistemskim pozivom `fork()` koji dijeli postojeći proces u dva. Time se cijeli sustav procesa organizira kao stablo s `init`-om u korijenu. O ovome će biti više riječi u poglavlju o procesima.

1.7 Shell skripte

Pored interaktivnog načina rada, ljusku možemo koristiti i **programski**. Niz naredbi zapisan u tekstualnoj datoteci kojoj smo dali pravo izvršavanja postaje cjelovit program — **shell skripta**. Skripte koristimo za automatizaciju, od jednostavnih svakodnevnih radnji do složenih administracijskih i programerskih zadataka.

Pri izradi skripti potrebno je imati na umu **tip ljuske** za koji je skripta pisana. Iako su osnovne naredbe iste, detalji poput definiranja varijabli, uvjetnog grananja i petlji razlikuju se između ljuski. Najčešće ljuske su `bash` (Bourne Again Shell, najraširenija na Linuxu) i `csh` (C Shell). Razlike u sintaksi:

Operacija	bash	csh
Prva linija skripte	<code>#!/bin/bash</code>	<code>#!/usr/bin/csh</code>
Dodjela varijable	<code>ime="Marko"</code>	<code>set ime="Marko"</code>
Čitanje varijable	<code>echo \$ime</code>	<code>echo \$ime</code>
Uvjetni izraz	<code>if [\$a = \$b]; then</code>	<code>if (\$a == \$b) then</code>
Kraj if bloka	<code>fi</code>	<code>endif</code>
for petlja	<code>for i in lista; do</code>	<code>foreach i (lista)</code>
Kraj for petlje	<code>done</code>	<code>end</code>
while petlja	<code>while [uvjet]; do</code>	<code>while (uvjet)</code>
Izlazni status	<code>\$?</code>	<code>\$status</code>
Argumenti skripte	<code>\$1, \$2, ...</code>	<code>\$argv[2], \$argv[1], ...</code>

Prvi redak skripte (`#!/bin/bash`) zove se **shebang** — to je posebna direktiva koju jezgra prepoznaje pri pokretanju skripte i koristi za odabir interpretera. Komentari u skripti počinju znakom `#` i traju do kraja retka.

Da bi se ilustrirale razlike u sintaksi između `bash` i `csh` ljusaka, u nastavku su dani isti primjeri u oba okruženja jedan pored drugog.

if/else grananje — provjera postoji li datoteka zadana kao argument skripte:

bash	csh
<pre>#!/bin/bash if [-f "\$1"]; then echo "\$1 postoji" else echo "\$1 ne postoji" fi</pre>	<pre>#!/usr/bin/csh if (-f "\$argv[1]") then echo "\$argv[1] postoji" else echo "\$argv[1] ne postoji" endif</pre>

while petlja — odbrojavanje unazad od broja zadanog kao argument:

bash	csh
#!/bin/bash a=\$1 while [\$a -gt 0]; do echo \$a ((a--)) done	#!/usr/bin/csh set a=\$1 while (\$a > 0) echo \$a @ a-- end

for/foreach petlja — listanje sadržaja direktorija sortiranog po vremenu izmjene:

bash	csh
#!/bin/bash a=\$(ls -t \$1) for i in \$a; do echo \$i done	#!/usr/bin/csh set a=`ls -t \$1` foreach i (\$a) echo \$i end

case/switch grananje — listanje s izborom načina prikaza (po veličini ili po vremenu), ovisno o prvom argumentu skripte:

bash	csh
#!/bin/bash case \$1 in size) a=\$(ls -S \$2) ;; time) a=\$(ls -t \$2) ;; *) a=\$(ls \$2) ;; esac for i in \$a; do echo \$i done	#!/usr/bin/csh switch (\$1) case size: set a=`ls -S \$2` breaksw case time: set a=`ls -t \$2` breaksw default: set a=`ls \$2` endsw foreach i (\$a) echo \$i end

Naredba `break` prekida izvršavanje petlje, a naredba `continue` preskače ostatak trenutne iteracije i nastavlja s idućom. Obje naredbe rade jednako u `bash` i `csh` ljusci.

Da bi skripta postala izvršna, treba joj dati pravo izvršavanja (x):

```
$ chmod +x skripta.sh
$ ./skripta.sh
```

U ovom direktoriju nalazi se nekoliko jednostavnih `bash` skripti koje ilustriraju ključne koncepte:

1.7.1 Najjednostavnija skripta

- **pozdrav.sh** — minimalan primjer skripte: shebang, komentar, `echo` poziv, korištenje varijabli okruženja i naredbenih supstitucija.

```
#!/bin/bash
# Najjednostavnija shell skripta - ispisuje pozdrav korisniku.

echo "Pozdrav, $USER!"
echo "Trenutni radni direktorij: $(pwd)"
echo "Datum i vrijeme: $(date)"
```

Prvi redak je shebang. Drugi redak je komentar. U pozivima `echo` koristimo dva mehanizma za umetanje vrijednosti u string:

- **\$USER** — varijabla okruženja koju ljuska automatski postavlja na ime trenutnog korisnika.
- **\$(naredba)** — naredbena supstitucija (engl. *command substitution*): ljuska izvrši naredbu unutar **\$()**, pa njezin izlaz umetne na to mjesto. U ovom primjeru **\$(pwd)** zamjenjuje se ispisom trenutnog radnog direktorija.

Pokretanje:

```
$ chmod +x pozdrav.sh
$ ./pozdrav.sh
Pozdrav, dkrst!
Trenutni radni direktorij: /home/dkrst/Programiranje_z_UX/P01-Osnove_UNIXa
Datum i vrijeme: Mon May 4 11:33:43 UTC 2026
```

1.7.2 Petlje, varijable i argumenti

- **brojac.sh** — broji od 1 do N, gdje N može biti zadan kao argument naredbenog retka, ili je 5 ako argument nije zadan.

```
#!/bin/bash
# Broji od 1 do N (N se daje kao argument, ili 5 ako nije zadan).

# Provjera broja argumenata
if [ $# -eq 0 ]; then
    n=5
else
    n=$1
fi

echo "Brojim od 1 do $n..."

# for petlja
for i in $(seq 1 $n); do
    echo " $i"
done

echo "Gotovo!"
```

Skripta uvodi nekoliko bitnih koncepata:

- **\$#** — broj argumenata kojima je skripta pozvana (analogno `argc` u C-u).
- **\$1** — prvi argument (analogno `argv[2]`). Slično tome **\$2**, **\$3** itd. su drugi, treći argument; **\$0** je ime skripte (analogno `argv[0]`).
- **if [uvjet]; then ... else ... fi** — uvjetno grananje. Razmaci unutar zagrada su obavezni.
- **-eq** — operator jednakosti za cijele brojeve. Za znakovne nizove koristimo **=** (npr. `if [ "$ime" = "Marko" ]`).
- **for i in lista; do ... done** — petlja kroz vrijednosti u listi. Lista se može zadati eksplicitno (`for i in 1 2 3; do`), ili generirati naredbom (`for i in $(seq 1 5); do`), ili kao popis datoteka (`for f in *.txt; do`).
- **\$(seq 1 \$n)** — naredbena supstitucija; `seq` generira slijed brojeva.

Pokretanje:

```
$ ./brojac.sh
Brojim od 1 do 5...
1
2
3
4
5
Gotovo!
$ ./brojac.sh 3
Brojim od 1 do 3...
1
```

```
2
3
Gotovo!
```

1.7.3 Praktičan primjer — backup direktorija

- **backup.sh** — stvara komprimiranu arhivu zadanog direktorija, s timestampom u imenu arhive. Tipičan primjer skripte kakvu bismo zaista mogli koristiti svakodnevno.

```
#!/bin/bash
# Stvara komprimiranu arhivu zadanog direktorija s timestampom u imenu.
#
# Korištenje: ./backup.sh <direktorij>

# Provjera argumenata
if [ $# -ne 1 ]; then
    echo "Korištenje: $0 <direktorij>"
    exit 1
fi

dir=$1

# Provjera da direktorij postoji
if [ ! -d "$dir" ]; then
    echo "Greška: '$dir' nije direktorij ili ne postoji."
    exit 2
fi

# Ime arhive: ime-direktorija_godina-mjesec-dan_sat-min.tar.gz
basename=$(basename "$dir")
timestamp=$(date +%Y-%m-%d_%H-%M)
arhiva="${basename}_${timestamp}.tar.gz"

echo "Stvaram arhivu '$arhiva' iz direktorija '$dir'..."
tar -czf "$arhiva" "$dir"

if [ $? -eq 0 ]; then
    velicina=$(du -h "$arhiva" | cut -f1)
    echo "Gotovo. Arhiva '$arhiva' (veličina: $velicina) uspješno stvorena."
else
    echo "Greška pri stvaranju arhive."
    exit 3
fi
```

Skripta uvodi nove koncepte:

- **exit N** — završi skriptu s izlaznim statusom N. Konvencija: 0 znači uspjeh, ostale vrijednosti (1-255) različite tipove grešaka.
- **-d "\$dir"** — test je li putanja direktorij. Slično: -f za običnu datoteku, -e za bilo što (samo postoji li), -r/-w/-x za prava pristupa, -z za prazan string. Operator ! negira test.
- **navodnici oko "\$dir"** — uvijek navodimo varijable u dvostrukim navodnicima da bismo izbjegli probleme ako vrijednost sadrži razmake ili specijalne znakove. Bez njih bi tar -czf "\$arhiva" \$dir puknuo na imenima poput Moji dokumenti.
- **?** — izlazni status zadnje izvršene naredbe. 0 je uspjeh, sve drugo je neka greška.
- **\${ime}_dodatak** — vitičaste zagrade oko imena varijable kad se nadovezuje s tekstom koji bi se inače mogao tumačiti kao dio imena.
- **tar -czf** — alat za arhiviranje: c create, z gzip kompresija, f ime datoteke.

Pokretanje:

```
$ chmod +x backup.sh
$ ./backup.sh slike
```

Stvaram arhivu 'slike_2026-05-04_11-33.tar.gz' iz direktorija 'slike'...
 Gotovo. Arhiva 'slike_2026-05-04_11-33.tar.gz' (veličina: 120K) uspješno stvorena.

Bez argumenta ili s neispravnim argumentom skripta uredno javlja grešku i izlazi:

```
$ ./backup.sh
Korištenje: ./backup.sh <direktorij>
$ ./backup.sh nepostojeci
Greška: 'nepostojeci' nije direktorij ili ne postoji.
```

1.7.4 Pretraga datoteka — find, grep, brojanje

- **trazi.sh** — pretražuje sve .txt datoteke u zadanom direktoriju (rekurzivno) i ispisuje koliko puta se u svakoj pojavljuje zadana ključna riječ. Skripta kombinira većinu dosad spomenutih alata: testovi, petlje, find, grep, aritmetika.

```
#!/bin/bash
# Pretražuje sve tekstualne datoteke u zadanom direktoriju koje sadrže
# zadanu ključnu riječ i ispisuje broj pojavljivanja u svakoj datoteci.
#
# Korištenje: ./trazi.sh <direktorij> <kljucna_rijec>

if [ $# -ne 2 ]; then
    echo "Korištenje: $0 <direktorij> <kljucna_rijec>"
    exit 1
fi

dir=$1
rijec=$2

if [ ! -d "$dir" ]; then
    echo "Greška: '$dir' nije direktorij."
    exit 2
fi

echo "Tražim '$rijec' u tekstualnim datotekama unutar '$dir'..."
echo

ukupno=0
nadenou=0

# Pretraga svih .txt datoteka u direktoriju i poddirektorijima
for f in $(find "$dir" -type f -name "*.txt"); do
    broj=$(grep -c "$rijec" "$f")
    if [ "$broj" -gt 0 ]; then
        echo "  $f: $broj"
        ukupno=$((ukupno + broj))
        nadenou=$((nadenou + 1))
    fi
done

echo
echo "Ukupno pojavljivanja: $ukupno (u $nadenou datoteka)"
```

Novi koncepti:

- **find <gdje> -type f -name "*.txt"** — moćan alat za pretragu datoteka. -type f traži obične datoteke, -name "*.txt" traži po imenu, -mtime, -size po vremenu/veličini itd.
- **grep -c "\$rijec" "\$f"** — -c (count) vraća broj redaka koji sadrže uzorak (umjesto same retke).
- **\$((aritmetika))** — aritmetička evaluacija. \$((a + b)) zbraja, \$((a * b)) množi, \$((a / b)) cjelobrojno dijeli.

Pokretanje:

```
$ ./trazi.sh /tmp/dokumenti linux
Tražim 'linux' u tekstualnim datotekama unutar '/tmp/dokumenti'...

/tmp/dokumenti/biljeske.txt: 5
/tmp/dokumenti/projekt/readme.txt: 2

Ukupno pojavljivanja: 7 (u 2 datoteka)
```

1.7.5 Provjera ispravnosti argumenata i izlazni status

- **provjeri.sh** — provjerava je li skripta pozvana ispravno (točno jedan argument) i postoji li zadana datoteka, pa ispisuje njezin broj redaka i zadnjih 5 redaka. Demonstrira tipičan oblik provjere argumenata i razlikovanje različitih razloga greške različitim izlaznim kodovima.

```
#!/bin/bash

# Provjeri je li zadan točno jedan argument
if [ $# -ne 1 ]; then
    echo "Koristenje: $0 <ime_datoteke>"
    exit 1
fi

# Provjeri postoji li zadana datoteka
if [ ! -f "$1" ]; then
    echo "Greska: datoteka '$1' ne postoji."
    exit 2
fi

# Sve je u redu -- obradi datoteku
echo "Obrada datoteke: $1"
echo "Broj redaka: $(wc -l < $1)"
echo "Zadnjih 5 redaka:"
tail -5 "$1"

exit 0
```

Izlazni status zadnje izvršene naredbe dostupan je kroz varijablu `?` (bash) ili `$status` (csh):

```
$ ./provjeri.sh datoteka.txt
Obrada datoteke: datoteka.txt
Broj redaka: 42
...
$ echo $?
0
$ ./provjeri.sh nepostojeca.txt
Greska: datoteka 'nepostojeca.txt' ne postoji.
$ echo $?
2
```

Dobra je praksa različitim greškama dodjeljivati različite izlazne kodove kako bi ih pozivajuća skripta (ili ljuska) mogla međusobno razlikovati. Vrijednost 0 po dogovoru znači uspješan završetak, a sve vrijednosti različite od nule označavaju neku grešku.

1.7.6 Obrada svih datoteka u direktoriju

- **prebroji.sh** — iterira po svim `.c` datotekama u trenutnom direktoriju, za svaku ispisuje broj redaka te na kraju daje ukupan zbroj. Demonstrira `for` petlju nad uzorkom datoteka, naredbenu supstituciju i aritmetičko zbrajanje u akumulator.

```
#!/bin/bash

ukupno=0
```

```

for datoteka in *.c; do
    # Provjeri postoji li uopce neka .c datoteka
    # (ako nema, for-petlja u bashu prolazi s doslovnim "*.c")
    if [ ! -f "$datoteka" ]; then
        echo "Nema .c datoteka u direktoriju."
        exit 1
    fi

    redci=$(wc -l < "$datoteka")
    echo "$datoteka: $redci redaka"
    (( ukupno += redci ))
done

echo "---"
echo "Ukupno: $ukupno redaka u svim .c datotekama"

```

Pokretanje (npr. iz direktorija s .c izvornim kodom):

```

$ ./prebroji.sh
funkcije.c: 5 redaka
niz.c: 21 redaka
nizfn.c: 40 redaka
pozdrav.c: 7 redaka
pozdrav_fn.c: 7 redaka
---
Ukupno: 80 redaka u svim .c datotekama

```

1.8 Što dalje?

U sljedećem poglavlju krećemo s osnovama programiranja u C-u i procesa prevođenja, što su preduvjeti za C programiranje na UNIX sustavima koje obrađujemo u ostatku skripte. Sve što smo naučili u ovom poglavlju — rad u terminalu, prava pristupa, preusmjeravanje — koristit ćemo praktično u radu s primjerima.

Čitatelju koji želi proširiti svoje znanje izvan dosega ove skripte preporučujemo nekoliko klasičnih referenci. Za dublji uvod u UNIX okruženje, alate i filozofiju ulančavanja, klasik je *The UNIX Programming Environment*, Kernighan & Pike [1] — knjiga koju mnogi smatraju najboljim uvodom u “UNIX način razmišljanja”. Za napredne teme koje će nas zaokupiti u idućim poglavljima — systemske pozive, procese, datotečni sustav i međuprocenu komunikaciju — referenca koja prati skriptu na više mjesta je *Advanced Programming in the UNIX Environment*, Stevens & Rago [2]. Konačno, za temeljito razumijevanje općih koncepata operacijskih sustava (procesi, niti, sinkronizacija, memorija, raspoređivanje) na hrvatskom jeziku preporučujemo sveučilišni udžbenik *Operacijski sustavi*, Budin, Golub, Jakobović i Jelenković [3], standardno štivo na hrvatskim sveučilištima.

1.9 Bibliografija

[1] B. W. Kernighan and R. Pike, *The UNIX Programming Environment*. Englewood Cliffs, NJ, USA: Prentice Hall, 1984.

[2] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2013.

[3] L. Budin, M. Golub, D. Jakobović, and L. Jelenković, *Operacijski sustavi*, 3. izd. Zagreb, Hrvatska: Element, 2013.

[4] TOP500 Project, "TOP500 Supercomputer Sites — Operating System Share." [Online]. Dostupno: <https://en.wikipedia.org/wiki/TOP500>. Pristupljeno: svibanj 2026.

[5] H. Horvat, *Kratka povijest UNIX-a — Od UNICS-a do FreeBSD-a i Linuxa*. Osijek: vlastita naklada, 2017. ISBN 978-953-59438-0-8. [Online]. Dostupno: <https://www.opensource-osijek.org>. Pristupljeno: svibanj 2026.

Poglavlje 2

Osnove programiranja

U ovom poglavlju opisani su osnovni koraci u procesu stvaranja programske podrške — od prve napisane linije izvornog koda do trenutka kada korisnik pokrene gotovu aplikaciju. Obrađeni su sljedeći koraci i alati:

- **prevođenje i povezivanje** — postupak kojim izvorni kod postaje izvršna datoteka, te tipovi datoteka koji u tom procesu sudjeluju,
- **GCC** — GNU prevodilac za C i C++, njegova sintaksa i najvažnije opcije,
- **make** — alat za automatiziranje prevođenja u projektima s više datoteka izvornog koda,
- **archive objektnog koda** — stvaranje i korištenje statičkih biblioteka funkcija.

Iako se navedeni postupci i primjeri u prvom redu odnose na razvoj programske podrške za operacijski sustav UNIX u programskom jeziku **C**, opisani principi prevođenja i povezivanja, uz manje razlike, vrijede za većinu operacijskih sustava i prevodilačkih programskih jezika.

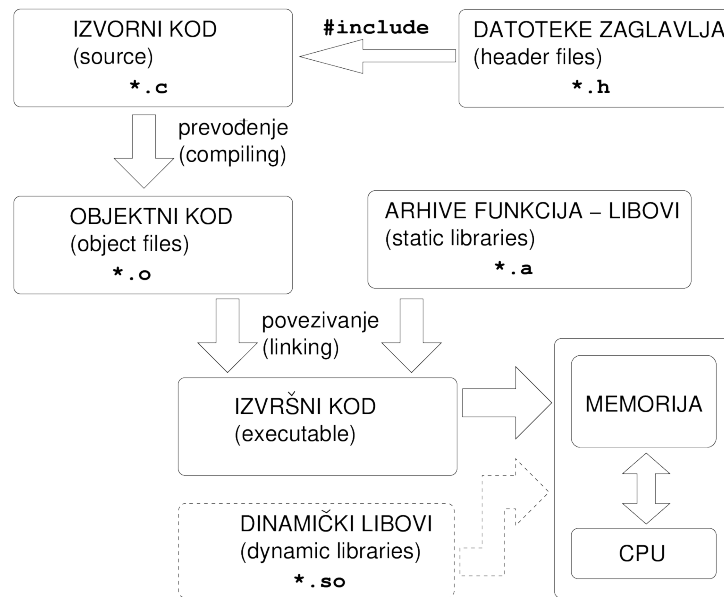
Prevodilački i interpreterski jezici

Pored prevodilačkih programskih jezika, kod kojih se izvorni kod postupkom prevođenja i povezivanja pretvara u izvršnu datoteku, tj. strojni kôd, postoje i interpreterski jezici. Kod interpreterskih jezika naredbe izvornog koda analiziraju se i izvršavaju u trenutku izvođenja — dobar primjer su shell skripte opisane u poglavlju 1.

Sintaksu C-a podrazumijevamo poznatom — za sustavno učenje samog jezika preporučujemo klasičnu *The C Programming Language*, Kernighan & Ritchie [1], odnosno na hrvatskom jeziku FESB-ovu nastavnu skriptu *Programiranje C jezikom*, Mateljan [2], dostupnu i u digitalnom obliku.

2.1 Prevođenje i povezivanje izvornog koda

Postupak dobivanja izvršne datoteke, tj. programa koji možemo učitati u memoriju i pokrenuti, sastoji se od dva osnovna koraka i uključuje više različitih tipova datoteka. Ovaj proces započinje pisanjem izvornog koda koji, u slučaju programskog jezika **C**, uključuje datoteke izvornog koda (datoteke s ekstenzijom **.c**) i datoteke zaglavlja (ekstenzija **.h**). Osnovni koraci ovog procesa prikazani su na slici 2.1:



Slika 2.1: Prevođenje i povezivanje

- **Prevođenje** (*Compiling*) Programski prevodilac analizira izvorni kod i prevodi ga u objektni. Ovaj postupak započinje obradom *predprocesorom* koji kod priprema za prevođenje, dio čega je i uključivanje datoteka zaglavlja (**.h**) u izvorni kod. Svaka datoteka izvornog koda prevodi se u datoteku objektnog koda (ekstenzija **.o**).
- **Povezivanje** (*Linking*) Datoteke objektnog koda povezuju se u izvršnu datoteku. Pored objektnih datoteka, u postupku povezivanja kao ulazne datoteke koriste se i arhive funkcija ili *statički libovi*. Libovi su kolekcije objektnih datoteka pohranjenih na način da je iz njih moguće izdvojiti objektne datoteke u izvornom obliku. Da bi u postupku povezivanja iz datoteka objektnog koda mogla biti generirana izvršna datoteka, u točno jednoj objektnoj datoteci mora biti definirana funkcija *main*, koja predstavlja ulaznu točku programa.

Integrirana razvojna okruženja (IDE)

Moderna integrirana razvojna okruženja (*Integrated Development Environment* — IDE) programeru olakšavaju postupak prevođenja izvornog koda u izvršnu datoteku. Ovi programi automatski generiraju datoteku s pravilima za prevođenje i povezivanje, a korisnik proces stvaranja izvršne datoteke pokreće jednostavnim odabirom opcije *build*. Pored automatiziranog prevođenja i povezivanja, integrirana razvojna okruženja obično uključuju editor, program za pronalaženje pogrešaka (*debugger*), alate za automatsko generiranje koda (*wizards*), alate za kreiranje korisničkog sučelja (*Graphical User Interface* — GUI) i druge dodatke koji značajno olakšavaju i ubrzavaju proces razvoja programske podrške.

Kod ovih je sustava dobar dio postupaka i koraka u stvaranju programa skriven od programera, pa mnogi, čak i iskusniji programeri slabo poznaju procese koji su potrebni da bi od izvornog koda nastala njihova najnovija aplikacija. Iako je ovim život programera značajno olakšan, svaki programer koji drži do sebe trebao bi ovladati osnovnim principima i koracima u prevođenju i povezivanju izvornog koda, kao i shvatiti odnose među tipovima datoteka koje u tom procesu sudjeluju, čime

stvara temelj za kvalitetniji razvoj programerske vještine.

Za razvoj programske podrške na UNIX sustavima danas je na raspolaganju više kvalitetnih integriranih razvojnih okruženja. Među najpopularnijima su *Visual Studio Code* (besplatan, s bogatim skupom dodataka), *CLion* (komercijalan, namijenjen C i C++ razvoju) te *Eclipse CDT* (besplatan, otvorenog koda).

2.2 GCC — GNU projekt C i C++ prevodilac

GCC, besplatan prevodilac otvorenog koda, dio je **GNU** paketa prevodilaca koji uključuje prevodilace za **C**, **C++**, **Fortran**, **Ada**, **Go** i druge programske jezike, kao i libove potrebne za povezivanje programa. Dostupan je pod **GPL** (*General Public License*) licencom za brojne platforme s ciljem omogućavanja razvoja prenosivog koda za različite arhitekture i okruženja. **GCC** prevodilac se koristi za prevođenje i povezivanje (*compiling & linking*) **C** i **C++** izvornog koda.

Što je GNU?

GNU (rekurzivni akronim za *GNU's Not UNIX*) je projekt slobodnog softvera koji je 1983. godine pokrenuo Richard Stallman s ciljem stvaranja potpuno slobodnog UNIX-kompatibilnog operacijskog sustava. GNU projekt razvio je velik broj ključnih alata koje danas svakodnevno koristimo: `gcc` (prevodilac), `make`, `bash`, `gdb` (debugger), `emacs`, `coreutils` (`ls`, `cp`, `mv`, ...) i mnoge druge. Kad je 1991. Linus Torvalds objavio Linux jezgru, ona se prirodno kombinirala s GNU alatima, što je dalo cjelovit slobodan operacijski sustav koji mnogi nazivaju **GNU/Linux**. Zaklada za slobodan softver (engl. *Free Software Foundation*, FSF), koja stoji iza GNU projekta, danas također održava i licencu GPL (engl. *GNU General Public License*) pod kojom se objavljuje velik dio slobodnog softvera.

2.2.1 Sintaksa i osnovne opcije

```
gcc [opcije] ulazne_datoteke
```

U daljnjem tekstu navedene su osnovne opcije i tipovi datoteka koje **gcc** koristi. Detaljnu uputu za korištenje **gcc** prevodilaca moguće je dobiti naredbom `man gcc`.

- **-c** — samo prevođenje, bez povezivanja (rezultat je objektna datoteka)
- **-O**razina — razina optimizacije (0-3)
- **-g** — generira informaciju za *debugger* (program za pronalaženje grešaka)
- **-Idir** — uključi `dir` u listu direktorija s datotekama zaglavlja
- **-Ldir** — uključi `dir` u listu direktorija s arhivama objektnog koda
- **-Wall** — prikaži sva upozorenja (*Warning all*)
- **-o** datoteka — naziv izlazne datoteke

Ukoliko ime izlazne izvršne datoteke nije eksplicitno navedeno opcijom **-o**, uobičajeno ime izvršne datoteke je `a.out`. Ukoliko se koristi opcija **-c**, izlazna datoteka je datoteka objektnog koda. Ime objektnog datoteke stvorene iz datoteke izvornog koda imena `ime.sufiks` (na primjer `ime.c`) je `ime.o`.

2.2.2 Ulazni tipovi datoteka

Nakon pokretanja, **gcc** obavlja predprocesiranje, prevođenje i povezivanje. Tipovi datoteka koji se mogu pojaviti kao ulazne datoteke za **gcc** uključuju:

- **datoteka.c** — C izvorni kod koji je potrebno predprocesirati
- **datoteka.i** — C izvorni kod koji se ne predprocesira
- **datoteka.h** — C ili C++ datoteka zaglavlja, obrađuje se u fazi predprocesiranja
- **datoteka.cc**, **datoteka.cp**, **datoteka.cpp**, **datoteka.CPP**, **datoteka.cxx**, **datoteka.c++**, **datoteka.C** — C++ izvorni kod koji je potrebno predprocesirati
- **datoteka.ii** — C++ izvorni kod koji se ne predprocesira
- **datoteka.hh**, **datoteka.H** — C++ datoteka zaglavlja, obrađuje se u fazi predprocesiranja
- **datoteka.o**, **datoteka.a** — objektni kod, koristi se u fazi povezivanja. Pored datoteka s ekstenzijom **.o** (objektne datoteke) i **.a** (arhive objektnog koda), **gcc** sve datoteke koje ne prepoznaje kao jedan od uobičajenih ulaznih tipova (pored ovdje navedenih postoje i drugi) tretira kao datoteke objektnog koda.

2.2.3 Primjeri korištenja

U sljedećim sekcijama upoznajemo se s dva primjera C programa — najprije najjednostavnijim, a zatim primjerom s funkcijom razdvojenom u zasebnu datoteku — te kroz njih obrađujemo praktične načine korištenja **gcc**-a, ručnim pokretanjem i automatizacijom pomoću alata **make**.

2.3 Najjednostavniji program

Sljedeći primjer demonstrira osnovnu strukturu C programa i tok prevođenja izvornog koda u izvršnu datoteku:

- **pozdrav.c** — program u jednoj datoteci koji ispisuje poruku na standardni izlaz. Najmanji smisleni C program koji se može prevesti i pokrenuti; služi za demonstraciju osnovne sintakse (**main**, **#include**, **printf**) i osnovnog toka prevođenja.

```
#include <stdio.h>

int main() {
    printf("Dobar jutro!\n");

    return 0;
}
```

2.3.1 Prevođenje u jednom koraku

Za **pozdrav.c**, koji ne ovisi o drugim datotekama izvornog koda, dovoljan je jedan poziv prevodioca:

```
gcc -Wall pozdrav.c -o pozdrav
```

Zastavica **-Wall** uključuje tipična upozorenja prevodioca; **-o pozdrav** određuje ime izlazne izvršne datoteke. Bez **-o**, rezultat bi se zvao **a.out**.

2.3.2 Prevođenje u dva koraka

Postupak generiranja izvršne datoteke zapravo se sastoji od dvije odvojene faze. U prvoj fazi (**prevođenje**, engl. *compilation*) prevodilac iz datoteke izvornog koda proizvodi datoteku objektnog koda. U drugoj fazi (**povezivanje**, engl. *linking*) jedna ili više objektnih datoteka spaja se u izvršni program, uz eventualno uključivanje simbola iz vanjskih biblioteka. Oba koraka mogu se pokrenuti odvojeno:

```
gcc -Wall -c pozdrav.c           # prevođenje: pozdrav.c -> pozdrav.o
gcc -Wall pozdrav.o -o pozdrav  # povezivanje: pozdrav.o -> pozdrav
```

Zastavica `-c` nalaže prevodiocu da stane nakon faze prevođenja i ne pokreće povezivanje.

2.4 Program u više datoteka

Funkcionalno, primjer iz ovog odjeljka radi potpuno isto što i prethodni pozdrav — ispisuje istu poruku na standardni izlaz. Razlika je u tome što je kod razbijen u tri datoteke: zaglavlje s deklaracijom funkcije, C datoteku izvornog koda s njezinom definicijom i glavni program koji funkciju poziva. Ovaj umjetno složeniji oblik služi nam da objasnimo proces prevođenja i povezivanja u prisutnosti više prevodbenih jedinica. U realnim programima organizacija koda u više datoteka je gotovo univerzalno pravilo, pa je ovaj minimalni primjer dobra polazna točka za razumijevanje takvog načina rada.

- **funkcije.h** — zaglavlje s deklaracijom funkcije `dobar_jutar()`. Pretprocesorska direktiva `#include` doslovno čita sadržaj navedene datoteke i “zalijepi” ga na svoje mjesto, prije nego što prevodilac počne s pravim prevođenjem. Zato bi se zaglavlje koje se u istoj prevodbenoj jedinici uključi više puta (npr. preko više drugih zaglavlja koja ga svako sa svoje strane uključuju) doslovno višestruko zalijepilo, što bi izazvalo greške višestruke deklaracije. Da bismo to spriječili, koristimo zaštitu od višestrukog uključivanja (engl. *include guards*) — par pretprocesorskih direktiva (`#ifndef _FUNKCIJE_H_ / #define ... / #endif`) koji osigurava da se sadržaj zaglavlja ubaci samo prilikom prvog uključivanja, a sva kasnija ignoriraju.

```
#ifndef _FUNKCIJE_H_
#define _FUNKCIJE_H_

void dobar_jutar();

#endif
```

- **funkcije.c** — definicija (implementacija) funkcije `dobar_jutar()`.

```
#include <stdio.h>

void dobar_jutar() {
    printf("Dobar jutar!\n");
}
```

- **pozdrav_fn.c** — glavni program koji uključuje `funkcije.h` i poziva `dobar_jutar()`.

```
#include "funkcije.h"

int main() {
    dobar_jutar();
}
```

```
    return 0;
}
```

Ovaj skup datoteka ilustrira osnovnu organizaciju projekta u više prevodbenih jedinica: zaglavlje `.h` sadrži deklaraciju i dijeli se između jedinica koje funkciju pozivaju i one koja ju definira, dok se `.c` datoteke prevode neovisno i kasnije povezuju u izvršni program.

2.4.1 Prevođenje programa iz više datoteka

Za pozdrav_fn potrebno je prevesti dvije izvorne datoteke u objektne, a zatim ih povezati u jedan izvršni program:

```
gcc -Wall -c pozdrav_fn.c          # -> pozdrav_fn.o
gcc -Wall -c funkcije.c           # -> funkcije.o
gcc -Wall pozdrav_fn.o funkcije.o -o pozdrav_fn
```

Ovdje se jasno vidi razlika između dviju faza koje smo ranije spomenuli — **prevođenja** i **povezivanja**. Prevođenje je sporiji proces u kojem prevodilac iz svake `.c` datoteke generira pripadnu `.o` (objektnu) datoteku — strojni kod te jedinice u oblik pripravan za povezivanje. Povezivanje je relativno brza operacija u kojoj se sve `.o` datoteke spajaju u izvršni program. Razdvajanje na dvije faze ima dvije važne koristi: **objektne datoteke možemo međusobno povezivati i u različitim kombinacijama** (ista funkcije.o može završiti u više različitih programa); još važnije, **kad u nekoj .c datoteci promijenimo neki redak, dovoljno je iznova prevesti samo tu jednu datoteku**, dok preostale `.o` datoteke ostaju kakve su bile. Povezivanje, međutim, uvijek treba ponoviti — ono je obavezni zadnji korak nakon svake izmjene.

Ovaj uvid je upravo razlog postojanja alata `make`. U projektu od nekoliko datoteka, ručno paziti koja se `.c` smije propustiti pri ponovnom prevođenju brzo postaje naporno i podložno greškama. `make` taj posao automatizira: na temelju vremena zadnje izmjene svake datoteke, sam odluči koje `.o` treba ponovno generirati, a koje su već svježije, i izvodi samo nužne korake.

2.5 Korištenje alata `make`

`make` je alat koji automatizira upravo takav proces. Iz datoteke s pravilima (Makefile) čita koje ulazne datoteke čine projekt, kako ovise jedna o drugoj i kojim se naredbama iz njih generiraju izlazne datoteke. Na temelju vremena zadnje izmjene datoteka, `make` ponovno prevodi samo one koje su mijenjane od posljednjeg prevođenja, i ništa više. U nastavku obrađujemo samo onoliko mogućnosti koliko nam je potrebno za vlastite primjere — `make` je znatno bogatiji alat, a definitivna referenca za sve njegove mogućnosti je *GNU Make Manual*, Stallman, McGrath & Smith [3], opsežan priručnik besplatno dostupan online na stranicama GNU projekta. Pokreće se tako da se u direktoriju s Makefile-om zada:

```
make          # izvršava prvo pravilo u Makefileu
make ime_pravila # izvršava navedeno pravilo
```

2.5.1 Struktura pravila

Svako pravilo ima oblik:

cilj: ovisnosti
naredbe

- **cilj** je najčešće ime datoteke koja nastaje izvršavanjem pravila (u pravilu izvršna ili objektna datoteka).
- **ovisnosti** je popis datoteka o kojima cilj ovisi — ako je bilo koja od njih novija od cilja, pravilo se izvršava.
- **naredbe** su naredbe ljske koje pravilo izvršava. **Moraju** biti uvučene tabulatorom, ne razmacima.

2.5.2 Gradnja Makefilea korak po korak

Krenut ćemo od najjednostavnije verzije i postupno je poboljšavati.

2.5.2.1 Korak 1: jednostavna pravila za svaki primjer

Prevedemo li logiku dvofaznog prevođenja iz prethodnog odjeljka u make pravila, za pozdrav dobivamo dva pravila — jedno za povezivanje (cilj pozdrav) i jedno za prevođenje (cilj pozdrav.o):

```
pozdrav: pozdrav.o
gcc -Wall pozdrav.o -o pozdrav

pozdrav.o: pozdrav.c
gcc -Wall -c pozdrav.c
```

Isti pristup za pozdrav_fn, koji se povezuje iz dvije objektna datoteke:

```
pozdrav_fn: pozdrav_fn.o funkcije.o
gcc -Wall pozdrav_fn.o funkcije.o -o pozdrav_fn

pozdrav_fn.o: pozdrav_fn.c
gcc -Wall -c pozdrav_fn.c

funkcije.o: funkcije.c
gcc -Wall -c funkcije.c
```

Promotrimo strukturu i redoslijed izvođenja pravila. Kada korisnik zatraži make pozdrav_fn, make čita Makefile u potrazi za pravilom čiji je cilj pozdrav_fn. To pravilo kao ovisnosti navodi pozdrav_fn.o i funkcije.o. Kako ove dvije datoteke ne postoje, make traži pravila u kojima su one ciljevi, provjerava njihove ovisnosti, i izvršava zadane naredbe — najprije za pozdrav_fn.o, zatim za funkcije.o — i tek na kraju za pozdrav_fn.

Što ako neka od objektnih datoteka navedena u ovisnostima već postoji? U tom slučaju make u pravilu kojem je ta datoteka cilj provjerava ovisnosti, odnosno odgovarajuću datoteku izvornog koda: ako je datoteka izvornog koda (ovisnost) **novija** od objektna datoteke (cilj), naredbe iz pravila izvršit će se iznova kako bi objektna datoteka uključila izmjene u izvornom kodu nastale od posljednje primjene pravila. Ako je objektna datoteka novija od izvora — odnosno već “svježā” — make taj korak preskače. Na ovaj način make rekurzivno kroz Makefile provjerava jesu li pojedini koraci “zastarjeli”, tj. postoje li izmjene u kodu od posljednje primjene pravila, i provodi samo one korake koji su nužni.

Ovakav Makefile je funkcionalan, ali pokazuje dva problema: (a) pravila za generiranje .o datoteka su praktički identična — razlikuju se samo po imenu datoteke, i (b) naredba gcc -Wall je ponovljena u svakom pravilu, pa promjena prevodioca ili zastavica traži izmjenu na više mjesta.

2.5.2.2 Korak 2: implicitna pravila

Postupak prevođenja `.c` → `.o` uvijek je isti: iz datoteke `ime.c` generira se `ime.o` istim pozivom prevodioca. Za takve unificirane postupke `make` nudi **implicitna pravila** koja se primjenjuju na temelju uzorka ekstenzije. Pravilo:

```
.C.o:
    gcc -Wall -c $<
```

govori `make`-u kako iz bilo koje `.c` datoteke izgraditi istoimenu `.o` datoteku. Unutar naredbe, automatska varijabla `$<` zamjenjuje se imenom ulazne datoteke (one iz popisa ovisnosti). Isto se tako može koristiti `$@` za ime cilja.

Ovim jednim implicitnim pravilom zamijenjena su sva pojedinačna `.c` → `.o` pravila. `Makefile` sad izgleda ovako:

```
pozdrav: pozdrav.o
    gcc -Wall pozdrav.o -o pozdrav

pozdrav_fn: pozdrav_fn.o funkcije.o
    gcc -Wall pozdrav_fn.o funkcije.o -o pozdrav_fn

.C.o:
    gcc -Wall -c $<
```

2.5.2.3 Korak 3: varijable

I dalje je prevodilac ugrađen u pravila kao `gcc`, a zastavica `-Wall` ponovljena na više mjesta. Uvođenjem **varijabli** se dio koji se ponavlja izvuče iz pravila i centralizira:

```
CC = /usr/bin/gcc
CFLAGS = -Wall

pozdrav: pozdrav.o
    $(CC) $(CFLAGS) pozdrav.o -o pozdrav

pozdrav_fn: pozdrav_fn.o funkcije.o
    $(CC) $(CFLAGS) pozdrav_fn.o funkcije.o -o pozdrav_fn

.C.o:
    $(CC) $(CFLAGS) -c $<
```

Varijabla se deklarira imenom, znakom jednakosti i tekstualnom vrijednošću. Vrijednost se dohvaća kao `$(IME)`. Promjena prevodioca ili zastavica sada se svodi na izmjenu jednog retka.

Postoji konvencija iz GNU svijeta prema kojoj se zastavice za **prevođenje** drže u varijabli `CFLAGS`, a zastavice za **povezivanje** (npr. `-L/put/do/lib` ili biblioteke) u zasebnoj varijabli `LDFLAGS`. Naši primjeri u ovoj fazi ne trebaju posebne zastavice za linker, pa `LDFLAGS` ostaje prazna — ali ju uvodimo radi konzistentnosti i lakšeg proširivanja:

```
CC = /usr/bin/gcc
CFLAGS = -Wall
LDFLAGS =

pozdrav: pozdrav.o
    $(CC) $(LDFLAGS) pozdrav.o -o pozdrav

pozdrav_fn: pozdrav_fn.o funkcije.o
    $(CC) $(LDFLAGS) pozdrav_fn.o funkcije.o -o pozdrav_fn

.C.o:
    $(CC) $(CFLAGS) -c $<
```

2.5.2.4 Korak 4: specijalna pravila `default`, `all`, `clean`

Ostaje još nekoliko praktičnih poboljšanja kojima se uvodi nekoliko konvencionalnih pravila. Bitno je odmah napomenuti: **imena `default`, `all` i `clean` nisu rezervirane ključne riječi** — pravilo `all` jednako bismo mogli nazvati `sve`, `clean` bismo mogli nazvati `ocisti`, i sve bi radilo identično. Ova imena su dogovorna konvencija koje se programeri drže kako bi Makefile bio čitljiv i predvidljiv svakome tko ga pogleda. Slično kao s imenima varijabli u kodu ili s imenima programa, dobra je praksa odabrati imena iz kojih se odmah vidi što pravilo radi; držanje navedene konvencije čini naš kod razumljivim drugima — a često i nama samima kad se vratimo svom kodu nakon izvjesnog vremena.

`default` — pravilo koje se izvršava kad korisnik pokrene `make` bez argumenata. `make` u tom slučaju jednostavno izvršava prvo pravilo navedeno u Makefile-u, bez obzira na njegovo ime — pa se po konvenciji to prvo pravilo zove `default` i smješta na vrh datoteke. Da smo mu dali bilo koje drugo ime, svedeno bi bilo pokrenuto kad korisnik upiše `make` bez argumenata.

```
default: pozdrav_fn
```

Ovo pravilo ima samo cilj (`default`) i ovisnost (`pozdrav_fn`), ali nema naredbi. `make` provjerava ovisnosti i, prema ranije spomenutoj logici, traži pravilo u kojem je `pozdrav_fn` cilj te ga izvršava (rekurzivno, kako smo opisali — samo ako su pripadne datoteke zastarjele). Nakon toga `default` ne radi ništa jer nema vlastitih naredbi. Pravilo `default` koristi upravo taj trik da nas vodi do “korisnog” pravila koje stvarno gradi program na kojem trenutno radimo.

`all` — koristi isti trik za izgradnju svih ciljeva u direktoriju odjednom. Za lakše održavanje koristimo varijablu `TARGETS` koja popisuje sve izvršne datoteke:

```
TARGETS = pozdrav pozdrav_fn
all: $(TARGETS)
```

I ovo pravilo ima samo ovisnosti, bez naredbi: `make` rekurzivno gradi svaku navedenu izvršnu datoteku, a samo `all` ne radi ništa.

`clean` — pravilo koje briše sve izvršne, objektne i privremene datoteke iz direktorija. Za razliku od `default` i `all`, ovdje imamo cilj i naredbe, ali **nema ovisnosti**. To znači da se pravilo izvršava bezuvjetno svaki put kad korisnik upiše `make clean` — neovisno o stanju datoteka u direktoriju.

```
clean:
    rm -f $(TARGETS) *.o *~ a.out
```

I ovo pravilo bi se moglo nazvati drukčije (npr. `ocisti`), ali ime `clean` toliko je rasprostranjena konvencija da bi nestandardno ime samo zbunjivalo druge programere — pa ga zato i koristimo.

2.5.2.5 Konačni Makefile

Objedinimo sve navedeno u finalnu Makefile datoteku u ovom direktoriju:

```
CC = /usr/bin/gcc
CFLAGS = -Wall
LDFLAGS =
TARGETS = pozdrav pozdrav_fn

default: pozdrav_fn
```

```
all: $(TARGETS)

pozdrav: pozdrav.o
    $(CC) $(LDFLAGS) pozdrav.o -o pozdrav

pozdrav_fn: pozdrav_fn.o funkcije.o
    $(CC) $(LDFLAGS) pozdrav_fn.o funkcije.o -o pozdrav_fn

clean:
    rm -f $(TARGETS) *.o *~ a.out

.C.o:
    $(CC) $(CFLAGS) -c $<
```

Tipična uporaba:

```
make          # izvršava "default", tj. gradi pozdrav_fn
make all      # gradi oba primjera
make pozdrav  # gradi samo pozdrav
make clean    # briše izvršne i objektne datoteke
```

2.6 Arhive objektnih datoteka — libovi

Napomena: ovo je gradivo za napredne. Možete ga preskočiti pri prvom čitanju, osobito ako ste se tek upoznali s procesom prevođenja i povezivanja programa — ostatak skripte ne ovisi o njemu. Na ovaj dio možete se vratiti kasnije, kada osjetite potrebu za organizacijom koda u veće biblioteke. Naravno, ako je do sada izloženo bilo “šala mala” za vas, nema razloga da ne uđete dublje u organiziranje objektnih datoteka u biblioteke funkcija, o čemu će u sljedećoj sekciji biti više riječi.

Kako program raste, izvorni kod se često dijeli u veći broj datoteka. Pri svakom prevođenju, prevodilac mora svaku izmijenjenu .c datoteku ponovno prevesti u objektni kod, a zatim u fazi povezivanja sastaviti sve objektne datoteke u jednu izvršnu datoteku. Za male projekte to je sasvim u redu — ali kad imamo *desetke* ili *stotine* datoteka, manipulacija svakim pojedinim .o postaje nepregledna.

Rješenje su **arhive objektnih datoteka**, koje u UNIX terminologiji nazivamo **libovima** (engl. *libraries*, biblioteke funkcija). Lib je jedna datoteka u koju je upakirano više objektnih datoteka, organiziranih po nekom tematskom kriteriju — npr. sve funkcije za rad s mrežom u jednom libu, sve funkcije za rad sa slikama u drugom, sve naše vlastite pomoćne funkcije u trećem. Najpoznatiji primjer je već viđena standardna C biblioteka `libc` koja sadrži funkcije poput `printf`, `fopen`, `malloc`, itd.

Postoje dvije temeljne vrste libova:

	Statičke biblioteke (.a)	Dinamičke biblioteke (.so)
Povezivanje	u fazi prevođenja — objektni kod se kopira u izvršnu datoteku	u fazi pokretanja — izvršna datoteka samo sadrži referencu na lib
Veličina izvršne datoteke	veća (sadrži kod liba)	manja (lib se učitava odvojeno)
Ovisnost o sustavu	nikakva — izvršna datoteka samostalna	lib mora biti prisutan na sustavu pri pokretanju
Distribucija	jednostavna (jedna datoteka)	složenija (lib + verzioniranje)
Memorija pri više procesa	svaki proces ima svoju kopiju	svi procesi dijele isti lib u memoriji

	Statičke biblioteke (.a)	Dinamičke biblioteke (.so)
Tipična ekstenzija	libIME.a	libIME.so

U ovom poglavlju zadržavamo se na **statičkim** bibliotekama, koje su jednostavnije za razumijevanje i dovoljne za većinu početničkih primjena.

2.6.1 Korištenje tuđih libova

Kada koristimo lib koji je netko drugi izradio (npr. standardnu C biblioteku, ili razne specijalizirane biblioteke za obradu slike, mrežnu komunikaciju, kriptografiju, ...), gcc ga uključuje u proces povezivanja na dva načina.

Prvi je da eksplicitno navedemo putanju do .a datoteke kao da je objektna datoteka:

```
gcc -Wall prog.o /putanja/do/libjpeg.a -o izvrsna
```

Drugi, češći način je korištenjem opcije `-l<ime>`: ako se naš lib zove `libjpeg.a` i nalazi se u nekom od standardnih direktorija u kojima gcc traži libove (npr. `/usr/lib`), dovoljno je napisati:

```
gcc -Wall prog.o -ljpeg -o izvrsna
```

Uočite konvenciju: lib se na disku zove `libjpeg.a`, ali u gcc naredbenom retku navodimo ga kao `-ljpeg` — bez prefiksa `lib` i bez ekstenzije `.a`. Ovu konvenciju nameće sam gcc koji iz opcije `-lime` rekonstruira ime datoteke `libime.a`.

Ukoliko se traženi lib ne nalazi u standardnim direktorijima, dodatnu putanju zadajemo opcijom `-L<putanja>`:

```
gcc -Wall -L/putanja/na/direktorij prog.o -ljpeg -o izvrsna
```

Redoslijed datoteka u naredbenom retku je važan. U svakoj arhivi može biti više objektnih datoteka, a prevodilac iz arhive izdvaja samo one koje sadrže funkcije potrebne za generiranje izvršne datoteke. Kada pokrenemo gcc, datoteke se analiziraju onim redoslijedom kojim su navedene u naredbenom retku — pri čemu se stvara lista nedostajućih funkcija. Te se funkcije zatim traže u libovima koji *slijede* u naredbenom retku. Ako je neka arhiva navedena ranije od .o datoteke koja koristi njezine funkcije, taj dio koda neće biti izdvojen iz liba i gcc će prijaviti grešku u povezivanju. **Dobra je praksa arhive staviti na kraj naredbe za povezivanje, kako bi gcc znao koje sve objektne datoteke treba iz libova “povući”.** Posebno treba voditi računa o redoslijedu ako koristimo više libova od kojih jedan koristi funkcije drugoga — tada lib “više razine” (onaj koji ovisi o drugima) mora doći **prije** libova o kojima ovisi.

2.6.2 Stvaranje vlastite arhive — alat ar

Pored korištenja tuđih libova, jednako je jednostavno izraditi vlastiti lib u koji ćemo upakirati svoje funkcije. Ovo je dobra praksa ukoliko često koristimo iste funkcije u različitim projektima: arhiviranjem funkcija u lib sistematiziramo kod koji smo ranije razvili, na način da umjesto velikog broja objektnih datoteka koristimo jednu ili više tematskih arhiva. Vjerojatno najlošija praksa je kopiranje izvornog koda u svaki novi projekt, pri čemu se vrlo lako izgubiti u šumi izmjena koje unosimo u različitim verzijama.

Za rad s arhivama koristimo alat `ar`:

```
ar [-opcije] arhiva [ulazne_datoteke]
```

Detaljnu uputu za korištenje dobivamo s `man ar`. Najčešće opcije:

Opcija	Značenje
<code>r</code>	dodavanje nove ili zamjena postojeće datoteke u arhivi (ako član s istim imenom postoji, prethodno se briše)
<code>d</code>	brisanje datoteke člana iz arhive
<code>x</code>	izdvajanje datoteke člana iz arhive
<code>t</code>	ispis liste datoteka članova arhive
<code>v</code>	mod rada s ispisom dodatnih informacija (<i>verbose</i>)

2.6.3 Primjer

U ovom poglavlju nalaze se tri datoteke koje ćemo iskoristiti za prikaz triju različitih načina povezivanja:

- **nizfn.c** — implementacija triju funkcija za rad s nizovima cijelih brojeva.

```
#include <stdlib.h>
#include <time.h>

/* Generira slucajni niz duzine broj_elemenata */
void slucajni_niz(int *niz, int broj_elemenata) {
    int i;
    srand(time(NULL));
    for (i = 0; i < broj_elemenata; i++)
        niz[i] = rand();
}

/* Pronalazi najveći element u nizu */
int najveći_element(int *niz, int broj_elemenata) {
    int i, maxel = niz[0];
    for (i = 1; i < broj_elemenata; i++) {
        if (maxel < niz[i])
            maxel = niz[i];
    }
    return maxel;
}

/* Pronalazi najmanji element u nizu */
int najmanji_element(int *niz, int broj_elemenata) {
    int i, minel = niz[0];
    for (i = 1; i < broj_elemenata; i++) {
        if (minel > niz[i])
            minel = niz[i];
    }
    return minel;
}
```

- **nizfn.h** — zaglavlje s deklaracijama funkcija.

```
#ifndef _NIZFN_H_
#define _NIZFN_H_

void slucajni_niz(int *niz, int broj_elemenata);
int najveći_element(int *niz, int broj_elemenata);
int najmanji_element(int *niz, int broj_elemenata);

#endif
```

- **niz.c** — glavni program koji generira slučajni niz, ispisuje sve elemente i pronalazi najveći i najmanji element.

```
#include <stdio.h>
#include <nizfn.h>
#define N_EL 10

int main() {
    int i, mx, mi;
    int niz[N_EL];

    slucajni_niz(niz, N_EL);
    printf("Elementi niza:\n");
    printf("-----\n");

    for (i = 0; i < N_EL; i++)
        printf("%3d. element: %12d\n", i + 1, niz[i]);

    mx = najveći_element(niz, N_EL);
    mi = najmanji_element(niz, N_EL);
    printf("\n\nMax: %d; Min: %d\n", mx, mi);

    return 0;
}
```

Sada možemo prevesti i povezati ovaj primjer na tri različita načina — sva tri daju identičnu izvršnu datoteku.

1. Direktno povezivanje s objektom datotekom (kao u prethodnim primjerima):

```
$ gcc -Wall -c nizfn.c
$ gcc -Wall -c -I. niz.c
$ gcc -Wall niz.o nizfn.o -o niz1
```

Uočite da smo pri prevodenju niz.c koristili zastavicu `-I.` — njom prevoditelju naglašavamo da datoteke zaglavlja uključene s preprocesorskom direktivom `#include <>` potraži i u trenutnom radnom direktoriju (jer naš niz.c uključuje `<nizfn.h>` u šiljatim zgradama, što inače znači da se traži samo u standardnim direktorijima).

2. Eksplicitno povezivanje s arhivom. Korištenjem alata ar najprije stvorimo statičku biblioteku libniz.a u koju ubacimo objektni kod naših funkcija:

```
$ gcc -Wall -c nizfn.c
$ ar -r libniz.a nizfn.o
```

Ovime smo stvorili arhivu koja sadrži jednu objektnu datoteku s tri funkcije. Sada izvršnu datoteku generiramo navodeći libniz.a umjesto objektna datoteke:

```
$ gcc -Wall niz.o libniz.a -o niz2
```

3. Povezivanje preko `-L -l` opcija — gcc sam pronalazi libniz.a u zadanom direktoriju:

```
$ gcc -Wall -L. niz.o -lniz -o niz3
```

Sve tri izvršne datoteke su **identične**. To se može provjeriti alatom diff koji ispisuje razlike između datoteka:

```
$ diff niz1 niz2
$ diff niz1 niz3
```

U oba slučaja diff ne ispisuje ništa — datoteke se ne razlikuju.

2.6.4 Makefile za sva tri načina

U Makefile datoteci proširenoj za ovaj primjer dana su pravila za sva tri načina povezivanja, kao i pravilo za stvaranje arhive libniz.a:

```
CC = /usr/bin/gcc
CFLAGS = -Wall
LIBDIR = .
TARGETS = pozdrav pozdrav_fn niz1 niz2 niz3

niz1: niz.o nizfn.o
    $(CC) $(LD_FLAGS) niz.o nizfn.o -o niz1

niz2: niz.o libniz.a
    $(CC) $(LD_FLAGS) niz.o libniz.a -o niz2

niz3: niz.o libniz.a
    $(CC) $(LD_FLAGS) -L$(LIBDIR) niz.o -lniz -o niz3

libniz.a: nizfn.o
    ar -r $(LIBDIR)/libniz.a nizfn.o

.C.O:
    $(CC) $(CFLAGS) -I$(LIBDIR) -c $<
```

Pozivom `make niz2` (ili `make niz3`), `make` najprije provjerava postoje li potrebne ovisnosti: ukoliko ne postoje, generira `nizfn.o` korištenjem implicitnog pravila, te potom `libniz.a` korištenjem alata `ar`. Nakon toga, ovisno o pravilu, povezuje izvršnu datoteku.

Jednom kad imamo statičku biblioteku, možemo ju koristiti i u drugim programima. Možemo je i **distribuirati** drugim programerima — bez dijeljenja izvornog koda, ako to ne želimo.

2.7 Pokretanje

Izvršni programi pokreću se navođenjem relativne putanje (`./`) iz istog direktorija:

```
./pozdrav
./pozdrav_fn
```

Oba primjera ispisuju istu poruku; razlika je isključivo u unutarnjoj organizaciji koda.

Ukoliko ste prošli i sekciju o libovima, na isti način možete pokrenuti i sve tri varijante `niz` primjera:

```
./niz1
./niz2
./niz3
```

Sve tri varijante daju **identičan** ispis — slučajni niz od deset cijelih brojeva, te njegov najveći i najmanji element.

2.8 Bibliografija

[1] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Englewood Cliffs, NJ, USA: Prentice Hall, 1988.

[2] I. Mateljan, *Programiranje C jezikom*. Split, Hrvatska: Fakultet elektrotehnike, strojarstva i brodogradnje, 2006. [Online]. Dostupno na: <http://programiranje.yola>

[site.com/resources/Programiranje_C_jezikom.pdf](https://www.gnu.org/software/make/manual/). Pristupljeno: svibanj 2026.

[3] R. M. Stallman, R. McGrath, and P. D. Smith, *GNU Make Manual*. Boston, MA, USA: Free Software Foundation, 2023. [Online]. Dostupno na: <https://www.gnu.org/software/make/manual/>. Pristupljeno: svibanj 2026.

Poglavlje 3

Ulazno/izlazne operacije

Ulazno-izlazne operacije jedna su od standardnih funkcionalnosti gotovo svih programskih jezika. Bez mogućnosti pohrane podataka u datoteku i čitanja iz datoteke, skup zadataka koji bi se takvim jezikom mogli riješiti bio bi značajno ograničen. U programskom jeziku C za rad s datotekama koriste se funkcije poput `fopen`, `fscanf`, `fprintf`, `fread` i druge, dok za komunikaciju s korisnikom putem naredbenog retka najčešće koristimo `printf` i `scanf`. Sve navedene funkcije dio su standardne C biblioteke funkcija definirane kroz ISO C standard [3]. Ove funkcije postoje i u drugim programskim jezicima, iako im se sintaksa i način pozivanja mogu razlikovati.

Bez obzira za koji operacijski sustav razvijamo program, navedene funkcije u izvornom kodu pozivamo na isti način. Ovo je moguće jer je svaka od ovih funkcija u osnovi **funkcija omotač** (*wrapper function*) koja se u postupku prevođenja i povezivanja prevodi u niz sistemskih poziva. Kako je detaljno objašnjeno u poglavlju 1, sistemski pozivi jedino su sučelje za upravljanje jezgrom operacijskog sustava. Svaki poziv funkcije koji uključuje ulazno/izlazne operacije, bez obzira na programski jezik koji koristimo, u konačnici se prevodi u jedan ili više sistemskih poziva koji su definirani na razini operacijskog sustava i jedino su sučelje za upravljanje funkcionalnostima jezgre.

U ovom poglavlju obrađeni su sistemski pozivi za upravljanje ulazno/izlaznim operacijama na UNIX-u. Važno je naglasiti da razumijevanje sistemskih poziva nije samo akademska vježba — predstavlja temelj za razumijevanje principa na kojima počiva operacijski sustav. Ovo osobito vrijedi na UNIX-u, koji slijedi načelo “*sve je datoteka*”: isti skup sistemskih poziva koristi se za komunikaciju s korisnikom putem terminala, rad s datotekama na disku, upravljanje uređajima, mrežnu komunikaciju (socketi) i komunikaciju između aktivnih procesa u memoriji. Upravo iz tog razloga ovo poglavlje predstavlja jedan od temelja za razumijevanje UNIX logike, a ono što ćete ovdje naučiti ima daleko širu primjenu nego što sam naslov poglavlja daje naslutiti.

Prije nego što krenemo s tehničkim detaljima, zapitajmo se jedno jednostavno pitanje: **što sve možemo s datotekom?** Odgovori na ovo pitanje često uključuju: možemo je obrisati, promijeniti joj ime, kopirati i slično. Naravno, ni jedna od ovih tvrdnji nije pogrešna — ali je jednako točna kao da kažemo da stolicu možemo prebojati, premjestiti u drugu sobu, ili je nacijepati sjekirom ako nam više nije potrebna. Sve navedeno je točno, ali ne otkriva temeljnu namjenu stolice: na stolicu možemo **sjesti**.

Ako istu logiku primijenimo na datoteku, jedine dvije stvari za koje datoteku zapravo koristimo su: u datoteku možemo nešto **upisati**, ili možemo nešto iz nje **pročitati**. I to je to. Stoga svaki programski jezik, ali i svaki operacijski sustav putem sistemskih poziva, mora osigurati najmanje te dvije funkcije. Da bi ovo bilo moguće, potrebno nam je još svega nekoliko funkcionalnosti: **otvaranje** i **zatvaranje** datoteke te **pozicioniranje** unutar datoteke.

U ovom poglavlju obrađene su upravo ove osnovne funkcionalnosti za rad s datotekama koje svaki operacijski sustav mora osigurati da bi bio iole upotrebljiv za ozbiljniji rad: otvaranje, čitanje, pisanje i zatvaranje datoteka, uz pozicioniranje unutar datoteke. Obradeni su i **deskriptori datoteke** — apstraktna referenca putem koje proces upravlja svojim otvorenim datotekama — te strukture podataka u kojima UNIX jezgra čuva informacije o procesima i datotekama koje su u njima otvorene. Iako ovaj posljednji dio ulazi nešto dublje u detalje implementacije UNIX-a, čitatelju svakako savjetujemo da odvoji malo vremena i pokuša “povezati konce” — razumijevanje ovih mehanizama kasnije će bitno olakšati korištenje naprednih koncepata koji omogućuju da se složene stvari postignu relativno jednostavnim kodom. Za one nestrpljivije, tu su brojni primjeri na kojima su opisane funkcionalnosti ilustrirane.

Na kraju poglavlja dan je osvrt na odnos između sistemskih poziva i funkcija standardne C biblioteke, s ciljem da budući programer razumije kada i zašto koristiti koje. Svi primjeri pisani su u programskom jeziku C i pretpostavljaju UNIX (Linux) okruženje.

U ovom direktoriju nalaze se programi koji ilustriraju UNIX sistemske pozive za rad s datotekama: `open()`, `creat()`, `close()`, `read()`, `write()`, `lseek()` i `umask()`. Svi primjeri pisani su izravno korištenjem sistemskih poziva (bez uporabe funkcija standardne C biblioteke poput `fopen`, `fread`, `fprintf`), s ciljem da se prikaže kako operacijski sustav zapravo obavlja ulazno/izlazne operacije i na kojoj razini počiva koncept “sve je datoteka”.

Funkcije više razine pružaju zgodne dodatke poput interno upravljanog međuspremnika za smanjenje broja sistemskih poziva, formatiran ulaz/izlaz (`printf`, `fprintf`, `scanf`, ...), ili portabilnost između operacijskih sustava — ali fizička komunikacija s jezgrom uvijek se odvija kroz `read()`, `write()`, `open()` i ostale pozive opisane u nastavku. Razumijevanje sistemskih poziva otkriva što se zapravo događa “ispod” funkcija standardne biblioteke koje ste do sada vjerojatno koristili.

3.1 Deskriptori datoteka

Jezgra operacijskog sustava sve otvorene datoteke referencira pomoću nenegativnih cijelih brojeva koje nazivamo **deskriptorima datoteke** (engl. *file descriptors*). Deskriptor je apstraktna referenca koju proces dobiva u trenutku otvaranja datoteke i koristi za sve daljnje operacije na njoj (čitanje, pisanje, pozicioniranje), do njezinog zatvaranja.

Pri tom proces ne mora znati ništa o samoj datoteci otvorenoj na nekom deskriptoru: gdje se fizički nalazi, radi li se uopće o datoteci na disku, ili je možda riječ o standardnom izlazu na koji program ispisuje poruke korisniku, ili o otvorenom portu na mreži. Zahvaljujući konceptu “sve je datoteka”, svim tim resursima upravlja se na isti način, korištenjem malog skupa standardiziranih sistemskih poziva. Naravno, upisivanje u datoteku na lokalnom disku fizički se

ne odvija na isti način kao komunikacija s udaljenim računalom putem mreže — jezgra čuva informacije o svakom otvorenom deskriptoru i na temelju toga odabire odgovarajuću komunikacijsku rutinu. Programer te detalje ne vidi: sučelje je jedinstveno za sve tipove datoteka.

Pri pokretanju novog programa, najčešće su već zauzeti file deskriptori 0, 1 i 2. Ovi deskriptori koriste se za komunikaciju s korisnikom putem naredbenog retka:

Deskriptor	Naziv	Izvor / odredište
0	standardni ulaz (<i>standard input</i>)	naredbeni redak — unos korisnika putem tipkovnice
1	standardni izlaz (<i>standard output</i>)	naredbeni redak — ispis u korisnički terminal
2	standardni izlaz za greške (<i>standard error</i>)	naredbeni redak — ispis u korisnički terminal

Ova konvencija datira iz ranih UNIX sustava 1970-ih i danas je neodvojiv dio POSIX standarda. Budući da su deskriptori 0, 1 i 2 pri pokretanju programa u pravilu već zauzeti, novokreirani deskriptori — oni koje vraća `open()` ili `creat()` — u pravilu imaju vrijednost 3 ili veću. Sama jezgra pri otvaranju nove datoteke uvijek dodjeljuje **najniži slobodan deskriptor**.

Vrijednosti deskriptora 0, 1 i 2 definirane su u zaglavlju `<unistd.h>` kao simboličke konstante `STDIN_FILENO`, `STDOUT_FILENO` i `STDERR_FILENO`. Preporuka je u kodu koristiti ove konstante umjesto golih brojeva 0, 1 i 2 — kod time postaje čitljiviji.

3.2 Sistemski pozivi za rad s datotekama

3.2.1 Funkcija `open()`

Osnovna UNIX funkcija za otvaranje (i po potrebi kreiranje) datoteka.

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int open(const char *pathname, int oflag, /* mode_t mode */);
```

Povratna vrijednost: file deskriptor otvorene datoteke (nenegativni cijeli broj), ili -1 u slučaju greške.

Argumenti:

- **pathname** — putanja do datoteke koja se otvara ili kreira.
- **oflag** — kombinacija bitovnih konstanti koja specificira način otvaranja. Mora sadržavati **točno jednu** od sljedeće tri konstante:
 - `O_RDONLY` — otvori samo za čitanje,
 - `O_WRONLY` — otvori samo za pisanje,
 - `O_RDWR` — otvori za čitanje i pisanje.

Uz tu obaveznu konstantu može se kombinacijom bitovnog *OR*-a (`|`) dodati jedna ili više opcionalnih:

- `O_CREAT` — kreiraj datoteku ako ne postoji (zahtijeva treći argument `mode`),

- `O_APPEND` — pri svakom pisanju pomakni offset na kraj datoteke, bez obzira na trenutni file offset,
- `O_TRUNC` — ako je datoteka otvorena za pisanje, izbriši njezin sadržaj (dužina postaje 0),
- `O_EXCL` — u kombinaciji s `O_CREAT`: vrati grešku ako datoteka već postoji; provjera i kreiranje izvode se kao **atomska operacija**,
- `O_NONBLOCK` — postavi neblokirajući način rada za I/O operacije,
- `O_SYNC` — čekaj da se svaka operacija pisanja fizički dovrši na disku.

Tipične kombinacije zastavica koje ćete često sresti u UNIX kodu:

Kombinacija	Značenje
<code>O_WRONLY O_TRUNC</code>	otvori za pisanje i skрати na 0 (obriši postojeći sadržaj)
<code>O_WRONLY O_CREAT</code>	otvori za pisanje; ako datoteka ne postoji, stvori je
<code>O_WRONLY O_CREAT O_TRUNC</code>	otvori za pisanje i obriši postojeći sadržaj; ako datoteka ne postoji, stvori je
<code>O_WRONLY O_CREAT O_EXCL</code>	stvori novu datoteku i otvori je za pisanje; ako datoteka već postoji, vrati grešku
<code>O_WRONLY O_APPEND</code>	otvori za pisanje; svako pisanje dodaje se na kraj datoteke, bez obzira na trenutni file offset

- **mode** — opcionalni treći argument koji se navodi pri kreiranju nove datoteke (uz zastavicu `O_CREAT`). Specificira prava pristupa novostvorene datoteke. Može se zadati kao troznamenkasti oktalni broj (svaka znamenka definira prava za vlasnika, grupu i ostale korisnike), ili kombinacijom konstanti tipa `mode_t` definiranih u zaglavlju `<sys/stat.h>`:

Konstanta	Oktalna vrijednost	Značenje
<code>S_IRUSR</code>	<code>0400</code>	čitanje za vlasnika
<code>S_IWUSR</code>	<code>0200</code>	pisanje za vlasnika
<code>S_IXUSR</code>	<code>0100</code>	izvršavanje za vlasnika
<code>S_IRGRP</code>	<code>0040</code>	čitanje za grupu
<code>S_IWGRP</code>	<code>0020</code>	pisanje za grupu
<code>S_IXGRP</code>	<code>0010</code>	izvršavanje za grupu
<code>S_IROTH</code>	<code>0004</code>	čitanje za ostale
<code>S_IWOTH</code>	<code>0002</code>	pisanje za ostale
<code>S_IXOTH</code>	<code>0001</code>	izvršavanje za ostale

Tako je primjerice `S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH` ekvivalentno oktalnom zapisu `0644` — vlasnik smije čitati i pisati, grupa i ostali samo čitati.

3.2.2 Funkcija `creat()`

Namijenjena je kreiranju novih datoteka.

Zašto `creat` bez završnog `e`?

Ime funkcije `creat` (bez završnog `e`) nije pogreška — riječ je o jednoj od najpoznatijih neobičnosti UNIX-a, naslijeđenoj još iz prvih verzija s početka 1970-ih, kad su rani UNIX sustavi imali ograničenje od najviše šest znakova u imenu sistemskog poziva. Anegdota: na pitanje što bi

promijenio u UNIX-u kad bi mogao, **Ken Thompson** — jedan od izvornih autora UNIX-a — odgovorio je: *“I’d spell creat with an e.”*

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int creat(const char *pathname, mode_t mode);
```

Povratna vrijednost: file deskriptor otvorene datoteke, ili -1 u slučaju greške.

Argumenti:

- **pathname** — putanja do datoteke koja se kreira.
- **mode** — prava pristupa novostvorene datoteke (kombinacija konstanti tipa `mode_t`, kako je opisano kod `open()`-a).

Ova funkcija ekvivalentna je pozivu:

```
open(pathname, O_WRONLY | O_CREAT | O_TRUNC, mode);
```

Iz definicije slijede dva ograničenja: `creat()` datoteku otvara **samo za pisanje**, a ako datoteka već postoji, njezin sadržaj se briše (`O_TRUNC`). Zato je `creat()` jednostavno kratki oblik za vrlo čest slučaj korištenja `open()`-a — u modernom kodu obično se ide direktno preko `open()`-a koji nudi punu kontrolu.

3.2.3 Funkcija `close()`

Zatvara datoteku otvorenu na file deskriptoru `filedes`, čime jezgra oslobađa deskriptor za ponovnu upotrebu.

```
#include <unistd.h>

int close(int filedес);
```

Povratna vrijednost: 0 u slučaju uspjeha, -1 u slučaju greške.

Argumenti:

- **filedes** — deskriptor otvorene datoteke koju treba zatvoriti.

Kada proces završi s radom, automatski se zatvaraju sve otvorene datoteke, pa eksplicitno pozivanje `close()` nije *strogo* nužno. Ipak, preporučuje se eksplicitno zatvaranje jer sustav ima ograničen broj deskriptora po procesu, a za neke tipove datoteka (npr. mrežne sockete) zatvaranje pokreće važne završne operacije.

3.2.4 Funkcija `read()`

Čita podatke iz datoteke i upisuje ih u memorijski prostor na koji pokazuje `buff`.

```
#include <unistd.h>

ssize_t read(int filedес, void *buff, size_t nbytes);
```

Povratna vrijednost: broj stvarno pročitanih bajtova (može biti manji od `nbytes`), 0 po dosizanju kraja datoteke (*EOF*), ili -1 u slučaju greške.

Povratni tip `ssize_t` (*signed size*) je POSIX-om definiran ekvivalent `size_t` koji može imati i negativnu vrijednost — neophodan da bi funkcija mogla vratiti -1 kao oznaku greške.

Argumenti:

- **filedes** — deskriptor otvorene datoteke.
- **buff** — pokazivač na statički ili dinamički alociran memorijski blok u koji se upisuju pročitani podaci. `read()` ne alocira memoriju sam — to je zadaća programera.
- **nbytes** — maksimalan broj bajtova koji će se pokušati pročitati.

Čitanje počinje na trenutnom file offsetu. Ova vrijednost je nakon otvaranja datoteke uvijek 0, tj. pokazuje na početak datoteke. Nakon svakog uspješnog poziva `read()`-a offset se automatski pomiče za broj pročitanih bajtova. Razlozi zbog kojih stvarno pročitani broj može biti manji od traženog uključuju dostignut kraj datoteke, čitanje s terminala (koji vraća redak koji je korisnik utipkao odjednom), ili čitanje s mrežnog socketa.

3.2.5 Funkcija `write()`

Upisuje podatke iz memorijskog bloka u datoteku identificiranu deskriptorom `filedes`.

```
#include <unistd.h>

ssize_t write(int filedes, const void *buff, size_t nbytes);
```

Povratna vrijednost: broj stvarno upisanih bajtova (po POSIX-u garantirano jednak `nbytes` u uspješnom slučaju za obične datoteke; može biti manji za neke tipove datoteka poput mrežnih socketa), ili -1 u slučaju greške.

Argumenti:

- **filedes** — deskriptor otvorene datoteke.
- **buff** — pokazivač na memorijski blok koji se upisuje.
- **nbytes** — broj bajtova koji se piše.

Pisanje počinje na trenutnom file offsetu, a nakon uspješnog upisa offset se pomiče za broj upisanih bajtova. Iznimka je kad je datoteka otvorena s `O_APPEND` zastavicom — tada jezgra prije svakog pisanja postavlja offset na kraj datoteke.

3.3 Primjeri

3.3.1 Otvaranje, čitanje i pisanje

Na primjerima koji slijede ilustrirano je korištenje sistemskih poziva za rad s datotekama:

- **read_file.c** — otvara datoteku `moja_datoteka.txt` i ispisuje njen sadržaj na standardni izlaz.

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
    int n, fd;
    char s;

    fd = open("moja_datoteka.txt", O_RDONLY);
```

```

if (fd == -1) {
    perror("open");
    return 1;
}

while ((n = read(fd, &s, 1)) > 0) {
    if (write(STDOUT_FILENO, &s, 1) != 1) {
        perror("write");
        return 1;
    }
}

close(fd);
return 0;
}

```

Datoteka se otvara sistemskim pozivom `open()` u načinu samo za čitanje (`O_RDONLY`), čita se znak po znak pozivima `read()`, a svaki znak se prosljeđuje na standardni izlaz pozivom `write()`. Program demonstrira osnovni slijed rada s datotekom (`open` → `read/write` → `close`) i rukovanje greškama preko povratnih vrijednosti sistemskih poziva.

```
./read_file
```

- **io_copy.c** — kopira standardni ulaz na standardni izlaz, čitajući u međuspremnik konstantne veličine (`BUFSIZE`).

```

#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

#define BUFSIZE 1024

int main() {
    int n;
    char buf[BUFSIZE];

    while ((n = read(STDIN_FILENO, buf, BUFSIZE)) > 0) {
        if (write(STDOUT_FILENO, buf, n) != n) {
            perror("write");
            return 1;
        }
    }

    if (n < 0)
        perror("read");

    return 0;
}

```

Za razliku od `read_file.c` koji čita znak po znak, ovdje se u jednom pozivu `read()` pokušava pročitati cijeli blok bajtova, čime se višestruko smanjuje broj sistemskih poziva potrebnih za istu količinu prenesenih podataka. Primjer ujedno ilustrira koncept “sve je datoteka”: pri pokretanju bez preusmjeravanja, standardni ulaz vezan je na tipkovnicu, a standardni izlaz na terminal — isti `read()` i `write()` koji bi radili s običnim datotekama na disku ovdje rade s terminalom. Svaki redak koji korisnik utipka program odmah ispiše natrag, a petlja se prekida pritiskom `Ctrl+D` (oznaka kraja ulaza, EOF):

```

$ ./io_copy
Prvi red teksta
Prvi red teksta
Drugi red teksta
Drugi red teksta

```

```
^D
$
```

U kombinaciji s preusmjeravanjem standardnog izlaza, isti program može poslužiti kao jednostavan alat za upis teksta u datoteku — sve što korisnik utipka do Ctrl+D završi kao sadržaj datoteke, a terminal u međuvremenu ne prikazuje ništa dodatno jer je standardni izlaz preusmjeren:

```
$ ./io_copy > datoteka.txt
Prvi red teksta
Drugi red teksta
^D
$ cat datoteka.txt
Prvi red teksta
Drugi red teksta
```

3.3.2 Pozicioniranje unutar datoteke (lseek)

Sistemske pozive `lseek()` mijenja file offset otvorene datoteke. Koristi se kad želimo čitati ili pisati na specifičnoj poziciji bez sekvencijalnog prolaska.

```
#include <sys/types.h>
#include <unistd.h>

off_t lseek(int filedes, off_t offset, int whence);
```

Povratna vrijednost: novi file offset (broj bajtova od početka datoteke), ili (`off_t`) -1 u slučaju greške.

Argumenti:

- **filedes** — deskriptor otvorene datoteke.
- **offset** — pomak; tumači se relativno prema vrijednosti `whence`.
- **whence** — referentna točka:
 - `SEEK_SET` — pomak je apsolutni, mjeran od početka datoteke,
 - `SEEK_CUR` — pomak je relativan u odnosu na trenutni offset,
 - `SEEK_END` — pomak je relativan u odnosu na kraj datoteke (može biti i negativan da se vrati unazad).

`lseek()` se zove “seek” jer pomiče glavu za čitanje na proizvoljnu poziciju u datoteci — ali to “pomicanje” je samo logičko, na razini jezgrinog file offseta. Stvarni pristup disku događa se tek pri sljedećem `read()`-u ili `write()`-u.

- **f_strip.c** — demonstrira `lseek()` s `SEEK_SET` (apsolutno pozicioniranje od početka datoteke).

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int main() {
    char buf1[] = "Prvi redak teksta";
    char buf2[] = "Drugi redak teksta";
    int fd;

    fd = open("file.strip", O_WRONLY | O_CREAT | O_TRUNC, (mode_t)0644);
    if (fd == -1) {
        perror("open");
        return 1;
    }
}
```

```

if (write(fd, buf1, strlen(buf1)+1) != strlen(buf1)+1) {
    perror("write buf1");
    return 1;
}

if (lseek(fd, 15, SEEK_SET) == -1) {
    perror("lseek");
    return 1;
}

if (write(fd, buf2, strlen(buf2)+1) != strlen(buf2)+1) {
    perror("write buf2");
    return 1;
}

close(fd);
exit(0);
}

```

Program otvara datoteku za pisanje, upisuje prvi niz znakova, lseek-om postavlja offset na 15. bajt i upisivanjem drugog niza **prepisuje** dio postojećeg sadržaja. Rezultat pokazuje da se pozicioniranje unutar otvorene datoteke može slobodno kombinirati s čitanjem i pisanjem — offset koji jezgra pamti nije povezan s fizičkim rasporedom blokova na disku.

Rezultat se može provjeriti UNIX naredbom cat, koja ispisuje sadržaj jedne ili više datoteka na standardni izlaz:

```

$ ./f_strip
$ cat file.strip
Prvi redak teksDrugi redak teksta

```

Prvih 15 bajtova (Prvi redak teks) ostalo je netaknuto iz prvog upisa, a od 15. bajta nadalje vidljiv je sadržaj drugog niza koji je lseek + write upisao preko ostatka prethodnog teksta.

- **f_hole.c** — demonstrira lseek() s SEEK_CUR (relativno pomicanje od trenutne pozicije).

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int main() {
    char buf1[] = "Prvi redak teksta";
    char buf2[] = "Drugi redak teksta";
    int fd;

    fd = creat("file.hole", (mode_t)0644);
    if (fd == -1) {
        perror("creat");
        return 1;
    }

    if (write(fd, buf1, strlen(buf1)+1) != strlen(buf1)+1) {
        perror("write buf1");
        return 1;
    }

    if (lseek(fd, 15, SEEK_CUR) == -1) {
        perror("lseek");
        return 1;
    }
}

```

```

if (write(fd, buf2, strlen(buf2)+1) != strlen(buf2)+1) {
    perror("write buf2");
    return 1;
}

close(fd);
exit(0);
}

```

Nakon upisa prvog niza, offset se pomiče 15 bajtova naprijed — **iza** trenutnog kraja datoteke — i tek se tada upisuje drugi niz. Petnaest bajtova između prve i druge linije ostaje kao **rupa**, područje koje pri čitanju iščitava kao bajtovi s vrijednošću 0 (null terminator), iako `write()` ondje ništa nije upisao.

```

$ ./f_hole
$ ls -l file.hole
-rw-r--r-- 1 dkrst users 52 Apr 22 12:54 file.hole
$ du -h file.hole
4.0K    file.hole
$ od -c file.hole
00000000  P  r  v  i      r  e  d  a  k      t  e  k  s  t
00000020  a  \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0 \0
00000040  \0 D  r  u  g  i      r  e  d  a  k      t  e  k
00000060  s  t  a  \0
00000064

```

Tri naredbe daju pogled na datoteku iz tri različita kuta:

- `ls -l` prijavljuje **logičku veličinu** datoteke — 52 bajta. To je raspon od početka datoteke do posljednjeg upisanog bajta, uključujući i bajtove kroz rupu koje `write()` nikad nije dotaknuo. Isti broj vratio bi i `lseek(fd, 0, SEEK_END)` u programu.
- `du -h` prijavljuje **stvarno zauzeće diska** — 4.0K (odnosno 4096 bajtova, tipična veličina bloka Linux datotečnih sustava poput *ext4*; na drugim sustavima može biti 512 B, 8 KB ili više). Razlog zašto datoteka od 52 bajta zauzima puni 4 KB blok leži u činjenici da su tvrdi diskovi u UNIX-u **blok specijalni uređaji** (vidi poglavlje *Osnove UNIX-a*): podaci se na njima prenose i adresiraju u blokovima fiksne veličine, ne bajt po bajt. Datotečni sustav slijedi tu granularnost — najmanja jedinica alokacije za svaku datoteku je jedan cijeli blok, pa i datoteka od jednog jedinog bajta zauzima koliko i puni blok.
- `od -c` otkriva unutarnju strukturu — niz od 15 uzastopnih `\0` bajtova između dva upisana niza točno je područje rupe. S gledišta procesa, tih 15 bajtova postoji i čitanjem bi se dobile nule; jezgra ih pri čitanju transparentno popunjava bez da su ti podaci ikad bili fizički upisani na disk.

3.3.3 Prava pristupa i maska (`umask`)

Sistemske poziv `umask()` postavlja masku kreiranja datoteka za trenutni proces. Maska predstavlja bitove **koji će biti uklonjeni** iz prava pristupa kada proces stvara nove datoteke (npr. preko `open()`-a s `O_CREAT` ili `creat()`-a).

```

#include <sys/types.h>
#include <sys/stat.h>

mode_t umask(mode_t cmask);

```

Povratna vrijednost: prethodna vrijednost maske.

Argumenti:

- **cmask** — nova vrijednost maske; kombinacija istih konstanti kao za mode u `open()`-u.

Algoritam je jednostavan: za svako kreiranje datoteke, jezgra uzima mode koji je program tražio i **iz njega ukloni** sve bitove koji su postavljeni u maski. Tako je rezultatni način pristupa `mode & ~cmask`. Maska se nasljeđuje od roditelja procesa; tipična vrijednost u shellu je `0022` (oduzima pravo pisanja grupi i ostalima).

- **perm_mask.c** — ilustrira djelovanje maske kreiranja datoteke (`umask`) na prava pristupa pri pozivu `creat()`.

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>

#define PRAVA S_IRUSR | S_IWUSR | S_IRGRP | S_IWGRP | S_IROTH

int main() {
    umask(0);
    if (creat("datoteka1", PRAVA) < 0)
        perror("creat datoteka1");

    umask(S_IRGRP | S_IWGRP | S_IROTH | S_IWOTH);
    if (creat("datoteka2", PRAVA) < 0)
        perror("creat datoteka2");

    return 0;
}
```

Program dva puta stvara datoteku s istim zatraženim pravima (`rw-rw-r--`), jednom s maskom postavljenom na 0 (nikakva prava se ne oduzimaju), a jednom s maskom koja eksplicitno isključuje pisanje za grupu i čitanje/pisanje za ostale. Usporedbom stvarno dobivenih prava vidi se učinak maske:

```
$ ./perm_mask
$ ls -al datoteka1 datoteka2
-rw-rw-r-- 1 dkrt users 0 Jan 16 16:23 datoteka1
-rw----- 1 dkrt users 0 Jan 16 16:23 datoteka2
```

Datoteka `datoteka1` zadržala je sva zatražena prava jer je maska bila 0, dok su u slučaju datoteke `datoteka2` maskom isključena sva prava za grupu i ostale korisnike.

3.3.4 Argumenti naredbenog retka

U svim primjerima koje smo do sada obradili u ovom poglavlju ime datoteke nad kojom program radi zapisano je izravno u izvornom kodu. `read_file.c` uvijek čita `moja_datoteka.txt`; `f_strip.c` uvijek piše u `file.strip`; `f_hole.c` u `file.hole`. U praksi ovakav pristup brzo postaje neupotrebljiv — teško je zamisliv iole koristan program kojem je ime ulazne ili izlazne datoteke trajno ugrađeno u izvorni kod i zahtijeva mijenjanje izvornog koda te ponovno prevođenje svaki put kada treba raditi s drugom datotekom.

Rješenje je da ime datoteke, kao i bilo koji drugi podatak ili parametar koji određuje ponašanje programa, zadaje korisnik pri pokretanju programa. Mehanizam koji UNIX nudi za to su **argumenti naredbenog retka** (*command-line arguments*) — niz tokena koje korisnik upisuje iza imena naredbe, a koje ljuska prosljeđuje programu pri njegovom pokretanju. Primjer iz svakodnevnog rada s ljuskom je `cp izvor.txt cilj.txt`, gdje `cp` prima dva argumenta. Da bi C program mogao iščitati

te argumente, `main` se deklarira u proširenom obliku:

```
int main(int argc, char *argv[])
```

Ovdje `argc` sadrži broj argumenata, a `argv` je polje nizova znakova u kojemu je svaki argument zaseban niz. Bitno je naglasiti da `argc` uključuje i samu naredbu, ne samo argumente koje korisnik dodaje uz nju. Po konvenciji, `argv[0]` je ime kojim je program pokrenut (uključujući i putanju ako je upisana), a stvarni argumenti slijede od `argv[1]` nadalje. Tako pri pozivu `./f_write izlaz.txt`, vrijednosti će biti `argc = 2`, `argv[0] = "./f_write"`, `argv[1] = "izlaz.txt"` — `argc` je 2 jer broji i samo ime programa i jedan argument koji mu je dan. Programi obično odmah na početku provjeravaju je li `argc` u očekivanom rasponu i ako nije, ispišu uputu o korištenju te uredno završe.

- **f_write.c** — čita sa standardnog ulaza i upisuje u novokreiranu datoteku čije se ime zadaje kao argument naredbenog retka.

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

#define FMODE S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH

int main(int argc, char *argv[]) {
    int fd, n;
    char s;

    if (argc != 2) {
        printf("korištenje: %s <ime_datoteke>\n", argv[0]);
        return 0;
    }

    fd = creat(argv[1], FMODE);
    if (fd == -1) {
        perror("creat");
        return 1;
    }

    while ((n = read(STDIN_FILENO, &s, 1)) > 0)
        if (write(fd, &s, 1) < 0) {
            perror("write");
            return 1;
        }

    close(fd);
    return 0;
}
```

Odmah na početku programa provjerava se `argc != 2` — očekuje se točno jedan argument (ime izlazne datoteke) uz ime programa. Ako korisnik program pozove bez argumenta ili s previše njih, program javlja uputu o korištenju i završava. Konverzija `%s` zamjenjuje se upravo vrijednošću `argv[0]` — imenom kojim je program pokrenut — pa poruka korisniku uvijek automatski odražava točan naziv pod kojim je program bio zvan, neovisno o tome je li preimenovan ili pozvan preko simboličke veze. Ovakva provjera ulaznih argumenata uobičajen je uzorak u svim UNIX programima.

Program se poziva s imenom izlazne datoteke kao jedinim argumentom; nakon toga sve što korisnik utipka do oznake kraja ulaza (Ctrl+D) zapisuje se u tu datoteku:

```
$ ./f_write izlaz.txt
```

```

Prvi red teksta
Drugi red teksta
^D
$ cat izlaz.txt
Prvi red teksta
Drugi red teksta

```

- **f_cat.c** — pojednostavljena implementacija UNIX naredbe cat.

```

#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

int rw(int fdin, int fdout) {
    int n;
    char s;
    while ((n = read(fdin, &s, 1)) > 0)
        write(fdout, &s, 1);

    return n;
}

int main(int argc, char **argv) {
    int k, fd;
    if (argc < 2) {
        rw(STDIN_FILENO, STDOUT_FILENO);
    } else {
        for (k = 1; k < argc; k++) {
            fd = open(argv[k], O_RDONLY);
            if (fd < 0) /* ne mogu otvoriti */
                perror("open");
            else {
                rw(fd, STDOUT_FILENO);
                close(fd);
            }
        }
    }

    return 0;
}

```

Ključna ideja u implementaciji je pomoćna funkcija `rw()` koja sadrži osnovnu petlju čitanja s jednog deskriptora i pisanja na drugi. Ista petlja koja je u prethodnim primjerima bila u `main()` ovdje je izdvojena kao zasebna funkcija kako bi se mogla pozvati u više različitih konteksta.

Analizirajmo glavni program:

- **Uvjet `argc < 2`** — ako je program pokrenut bez imena datoteke, poziva se `rw(STDIN_FILENO, STDOUT_FILENO)` — standardni ulaz se prepisuje na standardni izlaz, potpuno isto kao u primjeru `io_copy.c`, ali ovaj put znak po znak.
- **for petlja po argumentima** — u slučaju da je navedena jedna ili više datoteka, petlja redom prolazi kroz `argv[1]` do `argv[argc-1]`, otvara svaku pozivom `open()` s `O_RDONLY`, i njezin sadržaj ispisuje na standardni izlaz pozivom iste funkcije `rw()` — ali sada s file deskriptorom otvorene datoteke umjesto `STDIN_FILENO`. Nakon ispisa datoteka se zatvara pozivom `close()`, a petlja nastavlja sa sljedećom.

Ovaj primjer ilustrira jednu od temeljnih posljedica UNIX načela “sve je datoteka”: ista funkcija `rw()`, napisana nad općim file deskriptorima, radi bez izmjena kako s običnim datotekama tako i sa standardnim ulazno/izlaznim tokovima. Programeru je svejedno odakle podaci dolaze — logika

i implementacija čitanja i pisanja je jedinstvena.

Pokretanjem programa s različitim argumentima dobivamo različito ponašanje:

```
$ ./f_cat
```

Bez argumenata, program se ponaša kao `io_copy` — čita sa standardnog ulaza i ispisuje na standardni izlaz, sve dok ne dobije EOF.

```
$ ./f_cat f_cat.c
```

S jednim argumentom, program ispisuje sadržaj te datoteke na standardni izlaz. U gornjem primjeru program ispisuje vlastiti izvorni kod — i to bez ičega posebnoga: jednako bi ispisao bilo koju drugu tekstualnu datoteku navedenu kao argument.

```
$ ./f_cat datoteka1.txt datoteka2.txt
```

Sa više argumenata, program ispisuje navedene datoteke jednu za drugom — u ovom slučaju najprije sadržaj `datoteka1.txt`, a odmah za njim sadržaj `datoteka2.txt` (naravno, pod uvjetom da obje datoteke postoje u radnom direktoriju). Time se reproducira ponašanje UNIX naredbe `cat` po kojoj je program i nazvan: spaja (engl. *concatenate*) sadržaje više datoteka u jedinstveni izlazni tok. Ako neka od navedenih datoteka ne postoji, program će za nju ispisati poruku o grešci pozivom `perror()`, ali **neće prekinuti izvođenje** — sve ostale datoteke koje postoje uredno će biti ispisane.

3.4 Ulazno-izlazne strukture podataka

U primjerima iznad držali smo se praktične razine — otvorimo datoteku, dobijemo deskriptor, čitamo i pišemo. U tekstu koji slijedi dat ćemo malo više “teorije”: objasniti ćemo kako UNIX upravlja otvorenim datotekama, koje interne strukture jezgra pri tom koristi i kako su one međusobno povezane. Detaljniji prikaz ovih internih struktura, kao i mnogih drugih tema iz ovog poglavlja, čitatelj će pronaći u temeljnom djelu *Advanced Programming in the UNIX Environment*, Stevens & Rago [1] — knjizi koja prati gotovo svako poglavlje ove skripte; istu materiju, ali iz šire perspektive datotečnog podsustava operacijskih sustava, obrađuje i sveučilišni udžbenik *Operacijski sustavi*, Budin, Golub, Jakobović & Jelenković [2]. Nestrpljiv čitatelj koji želi što prije vidjeti dodatne primjere može nastaviti dalje, ali valja naglasiti da je razumijevanje ovih struktura važno ne samo za rad s datotekama, nego i za razumijevanje UNIX-a kao operacijskog sustava u cjelini. Mnogi koncepti koje ćemo susresti kasnije (dijeljenje datoteka među procesima, nasljeđivanje deskriptora, preusmjeravanje ulaza i izlaza, kao i temeljno UNIX načelo “sve je datoteka”) izravno proizlaze iz organizacije struktura opisanih u nastavku.

Govoriti ćemo o **tablici procesa**, **tablici datoteka** i **v-node tablici** — internim strukturama UNIX jezgre. Bitno je odmah naglasiti da korisnički proces tim strukturama ne može pristupiti izravno, putem varijable ili pokazivača, jer se radi o internim strukturama smještenim u memoriji jezgre; jedini način na koji proces može s njima komunicirati jesu sistemski pozivi.

3.4.1 Tablica procesa, tablica datoteka i v-node tablica

Jezgra operacijskog sustava svakom aktivnom procesu dodjeljuje jedan zapis u **tablici procesa** (engl. *process table*). Taj zapis sadrži sve informacije koje jezgra treba o procesu — od identifikatora i stanja do informacija o memoriji i otvorenim

datotekama. Uz tablicu deskriptora, o kojoj govorimo u nastavku, u zapis procesa spada i niz atributa koji utječu na ulazno-izlazne operacije — među njima i trenutna vrijednost maske `umask`, koju smo obradili ranije. Upravo zato svaki proces može neovisno mijenjati svoju masku, a novostvoreni proces nasljeđuje vrijednost od procesa koji ga je stvorio (tzv. **procesa roditelja**, engl. *parent process*). Tablicu procesa, kao i mehanizam stvaranja novih procesa, detaljnije ćemo razraditi u poglavlju o procesima; ovdje nas zanima samo jedan dio tablice procesa: **tablica deskriptora** (engl. *descriptor table*) unutar zapisa procesa, u kojoj se čuva informacija o svim otvorenim datotekama promatranog procesa, odnosno o svim njegovim deskriptorima datoteka. Podsjetimo se — pri pokretanju programa na deskriptorima 0, 1 i 2 u pravilu su otvoreni standardni ulaz, standardni izlaz i standardni izlaz za greške; svi kasnije otvoreni deskriptori dobivaju vrijednosti 3 i veće.

Svaki zapis u tablici deskriptora sadrži dvije stavke: zastavice deskriptora specifične za proces (*fd flags*) te pokazivač na odgovarajući zapis u globalnoj tablici datoteka. Polje *fd flags* privatno je za pojedini proces i u praksi sadrži samo jednu zastavicu, `FD_CLOEXEC`, koja određuje hoće li se deskriptor (tj. datoteka koja je na njemu otvorena) automatski zatvoriti ukoliko unutar ovog procesa pokrenemo novi program — odnosno učitamo s diska novu izvršnu datoteku i započnemo izvršavanje od prve naredbe u njezinu kodu. Pokretanje novog programa u postojećem procesu postiže se jednim od sistemskih poziva iz obitelji `exec`, kojima se detaljnije bavimo u poglavlju o okruženju procesa.

Bitno je odmah razlikovati ove zastavice od **statusnih zastavica datoteke** (*file status flags*) o kojima ćemo govoriti u nastavku: *fd flags* nalaze se u tablici deskriptora i privatne su za proces, dok su *file status flags* dio zapisa u tablici datoteka i dijele ih svi deskriptori koji pokazuju na isti zapis.

Dok je tablica deskriptora privatna za svaki proces, **tablica datoteka** (engl. *file table*) je globalna — jezgra održava jednu takvu tablicu u koju se upisuje zapis za svako otvaranje neke datoteke. Svaki takav zapis sadrži:

- **statusne zastavice datoteke** — postavljaju se pri otvaranju datoteke pozivima `open` ili `creat` i određuju način pristupa datoteci (npr. `O_RDONLY`, `O_WRONLY`, `O_RDWR`, `O_APPEND`, `O_NONBLOCK` i sl.). Za razliku od ranije spomenutih *fd flags*, ove zastavice nisu vezane uz pojedini deskriptor već uz otvorenu instancu datoteke, pa ih dijele svi deskriptori koji pokazuju na isti zapis u tablici datoteka;
- **file offset** — trenutnu poziciju unutar datoteke, koja se automatski pomiče pri svakom pozivu `read` ili `write`, a može se eksplicitno postaviti pozivom `lseek`;
- **pokazivač na zapis u v-node tablici** — referencu na strukturu koja opisuje samu datoteku.

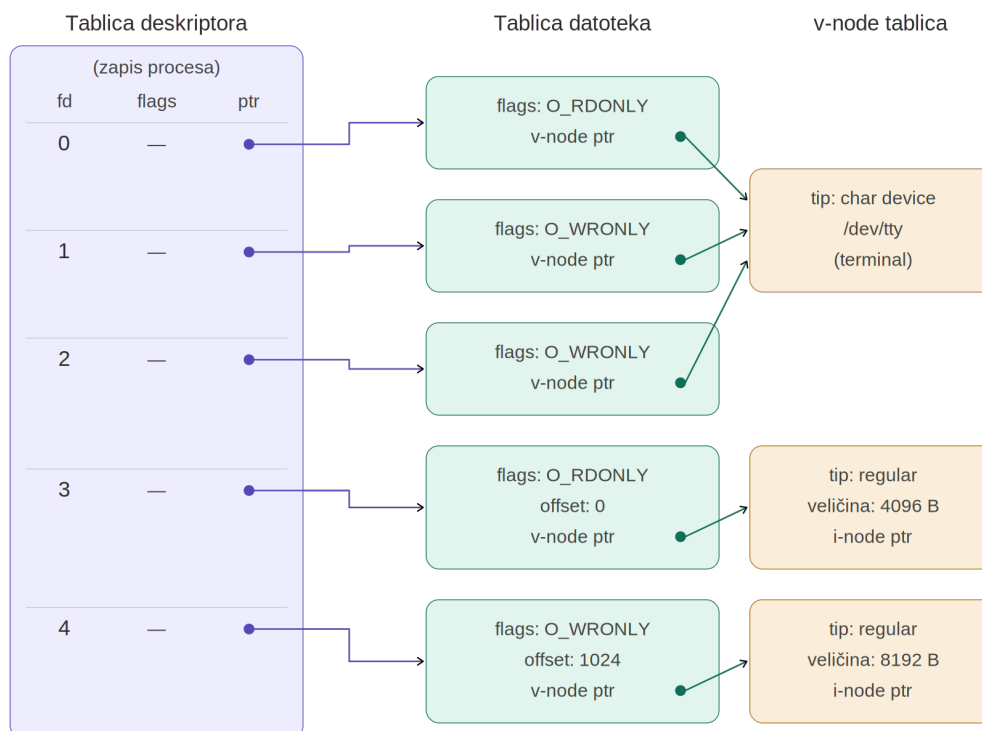
v-node tablica (engl. *v-node table*) predstavlja apstrakciju same datoteke. Riječ je o dinamičkoj strukturi koja nastaje u trenutku prvog otvaranja datoteke i oslobađa se kad ju zatvori posljednji proces koji ju drži otvorenom. Svaki zapis u v-node tablici sadrži informacije kao što su tip datoteke (obična datoteka, direktorij, simbolička veza, uređaj, socket, ...). Uz to, v-node zapis obično sadrži i kopiju podataka poput vlasnika, prava pristupa i veličine datoteke; ti se podaci u trenutku otvaranja datoteke čitaju iz **i-node** zapisa, gdje su trajno pohranjeni. Najvažnije — v-node zapis sadrži **pokazivače na stvarne funkcije za rad s datotekom** koje odgovaraju tipu te datoteke. Upravo ovaj mehanizam omogućuje da kad program pozove `read` ili `write` na nekom deskriptoru, jezgra zna koju stvarnu rutinu treba izvršiti da bi se pristupilo podacima, bez obzira na to radi li se o datoteci na disku,

terminalu, mrežnom socketu ili komunikacijskoj točki između dva aktivna procesa. Štoviše, programer ne mora ni znati kakva se datoteka krije iza deskriptora — v-node tablica stvara sloj apstrakcije koji omogućuje da se svim tipovima datoteka pristupa na isti način. Ovo je tehnička osnova temeljnog UNIX načela “sve je datoteka”.

Za datoteke koje se nalaze na datotečnom sustavu (obične datoteke, direktoriji, simboličke veze, uređaji, imenovane cijevi) v-node zapis dodatno sadrži pokazivač na odgovarajući **i-node** (engl. *index node*) zapis. I-node tablica trajno (statički) čuva implementacijske detalje vezane uz stvarnu datoteku: vlasnika, prava pristupa, veličinu, vremena pristupa, kao i — kod običnih datoteka — točne adrese blokova podataka od kojih je datoteka sastavljena na jedinici za pohranu. Kod posebnih datoteka (uređaja, imenovanih cijevi) i-node umjesto pokazivača na blokove podataka sadrži podatke koji jezgri omogućuju da identificira odgovarajući uređaj, odnosno mehanizam komunikacije s datotekom. Za datoteke koje nisu smještene na datotečnom sustavu (npr. mrežne utičnice — *socketi*, ili anonimne cijevi — dinamički stvorene komunikacijske strukture za razmjenu podataka među procesima) i-node ne postoji — u tom slučaju v-node zapis pokazuje izravno na odgovarajuće interne strukture jezgre koje opisuju taj resurs.

Važno je naglasiti da je većina opisanih struktura **dinamička**: kreiraju se u trenutku otvaranja datoteke i grade se od v-node tablice prema višim razinama apstrakcije (tablica datoteka, pa zatim tablica deskriptora u zapisu procesa). Kad se datoteka zatvori, odgovarajući zapisi u tim tablicama se oslobađaju. I-node tablica je, za razliku od njih, statična struktura koja čuva informaciju o stvarnim podacima datoteke na disku — ona postoji i onda kad datoteka nije otvorena ni u jednom procesu. Kad se datoteka po prvi put otvori, jezgra kopira odgovarajući i-node zapis s diska u memoriju (u takozvanu *in-core i-node tablicu*) i u tom trenutku se on ponaša kao dinamička struktura — sve dok datoteka ostaje otvorena.

Opisane strukture i njihove međusobne veze za jedan proces s dvije otvorene datoteke shematski su prikazane na slici 3.1:



Slika 3.1: Interne strukture jezgre za proces s dvije otvorene datoteke. Deskriptori datoteka 0 (standard input), 1 (standard output) i 2 (standard error) otvoreni su automatski pri pokretanju.

Slika prikazuje stanje internih struktura jezgre za jedan proces koji je sistemskim pozivima `open` ili `creat` otvorio dvije datoteke — jednu za čitanje (deskriptor 3) i jednu za pisanje (deskriptor 4). Uz njih, u tablici deskriptora postoje i zapisi za standardne tokove na deskriptorima 0, 1 i 2 koji su otvoreni automatski pri pokretanju procesa. Svih pet deskriptora ima vlastite zapise u tablici datoteka, no zanimljivo je primijetiti da svi zapisi za standardne tokove pokazuju na isti v-node zapis — onaj koji predstavlja korisnički terminal (znakovni uređaj `/dev/tty`). Zapisi u tablici datoteka za standardne tokove ne sadrže file offset jer kod znakovnog uređaja apstrakcija apsolutne pozicije unutar datoteke nema smisla — podaci pristižu (ili se šalju) onim redoslijedom kojim ih korisnik unosi (odnosno program ispisuje), bez mogućnosti vraćanja ili preskakanja. Nasuprot tome, deskriptori 3 i 4 pokazuju na zasebne v-node zapise koji predstavljaju regularne datoteke na disku, a u njihovim zapisima u tablici datoteka čuvaju se i odgovarajući file offseti.

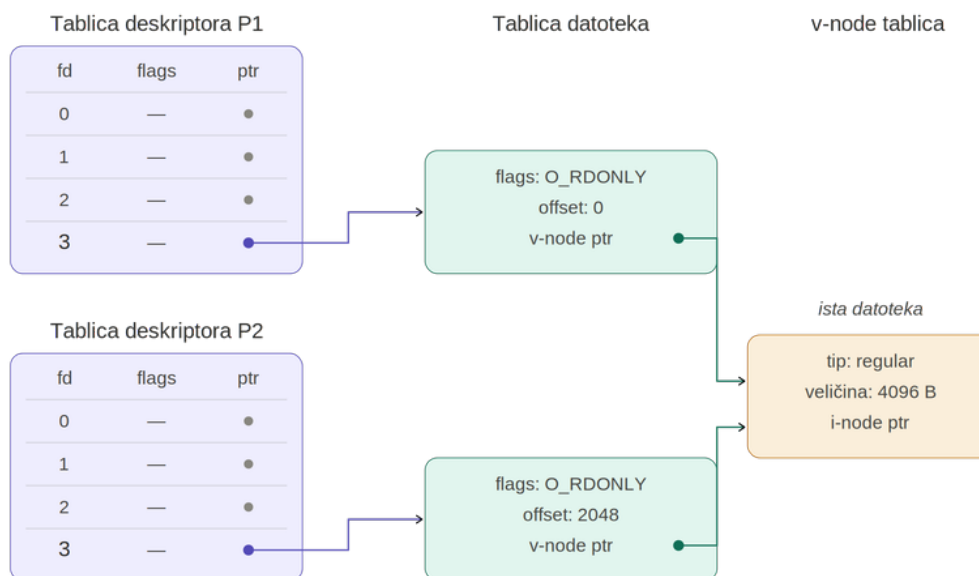
Ovakva troslojna organizacija struktura podataka ima nekoliko važnih posljedica. Dva različita deskriptora unutar istog procesa mogu pokazivati na iste ili različite zapise u tablici datoteka — ovisno o tome kako su nastali. Proces koji dijele istu datoteku mogu, ovisno o načinu na koji su je otvorili, imati ili vlastite zapise u tablici datoteka (a samim time i neovisne file offsete) ili dijeliti isti zapis u tablici datoteka (i samim time isti file offset). Konačno, zastavice deskriptora (`fd flags` u tablici procesa) vidljive su samo unutar jednog procesa i odnose se samo na taj deskriptor, dok statusne zastavice u tablici datoteka vrijede za sve deskriptore koji pokazuju na isti zapis u toj tablici. Sve ove posljedice obrađujemo u nastavku.

3.5 Dijeljenje datoteka između procesa

UNIX-ova troslojna organizacija struktura podataka prirodno omogućuje različite oblike dijeljenja datoteka između procesa. Dva procesa mogu pristupati istoj fizičkoj datoteci dijeleći samo v-node zapis, ali uz vlastite, neovisne file offseete. S druge strane, dva procesa (ili dva deskriptora unutar istog procesa) mogu dijeliti i sam zapis u tablici datoteka, pa tako i zajednički file offset. U nastavku ćemo razmotriti oba slučaja.

3.5.1 Dijeljenje na razini v-node tablice

Najjednostavniji i najosnovniji oblik dijeljenja prikazan je na slici 3.2: dva procesa, P1 i P2, neovisno jedan o drugome otvaraju istu datoteku, svaki vlastitim pozivom open. Zanimljivo je primijetiti da bi i u slučaju kad isti proces pozove open dva puta s imenom iste datoteke, rezultat bila dva nezavisna zapisa u tablici datoteka, od kojih svaki ima svoje statusne zastavice i svoj file offset.



Slika 3.2: Dva procesa neovisno otvaraju istu datoteku: vlastiti zapis u tablici datoteka, zajednički v-node

Iako se radi o istoj datoteci, jezgra svakom procesu dodjeljuje zaseban zapis u tablici datoteka — a samim time i vlastiti, neovisni file offset i vlastite statusne zastavice. Dijeljenje se događa na razini v-node tablice: v-node predstavlja apstrakciju same fizičke datoteke, koja je samo jedna. Praktična posljedica je da svaki proces čita ili piše po istoj datoteci nezavisno — pomak jednog procesa u datoteci ne utječe na poziciju drugoga — dok eventualne izmjene sadržaja postaju vidljive obama procesima jer dijele istu fizičku datoteku.

Ovakav način dijeljenja sasvim dobro funkcionira ukoliko procesi datoteku samo čitaju. Međutim, problemi mogu nastati ukoliko procesi datoteku otvore za pisanje. Budući da svaki proces ima vlastiti file offset koji se čuva u zasebnom zapisu u tablici datoteka, pisanje u datoteku jednog procesa nema nikakvog efekta na offset drugog procesa. Vrlo lako se može dogoditi da podaci koje je jedan proces upisao

u datoteku budu “pregaženi” podacima koje nakon toga upiše drugi proces. Ovaj problem detaljnije ćemo razmotriti u nastavku.

3.5.2 Race condition

Razmotrimo sada detaljnije problem koji smo nagovijestili na kraju prethodnog odjeljka. Dva nezavisna procesa otvorila su istu datoteku za pisanje pozivima `open`; svaki proces ima vlastiti zapis u tablici datoteka, s vlastitim file offsetom i vlastitim statusnim zastavicama, dok dijele isti v-node jer se radi o jednoj te istoj fizičkoj datoteci.

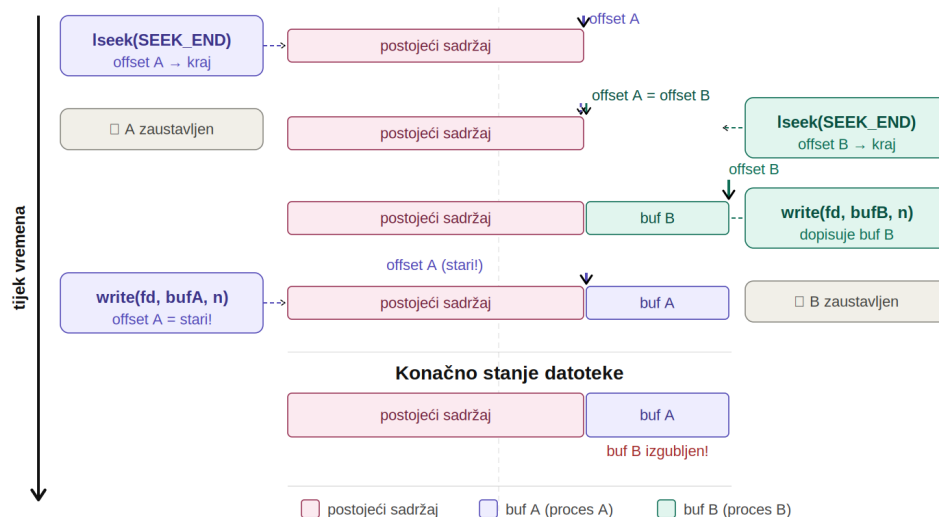
Zamislimo scenarij u kojem oba procesa, primjerice, u istu log datoteku upisuju informacije o vlastitom statusu, ili svaki od njih upisuje rezultate obrade dijela nekog velikog skupa podataka. Oba procesa žele svoje nove podatke dopisati na kraj datoteke, tako da se ne prepisuju postojeći podaci. Uobičajeni način da se to postigne je da se prije svakog pisanja eksplicitno pozicioniraju na kraj datoteke pomoću `lseek`:

```
lseek(fd, 0, SEEK_END);    /* pozicioniranje na kraj datoteke */
write(fd, buf, n);         /* upisivanje podataka */
```

Kao što smo ranije rekli, UNIX je višekorisnički, višezadaćni operacijski sustav, a naši procesi odvijaju se u tzv. **dijeljenom vremenu** (engl. *time-sharing*): svaki proces dobiva računalne resurse na raspolaganje određeni dio vremena, nakon čega biva pauziran i čeka svoj red dok se ostali procesi izvršavaju. Ovo upravljanje resursima, naravno, obavlja jezgra, odnosno njezin specijalizirani dio koji nazivamo **raspoređivač** (engl. *scheduler*). Same izmjene procesa koji se izvršava događaju se u pravilu jako brzo — proces ni u jednom trenutku nema spoznaju o tome da je njegovo izvršavanje nakratko bilo zaustavljeno, a korisnik to obično niti ne primjećuje. Tek u slučaju značajno opterećenog sustava može doći do vidljivog usporavanja i primjetnog čekanja.

Važno je naglasiti da do zaustavljanja procesa može doći u bilo kojem trenutku — na to proces nema nikakav utjecaj i zaustavljanje će nastupiti kad jezgra odluči da je vrijeme da proces prepusti resurse nekom drugom. Konkretno u scenariju koji razmatramo, to znači da raspoređivač može prekinuti izvođenje našeg procesa i između poziva `lseek` i `write`, što može dovesti do scenarija ilustriranog na slici 3.3:

1. Proces A poziva `lseek(fd, 0, SEEK_END)` i pozicionira svoj offset na kraj datoteke, recimo na offset 200.
2. Raspoređivač prekida proces A i aktivira proces B.
3. Proces B poziva `lseek(fd, 0, SEEK_END)` — budući da ima vlastiti offset, i on se pozicionira na kraj datoteke, također na offset 200.
4. Proces B poziva `write` i upisuje svoje podatke na offset 200, čime pomiče svoj offset dalje.
5. Raspoređivač vraća kontrolu procesu A.
6. Proces A poziva `write` — ali njegov offset još uvijek pokazuje na 200, jer se nije ažurirao nakon što je proces B pisao u datoteku (offset procesa A čuva se u zasebnom zapisu u tablici datoteka).
7. Posljedica: podaci koje je upisao proces B prepisani su podacima procesa A.



Slika 3.3: Stanje utrke (race condition) pri dopisivanju na kraj datoteke bez zastavice `O_APPEND`

Ključna spoznaja je da problem ne leži u samom pisanju — problem je što između `lseek` i `write` raspoređivač može aktivirati drugi proces koji mijenja sadržaj datoteke. U tom trenutku offset koji je sačuvao proces A više ne pokazuje na kraj datoteke. Ovakvu situaciju — u kojoj konačni ishod ovisi o tome kojim se redoslijedom odvijaju operacije dvaju ili više procesa — nazivamo **race condition**.

Rješenje ovog problema leži u mehanizmu **atomske operacije**, o kojima govorimo u nastavku.

3.5.3 Atomske operacije

Atomska operacija jest niz koraka koji se ili u potpunosti izvode ili se ne izvodi niti jedan korak — nema mogućnosti da operacija bude prekinuta na pola.

Rješenje race condition problema opisanog u prethodnom odjeljku jest spriječiti da proces bude zaustavljen između pozicioniranja i pisanja. Ovo možemo osigurati ukoliko datoteku otvorimo s uključenom zastavicom `O_APPEND`. U tom slučaju jezgra svaki poziv `write` pretvara u dvije operacije:

1. pomicanje offseta na kraj datoteke,
2. upisivanje podataka.

Ono što je ključno — jezgra u ovom slučaju garantira da naš proces neće biti prekinut između pozicioniranja i pisanja. Ova složena operacija koja se sastoji od dva koraka izvodi se atomski, kao jedna nedjeljiva operacija koju nije moguće prekinuti.

```
/* Bez O_APPEND -- nesigurno pri paralelnom radu: */
int fd = open("log.txt", O_WRONLY | O_CREAT, 0644);
lseek(fd, 0, SEEK_END); /* korak 1: pozicioniranje */
/* mogućnost prekida između naredbi! */
write(fd, buf, n); /* korak 2: pisanje */

/* S O_APPEND -- atomska garancija: */
int fd = open("log.txt", O_WRONLY | O_CREAT | O_APPEND, 0644);
write(fd, buf, n); /* pozicioniranje + pisanje, atomski */
```

Drugi primjer atomske operacije u radu s datotekama jest zastavica `O_EXCL` u kombinaciji s `O_CREAT`. Sjetimo se da kombinacijom ovih zastavica osiguravamo da se datoteka kreira **samo ako već ne postoji** — u suprotnom poziv `open` vraća grešku. Iako na prvi pogled ovo možda izgleda sigurno, ukoliko operacija provjere postojanja datoteke i kreiranja datoteke ne bi bila atomska, može doći do *race conditiona*: kao i u ranijem primjeru, raspoređivač može zaustaviti naš proces nakon što jezgra provjeri da datoteka ne postoji, a prije nego je stigne stvoriti. Ako u međuvremenu drugi proces sa svoje strane stvori datoteku s istim imenom (jer je i kod njega provjera završila s rezultatom “ne postoji”), oba bi procesa mislila da su uspješno stvorila novu datoteku, a zapravo bi jedan od njih nesvjesno otvorio postojeću datoteku drugog procesa. Zastavica `O_EXCL` jezgri govori da se provjera postojanja datoteke i njezino kreiranje moraju izvesti atomski — bez ikakve mogućnosti prekida između ova dva koraka — čime se opisani *race condition* u potpunosti izbjegava.

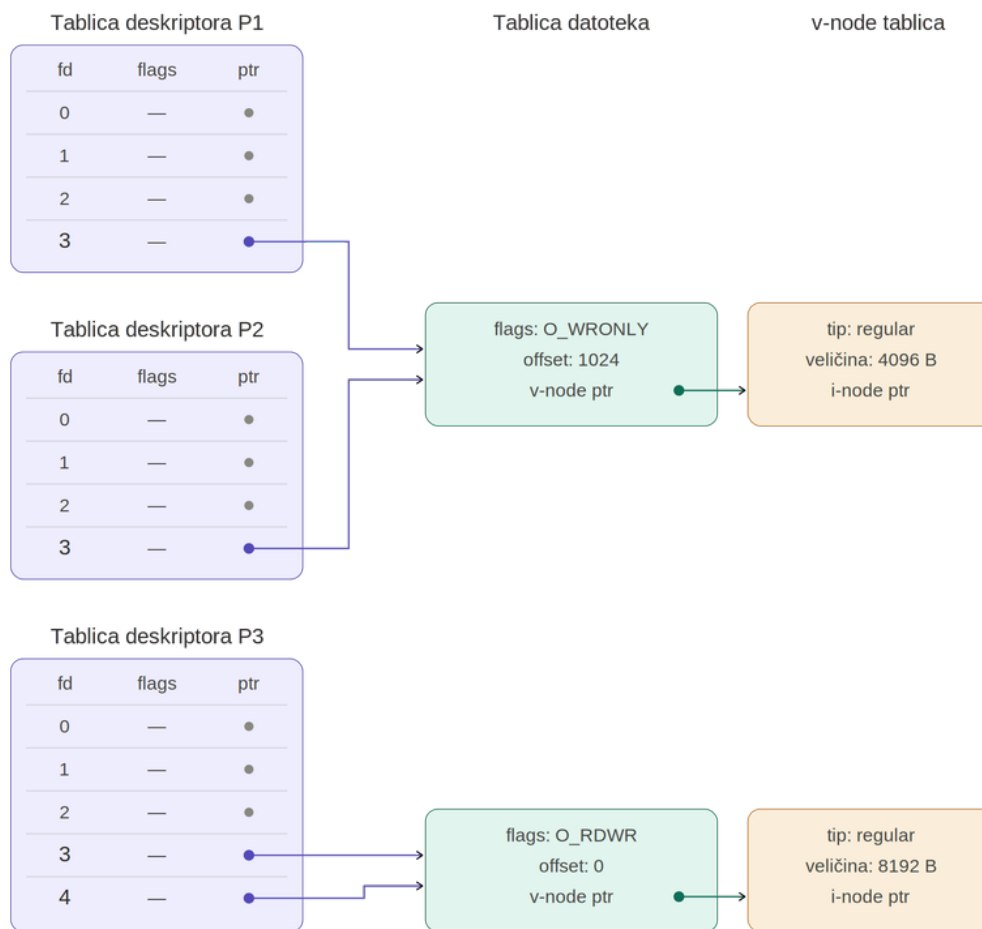
3.5.4 Dijeljenje na razini tablice datoteka

Vratimo se sad na priču o dijeljenju datoteka između procesa. Osim dijeljenja na razini *v-node* tablice, koje smo obradili u prethodnoj podsekciji, do dijeljenja datoteka u UNIX-u može doći i na razini same tablice datoteka: više deskriptora pokazuje na isti zapis u tablici datoteka, pa tako dijele i isti *file offset* i iste statusne zastavice. Kao što ćemo vidjeti, ovo ujedno predstavlja i drugi način izbjegavanja *race condition* problema — uz atomske operacije obrađene u prethodnoj podsekciji.

Postoje dva načina na koja do ovog oblika dijeljenja može doći.

Prvi način vezan je uz nastanak procesa. Svaki proces u UNIX-u nastaje na način da ga proces roditelj kopira sistemskim pozivom `fork`. Ovim sistemskim pozivom detaljno ćemo se baviti u poglavlju o procesima; ovdje je dovoljno znati da novostvoreni proces nasljeđuje sve otvorene deskriptore roditeljskog procesa, a naslijeđeni deskriptori u djetetu pokazuju na **iste zapise** u tablici datoteka kao i kod roditelja.

Drugi način je dupliciranje deskriptora unutar istog procesa korištenjem funkcija `dup` ili `dup2`. Rezultat je da dva (ili više) deskriptora unutar istog procesa pokazuju na isti zapis u tablici datoteka.



Slika 3.4: Dijeljenje zapisa u tablici datoteka: procesi P1 i P2 dijele zapis nakon fork-a, proces P3 ima dva deskriptora koji pokazuju na isti zapis nakon dup (ili dup2)

Oba slučaja prikazana su na slici 3.4. Gornji dio ilustrira prvi slučaj: procesi P1 i P2 dijele jedan zapis u tablici datoteka — P2 je dijete od P1, stvoreno opisanim mehanizmom. Iako se radi o dva odvojena procesa, oni dijele isti file offset, što znači da pisanje jednog procesa pomiče offset koji vidi i drugi proces. Upravo zato ovakav mehanizam predstavlja jedan od načina rješavanja problema prepisivanja podataka opisanog u podsekciji o race conditionu: umjesto da svaki proces ima vlastiti offset koji može “zaostati” za stvarnim krajem datoteke, svi procesi dijele jedan zajednički offset koji se ažurira pri svakoj operaciji pisanja.

Donji dio slike 3.4 prikazuje drugi slučaj: proces P3 ima dva deskriptora — fd 3 i fd 4 — koji pokazuju na isti zapis u tablici datoteka. Do ove situacije dolazi pozivom dup ili dup2, funkcija koje detaljno opisujemo u nastavku.

3.5.5 Dupliciranje deskriptora — dup() i dup2()

Iako na prvi pogled nije jasno čemu bi unutar istog procesa služila dva deskriptora koja pokazuju na isti zapis u tablici datoteka, ovaj mehanizam ima vrlo važnu praktičnu primjenu — koristi se za **preusmjerenje standardnog ulaza i izlaza**.

U UNIX-u postoje dvije funkcije za dupliciranje deskriptora — dup i dup2. Obje stvaraju novi deskriptor koji dijeli isti zapis u tablici datoteka s originalnim,

uključujući isti file offset i statusne zastavice.

```
#include <unistd.h>

int dup(int filedes);
int dup2(int filedes, int filedes2);
```

Povratna vrijednost: novi deskriptor datoteke, ili -1 u slučaju greške.

Argumenti:

- **filedes** — otvoreni deskriptor koji se duplicira.
- **filedes2** (samo za dup2) — ciljani deskriptor na koji se duplicira filedes. Ako je filedes2 već otvoren, dup2 ga **atomski zatvara** prije dupliciranja. Ako je filedes2 == filedes, poziv nema učinka i dup2 vraća filedes bez promjene.

Funkcija dup jednostavniji je način dupliciranja deskriptora datoteke: postojeći deskriptor, koji je zadan kao argument funkcije, duplicira se na **najniži slobodni deskriptor**. Na primjer, ukoliko je datoteka otvorena na deskriptoru 3 (sjetimo se da su deskriptori 0, 1 i 2 najčešće zauzeti već pri pokretanju procesa), poziv funkcije dup vratit će vrijednost 4, što je ujedno i novi deskriptor datoteke koji je povezan s istim zapisom u tablici datoteka na koji je prethodno pokazivao deskriptor 3. Upravo ova situacija prikazana je na donjem dijelu slike 3.4 u prethodnoj podsekciji.

Promislimo što bi bio rezultat ako prije pozivanja funkcije dup zatvorimo neki od nižih deskriptora datoteke. Ova situacija prikazana je u sljedećem primjeru:

- **dup_redirect.c** — preusmjerava standardni izlaz na datoteku korištenjem close + dup.

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    int fd, newfd;

    fd = open("izlaz.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) {
        perror("open");
        return 1;
    }

    close(STDOUT_FILENO); /* zatvori standardni izlaz */
    newfd = dup(fd);      /* dup vraća najnižu slobodnu vrijednost = 1 */

    if (newfd != fd)
        close(fd);

    printf("Ovaj tekst nece završiti u terminalu!\n");

    return 0;
}
```

Nakon što smo pozivom close(1) zatvorili standardni izlaz (deskriptor 1 time postaje slobodan), poziv dup(fd) duplicira fd upravo na deskriptor 1 (STDOUT_FILENO), koji je u tom trenutku najniži slobodni deskriptor. Posljedica je da deskriptor 1, koji se po konvenciji koristi za standardni izlaz procesa, sad pokazuje na isti zapis u tablici datoteka kao i fd — tj. na za pisanje otvorenu fizičku datoteku na disku. Nakon dupliciranja originalni deskriptor fd više nam ne treba: oba deskriptora pokazuju na isti zapis u tablici datoteka, pa je dovoljno zadržati samo jedan. Stari fd zatvaramo pozivom close(fd), ali tek uz provjeru if (newfd != fd) — njome se štitimo od posebnog slučaja u kojem je novi

deskriptor vraćen na istom mjestu kao stari, pa bi bezuvjetan `close(fd)` zatvorio jedini preostali deskriptor.

```
$ ./dup_redirect
$ cat izlaz.txt
Ovaj tekst neće završiti u terminalu!
```

Svako pisanje na standardni izlaz zbog toga efektivno završava u datoteci na disku. Čak i ako se u programu koriste funkcije standardne C biblioteke poput `printf`, one se u procesu prevođenja prevode u niz poziva `write` s deskriptorom `STDOUT_FILENO` (tj. 1) kao prvim argumentom — `write` je jedini sistemski poziv kojim se podaci mogu slati na bilo koji izlazni komunikacijski tok na UNIX-u. Tekst koji je programer namjeravao ispisati u terminal korisnika u ovom slučaju završava u datoteci `izlaz.txt`.

Upravo ovaj “trik” koristi se za preusmjeravanje standardnih ulaza i izlaza: programer koji je napisao program ne mora znati ništa o tome gdje će njegov ispis završiti — dovoljno je da prije pokretanja programa opisanom tehnikom “podmetnemo” datoteku na deskriptor 1 i svi ispisi nakon toga, umjesto u terminal korisnika, završavaju u toj datoteci. Ovu tehniku koristi UNIX ljuska kad korištenjem operatora `<` ili `>` preusmjeravamo standardne ulaze ili izlaze.

Iako program funkcionalno obavlja zadatak, u programima s više niti **nije siguran**: uzastopni pozivi `close(1)` i `dup(fd)` čine **dvije odvojene operacije**. O osnovama UNIX niti više ćemo govoriti u zasebnom poglavlju, no ovdje je ključno naglasiti da niti unutar istog procesa dijele iste deskriptore. Niti unutar procesa, kao i procesi međusobno, dijele računalne resurse, a resursima upravlja raspoređivač. Slično kao u ranije razmatranom primjeru race conditiona kod kojeg su dva procesa pisala u istu datoteku, i ovdje se može dogoditi situacija da nit koja je pozvala `close(1)` bude zaustavljena prije pozivanja funkcije `dup(fd)`. Ukoliko nakon toga neka druga nit unutar istog procesa pozove funkcije `open` ili `creat`, ove će funkcije traženu datoteku otvoriti na najnižem slobodnom deskriptoru i preuzeti upravo oslobođeni deskriptor 1 za sebe.

Rješenje je u korištenju naprednijeg poziva `dup2`, koji ima dva argumenta: deskriptor koji dupliciramo i deskriptor na koji želimo duplicirati postojeći deskriptor. Za razliku od funkcije `dup`, **`dup2` je atomska operacija** i sigurni smo da ju raspoređivač neće prekinuti. Ako je zadani odredišni deskriptor već otvoren, `dup2` ga atomski zatvara prije dupliciranja. Štoviše, eksplicitno se navodi i deskriptor na koji želimo duplicirati postojeći pa se ne moramo oslanjati na pretpostavku da je to ujedno i najniži slobodni deskriptor. Upravo iz ovih razloga `dup2` je preferirana funkcija.

- **`dup2_redirect.c`** — isti zadatak, samo s `dup2`.

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    int fd;

    fd = open("izlaz.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) {
        perror("open");
        return 1;
    }

    if (dup2(fd, STDOUT_FILENO) != fd)
        close(fd);
```

```
printf("Ovaj tekst završava u datoteci izlaz.txt!\n");  
  
return 0;  
}
```

Pozivom funkcije `dup2` atomski se zatvara datoteka otvorena na deskriptoru 1 (standardni izlaz), nakon čega se na njega duplicira `fd` — što efektivno povezuje deskriptor 1 sa zapisom u tablici datoteka koji je povezan s otvorenom fizičkom datotekom na disku. I u ovom slučaju izvodi se više koraka potrebnih da se cijela operacija zatvaranja i dupliciranja deskriptora izvrši, ali koncept atomske operacije osigurava da taj postupak neće biti prekinut dok se ne izvrši u cijelosti.

Kao i u prethodnom slučaju, nakon dupliciranja zatvaramo deskriptor koji nam više ne treba, uz prethodnu provjeru. Uočite da u ovom primjeru ne koristimo privremenu varijablu `newfd` — povratna vrijednost poziva `dup2` ispituje se izravno unutar `if` uvjeta, bez potrebe da je pamtimo. Isti pristup mogli smo upotrijebiti i u prethodnom primjeru.

```
$ ./dup2_redirect  
$ cat izlaz.txt  
Ovaj tekst završava u datoteci izlaz.txt!
```

3.6 Sistemski pozivi i funkcije C biblioteke

Kroz cijelo ovo poglavlje koristili smo isključivo sistemske pozive za sve ulazno-izlazne operacije. Vrijedi se osvrnuti na odnos između sistemskih poziva i funkcija C standardne biblioteke te razjasniti zašto je poznavanje sistemskih poziva temeljno znanje svakog UNIX programera — čak i kad se u svakodnevnom radu koriste funkcije više razine.

Cilj ove skripte jest upoznati budućeg programera s logikom UNIX-a: kako jezgra organizira procese, datoteke i komunikaciju između njih. Sistemski pozivi su sučelje između korisničkih programa i jezgre — sve što se događa na razini operacijskog sustava prolazi kroz njih. Razumijevanjem sistemskih poziva razumijemo što se zapravo događa kad program čita datoteku, ispisuje tekst na zaslon ili komunicira s drugim procesom. Upravo iz ovih razloga, u većini primjera u ovoj skripti koristimo sistemske pozive izravno.

Međutim, ovo ne znači da je takav pristup jedini ispravan, ili čak uopće smislen u svakodnevnom radu — upravo suprotno. Uzmimo za primjer ispis formatiranog teksta koji u sebi može sadržavati varijable i specijalne znakove (kontrolne sekvence kao npr. `\n`). Sistemski poziv `write` jednostavan je i izravan — upisuje točno određeni broj bajtova iz memorijskog međuspremnika u datoteku:

```
write(STDOUT_FILENO, "Pozdrav!!", 9);
```

Ako želimo ispisati, primjerice, vrijednost varijable zajedno s tekstom, `write` sam po sebi to ne podržava. Morali bismo ručno u memorijskom međuspremniku formatirati niz znakova u koji bismo ugradili vrijednost varijable. Pri tom bi bilo potrebno vrlo pažljivo analizirati samu varijablu i njenu vrijednost, npr. broj znamenaka, ili decimala kod brojeva. Dodajmo na to još formatiranje ispisa korištenjem kontrolnih sekvenci i potpuno je jasno koliko bi nam truda trebalo za formatiranje iole složenijeg ispisa.

Funkcija `printf` sve ovo za nas rješava u jednom pozivu:

```
printf("Rezultat: %d (hex: 0x%x)\n", vrijednost, vrijednost);
```

`printf` je složena funkcija koja pruža bogat skup mogućnosti formatiranja — decimalni, heksadecimalni i oktalni ispis brojeva, poravnanje, preciznost za decimalne vrijednosti, ispis nizova znakova i mnogo više. Implementacija iste funkcionalnosti korištenjem samog `write`-a zahtijevala bi niz poziva i bila bi nefleksibilna i podložna greškama.

S druge strane, `printf` i ostale funkcije C biblioteke u pozadini se oslanjaju upravo na `write` — one su samo praktičan, prenosiv sloj iznad sistemskog poziva. Pored toga, funkcije C biblioteke tipično koriste vlastiti međuspremnik (engl. *buffering*) kako bi smanjile broj skupih sistemskih poziva: umjesto poziva `write` za svaki znak, akumuliraju podatke i šalju ih u jednom bloku.

Sistemske pozivi i funkcije C biblioteke dakle nisu konkurenti — oni se nadopunjuju. Sistemske pozivi daju nam kontrolu i uvid u rad operacijskog sustava; funkcije biblioteke daju nam produktivnost i prenosivost. Iskusan UNIX programer zna kad koristiti koje.

3.7 Prevođenje

Direktorij dolazi s priloženim `Makefile`-om koji prati iste konvencije kao i `Makefile` u `osnove_programiranja/` (varijable `CC`, `CFLAGS`, `LDFLAGS`, `TARGETS`; implicitno pravilo `.c.o`; pravila `default`, `all`, `clean`). Detaljan opis strukture i korake gradnje `Makefilea` vidjeti u `../osnove_programiranja/README.md`.

Tipična uporaba:

```
make          # gradi zadani cilj (perm_mask)
make all      # gradi sve primjere
make f_strip  # gradi samo zadani primjer
make clean    # briše izvršne, objektne i generirane datoteke
```

Pravilo `clean` briše sve izvršne datoteke, objektne datoteke, privremene `*~` datoteke, `a.out`, kao i datoteke koje primjeri sami stvaraju pri pokretanju — `file.strip`, `file.hole`, `datoteka1`, `datoteka2` — kako bi se radni direktorij vratio u čisto stanje.

3.8 Bibliografija

- [1] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2013.
- [2] L. Budin, M. Golub, D. Jakobović, and L. Jelenković, *Operacijski sustavi*, 3. izd. Zagreb, Hrvatska: Element, 2013.
- [3] *Information technology — Programming languages — C*, ISO/IEC 9899:2018, International Organization for Standardization, Geneva, Switzerland, 2018.

Poglavlje 4

Upravljanje datotekama

U ovom poglavlju upoznat ćemo dublji UNIX pogled na datoteke: njihove **atribute**, kako jezgra organizira informacije o njima, te kako iz programa dohvatiti i mijenjati ta svojstva. Dok smo se u prethodnom poglavlju bavili **sadržajem** datoteka — čitanjem i pisanjem bajtova — ovdje stojimo “stepenicu više” i gledamo **metapodatke**: tip datoteke, vlasnika, prava pristupa, vremena pristupa, broj linkova, fizičku organizaciju na disku. Sve teme koje slijede klasično su pokrivene u *Advanced Programming in the UNIX Environment*, Stevens & Rago [1], i čitatelj koji želi šire i detaljnije obrade može tamo pronaći dodatne primjere i rasprave o specifičnim slučajevima.

4.1 Tipovi datoteka

UNIX-ova filozofija “sve je datoteka” znači da kroz datotečno sučelje pristupamo različitim vrstama resursa. Sustav podržava sljedeće tipove datoteka:

Tip	Opis
regularna datoteka	“obična” datoteka — niz bajtova: tekstualne datoteke, binarni programi, slike, arhive
direktorij	datoteka koja sadrži popis imena drugih datoteka i pripadnih i-node brojeva
simbolički link	datoteka koja sadrži putanju do druge datoteke (UNIX-ov “prečac”)
blok specijalna datoteka	uređaj kojemu se pristupa u blokovima fiksne veličine (npr. disk: /dev/sda)
karakter specijalna datoteka	uređaj kojemu se pristupa znak po znak (npr. terminal: /dev/tty)
FIFO (imenovani cjevovod)	jednosmjerni komunikacijski kanal između procesa, vidljiv u datotečnom sustavu
socket	krajnja točka za lokalnu ili mrežnu međuprocesnu komunikaciju

Bez obzira na tip, svim ovim resursima upravlja se istim malim skupom sistemskih poziva (open, read, write, close, lseek) — što je ujedno temelj UNIX-ove jednostavnosti i moći.

Posebno se zaustavljamo na **simboličkim linkovima** jer ćemo ih sresti kroz cijelo poglavlje. Simbolički link je u osnovi obična datoteka koja sadrži tekst — doslovno

putanju do druge datoteke. Pri tom datoteka na koju link pokazuje ne mora uopće postojati: link će svejedno čuvati string koji predstavlja tu putanju, čak i ako na toj putanji nema ničega. Ako se datoteka na koju link pokazuje stvori naknadno, link će na nju automatski pokazivati. Ovaj mehanizam temelji se isključivo na razrješavanju puta — jezgra svaki put kad pristupa linku iznova čita njegov sadržaj i koristi ga kao putanju.

4.2 Svojstva datoteka

Svaka datoteka u UNIX sustavu, neovisno o tipu, ima skup **atributa** (engl. *file attributes*) koje jezgra održava neovisno o samom sadržaju datoteke: tko je vlasnik, kakva su prava pristupa, kolika je veličina, kada je posljednji put mijenjana, koliko ima linkova... Iz programa, ovi se atributi dohvaćaju jednim od tri systemska poziva iz `stat` obitelji:

```
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>

int stat(const char *path, struct stat *buf);
int fstat(int fd, struct stat *buf);
int lstat(const char *path, struct stat *buf);
```

Povratna vrijednost (sve tri funkcije): 0 u slučaju uspjeha, -1 u slučaju greške.

Tri funkcije razlikuju se po načinu na koji identificiraju datoteku i po tome kako se ponašaju ako je riječ o simboličkom linku:

- **stat()** — datoteka se zadaje putanjom; ako je putanja simbolički link, funkcija ga slijedi i vraća attribute datoteke na koju link pokazuje (umjesto atributa za simbolički link — datoteku koja je navedena kao argument funkcije).
- **fstat()** — atributi otvorene datoteke; umjesto putanje kao argument se zadaje deskriptor na kojem je datoteka otvorena.
- **lstat()** — kao `stat()`, ali ako je putanja simbolički link, ne slijedi ga nego vraća attribute samog linka (njegovu veličinu, vrijeme stvaranja itd.).

U svim trima slučajevima, drugi argument je pokazivač na strukturu `struct stat` u koju jezgra upisuje attribute. Strukturu prethodno alocira pozivatelj (najčešće kao lokalnu varijablu na stogu).

4.2.1 Struktura `struct stat`

```
struct stat {
    mode_t    st_mode;    // tip datoteke i prava pristupa
    ino_t     st_ino;     // i-node broj
    dev_t     st_dev;     // identifikator uređaja na kojem datoteka leži
    dev_t     st_rdev;    // identifikator uređaja (samo za specijalne datoteke)
    nlink_t   st_nlink;   // broj hard linkova
    uid_t     st_uid;     // user ID vlasnika
    gid_t     st_gid;     // group ID vlasnika
    off_t     st_size;    // velicina datoteke u bajtovima
    time_t    st_atime;   // vrijeme zadnjeg pristupa sadržaju
    time_t    st_mtime;   // vrijeme zadnje izmjene sadržaja
    time_t    st_ctime;   // vrijeme zadnje promjene statusa (atributa)
    blksize_t st_blksize; // optimalna velicina I/O bloka
    blkcnt_t  st_blocks;  // broj alociranih blokova na disku
};
```

Pojašnjenje pojedinih polja:

- **st_mode** — sadrži dvije informacije u istom polju: tip datoteke (regularna, direktorij, link...) i prava pristupa (devet bitova `rw-rw-rw-` plus posebni bitovi). Tip se ne čita izravno iz brožčane vrijednosti, nego pomoću posebnih makro funkcija opisanih u nastavku.
- **st_ino** — broj i-node zapisa, jedinstvene strukture u kojoj jezgra čuva sve attribute datoteke. Dvije datoteke s istim i-node brojem na istom datotečnom sustavu su zapravo **ista datoteka** (vidi hard linkove).
- **st_dev** — identifikator uređaja (datotečnog sustava) na kojem datoteka leži. Par (**st_dev**, **st_ino**) jedinstveno identificira datoteku na cijelom sustavu.
- **st_rdev** — koristi se samo za blok i karakter specijalne datoteke (uređaje); u njemu su kodirani tzv. *major* i *minor* brojevi uređaja, što izlazi izvan onoga što nas u ovom poglavlju zanima.
- **st_nlink** — broj hard linkova koji pokazuju na ovaj i-node. Datoteka može imati više imena u datotečnom sustavu i sva su ravnopravna — niti jedno nije “izvorno” ili “primarno”, svaki je obični zapis u nekom direktoriju koji upućuje na isti i-node. Kad brojač padne na 0 (kad se obriše i zadnje ime), datoteka se fizički briše s diska.
- **st_uid**, **st_gid** — brožčani identifikatori vlasnika i grupe vlasnika. Tekstualna imena (npr. `dkrst`, `users`) dohvaćaju se naknadno iz sistemskih datoteka `/etc/passwd` i `/etc/group`.
- **st_size** — veličina sadržaja u bajtovima. Za regularne datoteke je to broj bajtova, za direktorije veličina ovisi o implementaciji, a za simboličke linkove — sjetimo se ranije napomene da je link u osnovi tekstualna datoteka — to je broj znakova u stringu putanje koji link sadrži.
- **st_atime** — vrijeme zadnjeg **pristupa sadržaju** (čitanja). Neke UNIX implementacije ne ažuriraju ovo polje pri svakom čitanju zbog performansi.
- **st_mtime** — vrijeme zadnje **izmjene sadržaja** (pisanja u datoteku).
- **st_ctime** — vrijeme zadnje promjene **statusa** datoteke: ne sadržaja, nego nekog atributa (prava pristupa, vlasništvo, broj linkova). Razlika prema `mtime` je suptilna ali važna: `chmod` mijenja `ctime` ali ne `mtime`.
- **st_blksize** — optimalna veličina jedinice I/O prijenosa za ovu datoteku; često 4096 B na suvremenim sustavima.
- **st_blocks** — broj 512-bajtnih blokova fizički alociranih datoteci. Kod *rijetkih* datoteka (“rupa”) može biti znatno manji od `st_size / 512`.

4.2.1.1 Brojač linkova kod direktorija

Bilo bi prirodno očekivati da svaki novostvoreni direktorij ima `st_nlink` jednak 1 — ipak je riječ o samo jednom imenu u nadređenom direktoriju. Ipak, već u trenutku stvaranja, brojač linkova svakog direktorija je 2. Razlog je u dva posebna unosa koja jezgra automatski stvara unutar svakog direktorija: `.` (link na sam taj direktorij) i `..` (link na nadređeni direktorij). Tako svaki direktorij na samom početku ima dva imena koja na njega upućuju — jedno u direktoriju u kojem se promatrani direktorij nalazi (njegovo “stvarno” ime) i jedno unutar njega samoga (`.`).

Štoviše, svaki put kad se unutar nekog direktorija stvori novi poddirektorij, brojač linkova raste još za jedan jer poddirektorij sa sobom donosi i `..` koji pokazuje natrag. Pogledajmo to izravno u ljusci:

```
$ mkdir test_dir
$ cd test_dir
$ ls -al
total 8
drwxr-xr-x  2 dkrst users 4096 May  5 15:41 .
```

```
drwxr-xr-x  5 dkrst users 4096 May  5 15:41 ..
```

Tek što je stvoren, `test_dir` ima `nlink = 2` (drugi stupac u retku za `.`). Sad stvorimo poddirektorij:

```
$ mkdir poddir
$ ls -al
total 12
drwxr-xr-x  3 dkrst users 4096 May  5 15:41 .
drwxr-xr-x  5 dkrst users 4096 May  5 15:41 ..
drwxr-xr-x  2 dkrst users 4096 May  5 15:41 poddir
```

Brojač linkova za `test_dir` (vidljiv u retku za `.`) sad je 3, jer mu je `poddir/..` dodao novu referencu. Sam `poddir` također kreće od 2. Općenito vrijedi da brojač linkova direktorija jednak je `2 + broj poddirektorija` — što ujedno objašnjava zašto naredba `find -type d -links 2` (ili slične formulacije) može poslužiti za pronalazak direktorija koji nemaju poddirektorija.

4.2.2 Otkrivanje tipa datoteke

Tip datoteke kodiran je u `st_mode` polju, ali ne kao broj — provjera se radi pomoću makro funkcija definiranih u `<sys/stat.h>`:

Makro	Vraća true ako je datoteka...
<code>S_ISREG(m)</code>	regularna
<code>S_ISDIR(m)</code>	direktorij
<code>S_ISLNK(m)</code>	simbolički link
<code>S_ISBLK(m)</code>	blok specijalna
<code>S_ISCHR(m)</code>	karakter specijalna
<code>S_ISFIFO(m)</code>	FIFO (imenovani cjevovod)
<code>S_ISSOCK(m)</code>	socket

Tipičan obrazac uporabe: `if (S_ISREG(st.st_mode)) { ... }`.

4.2.3 Ispis atributa datoteke

- **fileinfo.c** — prima ime datoteke kao argument naredbenog retka i ispisuje njene osnovne attribute pozivom `stat()`.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <time.h>

int main(int argc, char *argv[]) {
    struct stat st;

    if (argc != 2) {
        printf("koristenje: %s <ime_datoteke>\n", argv[0]);
        return 1;
    }

    if (stat(argv[1], &st) < 0) {
        perror("stat");
        return 1;
    }

    printf("Datoteka: %s\n", argv[1]);
    printf(" tip:          ");
    if (S_ISREG(st.st_mode)) printf("regularna datoteka\n");
```

```

else if (S_ISDIR(st.st_mode)) printf("direktorij\n");
else if (S_ISLNK(st.st_mode)) printf("simbolički link\n");
else if (S_ISCHR(st.st_mode)) printf("karakter specijalna\n");
else if (S_ISBLK(st.st_mode)) printf("blok specijalna\n");
else if (S_ISFIFO(st.st_mode)) printf("FIFO\n");
else if (S_ISSOCK(st.st_mode)) printf("socket\n");

printf(" i-node broj:   %ld\n", (long)st.st_ino);
printf(" prava:        %o\n", st.st_mode & 0777);
printf(" vlasnik UID:   %d\n", st.st_uid);
printf(" grupa GID:     %d\n", st.st_gid);
printf(" velicina:      %ld B\n", (long)st.st_size);
printf(" broj linkova:   %ld\n", (long)st.st_nlink);
printf(" zadnji pristup: %s", ctime(&st.st_atime));
printf(" zadnja izmjena: %s", ctime(&st.st_mtime));
printf(" zadnja promjena: %s", ctime(&st.st_ctime));

return 0;
}

```

Konverzija `%o` ispisuje prava pristupa u oktalnom obliku (zajednički prikaz prava na UNIX sustavima — npr. 644 za `rw-r--r--`, 755 za `rw-r-xr-x`). Funkcija `ctime()` iz `<time.h>` pretvara `time_t` (sekunde od epohe, 1.1.1970) u čitljivi datum/vrijeme. Eksplicitne pretvorbe `((long)st.st_ino)` nužne su jer su `ino_t`, `off_t` i drugi tipovi različiti na različitim sustavima — bez pretvorbe `printf` može ispisati pogrešne brojeve.

Pokretanje:

```

$ ./fileinfo fileinfo.c
Datoteka: fileinfo.c
tip:          regularna datoteka
i-node broj:  165
prava:        644
vlasnik UID:  1000
grupa GID:    1000
velicina:     1300 B
broj linkova: 1
zadnji pristup: Tue May 5 14:45:04 2026
zadnja izmjena: Tue May 5 14:44:43 2026
zadnja promjena: Tue May 5 14:44:43 2026

```

Pokušajte i s drugim vrstama datoteka:

```

$ ./fileinfo /etc          # direktorij
$ ./fileinfo /dev/null     # karakter specijalna
$ ./fileinfo /dev/sda      # blok specijalna (ako postoji)

```

4.2.3.1 Ponašanje na simboličkom linku

Stvorimo iz ljuske simbolički link `drugoime.c` koji pokazuje na `fileinfo.c`:

```

$ ln -s fileinfo.c drugoime.c
$ ls -l fileinfo.c drugoime.c
lrwxrwxrwx 1 dkrtst users 10 May 5 16:14 drugoime.c -> fileinfo.c
-rw-r--r-- 1 dkrtst users 1300 May 5 15:22 fileinfo.c

```

Iz ispisa `ls -l` vidimo da je `drugoime.c` zaista simbolički link (početno slovo `l` u prvom stupcu) koji pokazuje na `fileinfo.c`. Sad nad njim pokrenimo naš program:

```

$ ./fileinfo drugoime.c
Datoteka: drugoime.c
tip:          regularna datoteka
i-node broj:  63
prava:        644
vlasnik UID:  1000
grupa GID:    1000

```

```

velicina:      1300 B
broj linkova:  1
zadnji pristup: Tue May  5 16:14:39 2026
zadnja izmjena: Tue May  5 15:22:01 2026
zadnja promjena: Tue May  5 16:14:38 2026

```

Iako smo u argumentu zadali `drugoime.c`, ispisani atributi pripadaju datoteci `fileinfo.c` — što vidimo po veličini od 1300 B (link sam ima samo 10 znakova) i tipu “regularna datoteka”. To je posljedica činjenice da `stat()` **slijedi simbolički link** i vraća attribute datoteke na koju link pokazuje, ne samog linka.

4.2.4 Razlika `stat` i `lstat`

Kako vidjeti attribute samog linka, a ne datoteke na koju on pokazuje? Tu na scenu stupa funkcija `lstat()`. Da bismo ilustrirali razliku, modificirajmo `fileinfo.c` tako da koristi `lstat` umjesto `stat` — sve ostalo neka ostane isto.

- **`fileinfo2.c`** — identičan programu `fileinfo.c`, jedina razlika je u tome što umjesto `stat()` koristi `lstat()`.

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <time.h>

int main(int argc, char *argv[]) {
    struct stat st;

    if (argc != 2) {
        printf("koristenje: %s <ime_datoteke>\n", argv[0]);
        return 1;
    }

    if (lstat(argv[1], &st) < 0) {
        perror("lstat");
        return 1;
    }

    printf("Datoteka: %s\n", argv[1]);
    printf(" tip: ");
    if (S_ISREG(st.st_mode)) printf("regularna datoteka\n");
    else if (S_ISDIR(st.st_mode)) printf("direktorij\n");
    else if (S_ISLNK(st.st_mode)) printf("simbolički link\n");
    else if (S_ISCHR(st.st_mode)) printf("karakter specijalna\n");
    else if (S_ISBLK(st.st_mode)) printf("blok specijalna\n");
    else if (S_ISFIFO(st.st_mode)) printf("FIFO\n");
    else if (S_ISSOCK(st.st_mode)) printf("socket\n");

    printf(" i-node broj:   %ld\n", (long)st.st_ino);
    printf(" prava:          %o\n", st.st_mode & 0777);
    printf(" vlasnik UID:    %d\n", st.st_uid);
    printf(" grupa GID:     %d\n", st.st_gid);
    printf(" velicina:      %ld B\n", (long)st.st_size);
    printf(" broj linkova:  %ld\n", (long)st.st_nlink);
    printf(" zadnji pristup: %s", ctime(&st.st_atime));
    printf(" zadnja izmjena: %s", ctime(&st.st_mtime));
    printf(" zadnja promjena: %s", ctime(&st.st_ctime));

    return 0;
}

```

Usporedimo izlaz oba programa nad našim simboličkim linkom `drugoime.c`:

```

$ ./fileinfo2.c drugoime.c
Datoteka: drugoime.c
 tip:      regularna datoteka
i-node broj: 63

```

```

prava:          644
vlasnik UID:    1000
grupa GID:      1000
velicina:       1300 B
broj linkova:   1
zadnji pristup: Tue May  5 16:14:39 2026
zadnja izmjena: Tue May  5 15:22:01 2026
zadnja promjena: Tue May  5 16:14:38 2026

$ ./fileinfo2 drugoime.c
Datoteka: drugoime.c
tip:           simbolicki link
i-node broj:   81
prava:         777
vlasnik UID:   1000
grupa GID:     1000
velicina:      10 B
broj linkova:  1
zadnji pristup: Tue May  5 16:14:39 2026
zadnja izmjena: Tue May  5 16:14:39 2026
zadnja promjena: Tue May  5 16:14:39 2026

```

Razlika je sad očita. `fileinfo` (koristi `stat`) slijedi simbolički link i pokazuje attribute datoteke `fileinfo.c` — i-node 63, veličina 1300 B, tip “regularna datoteka”. `fileinfo2` (koristi `lstat`) ostaje na samom linku — i-node 81, veličina 10 B (koliko je dugačak string “`fileinfo.c`”), tip “simbolicki link”.

Koju ćemo funkciju koristiti, `stat` ili `lstat`, ovisi o tome želimo li dobiti informaciju da je određena datoteka simbolički link te njezine attribute, ili informaciju o datoteci na koju simbolički link pokazuje. Odabir ovisi o namjeni našeg programa.

Međutim, potrebno je voditi računa o jednoj posebnoj situaciji koja se javlja kod programa koji pretražuju datotečni sustav (npr. `find` ili rekurzivni obilazak stabla direktorija). Ako u nekom direktoriju postoji simbolički link koji pokazuje natrag na taj isti direktorij — ili na bilo kojeg pretka u stablu — `stat()` će takav link slijediti i naš će program ponovno “ući” u već obišteni direktorij. Pri svakom novom ulasku link je opet tu, pa program ponovno ulazi, i tako u nedogled. Da bismo ovakvu beskonačnu petlju izbjegli, prilikom pretraživanja datotečnog sustava treba koristiti `lstat()` — koji će za simbolički link to upravo i prijaviti, pa naš program može odlučiti da ga ne slijedi.

4.3 Korisnici, grupe i prava pristupa

UNIX je višekorisnički sustav, što znači da na istom računalu istovremeno može raditi više korisnika. Svakom korisniku pridružen je jedinstveni cjelobrojni **UID** (*User ID*), a svaki korisnik pripada jednoj ili više **grupa**, od kojih svaka ima svoj **GID** (*Group ID*). Mapiranje imena u brojeve nalazi se u sistemskim datotekama `/etc/passwd` (korisnici) i `/etc/group` (grupe).

Kao što smo upoznali u prvom poglavlju, prava pristupa svakoj datoteci dijele se u tri razine — vlasnik (*user*), grupa (*group*), ostali (*others*) — i tri tipa prava: čitanje (*r*), pisanje (*w*), izvršavanje (*x*). Sveukupno **devet bitova** koji se zapisuju u `st_mode` polje strukture `stat`.

4.3.1 Procesi i njihovo vlasništvo

Vlasništvo nije samo svojstvo datoteka — nego i procesa. Uostalom, već smo naučili da je na UNIX-u sve datoteka. Vidjeli smo u prvom poglavlju da standardni izlaz

jednog procesa možemo preusmjeriti na standardni ulaz drugog i tako efektivno “pisati” u proces. Stoga je sasvim prirodno da i procesi imaju svog vlasnika i grupu kojoj pripadaju.

Svaki proces u sustavu ima pridruženu skupinu identifikatora koji određuju “u čije ime” radi. Najvažnija dva su:

- **stvarni** (*real*) **UID i GID** — tko je doslovno pokrenuo proces; preuzima se od ljuske u kojoj je naredba upisana.
- **efektivni** (*effective*) **UID i GID** — koje ovlasti proces zapravo ima u trenutku provjere.

Stvarni i efektivni vlasnik, kao i stvarno i efektivno grupno vlasništvo, u najvećem se broju slučajeva ne razlikuju. Međutim, postoje određene situacije u kojima se efektivno i stvarno vlasništvo mogu razlikovati. Važno je zapamtiti da jezgra prilikom pristupa datotekama uvijek provjerava prava **efektivnog** vlasnika i efektivne grupe koji su vlasnici procesa.

Logika pridruživanja vlasnika novom procesu prirodno slijedi iz toga kako razmišljamo o korištenju alata općenito: kada koristimo neki alat, sve posljedice — dobre i loše — našeg rada s tim alatom idu nama, nikako ne proizvođaču alata. **Stvarni vlasnik procesa u memoriji stoga je uvijek onaj tko je proces pokrenuo**, ne onaj tko je napisao izvršnu datoteku. Efektivni vlasnik je gotovo uvijek također onaj tko je proces pokrenuo, ali postoje specifične situacije u kojima efektivni vlasnik (i/ili efektivna grupa) može biti onaj korisnik sustava koji je vlasnik izvršne datoteke. Često je to root (superuser) koji ima neograničena prava na sustavu.

Klasičan primjer je promjena lozinke. Kad korisnik utipka naredbu `passwd`, ona mora upisati novu kriptiranu lozinku u datoteku `/etc/shadow`. Iz očiglednih sigurnosnih razloga, čitanje i pisanje u datoteku `/etc/shadow` dozvoljeno je samo root-u — niti jedan običan korisnik ne smije ni pročitati tuđe lozinke ni napisati novu:

```
$ ls -l /etc/shadow
-rw-r----- 1 root shadow 609 Apr 18 18:13 /etc/shadow
```

Ipak, svaki korisnik mora moći promijeniti **vlastitu** lozinku. Rješenje je **set-UID bit** — poseban bit u pravima izvršne datoteke koji jezgri kaže: “*kad se ovaj program pokrene, postavi efektivni UID procesa na UID vlasnika izvršne datoteke, ne na UID korisnika koji ga je pokrenuo*”. Set-UID bit u ispisu naredbe `ls -l` zauzima mjesto izvršnog bita za vlasnika i prikazuje se slovom `s`:

```
$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 64152 May 30 2024 /usr/bin/passwd
```

Datoteka `/usr/bin/passwd` u vlasništvu je root-a (četvrti i peti stupac) i ima postavljen set-UID bit (`s` umjesto `x` u korisničkim pravima). Posljedica je da se svaki put kad ju običan korisnik pokrene proces izvršava s **efektivnim UID-om root-a** (i može mijenjati `/etc/shadow`), dok **stvarni UID ostaje korisnikov** (pa program zna tko ga je pokrenuo i može mu mijenjati samo njegovu lozinku).

Set-UID bit moguće je postaviti i na izvršne datoteke koje sami napišemo — pomoću naredbe `chmod` (s kojom ćemo se pobliže upoznati malo kasnije u poglavlju). Pri tome treba biti **iznimno oprezan**: svaki nedostatak u programu s postavljenim set-UID bitom potencijalno se može iskoristiti tako da napadač izvrši proizvoljan kod s povišenim ovlastima. Zato se set-UID koristi samo na pažljivo provjerenim,

minimalističkim alatima poput `passwd`-a, a u svakodnevnom programiranju se gotovo nikad ne koristi.

Kako će programi koji rade s povišenim ovlastima provjeriti smije li *stvarni* korisnik (onaj koji je proces pokrenuo) zaista raditi neku akciju? Tu na scenu stupa sistemski poziv `access`, kojeg upoznajemo u sljedećoj sekciji.

4.3.2 Provjera prava pristupa — `access()`

Za testiranje da li trenutni proces ima određena prava nad datotekom (čitanje, pisanje, izvršavanje, ili samo provjera postojanja), POSIX nudi sistemski poziv `access`.

```
#include <unistd.h>

int access(const char *pathname, int mode);
```

Povratna vrijednost: 0 ako su sva tražena prava ostvarena (ili datoteka postoji u slučaju `F_OK`); -1 ako barem jedno pravo nije dopušteno ili je došlo do greške.

Argumenti:

- **pathname** — putanja do datoteke koja se provjerava.
- **mode** — kombinacija (bitovni OR) jedne ili više konstanti:
 - `R_OK` — provjeri pravo čitanja,
 - `W_OK` — provjeri pravo pisanja,
 - `X_OK` — provjeri pravo izvršavanja,
 - `F_OK` — provjeri samo postojanje datoteke (zanemaruje prava).

Bitno je razumjeti da `access` provjerava prava pomoću **stvarnog** UID-a i GID-a procesa, ne efektivnog. Razlika postaje važna kod programa s postavljenim set-UID bitom (npr. `passwd`): `access` će vratiti odgovor kakav bi imao stvarni korisnik, ne onaj kojeg je program privremeno preuzeo. Ovo je namjerno ponašanje — daje set-UID programima način da provjere “smije li to korisnik koji me je pokrenuo zaista raditi”.

- **provjeri.c** — prima ime datoteke kao argument i ispisuje koja prava (čitanje, pisanje, izvršavanje) trenutni korisnik nad njom ima.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("koristenje: %s <ime_datoteke>\n", argv[0]);
        return 1;
    }

    if (access(argv[1], F_OK) < 0) {
        printf("Datoteka '%s' ne postoji.\n", argv[1]);
        return 1;
    }

    printf("Prava korisnika nad datotekom '%s':\n", argv[1]);
    printf("  citanje:      %s\n", access(argv[1], R_OK) == 0 ? "DA" : "NE");
    printf("  pisanje:      %s\n", access(argv[1], W_OK) == 0 ? "DA" : "NE");
    printf("  izvrsavanje: %s\n", access(argv[1], X_OK) == 0 ? "DA" : "NE");

    return 0;
}
```

Program prvo pozivom `access(argv[1], F_OK)` provjerava postoji li datoteka uopće — to je preporučena praksa jer `access` s drugim zastavicama ne razlikuje “datoteka ne postoji” od “datoteka postoji ali nemam prava”. Ako datoteka postoji, slijede tri zasebne provjere za svako od tri prava.

Pokretanje (kao običan korisnik):

```
$ ./provjeri /etc/passwd
Prava korisnika nad datotekom '/etc/passwd':
  citanje:    DA
  pisanje:    NE
  izvršavanje: NE

$ ./provjeri /etc/shadow
Prava korisnika nad datotekom '/etc/shadow':
  citanje:    NE
  pisanje:    NE
  izvršavanje: NE

$ ./provjeri ./provjeri
Prava korisnika nad datotekom './provjeri':
  citanje:    DA
  pisanje:    DA
  izvršavanje: DA

$ ./provjeri nepostoji.txt
Datoteka 'nepostoji.txt' ne postoji.
```

Iz primjera je vidljivo da običan korisnik može čitati `/etc/passwd` (koji sadrži korisnička imena i UID-ove), ali ne i `/etc/shadow` (koji sadrži kriptirane lozinke i kojem pristup ima samo root).

4.3.3 Promjena prava pristupa — `chmod()` i `fchmod()`

```
#include <sys/stat.h>

int chmod(const char *path, mode_t mode);
int fchmod(int fd, mode_t mode);
```

Povratna vrijednost (obje funkcije): 0 u slučaju uspjeha, -1 u slučaju greške.

Argumenti:

- **path** — putanja do datoteke (`chmod`).
- **fd** — otvoreni file deskriptor (`fchmod`).
- **mode** — nova prava pristupa, zadana kao bitovni OR konstanti tipa `mode_t` (`S_IRUSR` | `S_IWUSR` | ...) ili kao oktalni broj (0644, 0755).

Da bi proces smio mijenjati prava pristupa neke datoteke, mora biti zadovoljen jedan od dva uvjeta: efektivni UID procesa mora biti jednak UID-u vlasnika datoteke, ili proces mora biti pokrenut kao root.

- **prava.c** — jednostavna inačica naredbe `chmod` iz ljuske: prima oktalni zapis prava pristupa i ime datoteke, te postavlja zadana prava.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>

int main(int argc, char *argv[]) {
    mode_t mode;

    if (argc != 3) {
        printf("koristenje: %s <oktalna_prava> <ime_datoteke>\n", argv[0]);
```

```

    printf("primjer: %s 644 dat.txt\n", argv[0]);
    return 1;
}

/* strtol s bazom 8 - korisnik zadaje oktalni zapis poput 644 */
mode = (mode_t)strtol(argv[1], NULL, 8);

if (chmod(argv[2], mode) < 0) {
    perror("chmod");
    return 1;
}

printf("Prava datoteke '%s' postavljena na %o.\n", argv[2], mode);
return 0;
}

```

Funkcija `strtol` iz standardne C biblioteke pretvara string u cijeli broj; treći argument (8) određuje brojčanu bazu — za oktalne brojeve to je 8. (Mogli bismo koristiti i `strtol(..., 0)` koji automatski prepoznaje bazu prema prefiksu: 0 za oktalni, 0x za heksadekadski, ali smo se odlučili za eksplicitnu bazu radi jasnoće.)

Ekvivalent u ljusci je naredba `chmod` koja prihvaća iste oktalne brojeve:

```

$ ls -l dat.txt
-rw-r--r-- 1 dkrst users 5 May  5 15:00 dat.txt

$ ./prava 600 dat.txt
Prava datoteke 'dat.txt' postavljena na 600.

$ ls -l dat.txt
-rw----- 1 dkrst users 5 May  5 15:00 dat.txt

$ chmod 644 dat.txt          # ekvivalentno ./prava 644 dat.txt
$ ls -l dat.txt
-rw-r--r-- 1 dkrst users 5 May  5 15:00 dat.txt

```

4.3.4 Promjena vlasništva — `chown()`, `fchown()`, `lchown()`

```

#include <unistd.h>

int chown(const char *path, uid_t owner, gid_t group);
int fchown(int fd, uid_t owner, gid_t group);
int lchown(const char *path, uid_t owner, gid_t group);

```

Povratna vrijednost (sve tri funkcije): 0 u slučaju uspjeha, -1 u slučaju greške.

Argumenti:

- **path** — putanja do datoteke (`chown`, `lchown`).
- **fd** — otvoreni file deskriptor (`fchown`).
- **owner** — novi UID vlasnika; ako se zadaje -1, vlasnik se ne mijenja.
- **group** — novi GID grupe; ako se zadaje -1, grupa se ne mijenja.

Razlika između `chown` i `lchown` analogna je razlici između `stat` i `lstat`: `chown` slijedi simbolički link i mijenja vlasništvo ciljane datoteke, dok `lchown` mijenja vlasništvo samog linka.

Bitna napomena: na većini suvremenih UNIX sustava, **promjena vlasništva ograničena je na korisnika root**. Običan korisnik **ne može** “preuzeti” tuđu datoteku (jer bi to predstavljao sigurnosni rizik), niti svoju datoteku “predati” drugome (jer bi mogućnost da nekome “uvalimo” kukavičje jaje također predstavljala sigurnosni rizik, ali i omogućilo zaobilaženje kvota i drugih ograničenja). U praksi to znači da običan korisnik ove systemske pozive uglavnom

ne koristi izravno; oni su prvenstveno alat administratora. Ekvivalent u ljusci je naredba `chown`. Pošto promjena vlasništva traži root ovlasti, naredba se najčešće poziva uz `sudo` — pomoćnu naredbu koja omogućuje da se ostatak naredbe izvrši s ovlastima root korisnika, pod uvjetom da korisnik ima takvo pravo (tipično definirano u datoteci `/etc/sudoers`):

```
$ sudo chown dkrst:users dat.txt # dodjeljuje datoteku korisniku dkrst, grupi users
```

4.4 Linkovi

UNIX podržava dva različita načina da jedna te ista datoteka bude dostupna pod više imena: **čvrste linkove** (engl. *hard link*) i **simboličke linkove** (engl. *symbolic link* ili *soft link*). Iako oba mehanizma pružaju “alias” za neku datoteku, oni rade na suštinski različite načine.

4.4.1 Čvrsti linkovi

Promislimo na trenutak o ulozi direktorija u datotečnom sustavu. Najjednostavnije ih možemo zamisliti kao registre neke velike arhive u kojima piše gdje se koji podatak nalazi: “*ako tražiš tu i tu mapu, pogledaj na trećoj polici lijevo, druga kutija od kraja*”. Nema nikakvog razloga da ista informacija o tome gdje se neki podatak nalazi ne piše u više različitih registara. U svakom od tih registara podatak je možda drugačije zaveden (recimo, računovodstvo fakulteta možda sistematizira studente na jedan, a referada na drugi način). Međutim, u svakom od njih zabilježena je ista lokacija mape na polici, i bez obzira u kojem registru tražimo, doći ćemo do istih podataka.

Na sličan način, iako datoteku čini jedan jedinstveni sadržaj pohranjen na disku, ne postoji nikakav razlog da informaciju o tome gdje se taj sadržaj nalazi ne pohranimo u više direktorija, pod istim ili različitim imenima. Bez obzira koje ime koristimo, sva ona u konačnici pokazuju na iste fizičke podatke na disku.

Tehnički, **čvrsti link** je upravo to: dodatni zapis u nekom direktoriju koji povezuje neko ime s istim i-node brojem kao već postojeća datoteka. Sjetimo se da u UNIX-u sve attribute datoteke (tip, prava, veličinu, vremena, lokaciju podataka na disku) jezgra čuva u i-node strukturi, a ime datoteke u nekom direktoriju je samo zapis koji to ime povezuje s odgovarajućim i-node brojem. Stvaranjem čvrstog linka jednostavno se dodaje novi takav zapis koji pokazuje na isti i-node — pa s gledišta jezgre, sva imena su jednako vrijedne reference na istu datoteku.

Iz ovog mehanizma slijedi nekoliko važnih svojstava čvrstih linkova:

- **Svi (čvrsti) linkovi pokazuju na isti i-node**, dijele identičan sadržaj i attribute. Promjena sadržaja kroz jedno ime vidljiva je odmah kroz drugo.
- **Svi (čvrsti) linkovi moraju biti na istom datotečnom sustavu** (istoj particiji), jer i-node brojevi vrijede samo unutar pojedinog datotečnog sustava.
- **Brisanjem nekog imena samo se uklanja njegov zapis u direktoriju** i smanjuje brojač linkova (`st_nlink`) na i-nodeu. Sama datoteka briše se s diska tek kad brojač padne na 0 — odnosno kad je obrisano i zadnje ime, jer više ništa ne “pokazuje” na te podatke.
- **Izvorno ime nije ničim “jače” od kasnije stvorenih linkova**. Brisanje izvornog imena je jednako brisanju bilo kojeg drugog linka.
- **Samo root smije stvarati čvrste linkove na direktorij**. Stvaranje takvog linka kreiralo bi cikluse u stablu direktorija — primjerice, link `/foo/bar` koji pokazuje

natrag na /foo značio bi da iz direktorija /foo možemo unedogled “spuštati” se kroz bar/bar/bar/... i uvijek smo u istom direktoriju. Alati poput find ili du koji rekurzivno obilaze stablo zaglavili bi u beskonačnoj petlji. Slično vrijedi i za posebne unose . i .. koji su u biti čvrsti linkovi, ali njih jezgra automatski stvara i njima upravlja sama.

```
#include <unistd.h>

int link(const char *oldpath, const char *newpath);
int unlink(const char *pathname);
```

Povratna vrijednost (obje): 0 u slučaju uspjeha, -1 u slučaju greške.

link() stvara novo ime (newpath) za već postojeću datoteku (oldpath) — povećava brojč linkova na i-nodeu i atomski dodaje novi zapis u direktorij. unlink() radi obrnuto: uklanja zapis iz direktorija i smanjuje brojč. Ako brojč padne na 0 i nema otvorenih deskriptora na datoteku, jezgra fizički briše datoteku s diska.

Obje funkcije imaju izravne ekvivalente u ljsuci: link() odgovara naredbi ln, a unlink() naredbi unlink ili — što je u praksi puno češće — naredbi rm. Naredba rm nudi sve što i unlink, ali i puno više: brisanje više datoteka odjednom, rekurzivno brisanje s opcijom -r, prisilno brisanje s -f. Ipak, s gledišta sustava, one rade istu temeljnu operaciju — uklanjaju zapis iz direktorija i smanjuju brojč linkova.

- **makelink.c** — stvara čvrsti link između dvije zadane putanje.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("koristenje: %s <postojeca_datoteka> <novi_link>\n", argv[0]);
        return 1;
    }

    if (link(argv[1], argv[2]) < 0) {
        perror("link");
        return 1;
    }

    printf("Stvoren hard link '%s' -> '%s'.\n", argv[2], argv[1]);
    return 0;
}
```

Pokrenimo cjeloviti scenarij koji ilustrira sva spomenuta svojstva. Stvorit ćemo datoteku, dodati joj dva čvrsta linka, pa pratiti brojč linkova dok ne dođe do nule:

```
$ echo "vrijedan sadrzaj" > orig.txt
$ ls -li orig.txt
1002 -rw-r--r-- 1 dkrst users 17 May  5 15:00 orig.txt

$ ./makelink orig.txt link1.txt
Stvoren hard link 'link1.txt' -> 'orig.txt'.
$ ln orig.txt link2.txt

$ ls -li orig.txt link1.txt link2.txt
1002 -rw-r--r-- 3 dkrst users 17 May  5 15:00 link1.txt
1002 -rw-r--r-- 3 dkrst users 17 May  5 15:00 link2.txt
1002 -rw-r--r-- 3 dkrst users 17 May  5 15:00 orig.txt
```

Drugi link je stvoren naredbom ln u ljsuci — ekvivalentno pozivu ./makelink orig.txt link2.txt.

Sva tri imena imaju **isti i-node 1002** (prvi stupac, koji se ispisiuje opcijom -i) i

brojač linkova jednak 3 (četvrti stupac). Sad obrišimo izvorno ime:

```
$ rm orig.txt
$ ls -li link1.txt link2.txt
1002 -rw-r--r-- 2 dkrst users 17 May  5 15:00 link1.txt
1002 -rw-r--r-- 2 dkrst users 17 May  5 15:00 link2.txt

$ cat link1.txt
vrijedan sadrzaj
```

Brojač je pao na 2, ali i-node, sadržaj i ostali atributi nepromijenjeni su — datoteka još uvijek živi pod druga dva imena. Brisanje “izvornog” imena nije imalo nikakve posebne posljedice. Pokušajmo dalje:

```
$ rm link1.txt
$ ls -li link2.txt
1002 -rw-r--r-- 1 dkrst users 17 May  5 15:00 link2.txt

$ cat link2.txt
vrijedan sadrzaj

$ rm link2.txt
$ cat link2.txt
cat: link2.txt: No such file or directory
```

Tek kad je brisano i zadnje ime — i brojač pao na 0 — jezgra je fizički obrisala datoteku s diska.

4.4.2 Simbolički linkovi

Simbolički link je datoteka koja sadrži tekst — putanju do druge datoteke. Ono što razlikuje simbolički link od bilo koje druge tekstualne datoteke jest njegov **tip**, zapisan u atributima datoteke (točnije, u `st_mode` polju i-noda). Po tom tipu jezgra zna da sadržaj ne treba čitati kao obične podatke — nego da tekst u datoteci treba interpretirati kao putanju do druge datoteke i operaciju nastaviti nad njom. Tako pri svakom otvaranju, čitanju ili pisanju kroz simbolički link, jezgra čita njegov sadržaj i operaciju izvršava nad datotekom čije je ime ondje pročitano.

Simbolički linkovi pružaju funkcionalnost sličnu čvrstim linkovima, ali rade na potpuno drugačiji način, što ima nekoliko važnih posljedica:

- **Simbolički linkovi mogu prelaziti granice datotečnih sustava.** Sadržaj je samo tekst (putanja), tako da nema problema kao kod čvrstih linkova.
- **Mogu pokazivati na nepostojeće datoteke** — u trenutku stvaranja jezgra ne provjerava da li ciljna datoteka postoji. Ako ciljna datoteka nikad nije stvorena, ili je obrisana, link postaje “razbijen” (engl. *dangling*).
- **Mogu pokazivati na direktorije** — bez ikakvih ograničenja koja vrijede za čvrste linkove na direktorije.
- **Brisanjem ciljne datoteke linkovi ostaju na mjestu**, ali postaju razbijeni. Obrnuto: brisanjem linka ciljnoj datoteci se ništa ne događa.
- **Imaju vlastite attribute** (vlasnika, vremena) odvojene od ciljne datoteke. Veličina linka je broj bajtova u stringu putanje koji sadrži.

```
#include <unistd.h>
```

```
int symlink(const char *target, const char *linkpath);
ssize_t readlink(const char *pathname, char *buf, size_t bufsize);
```

symlink() stvara simbolički link na putanji `linkpath` koji “pokazuje” na `target`. Vraća 0 u slučaju uspjeha, -1 u slučaju greške.

readlink() čita sadržaj simboličkog linka — odnosno **tekst (putanju) zapisan u datoteci tipa simbolički link** — i smješta ga u zadani međuspremnik. Vraća broj stvarno pročitanih bajtova, ili -1 u slučaju greške. Bitna posebnost: **readlink ne dodaje null-terminator** na kraj — pozivatelj ga mora dodati ručno.

- **makesymlink.c** — stvara simbolički link na zadanoj putanji koji “pokazuje” na zadanu metu.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("koristenje: %s <postojeca_datoteka> <novi_link>\n", argv[0]);
        return 1;
    }

    if (symlink(argv[1], argv[2]) < 0) {
        perror("symlink");
        return 1;
    }

    printf("Stvoren simbolicki link '%s' -> '%s'.\n", argv[2], argv[1]);
    return 0;
}
```

Pokretanje:

```
$ echo "ovo je sadrzaj" > orig.txt
$ ./makesymlink orig.txt sym.txt
Stvoren simbolicki link 'sym.txt' -> 'orig.txt'.

$ ls -li orig.txt sym.txt
1003 -rw-r--r-- 1 dkrst users 15 May  5 15:00 orig.txt
1004 lrwxrwxrwx 1 dkrst users  8 May  5 15:00 sym.txt -> orig.txt

$ cat sym.txt
ovo je sadrzaj
```

Bitno: i-node simboličkog linka (1004) različit je od i-noda ciljne datoteke (1003); brojač linkova je 1 na svakom (nisu hard linkovi); tip prvog je - (regularna), drugog l (link); veličina linka je 8 bajtova (duljina stringa "orig.txt"). Pokušajmo obrisati izvornu datoteku:

```
$ rm orig.txt
$ ls -l sym.txt
lrwxrwxrwx 1 dkrst users 8 May  5 15:00 sym.txt -> orig.txt
$ cat sym.txt
cat: sym.txt: No such file or directory
```

Link je sam ostao netaknut — ali ciljna datoteka više ne postoji, pa je link sad razbijen. Moderne UNIX ljuste razbijene (*dangling*) simboličke linkove često prikazuju drugom bojom kako bi odmah bilo vidljivo da putanja u linku pokazuje na nepostojeću datoteku.

Ekvivalent u ljusti je opet naredba **ln**, ovaj put s opcijom **-s**:

```
$ ln -s orig.txt sym.txt
```

Naredba **ln** ima i niz drugih opcija — ovdje smo upoznali samo dvije najčešće (osnovni oblik za hard linkove i **-s** za simboličke). Za potpunu uputu i popis svih opcija, kao i kod svih ostalih UNIX naredbi, čitatelj se upućuje na priručnik koji se otvara naredbom **man ln**.

- **readsymlink.c** — čita sadržaj simboličkog linka, odnosno doslovni tekst (putanju) koji je u njemu zapisan, pomoću funkcije **readlink**.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    char buf[256];
    ssize_t n;

    if (argc != 2) {
        printf("koristenje: %s <simbolicki_link>\n", argv[0]);
        return 1;
    }

    /* readlink ne dodaje null terminator, pa ga moramo sami dodati */
    n = readlink(argv[1], buf, sizeof(buf) - 1);
    if (n < 0) {
        perror("readlink");
        return 1;
    }
    buf[n] = '\0';

    printf("Sadržaj linka '%s': \"%s\"\n", argv[1], buf);
    return 0;
}

```

Da bismo program isprobali, stvorimo jednu datoteku u poddirektoriju (kako bi imala nešto složeniju putanju), pa na nju usmjerimo dva simbolička linka pisana s različito formuliranom putanjom — jednog napravimo pozivom `makesymlink`-a, a drugog izravno iz ljuske naredbom `ln -s`:

```

$ mkdir -p dokumenti
$ echo "ovo je sadržaj" > dokumenti/orig.txt

$ ./makesymlink dokumenti/orig.txt sym1
Stvoren simbolički link 'sym1' -> 'dokumenti/orig.txt'.

$ ln -s ./dokumenti/orig.txt sym2

$ ls -l sym1 sym2
lrwxrwxrwx 1 dkrst users 19 May  5 17:23 sym1 -> dokumenti/orig.txt
lrwxrwxrwx 1 dkrst users 21 May  5 17:23 sym2 -> ./dokumenti/orig.txt

```

Već iz `ls -l` ispisa vidimo da oba linka pokazuju na istu datoteku, ali se njihove veličine razlikuju — `sym1` zauzima 19 bajtova (duljina niza "dokumenti/orig.txt"), a `sym2` 21 bajt (duljina niza `./dokumenti/orig.txt` — dva bajta više za uvodno `./`). Time je vidljivo da svaki link doslovno čuva onaj tekst kojim je stvoren.

Sad pročitajmo sadržaj oba linka pozivom `readsymlink`:

```

$ ./readsymlink sym1
Sadržaj linka 'sym1': "dokumenti/orig.txt"

$ ./readsymlink sym2
Sadržaj linka 'sym2': "./dokumenti/orig.txt"

```

Vidimo da nam program ispisuje upravo onaj tekst koji smo prosljedili kao putanju pri stvaranju linka — nezavisno o tome koristi li se `symlink()` u C-u ili `ln -s` u ljusci. Različita formulacija putanje (dokumenti/orig.txt vs ./dokumenti/orig.txt) doslovno se čuva u sadržaju linka — jezgra niti normalizira putanju niti provjerava simboličke linkove pri stvaranju. Provjera se događa tek pri svakom pristupu kroz link.

4.5 Vremena pristupa

Već smo upoznali tri vremenska polja u struct stat: st_atime, st_mtime, st_ctime. Jezgra ih ažurira automatski — pri svakom čitanju, pisanju ili promjeni atributa datoteke. Ipak, ponekad želimo eksplicitno postaviti atime i mtime za određenu datoteku na neku zadanu vrijednost — najčešće da bismo “potvrdili” da je datoteka svjež, ili da Build sustavi (make) misle da je novija od neke druge.

```
#include <sys/types.h>
#include <utime.h>

int utime(const char *pathname, const struct utimbuf *times);

struct utimbuf {
    time_t actime;    // novo atime
    time_t modtime;   // novo mtime
};
```

Povratna vrijednost: 0 u slučaju uspjeha, -1 u slučaju greške.

Argumenti:

- **pathname** — datoteka kojoj mijenjamo vremena.
- **times** — pokazivač na strukturu s novim vremenima. Ako je NULL, oba vremena postavljaju se na **trenutno** vrijeme (što je zapravo ekvivalent pozivu naredbe touch bez argumenata koji zadaju vrijeme).

Pogledajmo kako se ova funkcija koristi u praksi:

- **dotakni.c** — pojednostavljena inačica naredbe touch: ako datoteka ne postoji, stvara je praznu; potom postavlja njena vremena na trenutno vrijeme.

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <utime.h>
#include <sys/stat.h>

int main(int argc, char *argv[]) {
    int fd;

    if (argc != 2) {
        printf("koristenje: %s <ime_datoteke>\n", argv[0]);
        return 1;
    }

    /* Ako datoteka ne postoji, stvori je praznu */
    fd = open(argv[1], O_WRONLY | O_CREAT, 0644);
    if (fd < 0) {
        perror("open");
        return 1;
    }
    close(fd);

    /* Postavi atime i mtime na trenutno vrijeme.
     * NULL kao drugi argument znaci "koristi trenutno vrijeme". */
    if (utime(argv[1], NULL) < 0) {
        perror("utime");
        return 1;
    }

    printf("Datoteka '%s' azurirana.\n", argv[1]);
    return 0;
}
```

Pokretanje:

```
$ ls nova.txt
ls: cannot access 'nova.txt': No such file or directory

$ ./dotakni nova.txt
Datoteka 'nova.txt' azurirana.

$ ls -l nova.txt
-rw-r--r-- 1 dkrst users 0 May  5 15:00 nova.txt

$ ./dotakni nova.txt          # vec postoji - samo azurira vremena
Datoteka 'nova.txt' azurirana.
```

Ekvivalent u ljusci je naredba `touch`:

```
$ touch nova.txt          # ekvivalent gornjeg poziva
```

Naredba `touch` u praksi je nešto bogatija od našeg programa. Po zadanom postavlja **oba vremena** (atime i mtime) na trenutni trenutak, ali nudi i niz opcija za finiju kontrolu:

- **-a** mijenja samo atime, dok mtime ostaje netaknut,
- **-m** mijenja samo mtime, dok atime ostaje netaknut,
- **-d** ili **-t** omogućuju zadavanje **proizvoljnog** datuma i vremena umjesto trenutnog (npr. `touch -d "2024-01-15 10:30:00" dat.txt`),
- **-r** preslikava vremena s neke druge datoteke kao referentne (`touch -r referenca.txt cilj.txt`),
- **-c** sprečava stvaranje datoteke ako ne postoji.

Funkcija `utime` u svojoj punoj verziji već ima podršku za sve ovo: drugi argument može biti pokazivač na struct `utimbuf` s eksplicitno zadanim vrijednostima atime i mtime (umjesto NULL koji znači “trenutno vrijeme”). Vrijednosti tipa `time_t` mogu se dobiti iz čovjeku čitljivog datuma pomoću funkcija `mktime()` ili `strptime()`. Naš `dotakni.c` dakle pokriva samo najjednostavniji slučaj; proširenje s navedenim opcijama može biti korisna **vježba za čitatelja** ako želi bolje razumjeti rad s vremenom u UNIX sustavima.

Tipična uporaba `touch` (i `dotakni`) — “natjerati” make da iznova prevede neki fajl, čak i ako mu sadržaj nije promijenjen, samo postavljanjem mtime na sadašnji trenutak.

4.6 Rad s direktorijima

U posljednjem dijelu poglavlja dat ćemo pregled funkcija koje u našim programima možemo koristiti za upravljanje datotečnim stablom — stvaranje i brisanje direktorija, kretanje kroz njih, te čitanje njihova sadržaja. Završit ćemo i s odgovarajućim primjerom korištenja kojim ćemo ilustrirati kako radi još jedna od standardnih UNIX naredbi koju iznimno često koristimo u svakodnevnom radu.

```
#include <unistd.h>
#include <sys/stat.h>

int mkdir(const char *pathname, mode_t mode);
int rmdir(const char *pathname);

char *getcwd(char *buf, size_t size);
int chdir(const char *pathname);
int fchdir(int fd);
```

4.6.0.1 Funkcija `mkdir()`

Stvara novi prazan direktorij. U njemu jezgra automatski stvara dva posebna unosa: `.` (pokazivač na sam taj direktorij) i `..` (pokazivač na nadređeni direktorij).

Povratna vrijednost: 0 u slučaju uspjeha, -1 u slučaju greške.

Argumenti:

- **pathname** — putanja na kojoj se stvara novi direktorij.
- **mode** — prava pristupa za novi direktorij (npr. 0755); konačna prava dobiju se kombinacijom s maskom procesa (umask).

4.6.0.2 Funkcija `rmdir()`

Briše direktorij — ali samo ako je **prazan** (sadrži samo `.` i `..`). Ukoliko želimo obrisati direktorij u kojem se nalaze druge datoteke i direktoriji, potrebno je ući u njega i rekurzivno obrisati cijeli njegov sadržaj, datoteku po datoteku (sistemskim pozivom `unlink`) te tek nakon toga obrisati i sam direktorij.

Povratna vrijednost: 0 u slučaju uspjeha, -1 u slučaju greške.

Argumenti:

- **pathname** — putanja do direktorija koji se briše.

4.6.0.3 Funkcija `getcwd()`

Vraća putanju do trenutnog radnog direktorija procesa (CWD - *Current Working Directory*). Bitno je razumjeti da je radni direktorij **svojstvo procesa**, ne korisnika; svaki proces ima svoj vlastiti CWD koji je naslijedio od svog roditelja u trenutku stvaranja (sistemskim pozivom `fork()`).

Povratna vrijednost: pokazivač na buf u slučaju uspjeha, NULL u slučaju greške (npr. ako je međuspremnik premali da primi punu putanju).

Argumenti:

- **buf** — međuspremnik u koji se upisuje putanja do trenutnog direktorija (kao null-terminirani string).
- **size** — veličina međuspremnika buf u bajtovima.

4.6.0.4 Funkcije `chdir()` i `fchdir()`

Mijenjaju radni direktorij procesa. `chdir` direktorij identificira putanjom, dok `fchdir` koristi otvoreni file deskriptor.

Bitan suptilan detalj: `chdir` u programu mijenja radni direktorij **samo tom procesu** — ne i ljusci koja ga je pokrenula. Zato naredba `cd` **nije implementirana kao "vanjska" naredba**, tj. ne postoji izvršna datoteka koja se poziva kad napišemo `cd` (za razliku od drugih naredbi, kao npr. `ls`, `touch` i brojnih drugih koje smo koristili). Ovako implementirana naredba bila bi beskorisna: pokretanje novog programa podrazumijeva i stvaranje novog procesa u kojem će se program izvršavati, dok ljuska čeka da on završi i prihvati sljedeću naredbu korisnika. Pozivanje funkcije `chdir()` u takvom novom procesu zapravo bi promijenilo trenutni radni direktorij (CWD) samo tog procesa, ne i ljuske koja je naredbu pozvala. Stoga je naredba `cd` realizirana kao ugrađena (*built-in*) naredba ljuske — ne izvršava se u novom

procesu, nego mijenja CWD same ljuske u kojoj je pozvana, pozivom `chdir()` “iznutra”.

Povratna vrijednost (obje funkcije): 0 u slučaju uspjeha, -1 u slučaju greške.

Argumenti:

- **pathname** — putanja do direktorija (`chdir`).
- **fd** — otvoreni file deskriptor (`fchdir`).

4.6.1 Čitanje sadržaja direktorija

Direktoriji su zapravo posebne datoteke koje sadrže popis imena i pripadnih i-node brojeva. Jezgra dopušta samo sebi da piše u njih — korisnički procesi smiju ih čitati, ali ne i izravno mijenjati. Najčešći oblik rada s direktorijem — sekvencijalno čitanje svih zapisa u njemu — obavlja se trojkom funkcija iz `<dirent.h>`:

```
#include <dirent.h>

DIR      *opendir(const char *pathname);
struct dirent *readdir(DIR *dp);
int      closedir(DIR *dp);
```

4.6.1.1 Funkcija `opendir()`

Otvora direktorij za čitanje i vraća pokazivač na DIR strukturu — **neprozirni** (*opaque*) tip, što znači da programer ne zna i ne treba znati kako struktura iznutra izgleda; samo je prosljeđuje ostalim funkcijama obitelji kao apstraktnu referencu, dok upravljanje sadržajem strukture u potpunosti preuzima biblioteka.

Povratna vrijednost: pokazivač na DIR strukturu u slučaju uspjeha, NULL u slučaju greške.

Argumenti:

- **pathname** — putanja do direktorija koji se otvara.

4.6.1.2 Funkcija `readdir()`

Vraća zapis koji se nalazi na **trenutnoj poziciji** u direktoriju, a zatim tu poziciju pomiče na sljedeći zapis. Pozivom `opendir()` pozicija se postavlja na **prvi zapis** u direktoriju, pa prvi poziv `readdir`-a vraća upravo njega. Svaki sljedeći poziv vraća sljedeći zapis i pomiče poziciju za jedno mjesto naprijed; kad više nema zapisa, `readdir` vraća NULL. Time u jednoj while petlji možemo prirodno pročitati cijeli direktorij od početka do kraja.

Mehanizam je analogan onomu što smo već vidjeli kod običnih datoteka: tamo `read()` čita podatke od trenutnog **offseta** u datoteci i pomiče ga za broj pročitanih bajtova; ovdje `readdir()` čita zapis s trenutne pozicije u direktoriju i pomiče tu poziciju na sljedeći zapis. Razlika je u tome što kod regularnih datoteka offsetom programer može slobodno upravljati funkcijom `lseek()`, dok je kod direktorija interna pozicija dio neprozirne DIR strukture i nije izravno dostupna programeru — on je može samo vratiti na početak pozivom `rewinddir()`, ili koristiti `tellldir()` i `seekdir()` o kojima ćemo govoriti malo kasnije.

Povratna vrijednost: pokazivač na `struct dirent` strukturu sa zapisom, ili NULL na kraju direktorija ili u slučaju greške.

POSIX standard zahtijeva da `struct dirent` sadrži samo **dva polja**:

```
struct dirent {
    ino_t  d_ino;      // i-node broj zapisa
    char   d_name[];   // ime datoteke (null-terminirano)
};
```

Polje `d_name` ima **neodređenu veličinu** — POSIX kaže samo “polje znakova koje sadrži ime od najviše `NAME_MAX` bajtova plus null-terminator”. Različite implementacije strukturu dopunjuju vlastitim poljima (`d_off`, `d_reclen`, `d_type` na Linuxu/glibc), ali ta polja **nisu prenosiva**. Zbog ovoga vrijedi nekoliko praktičnih napomena:

- **`sizeof(d_name)` ne radi pouzdano** — neke implementacije deklariraju polje kao `char d_name[1]`, druge kao `char d_name[256]`. Za pravu duljinu uvijek koristimo `strlen(d_name)`.
- **`sizeof(struct dirent)` ne predstavlja nužno stvarnu veličinu zapisa** — iz istog razloga kao gore.
- **Polje `d_type`** (regularna/direktorij/link/...) postoji na Linuxu, ali nije dio POSIX standarda. Ukoliko želimo pisati prenosivi kod, sigurnije je koristiti `lstat` sa imenom datoteke (koje možemo dobiti iz `d_name` polja) i nakon toga iz `st_mode` saznati tip datoteke.

Vraćeni pokazivač pokazuje na strukturu koju je sama biblioteka prethodno alocirala u svojoj internoj memoriji (najčešće u statičkom međuspremniku); pozivatelj ju nije ni alocirao, pa ju ni ne smije osloboditi pozivom `free()`. Sljedeći poziv `readdir`-a nad istim direktorijem prepíše taj međuspremnik novim zapisom — povratni pokazivač iz prethodnog poziva nakon toga još uvijek vrijedi, ali sadržaj na koji pokazuje više nije isti. Ako iz nekog razloga trebamo zadržati podatke iz zapisa duže (npr. spremiti listu svih imena u direktoriju), moramo ih kopirati u vlastitu memoriju.

Argumenti:

- **`dp`** — pokazivač na otvorenu DIR strukturu, dobiven prethodnim pozivom `opendir`.

4.6.1.3 Funkcija `closedir()`

Zatvara direktorij i oslobađa pripadne resurse.

Povratna vrijednost: 0 u slučaju uspjeha, -1 u slučaju greške.

Argumenti:

- **`dp`** — pokazivač na otvorenu DIR strukturu.

Veza sa standardnom C bibliotekom: uočite da je kombinacija `opendir` → `readdir` → `closedir` za direktorije analogna kombinaciji `fopen` → `fread` → `fclose` za regularne datoteke. U oba slučaja imamo “stream” apstrakciju — neprozirni objekt (DIR * odnosno FILE *) koji čuva interne podatke o napretku čitanja, plus funkcije za otvaranje, sekvencijalno čitanje i zatvaranje.

4.6.1.4 Pozicioniranje unutar direktorija

Pored osnovne trojke koju smo upravo opisali, `<dirent.h>` nudi i tri dodatne funkcije za izravno upravljanje internom pozicijom u direktoriju:

```
#include <dirent.h>

void rewinddir(DIR *dp);
long telldir(DIR *dp);
void seekdir(DIR *dp, long loc);
```

4.6.1.5 Funkcija rewinddir()

Vraća čitanje na početak direktorija — sljedeći poziv readdir-a opet će vratiti prvi zapis.

Povratna vrijednost: nema (funkcija je tipa void).

Argumenti:

- **dp** — pokazivač na otvorenu DIR strukturu.

4.6.1.6 Funkcije telldir() i seekdir()

telldir vraća trenutnu poziciju u direktoriju kao neprozirnu vrijednost; seekdir postavlja čitanje natrag na poziciju ranije zapamćenu telldir-om. Ove dvije funkcije rijetko se koriste u praksi.

Povratna vrijednost:

- **telldir** — trenutna pozicija (kao long vrijednost), ili -1 u slučaju greške.
- **seekdir** — nema povratne vrijednosti (void).

Argumenti:

- **dp** — pokazivač na otvorenu DIR strukturu.
- **loc** (samo seekdir) — pozicija ranije dobivena pozivom telldir.

Sada kad smo upoznali sve funkcije za rad s direktorijima, iskoristit ćemo ih u jednom konkretnom primjeru — implementaciji **pojednostavljene inačice standardne UNIX naredbe ls**, koja kao i izvorna naredba ispisuje sadržaj zadanog direktorija.

- **mojls.c** — pojednostavljena inačica naredbe ls. Bez argumenta lista trenutni direktorij; s argumentom lista zadani. Za svaki zapis ispisuje veličinu datoteke u bajtovima, te ime.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <dirent.h>

int main(int argc, char *argv[]) {
    DIR *dp;
    struct dirent *entry;
    struct stat st;
    char path[1024];
    const char *dir;

    /* bez argumenta, listamo trenutni direktorij */
    dir = (argc < 2) ? "." : argv[1];

    dp = opendir(dir);
    if (dp == NULL) {
        perror("opendir");
        return 1;
    }
}
```

```

/* readdir vraca po jedan zapis u svakom pozivu;
 * NULL znaci kraj direktorija (ili greska) */
while ((entry = readdir(dp)) != NULL) {
    /* preskoci skrivene datoteke (ukljucujuci . i ..) */
    if (entry->d_name[0] == '.')
        continue;

    /* sastavi punu putanju "dir/ime" za poziv lstat-u */
    snprintf(path, sizeof(path), "%s/%s", dir, entry->d_name);

    if (lstat(path, &st) < 0) {
        perror(path);
        continue;
    }

    printf("%10ld %s\n", (long)st.st_size, entry->d_name);
}

closedir(dp);
return 0;
}

```

Glavna petlja je tipična UNIX-ova “while-readdir” konstrukcija — funkcija u istom pozivu javlja i podatke (kroz povratnu vrijednost) i kraj pretrage (vraćanjem NULL-a). Ovakvu petlju, koja prolazi kroz niz podataka ili poziva zaključenih NULL pokazivačem, često ćete sresti u programima.

Bitno: entry->d_name sadrži samo **ime zapisa unutar otvorenog direktorija**, ne njegovu punu putanju. lstat() međutim treba putanju — pa prije svakog poziva snprintf-om sastavljamo string oblika “<dir>/<ime>”. Bez ovog koraka lstat bi tražio entry->d_name u trenutnom radnom direktoriju procesa, što bi naravno bilo pogrešno čim listamo neki drugi direktorij osim “.”. Format “%10ld” u printf-u poravnava broj u stupac širine 10 znakova radi urednijeg ispisa.

Pokretanje:

```

$ ./mojls
564 prava.c
373 makesymlink.c
595 provjeri.c
807 Makefile
484 readsymlink.c
1302 fileinfo2.c
942 mojls.c
58904 README.md
677 dotakni.c
1300 fileinfo.c
361 makelink.c

$ ./mojls /etc
4096 cron.d
4096 default
4096 X11
1111 passwd
597 group
609 shadow
219 hosts
11 hostname
12813 services
582 profile
656 fstab
122 resolv.conf
...

```

Direktorij /etc u praksi sadrži znatno više datoteka i poddirektorija nego što je prikazano (na tipičnom sustavu desetke i stotine zapisa); ostatak ispisa koji ovdje nije prikazan iz prostornih razloga označen je s ... na dnu.

Direktoriji (`cron.d`, `default`, `X11`) uobičajeno imaju veličinu 4096 bajtova, što je tipična veličina bloka datotečnog sustava na Linuxu — jedan blok dovoljan je za pohranjivanje popisa imena i pripadnih i-node brojeva uobičajenog direktorija. Bitno je razumjeti da ova vrijednost **ne predstavlja zbroj veličina datoteka unutar direktorija**, kao što čitatelj možda intuitivno očekuje, nego veličinu prostora koji direktorij sam zauzima na disku radi vlastite organizacije. Većina direktorija stoga u `lsstat()` ispisu pokazuje istih 4096 B bez obzira što se u njima nalazi (vrijednost može biti i veća kod direktorija s vrlo mnogo unosa, kad jedan blok više nije dovoljan). Za regularne datoteke (`passwd`, `services`, ...) `st_size` pokazuje stvarnu veličinu sadržaja u bajtovima.

Vježba za čitatelja: ispisivanje pravog `ls -l` sadrži znatno više informacija — tip datoteke, prava pristupa, vlasnika i grupu, vrijeme zadnje izmjene. Sve te informacije su nam već dostupne u `struct stat` strukturi koju smo popunili pozivom `lsstat()`. Pokušajte doraditi `mojls` tako da reproducira potpun `ls -l` ispis. Korisne funkcije iz standardne C biblioteke: `getpuid()` (pretvara UID u ime korisnika, definirana u `<pwd.h>`), `getgrgid()` (pretvara GID u ime grupe, definirana u `<grp.h>`), te `ctime()` ili `strftime()` za pretvorbu `time_t` vrijednosti u tekstualni datum.

4.7 Što smo zapravo radili

Vrijedi se na kraju ovog poglavlja na trenutak osvrnuti na ono što smo zapravo radili kroz dane primjere. Mali UNIX programi poput `dotakni`, `mojls`, `prava`, `makelink`, `makesymlink`, `readsymlink` — kao i `f_cat`, `f_strip` ili `f_write` iz prethodnog poglavlja — nisu tek umjetni vježbeni primjeri. Svaki od njih implementira temeljnu funkcionalnost neke od standardnih UNIX naredbi: `touch`, `ls`, `chmod`, `ln`, `ln -s`, `readlink`, `cat`. Time ilustriramo i jednu od najvažnijih osobina UNIX-a: skupina alata koje koristimo svaki dan zapravo je **tanki sloj iznad sistemskih poziva**. Kad ih sami napišemo u nekoliko desetaka linija C koda, vidimo iz prve ruke da iza naredbi koje na prvi pogled izgledaju “magično” stoje sasvim razumljivi pozivi `stat`, `chmod`, `link`, `symlink`, `utime` — a temeljne UNIX rutine i sistemski pozivi koji ih implementiraju isti su već gotovo pola stoljeća, što pokazuje koliko je dobro osmišljen UNIX sustav.

Ovo nam ukazuje na još jednu osobinu UNIX-a: koncepti na kojima UNIX počiva su složeni, ali kad ih jednom usvojite, naizgled kompleksne stvari možete realizirati na relativno jednostavan način, sa svega nekoliko sistemskih poziva. Ova osobina utkana je u temeljnu filozofiju izrade UNIX programa: *radi jednu stvar, ali radi je dobro!* Takav pristup potiče pisanje malih, modularnih programa specijaliziranih za obavljanje jedne konkretne zadaće, uz mogućnost njihova povezivanja u veće i složenije cjeline koristeći osnovno načelo UNIX-a — *sve je datoteka*.

4.8 Prevođenje

Direktorij dolazi s priloženim `Makefile`-om koji prati iste konvencije kao i u ostalim poglavljima (varijable `CC`, `CFLAGS`, `LDLAGS`, `TARGETS`; implicitno pravilo `.c.o`; pravila `default`, `all`, `clean`).

```
make all           # gradi sve primjere
make fileinfo      # gradi pojedinačni primjer
make clean         # čisti generirane datoteke
```

4.9 Bibliografija

[1] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2013.

Poglavlje 5

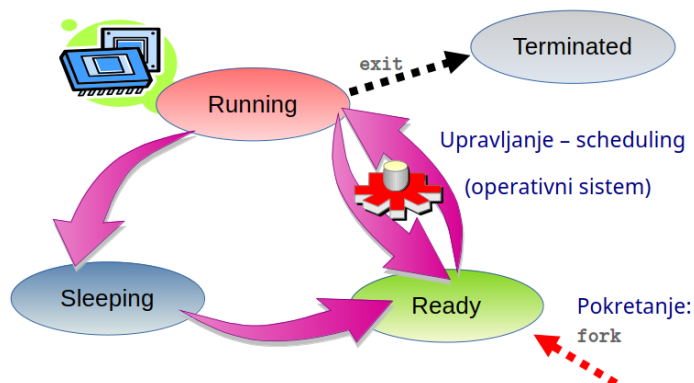
Okruženje procesa

Proces je jedan od fundamentalnih koncepata UNIX-a, kao i bilo kojeg drugog operacijskog sustava, te predstavlja osnovnu jedinicu izvršavanja koja omogućuje pokretanje programa. UNIX proces aktivni je entitet s obilježjima koja uključuju jedinstveni identifikator procesa (engl. *Process ID* — **PID**), vlasnika procesa, prioritet procesa, stanje procesa, vremena izvršavanja i pridijeljene resurse. Svaki UNIX proces samostalna je instanca izvršavanja zadanog programa, pri čemu više procesa istovremeno i nezavisno može izvršavati isti program.

Program i proces

Odnos između programa i procesa možemo usporediti s receptima i kuhanjem ručka. Svaki recept u osnovi predstavlja “program” za kuhanje: niz precizno definiranih koraka i postupaka s naredbama kontrole toka (`if (nedovoljno_slano): dodaj_soli`), petljama (`while(meso_tvrdo): nastavi_kuhanje`) i svim ostalim instrukcijama potrebnim da dobijemo očekivani rezultat u vidu ukusnog ručka.

Sam postupak kuhanja u osnovi je pokretanje procesa koji izvršava program, tj. prati zadani recept korak po korak. Pri tom ne postoji nikakva prepreka da više kuhara istovremeno ne pokrene proces kuhanja po istom receptu. Tada svaki od kuhara zapravo nezavisno izvršava svoj “proces”, ali svi kuhaju po istom receptu, tj. u svim procesima izvršava se isti “program”.



Slika 5.1: Životni ciklus UNIX procesa

Životni ciklus UNIX procesa, prikazan na slici 5.1, započinje sistemskim pozivom `fork` — jedinim načinom za stvaranje novog procesa na UNIX-u. U tom trenutku proces dobiva **PID**, koji predstavlja jedinstveni identitet procesa i nepromjenjiv je za cijelo vrijeme njegovog postojanja, te memorijski prostor u kojem se proces izvršava, nakon čega je spreman za izvršavanje. Kada će i koliko vremena proces stvarno dobiti na raspolaganje za izvršavanje na računalnom sklopovlju ovisi o opterećenosti sustava. Naime, UNIX je višezadaćni operacijski sustav i u pravilu u svakom trenutku postoji više aktivnih procesa od raspoloživih procesora i procesorskih jezgri. Kako bi se osiguralo da se više procesa izvršava istovremeno, procesi se izvršavaju u tzv. *dijeljenom vremenu*. Resursima upravlja **raspoređivač** (engl. *scheduler*), komponenta operacijskog sustava odgovorna za dijeljenje procesorskog vremena među procesima koji se istovremeno izvršavaju, pri čemu uzima u obzir prioritet svakog procesa zasebno. Tijekom izvršavanja, UNIX proces može se nalaziti u jednom od nekoliko stanja:

1. **Ready** — proces je spreman za izvršavanje i čeka da ga operacijski sustav prebaci u aktivno stanje i dodijeli mu njegov dio procesorskog vremena.
2. **Running** — proces se izvršava na računalnom sklopovlju. Raspoređivač u bilo kojem trenutku može privremeno zaustaviti izvršavanje procesa i prebaciti ga u stanje čekanja. Važno je naglasiti da proces — tj. programer koji je napisao program koji se u procesu izvršava — ne može predvidjeti u kojem će se trenutku to dogoditi.
3. **Sleeping** — za vrijeme izvršavanja proces može doći u situaciju da mora čekati nastup određenog uvjeta kako bi nastavio s izvršavanjem, npr. kada čeka na “spore” ulazno/izlazne operacije. U tom slučaju raspoređivač ga prebacuje iz stanja aktivnog izvršavanja u stanje spavanja. Kada se uvjeti za nastavak zadovolje, raspoređivač proces prebacuje u stanje čekanja (Ready) — raspoloživi resursi u tom su trenutku dodijeljeni drugim procesima pa ga nije moguće odmah vratiti u aktivno izvršavanje. Proces može i sam zatražiti da ga se uspava na određeno vrijeme sistemskim pozivom `sleep`.
4. **Terminated** — sistemski poziv `exit` prekida izvršavanje procesa. Prekidom se oslobađaju resursi koje je proces koristio, ali informacija o procesu, uključujući njegov *izlazni status*, još neko vrijeme može ostati zapisana u sustavu. Kao što je ranije spomenuto, funkcija `main` ima povratnu vrijednost tipa `int`, a sistemski poziv `exit` kao argument prima jednu cjelobrojnu vrijednost — upravo ta vrijednost predstavlja izlazni status procesa. Poziv `return` iz funkcije `main` ima isti učinak kao `exit` te prekida izvršavanje procesa.

U ovom poglavlju dani su primjeri koji demonstriraju upravljanje procesima na UNIX-u: dohvaćanje argumenata naredbenog retka i okruženja UNIX procesa, stvaranje procesa, pokretanje programa i upravljanje limitima. Svaki od ovih mehanizama vezan je uz jedan ili više sistemskih poziva — `fork()`, `exec` obitelj, `wait()`, `dup2()`, `setrlimit()` — koji zajedno tvore jezgru UNIX-ove filozofije upravljanja procesima. Primjeri su poredani tako da se teme grade postupno — od najjednostavnijih (ispis primljenih argumenata) prema složenijima (kombinacija `fork`, `exec`, `dup2` i `wait` u jednom programu). Tematski je ovo područje detaljno obrađeno u *Advanced Programming in the UNIX Environment*, Stevens & Rago [1], a šire koncepte procesa kao temeljne apstrakcije operacijskog sustava (životni ciklus, stanja, raspoređivanje) na hrvatskom jeziku obrađuje *Operacijski sustavi*, Budin, Golub, Jakobović & Jelenković [2].

5.0.1 Argumenti naredbenog retka

S argumentima naredbenog retka susreli smo se već u poglavlju o ulazno/izlaznim operacijama; ovdje ćemo detaljnije objasniti njihovu organizaciju u kontekstu memorijske slike procesa.

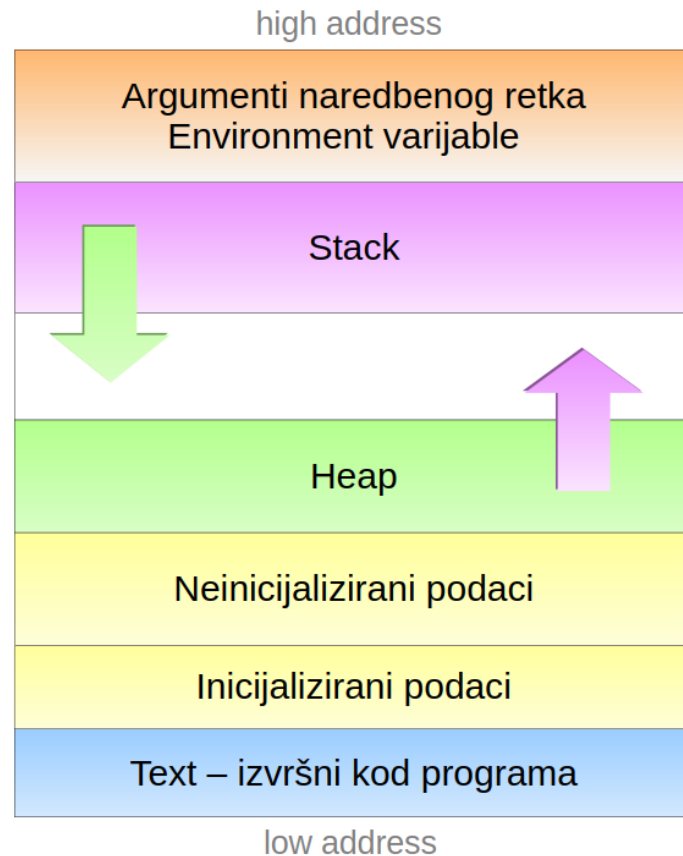
Funkcija `main` prema ISO C standardu može imati dva osnovna oblika:

```
int main(void)
int main(int argc, char *argv[])
```

Prvi oblik koristimo kada program ne očekuje dodatne opcije i argumente koje korisnik može zadati prilikom pokretanja programa. Međutim, često je korisniku ostavljena mogućnost da putem dodatnih argumenata, koji se zadaju kao stringovi u naredbenom retku iza same naredbe kojom je program pozvan, usmjerava način izvršavanja našeg programa. U ovom slučaju koristimo drugi oblik funkcije `main`, kako bi mogli pristupiti svim argumentima koji su u naredbenom retku zadani.

Prilikom pozivanja bilo koje naredbe u UNIX ljusci, ljuska analizira niz znakova kojim korisnik želi pokrenuti naredbu i dijeli ga na podnizove odvojene razmacima. Ovaj postupak nazivamo **tokenizacija**, a rezultat je niz **tokena** — stringova koji redom sadrže pozvanu naredbu i sve argumente koje je korisnik naveo. Ukoliko ovim stringovima želimo pristupiti iz našeg programa, koristimo drugi oblik funkcije `main`, kod kojeg funkcija prima dva argumenta: cjelobrojni `argc` u kojem je pohranjen ukupan broj stringova navedenih u naredbenom retku (uključujući i naredbu kojom je program pozvan), i polje pokazivača na znakovni niz `argv`, u kojem su ovi stringovi redom pohranjeni.

Argumenti naredbenog retka nalaze se na samom vrhu adresnog prostora procesa, kako je prikazano na slici 5.2:



Slika 5.2: Memorijska slika (adresni prostor) UNIX procesa

Najjednostavniji način pristupa je iteriranje kroz polje pokazivača `argv`, od prvog elementa (indeks 0) koji pokazuje na samu naredbu, do posljednjeg s indeksom `argc-1`.

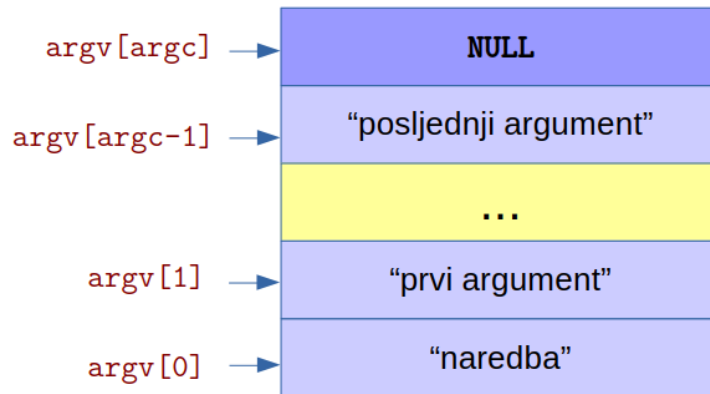
Uzmimo za primjer hipotetski program `mojprogram` kod kojeg korisnik može prilikom pokretanja zadati argumente naredbenog retka. Neka je naš program pozvan s:

```
./mojprogram prvi argument      drugi_argument
```

Vrijednost argumenta `argc` bila bi 4 i odgovarala bi broju tokena koji čine naredbeni redak. Polje pokazivača `argv` redom bi pokazivalo na adrese pri vrhu adresnog prostora procesa, na kojima bi bile sljedeće vrijednosti:

```
argv[0] = "./mojprogram"
argv[1] = "prvi"
argv[2] = "argument"
argv[3] = "drugi_argument"
argv[4] = NULL
```

Uočimo dva detalja: ljuška tokenizira naredbeni redak tako da tokene razdvaja temeljem razmaka (*space*) — praznog prostora između njih. Pri tom je svejedno koristimo li jedan ili više razmaka (više puta pritisnuta tipka *space* prilikom unosa). Dodatno, pored pokazivača `argv[0]` do `argv[argc-1]`, uvijek postoji i posljednji pokazivač u nizu s indeksom `argc`, koji pokazuje na vrijednost `NULL`, tj. `(void*)0`. Organizacija polja `argv` u memoriji shematski je prikazana na slici 5.3:



Slika 5.3: Organizacija argumenata naredbenog retka u memoriji procesa

- **argumenti.c** — najjednostavniji mogući primjer rada s argumentima naredbenog retka. Program u petlji prolazi kroz polje `argv[0]`, ..., `argv[argc-1]` i ispisuje indeks i vrijednost svakog argumenta. Koristi se za vizualnu provjeru kako ljuska prenosi naredbeni redak programu — posebno korisno za razumijevanje razdvajanja riječi po razmacima, ponašanja navodnika, ili kako `argv[0]` uvijek nosi ime kojim je program pokrenut.

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    int k;

    for (k = 0; k < argc; k++)
        printf("%d:\t %s\n", k, argv[k]);

    return 0;
}
```

```
$ ./argumenti jedan dva tri
0:      ./argumenti
1:      jedan
2:      dva
3:      tri
```

- **zbroyi.c** — program koji dva argumenta naredbenog retka tumači kao cijele brojeve i ispisuje njihov zbroj. Uvodi praksu **provjere broja argumenata** na početku programa (`argc < 3` → ispis upute za korištenje i izlaz) te funkciju `atoi()` iz standardne C biblioteke za konverziju stringa u `int`. Uobičajeni UNIX obrazac: poruka o korištenju uvijek koristi `argv[0]` kako bi točno odražavala naziv pod kojim je program pozvan.

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int a, b, zbroj;

    if (argc < 3) {
        printf("koristenje: %s <1.broj> <2.broj>\n", argv[0]);
        exit(0);
    }

    a = atoi(argv[1]);
    b = atoi(argv[2]);
    zbroj = a + b;

    printf("%d + %d = %d\n", a, b, zbroj);
}
```

```

    exit(0);
}

$ ./zbroji
koristenje: ./zbroji <1.broj> <2.broj>
$ ./zbroji 17 25
17 + 25 = 42

```

5.0.2 Varijable okruženja

Pored argumenata naredbenog retka, svaki UNIX proces u svom memorijskom prostoru sadrži i drugu vrstu konteksta — **varijable okruženja** (engl. *environment variables*). Ovaj skup vrijednosti proces nasljeđuje od svog roditelja — procesa koji je inicirao njegovo stvaranje korištenjem sistemskog poziva `fork()`, a može se mijenjati tijekom izvršavanja procesa.

Varijable okruženja nalaze se u memoriji procesa, pri samom vrhu adresnog prostora (odmah "ispod" argumenata naredbenog retka), a zadane su u formi niza parova oblika "IME=vrijednost". Ove vrijednosti procesu prenose informacije o sistemskoj konfiguraciji i korisničkim postavkama, bez potrebe da se eksplicitno zadaju kao argumenti naredbenog retka. Vrijednosti pojedinih varijabli postavljaju se u inicijalizacijskim skriptama ljuške, a sve naredbe pokrenute iz iste sesije ih dijele kao zajednički kontekst.

Najčešće varijable okruženja na svakom UNIX sustavu su:

Varijabla	Značenje
HOME	Apsolutna putanja korisničkog <i>home</i> direktorija.
PATH	Popis direktorija (odvojenih dvotočkom) u kojima ljuška traži izvršne datoteke.
USER	Korisničko ime trenutno prijavljenog korisnika.
SHELL	Apsolutna putanja korisnikove zadane ljuške.
LANG	Lokalizacijske postavke (jezik, kodna stranica).
PWD	Trenutno radni direktorij.
TERM	Tip terminala u kojem se sesija odvija.

Vrijednostima varijabli okruženja možemo pristupati i mijenjati ih direktno iz UNIX ljuške. Vrijednost varijable okruženja čitamo korištenjem prefiksa `$` ispred imena. Na primjer, ukoliko želimo saznati koji je naš korisnički direktorij, dovoljno je izvršiti:

```

$ echo $HOME
/home/dkrst

```

Za promjenu vrijednosti postojeće, ili dodavanje nove varijable okruženja koristimo naredbu `export`:

```

$ export MOJA_VAR="neka vrijednost"
$ echo $MOJA_VAR
neka vrijednost

```

Naredba `env` (bez argumenata) ispisuje sve varijable okruženja trenutne sesije.

Iz programa pisanih u C-u, varijablama okruženja pristupa se kroz funkcije iz standardne C biblioteke deklarirane u `<stdlib.h>`:

```
#include <stdlib.h>

char *getenv(const char *name);
int  setenv(const char *name, const char *value, int overwrite);
int  unsetenv(const char *name);
int  putenv(char *string);
```

5.0.2.1 Funkcija getenv()

Dohvaća vrijednost varijable okruženja prema njezinom imenu.

Povratna vrijednost: pokazivač na string s vrijednošću tražene varijable, ili NULL ako varijabla nije postavljena.

Argumenti:

- **name** — ime varijable okruženja čija se vrijednost dohvaća.

Vraćeni string ne smije se modificirati — pripada okruženju procesa kojim upravlja jezgra. Ako želimo zadržati vrijednost duže vremena, najsigurnije ju je odmah kopirati u vlastiti međuspremnik (npr. pozivom `strdup()` ili `strncpy()`).

5.0.2.2 Funkcija setenv()

Postavlja vrijednost varijable okruženja, eventualno prepisujući postojeću.

Povratna vrijednost: 0 u slučaju uspjeha, -1 u slučaju greške (pri čemu se postavlja `errno`).

Argumenti:

- **name** — ime varijable koja se postavlja.
- **value** — nova vrijednost varijable.
- **overwrite** — ponašanje kad varijabla već postoji: ako je 0, postojeća vrijednost se zadržava; inače se prepisuje.

5.0.2.3 Funkcija unsetenv()

Briše varijablu iz okruženja.

Povratna vrijednost: 0 u slučaju uspjeha, -1 u slučaju greške.

Argumenti:

- **name** — ime varijable koju brišemo. Ako varijabla ne postoji, poziv se smatra uspješnim (vraća 0).

5.0.2.4 Funkcija putenv()

Postavlja varijablu zadanu u obliku "ime=vrijednost".

Povratna vrijednost: 0 u slučaju uspjeha, ne-nula vrijednost u slučaju greške.

Argumenti:

- **string** — pokazivač na niz oblika "ime=vrijednost". Bitno je naglasiti da `putenv()` **ne kopira** zadani string nego pohranjuje sam pokazivač u tablicu okruženja. Stoga string mora biti trajno dostupan — najčešće se koriste statički ili dinamički alocirani nizovi, a **nikad lokalne varijable** funkcije koja poziva

putenv(). Zbog ovog ponašanja, setenv() je u modernom kodu preferiran jer kopira vrijednost u svoje vlasništvo.

Funkcije setenv, unsetenv i putenv koriste se za izmjenu okruženja samog procesa — promjene se **ne** propagiraju natrag u roditeljski proces (ljusku), nego ostaju lokalne tekućem procesu i njegovim potomcima.

- **readenv.c** — čita vrijednost jedne varijable okruženja čije se ime zadaje kao argument naredbenog retka, pozivom getenv() iz standardne C biblioteke. getenv() vraća pokazivač na string s vrijednošću varijable, ili NULL ako varijabla nije postavljena. Program razlikuje ta dva slučaja i prikazuje odgovarajuću poruku. Ilustrira temeljni način na koji program pristupa okolini koju je naslijedio od ljuske — varijable poput HOME, PATH, USER ili vlastite varijable postavljene naredbom export.

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    char *vrijednost;

    if (argc < 2) {
        printf("koristenje: %s <ime varijable>\n", argv[0]);
        return 0;
    }

    vrijednost = getenv(argv[1]);
    if (!vrijednost)
        printf("%s: environment varijabla ne postoji\n", argv[1]);
    else
        printf("%s = %s\n", argv[1], vrijednost);

    return 0;
}
```

```
$ ./readenv HOME
HOME = /home/dkrst
$ ./readenv NEPOSTOJECA
NEPOSTOJECA: environment varijabla ne postoji
```

5.0.2.5 Treći argument funkcije main — envp

Pored dva standardna oblika funkcije main opisana ranije, na UNIX sustavima česta je i sljedeća, proširena varijanta:

```
int main(int argc, char *argv[], char *envp[]) {
    // envp je ovdje lokalni parametar funkcije
}
```

Treći argument envp je polje pokazivača na stringove oblika "IME=vrijednost", završeno NULL pokazivačem — (void *)0 — i sadrži kompletno okruženje koje je proces naslijedio pri pokretanju.

Treba imati na umu da **envp nije dio ISO C standarda** — ISO/IEC 9899:2018 u Annexu J.5.1 (*Common extensions*) spominje ovaj treći argument samo kao uobičajenu implementacijsku ekstenziju, što znači da je implementacije smiju ali nisu dužne podržavati. Ni POSIX.1-2017 ne propisuje envp kao standardni oblik — naprotiv, izričito preporučuje korištenje varijable environ (opisane u nastavku). U praksi je ipak treći argument main-a podržan na praktički svim modernim UNIX i Linux distribucijama, kao i u Microsoft C kompajleru.

5.0.2.6 Vanjska varijabla environ

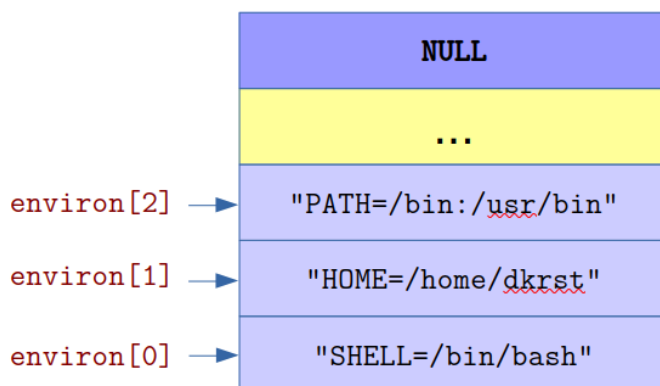
Drugi način pristupa okolini iz programa pisanog u jeziku C jest preko vanjske globalne varijable `environ`. Za razliku od `envp`, koji je lokalni parametar funkcije `main`, `environ` je dostupna iz bilo koje funkcije programa nakon što se na nju prethodno deklarira:

```
#include <unistd.h>
extern char **environ;    // deklaracija vanjske varijable

int main(int argc, char *argv[]) {
    // environ je dostupan ovdje iako nije u zagradama main-a
}
```

Pokazivač `environ` pokazuje na isti niz pokazivača kao i `envp` u trenutku pokretanja procesa — drugim riječima, oba mehanizma daju početno isti pogled na okruženje. Razlika postaje vidljiva tek nakon poziva funkcija `setenv()`, `putenv()` ili `unsetenv()`: te funkcije ažuriraju varijablu `environ` (eventualno realocirajući memoriju), dok `envp` ostaje pokazivati na izvornu, sada zastarjelu kopiju. Drugim riječima, `envp` je “snapshot” okoline u trenutku ulaska u `main`, a `environ` je “živa” referenca.

Organizacija varijabli okruženja u memoriji procesa shematski je prikazana na slici 5.4. Bez obzira pristupa li se okruženju kroz `environ` ili kroz `envp`, sama struktura u memoriji uvijek je ista — niz pokazivača na stringove oblika “IME=vrijednost” završen NULL-om. Razlikuje se samo ime varijable kroz koju mu pristupamo.



Slika 5.4: Organizacija varijabli okruženja u memoriji procesa

Za razliku od `envp`, varijabla `environ` **jest** standardizirana — propisuje je POSIX.1-2017. Zanimljivo, to je jedini objekt u POSIX-u kojeg ne deklarira nijedna sistemsko zaglavna datoteka, pa korisnik mora sam navesti deklaraciju `extern char **environ;` u svom programu (kao u primjeru iznad). ISO C, ponovo, ovu varijablu uopće ne spominje.

5.0.2.7 Koji oblik koristiti?

Preferirani oblik je `environ`, iz nekoliko razloga:

- **Standardiziran je u POSIX-u**, dok je `envp` samo implementacijska ekstenzija u Annexu ISO C standarda.
- **Uvijek odražava trenutno stanje okoline** — promjene napravljene pozivima `setenv()`/`putenv()`/`unsetenv()` reflektiraju se u `environ`-u, ali ne i u `envp`-u, pa korištenje `envp`-a nakon takvih izmjena vodi u nedefinirano ponašanje.

- **Dostupna je iz bilo koje funkcije programa**, ne samo iz `main`. Predaja `envp`-a niz funkcijske pozive zahtijeva da ga svaka funkcija koja ga treba primi kao argument.

U sljedećem ćemo paru primjera (`listenv1.c` i `listenv2.c`) prikazati oba mehanizma — funkcionalno daju isti rezultat, ali ilustriraju razliku u načinu pristupa okruženju iz koda.

- **`listenv1.c`** — ispisuje sve varijable okruženja procesa korištenjem trećeg argumenta funkcije `main` (`envp`):

```
int main(int argc, char *argv[], char *envp[])
```

Petlja iterira kroz polje `envp` sve dok ne naiđe na završni `NULL`. Funkcionalno je ekvivalentan UNIX naredbi `env`:

```
$ ./listenv1
SHELL=/bin/bash
HOME=/home/dkrst
PATH=/usr/local/bin:/usr/bin:/bin
LANG=hr_HR.UTF-8
...
```

- **`listenv2.c`** — ista funkcionalnost kao `listenv1.c`, ali umjesto trećeg argumenta `main`-a koristi vanjsku varijablu `environ`:

```
extern char **environ;

int main(int argc, char *argv[])
```

Funkcija `main` koristi standardni oblik s dva argumenta, a okruženju se pristupa preko globalne varijable. Ispis je identičan onom iz `listenv1`. Ovaj oblik je preporučen za stvarne programe iz razloga navedenih u prethodnom odjeljku.

U `listenv2.c` smo se uz to malo poigrali s pokazivačkom aritmetikom — umjesto da koristimo brojač i indeksiramo polje kao u `listenv1.c` (`envp[k]`), ovdje izravno pomičemo sam pokazivač `environ` izrazom `*environ++` u svakoj iteraciji petlje. Funkcionalno je rezultat potpuno isti — petlja redom prolazi kroz sve elemente niza dok ne naiđe na završni `NULL` — samo što sada uloga “indeksa” više nije zasebna varijabla `k`, nego sam pokazivač koji se pomiče po nizu.

5.0.3 Životni ciklus procesa

Kao što je opisano u uvodu, novi proces na UNIX-u nastaje isključivo sistemskim pozivom `fork` — njime započinje životni ciklus prikazan na slici 5.1. Pogledajmo sada `fork` detaljnije:

```
#include <unistd.h>

pid_t fork(void);
```

Povratna vrijednost: u roditeljskom procesu `fork()` vraća PID novostvorenog djeteta (pozitivna cjelobrojna vrijednost); u djetetu vraća `0`; u slučaju greške (npr. kada je dosegnut sistemski limit broja procesa), vraća `-1` (samo u parent procesu — nije stvoren child proces).

Sistemski poziv `fork()` nema argumenata i jednostavno kopira memorijsku sliku postojećeg procesa — procesa roditelja koji je pozvao `fork()`. Ova funkcija **poziva se jednom, a vraća dva puta**: u postojećem procesu (proces roditelj, *parent process*) i u novonastalom procesu (proces djetete, *child process*), koji je identična

kopija roditeljskog procesa — uključujući stanje stoga, hrpe i svih varijabli, ali i **brojača instrukcija** (engl. *program counter*, PC) — posebnog registra u UNIX procesu koji pokazuje na sljedeću instrukciju u strojnom kodu koja se treba izvršiti. Ovo efektivno znači da proces dijete nastavlja točno tamo gdje se proces roditelj nalazio u trenutku poziva funkcije `fork()` — kao da su se do tog trenutka oba procesa izvršavala na potpuno jednak način i nalaze se u identičnom stanju, iako je zapravo postojao samo jedan proces. Od tog trenutka dva procesa počinju živjeti odvojene živote — promjena bilo koje varijable u jednom od njih ne utječe na vrijednost varijabli u drugom procesu, jer se svaki proces izvršava u svom memorijskom prostoru.

U primjerima koji slijede koristit ćemo i dva pomoćna systemska poziva pomoću kojih proces može doznati vlastiti PID, kao i PID svog roditelja:

```
#include <unistd.h>

pid_t getpid(void);
pid_t getppid(void);
```

Povratna vrijednost: `getpid()` vraća PID procesa koji ga je pozvao, a `getppid()` PID njegovog roditelja. Ove funkcije ne mogu vratiti grešku.

- **nproc.c** — ilustrira upravo opisano svojstvo `fork()`-a: novi proces dobiva vlastitu kopiju memorijskog prostora roditelja, uključujući sve varijable na stogu i hrpi, te nastavlja izvršavanje točno tamo gdje se roditelj nalazio u trenutku poziva. Program u petlji poziva `fork()` tri puta. Nakon završetka petlje program ispisuje svoj jedinstveni process ID (PID) i proces ID procesa roditelja koristeći funkcije `getpid()` i `getppid()`. Ova informacija ispisuje se **8 puta** — što znači da izvršavanje petlje rezultira s ukupno 8 procesa, iako bi na prvu mogli pomisliti da je logičan broj procesa nakon izvršavanja petlje 4 (jedan izvorni i tri kopije, stvorene u tri iteracije petlje).

```
#include <stdio.h>
#include <unistd.h>

int main() {
    int k;
    for (k = 0; k < 3; k++)
        fork();
    printf("PID: %d\t PPID: %d\n", getpid(), getppid());

    return 0;
}
```

Ovo je posljedica upravo gore opisanog načina funkcioniranja systemskog poziva `fork()` — kopiranjem memorijske slike procesa kopira se i varijabla `k`, ali i brojač instrukcija (PC) koji pokazuje na sljedeću instrukciju koja se treba izvršiti. U svakoj iteraciji petlje iz jednog procesa nastaju dva potpuno identična procesa, koja se od te točke izvršavaju nezavisno — oba procesa nalaze se usred petlje i nastavljaju dalje kroz istu petlju. Razvoj kroz iteracije, uz broj procesa i vrijednost varijable `k` koja nakon `fork`-a postoji nezavisno u svakome:

prije petlje:	1 proces, k=0
iteracija k=0:	
fork() klonira postojeći proces	→ 2 procesa, k=0 u svakom
k++	→ 2 procesa, k=1 u svakom
iteracija k=1:	
fork() klonira sva 2 procesa	→ 4 procesa, k=1 u svakom
k++	→ 4 procesa, k=2 u svakom

```

iteracija k=2:
    fork() klonira sva 4 procesa      → 8 procesa, k=2 u svakom
    k++                               → 8 procesa, k=3 u svakom

uvjet k<3 neistinit u svima          → izlazak iz petlje
printf() izvršava se u svih 8

```

Stablo procesa nakon tri iteracije, s oznakama P0 (izvorni proces) do P7 (posljednje stvoreno dijete). Svaki `fork()` poziv udvostručuje skup aktivnih procesa — u svakoj iteraciji svi postojeći procesi paralelno kloniraju sebe, tako da nakon tri iteracije imamo 8 procesa:

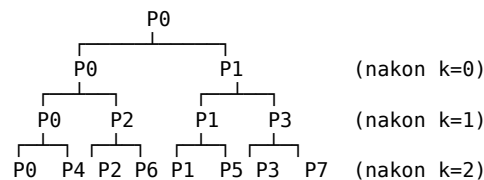
```

prije petlje:      P0

nakon k=0:         P0  P1
                   ↓   ↓      (svaki se u k=1 forka)
nakon k=1:         P0  P1  P2  P3
                   ↓   ↓   ↓   ↓      (svaki se u k=2 forka)
nakon k=2:         P0  P1  P2  P3  P4  P5  P6  P7

```

Odnos roditelj–dijete u stablu:



Formula za broj procesa nakon n uzastopnih `fork()` poziva je 2^n — svaki `fork()` udvostručuje broj aktivnih procesa. U ovom primjeru u petlju ulazi 1 proces, a iz nje izlazi 8. Iz ispisanih PPID-ova moguće je rekonstruirati cijelo stablo — svaki proces zna tko mu je neposredni roditelj.

- **novi.c** — korištenje povratne vrijednosti `fork()`-a za razlikovanje uloge roditelja i djeteta. Iako su oba procesa identične kopije jedan drugog, njih dva trebaju se u nastavku ponašati različito: roditelj obično nastavlja svoj prethodni posao, a dijete izvršava neki novi zadatak. Jedini mehanizam po kojem dva procesa mogu razaznati tko je tko jest upravo povratna vrijednost `fork()`-a — roditelj dobiva PID djeteta, a dijete dobiva 0. Program to iskorištava za granjanje logike — istim `printf`-om ispisuje različit tekst ovisno o tome tko ga izvršava:

```

#include <stdio.h>
#include <unistd.h>

int main() {
    int pid;

    pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    } else if (pid == 0) {
        printf("CHILD (%d)\t PID: %d\t PPID: %d\n",
            pid, getpid(), getppid());
    } else {
        printf("PARENT (%d)\t PID: %d\t PPID: %d\n",
            pid, getpid(), getppid());
    }

    return 0;
}

```

```

$ ./novi
PARENT (25017)    PID: 25016  PPID: 14567
CHILD (0)        PID: 25017  PPID: 25016

```

PID djeteta (25017) koji je roditelj dobio kao povratnu vrijednost `fork()`-a točno odgovara PID-u koji dijete prijavljuje pozivom `getpid()`; istovremeno dijete vidi PPID jednak PID-u roditelja, što potvrđuje odnos roditelj-dijete u procesnom stablu. Redoslijed ispisa između roditelja i djeteta nije određen — ovisi o raspoređivaču koji odlučuje koji će od dva spremna procesa dobiti procesor prvi.

5.0.3.1 Izlazni status procesa — sistemski poziv `wait`

Nakon stvaranja novog procesa sistemskim pozivom `fork`, jezgra zapisuje informacije o novonastalom procesu u **tablicu procesa** — istu onu u kojoj se čuva i informacija o svim otvorenim deskriptorima datoteka. Ovaj zapis u tablici procesa čuva se sve dok ima potrebe za tim — sve dok proces postoji, ali ponekad i duže. Sjetimo se da funkcija `main` ima povratnu vrijednost `int`. Vjerojatno ste se već zapitali čemu ova vrijednost služi i kome se uopće vraća, s obzirom da je `main` prva funkcija koja se poziva u novopokrenutom programu i ne postoji funkcija koja bi ju pozivala.

Upravo ova vrijednost osigurava mehanizam putem kojeg proces roditelj može saznati što se dogodilo s njegovim procesom djetetom: je li izvršio svoju zadaću ili je došlo do greške, i što je tu grešku uzrokovalo? Naime, poziv `return` u funkciji `main`, isto kao i poziv sistemskog poziva `exit` u bilo kojoj drugoj funkciji, prekidaju izvršavanje procesa i predaju jezgri **izlazni status procesa** — cjelobrojnu vrijednost od 0 do 255. Jezgra ovu vrijednost pohranjuje u slog u tablici procesa koji odgovara procesu koji je upravo završio. Zadatak procesa roditelja je da ovu vrijednost pročita i na temelju nje sazna sudbinu svog djeteta, a to može napraviti sistemskim pozivom `wait`:

```
#include <sys/wait.h>

pid_t wait(int *wstatus);
```

Povratna vrijednost: PID djeteta koje je završilo, ili -1 u slučaju greške (npr. proces nema djece koja bi mogla završiti). Izlazni status `child` procesa upisuje se u cjelobrojnu varijablu na koju pokazuje argument `wstatus`.

`wait()` blokira pozivajući proces sve dok jedno od njegove djece ne završi izvršavanje. Kad se to dogodi, jezgra puni varijablu na koju pokazuje `wstatus` informacijama o tome kako je dijete završilo: je li završilo normalno (npr. pozivom `return` ili `exit()`), je li prekinuto signalom, te koja je njegova povratna vrijednost odnosno koji je signal uzrokovao prekid. Ako pozivatelja zanima samo to da pričeka dijete, ali ne i sam status, kao argument se može proslijediti `NULL`.

Ovaj statusni broj **nije izravno** povratna vrijednost djeteta — riječ je o pakiranom cjelobrojnem polju u kojem su različite informacije o završetku procesa smještene u različite skupove bitova. Konkretno, izlazni status (vrijednost koju je dijete vratilo iz `main`-a ili predalo `exit()`-u, u rasponu **0 do 255**) zapisuje se u bitove 8–15 statusne riječi. Da bi se ta vrijednost izvukla, koristi se makro `WEXITSTATUS()` iz iste zaglavne datoteke `<sys/wait.h>`.

Posljedica je da bezuvjetan ispis statusnog broja kao cijele vrijednosti ne daje povratnu vrijednost djeteta, nego njezino “pomaknuto” prikazivanje: ako dijete vrati 5, `raw status bit` će `5 << 8 = 1280`; ako vrati 255, `raw status bit` će 65280.

- **ret_stat.c** — minimalan primjer koji ilustrira upravo opisani odnos između `raw statusnog` broja i `WEXITSTATUS`-a. Roditelj pozivom `fork()` stvara dijete koje

jednostavno vrati cjelobrojnu vrijednost zadanu kao argument naredbenog retka:

```
} else if (pid == 0) {
    return atoi(argv[1]); // child: vrati zadanu vrijednost
} else {
    wait(&rv);
    printf("CHILD EXIT STATUS: %d\n", rv);
    // printf("CHILD EXIT STATUS: %d\n", WEXITSTATUS(rv));
}
```

Pokrenimo program s nekoliko različitih vrijednosti u rasponu 0–255 (sve dopuštene povratne vrijednosti procesa):

```
$ ./ret_stat 0
CHILD EXIT STATUS: 0
$ ./ret_stat 1
CHILD EXIT STATUS: 256
$ ./ret_stat 5
CHILD EXIT STATUS: 1280
$ ./ret_stat 255
CHILD EXIT STATUS: 65280
```

Samo prva vrijednost (0) odgovara onome što je dijete uistinu vratilo. Ostali brojevi su pomnoženi sa 256 — odnosno, gledano kao niz bitova, izvorna vrijednost je pomaknuta osam mjesta ulijevo. To su upravo bitovi 8–15 statusne riječi, gdje jezgra po POSIX konvenciji pohranjuje izlazni status. Ostali bitovi statusnog broja rezervirani su za druge informacije (signal koji je prekinuo proces, oznaka core-dumpa itd.) — te ćemo informacije iskoristiti u sekciji **Pokretanje novog programa**.

Da bismo dobili izvornu povratnu vrijednost, koristi se makro `WEXITSTATUS()`. U primjeru `ret_stat.c` zakomentirajte prvi `printf` i otkomentirajte drugi:

```
// printf("CHILD EXIT STATUS: %d\n", rv);
printf("CHILD EXIT STATUS: %d\n", WEXITSTATUS(rv));
```

Nakon ponovnog prevođenja, ispis je upravo onakav kakav očekujemo:

```
$ ./ret_stat 5
CHILD EXIT STATUS: 5
$ ./ret_stat 255
CHILD EXIT STATUS: 255
```

Makro `WEXITSTATUS(s)` ne radi ništa “magično” — interno samo izvuče bitove 8–15 iz statusnog broja, što je ekvivalentno izrazu `(s >> 8) & 0xFF`. Korištenje makroa, međutim, čini kod jasnijim, neovisnim o konkretnoj implementaciji statusne riječi i prenosivim na različite UNIX sustave.

5.0.3.2 Zombi procesi

Slijed događaja koji smo upravo opisali — proces završi pozivom `exit()` ili `return` iz funkcije `main`, a njegov proces roditelj njegov izlazni status preuzme pozivom `wait()` — otvara jedno suptilno pitanje: **kada točno jezgra može osloboditi zapis o procesu u tablici procesa?** Resursi koje je proces koristio (memorija, otvoreni file deskriptori, itd.) oslobađaju se odmah po prekidu izvršavanja, ali to se ne odnosi na sam zapis u tablici procesa: u njemu je pohranjen izlazni status koji jezgra još mora moći predati procesu roditelju kad on (u nekom budućem trenutku) pozove `wait()`. Ako bi taj zapis bio izbrisan u trenutku prekida izvršavanja procesa djeteta, poziv `wait()` ne bi imao odakle pročitati izlazni status.

Rješenje koje UNIX koristi je sljedeće: kad dijete završi, jezgra **zadržava** njegov zapis u tablici procesa, ali u posebnom stanju u kojem proces više ne troši resurse

— postoji samo zapis o tome koji PID je imao i koji je njegov izlazni status. U uredno napisanim programima, ovaj period je vrlo kratak. Problem nastaje kad roditelj iz nekog razloga ne pozove `wait()` za svoju djecu. U ovom slučaju, imamo proces koji više nije živ, ali je još uvijek tu negdje i jezgra mora čuvati odgovarajući slog u tablici procesa. Štoviše, procesa u ovom stanju ne možemo se ni prisilno riješiti, npr. slanjem signala `KILL` koji bezuvjetno ubija proces kojem je poslan — jer ionako više nije živ. U UNIX terminologiji ovakvi procesi nazivaju se (prigodno) **zombi procesima** (engl. *zombie process*).

Zombi proces moguće je vidjeti naredbom `ps`: u stupcu `STAT` (status) prikazana je oznaka **Z**, a uz ime procesa stoji oznaka `<defunct>`. Problem nastaje ako se ovakvi procesi počnu gomilati: svaki od njih troši jedan slog u tablici procesa i koristi jedan PID — iako više nije aktivan. Kako tablica procesa ima ograničenu veličinu, kontinuirano stvaranje zombi procesa može dovesti do iscrpljenja sistemskih resursa — sustav neće moći pokrenuti nove procese sve dok netko ne “pokupi” zombije.

Stoga je dobra programerska praksa voditi računa o procesima koje ste stvorili: obavezno se pobrinite da po završetku izvršavanja vašeg `child` procesa pozovete `wait`, makar s argumentom `NULL` ukoliko vas njegov izlazni status ne zanima — ovo je jezgri potrebno da bi znala da ne mora više čuvati izlazni status procesa koji je završio s izvršavanjem.

Što se događa ako roditelj završi s izvršavanjem prije nego pozove `wait()`, ili jednostavno prekine izvršavanje dok je `child` još uvijek aktivan? Tu na scenu stupa mehanizam koji ćemo upoznati u sljedećem poglavlju, kod **osirotjelih procesa** (engl. *orphan processes*). Jezgra preuzima ulogu roditelja svim procesima čiji proces roditelj je završio izvršavanje — uključujući i zombije — i prebacuje ih procesu `init` (PID 1, ili u modernim Linux distribucijama `systemd`). `init` putem signala dobiva obavijest kada bilo koji od njegovih `child` procesa završi s izvršavanjem, uključujući i “usvojene” procese koji su ostali bez svojih roditelja. Svaki put kada primi takav signal, `init` poziva `wait()` te “čisti” iz tablice procesa sve zombije i pušta jezgru da oslobodi njihove zapise.

5.0.4 Pokretanje novog programa

Proces i program su dva povezana ali različita koncepta. **Proces** je aktivna instanca izvršavanja — entitet koji ima svoj PID, memorijski prostor i resurse. **Program** je statičan zapis: strojni kod pohranjen u izvršnoj datoteci, niz instrukcija koje proces izvršava.

Na UNIX-u proces “kuhanja” započinje s pripremom: stvaranjem novog procesa i pripadajućeg memorijskog prostora u kojem će se proces izvršavati — pozivom funkcije `fork()`, kao što smo vidjeli u prethodnoj sekciji. Nakon toga krećemo s kuhanjem po odgovarajućem receptu — programu zapisanom u izvršnoj datoteci — korištenjem sistemskog poziva `exec`, koji kao argument uzima izvršnu datoteku koju želimo pokrenuti. Valja napomenuti da je `exec` zapravo **obitelj funkcija**, koje imaju istu funkcionalnost (učitavanje programa-recepta), a razlikuju se u načinu kako se zadaju argumenti naredbenog retka i put do izvršne datoteke:

```
#include <unistd.h>

int execl(const char *path, const char *arg0, ... /*, NULL */);
int execlp(const char *file, const char *arg0, ... /*, NULL */);
int execle(const char *path, const char *arg0, ... /*, NULL, char *const envp[] */);
```

```
int execl(const char *path, char *const argv[]);
int execlp(const char *file, char *const argv[]);
int execve(const char *path, char *const argv[], char *const envp[]);
```

Razlike među varijantama kodirane su u sufiksima iza exec:

- **l** (*list*) — argumenti nove naredbe prenose se kao redom navedena **lista** pojedinačnih stringova, završena NULL-om (npr. `execlp("ls", "ls", "-al", NULL)`). Sjetimo se da polje pokazivača `argv` u funkciji `main` uvijek završava NULL pokazivačem — upravo zato i u pozivu `execl` NULL moramo navesti kao posljednji argument.
- **v** (*vector*) — argumenti se prenose kao **polje pokazivača** na stringove, završeno NULL-om (npr. `execvp("ls", argv)`). Prikladno kad broj argumenata nije poznat u trenutku pisanja koda, odnosno kada se argumenti dobivaju u istom obliku (kao u `argv`) od samog pozivatelja.
- **p** (*path*) — izvršna datoteka pronalazi se pretragom direktorija iz varijable okoline `PATH`, pa se umjesto pune putanje može navesti samo ime (npr. "ls" umjesto `"/bin/ls"`). Varijante bez `p` očekuju punu putanju.
- **e** (*environment*) — poziv prima dodatni posljednji argument `envp`, niz pokazivača na stringove "IME=vrijednost" završen NULL-om, čime se novoj naredbi može eksplicitno zadati drugačije okruženje od onog koje ima trenutni proces. Varijante bez `e` ne mijenjaju trenutno okruženje procesa.

Svih šest funkcija u pozadini završava u sistemskom pozivu `execve()` — ostalih pet je omotač koji pogodnije pakira argumente. Zajedničko svim varijantama je da **ne stvaraju novi proces**: PID ostaje isti, mijenja se samo kod koji se izvršava. Ako `exec` uspije, poziv se nikad ne vraća — izvorni kod programa zamijenjen je novim, učitanim iz izvršne datoteke, varijable na stogu i hrpi se brišu, a brojač instrukcija (PC) postavlja se na prvu instrukciju u novom programu. Vraćanje iz `exec`-a znači grešku (npr. izvršna datoteka nije pronađena ili nemamo pravo izvršavanja).

- **myls.c** — prvi primjer funkcija iz `exec` obitelji: poziv `execlp()` zamjenjuje trenutni programski kod procesa kodom nekog drugog programa — u ovom slučaju UNIX naredbe `ls`. Za razliku od `fork()`, koji stvara novi proces, `exec()` **ne stvara novi proces** — PID ostaje isti, promijeni se samo programski kod koji se izvršava.

```
#include <stdio.h>
#include <unistd.h>

int main(int argc, char **argv) {
    if (argc < 2)
        execlp("ls", "ls", "-al", NULL);
    else
        execlp("ls", "ls", "-al", argv[1], NULL);

    perror("execlp");
    return 1;
}
```

Bez argumenata, program pokreće `ls -al` nad trenutnim direktorijem; s jednim argumentom, pokreće `ls -al <argument>` kako bi nam ispisao sadržaj zadanog direktorija. Primijetite da su **prva dva argumenta poziva `execlp` identična**: prvi ("ls") je ime izvršne datoteke koju treba naći i pokrenuti, a drugi ("ls") je ono što će nova naredba dobiti kao svoj `argv[0]`. Podsjetimo se da po konvenciji svaka naredba u svom `argv[0]` očekuje vlastito ime (upravo iz tog razloga poruke o korištenju koriste `argv[0]`). Teoretski je moguće proslijediti i različito ime — tako bi program vidio da je pokrenut pod drugim imenom — ali u normalnoj

uporabi oba se argumenta podudaraju.

Ako `execvp()` uspije, poziv se nikad ne vraća — zato će se `perror("execvp")` izvršiti samo u slučaju greške (npr. naredba `ls` nije pronađena u `PATH-u`). Stoga nije potrebno provjeravati povratnu vrijednost funkcije `execvp`. Za razliku od poziva `fork()`, koji se poziva jednom a vraća dva puta, `execvp` (kao i ostale funkcije iz obitelji `exec`) ne vraća se nikad — osim u slučaju greške.

```
$ ./mys
ukupno 24
drwxr-xr-x 2 dkrst users 4096 Apr 23 15:10 .
drwxr-xr-x 8 dkrst users 4096 Apr 23 15:09 ..
-rwxr-xr-x 1 dkrst users 8760 Apr 23 15:10 mys
-rw-r--r-- 1 dkrst users 256 Apr 23 15:09 mys.c
...
```

- **pokreni.c** — uopćena verzija prethodnog primjera: program prima proizvoljnu naredbu s argumentima kao svoje argumente naredbenog retka i pokreće je pozivom `execvp()`.

```
#include <stdio.h>
#include <unistd.h>

int main(int argc, char **argv) {
    if (argc < 2) {
        printf("koristenje: %s <naredba> [arg1] [arg2] ... \n", argv[0]);
        return 0;
    }

    execvp(argv[1], &argv[1]);
    perror("execvp");

    return 1;
}
```

Kako funkcija `main` već dobiva argumente u obliku polja pokazivača (`argv`), upravo takav oblik očekuje i `execvp` — dovoljno je proslijediti pokazivač na `argv[1]`. Izraz `&argv[1]` je pokazivačka aritmetika — `argv` je polje pokazivača na stringove, a `&argv[1]` daje adresu **drugog elementa** tog polja, tj. pokazivač na pod-polje koje počinje od `argv[1]` i ide dalje. Funkcija `execvp` će to pod-polje tretirati kao svoje vlastito `argv`: `argv[1]` iz perspektive `pokreni`-ja postaje `argv[0]` nove naredbe (njezino ime), `argv[2]` postaje `argv[1]`, i tako redom. Ovako jednostavnim obrascem dobivamo zametak vlastite ljuske — program koji može pokrenuti bilo koju drugu UNIX naredbu.

Preporuka je pokušati razne kombinacije i promotriti ponašanje:

```
$ ./pokreni ls
$ ./pokreni ls -al
$ ./pokreni ls -al /root
$ ./pokreni asdhasd
```

Prva dva poziva rade očekivano. Treći poziv (`ls -al /root`) ovisi o tome je li korisnik `root` — ako nije, `ls` će ispisati poruku o greški jer nema pravo pristupa direktoriju `/root`. Zanimljivo je odgovoriti na pitanje: **tko ispisuje tu poruku?** U ovom slučaju to nije `pokreni` — `pokreni` je svoj posao obavio kad je pozvao `execvp`, koji je uspješno zamijenio njegov kod kodom naredbe `ls`. `pokreni` u doslovnom smislu više ne postoji kao program u tom procesu; poruku o grešci ispisuje sama naredba `ls`, nad svojim otvorenim standardnim izlazom za greške. U četvrtom pozivu (`./pokreni asdhasd`), poruku o grešci ispisuje `pokreni` sam — `execvp` je neuspješno pokušao pronaći izvršnu datoteku `asdhasd`, vratio se u `pokreni`, i idući redak koda je upravo `perror("execvp")`:

```
$ ./pokreni asdhasd
execvp: No such file or directory
```

Razlika je suptilna ali ključna: kad `exec` uspije, program koji je pozvao `exec` prestaje postojati; kad `exec` ne uspije, izvršavanje se nastavlja s naredbom iza njega.

Zabavan primjer rekurzivne zamjene:

```
$ ./pokreni ./pokreni ./pokreni ls -al
```

Jedan isti proces redom mijenja svoj sadržaj tri puta: prvi `pokreni` kod je zamijenjen drugim pozivom `pokreni`, taj pozivom trećeg `pokreni-a`, i on na kraju pozivom `ls -al`. PID ostaje isti kroz sve četiri metamorfoze, a ono što korisnik vidi kao rezultat — ispis sadržaja direktorija — potpuno je isto kao da smo `ls -al` pozvali direktno. Ova rekurzija ilustrira što `exec` zapravo jest: ne pokretanje novog procesa, nego **zamjena tijela postojećeg procesa drugim kodom**.

- **pokreni2.c** — pokretanje programa u novom procesu: **kombinacija `fork()` i `exec()`**.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char **argv) {
    int pid, s;

    if (argc < 2) {
        printf("koristenje: %s <naredba> [arg1] [arg2] ...\n", argv[0]);
        return 0;
    }

    pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    } else if (pid == 0) { /* dijete */
        execvp(argv[1], &argv[1]);
        perror("execvp");
        return 127;
    } else { /* roditelj */
        wait(&s);
        if (WIFEXITED(s))
            printf("Normalan izlaz, izlazni status: %d\n",
                WEXITSTATUS(s));
        else if (WIFSIGNALED(s))
            printf("Prekid signalom, signal: %d\n", WTERMSIG(s));
    }

    return 0;
}
```

U prethodnom primjeru `pokreni` je nakon `execvp()` prestao postojati kao `pokreni` — njegov je kod zamijenjen kodom pokrenute naredbe, pa nakon završetka naredbe nema ničeg na što bi se vratilo. U praksi često želimo zadržati roditelja živim kako bi nastavio izvršavati svoju primarnu zadaću, ili čak pokrenuo novu naredbu nakon što se program pokrenut s `exec` u child procesu završi. Ovaj obrazac slijedi UNIX ljuska svaki put kada korisnik utipka naredbu: ljuska pozivom `fork` stvori novi proces (kopiju same sebe), potom u tom novom procesu pozivom `exec` pokrene traženu naredbu, a u roditeljskom procesu čeka njen završetak pozivom `wait`. Ljuska ovo radi u petlji — nakon svakog `wait-a` korisniku daje novu ljuskinu oznaku (*prompt*) i mogućnost da unese sljedeću naredbu.

U prethodnom odjeljku, kod primjera `ret_stat.c`, već smo se susreli s makroom

WEXITSTATUS() koji iz statusne riječi izvlači izlazni status djeteta. U pokreni2 koristimo dva dodatna makroa iz <sys/wait.h> koji nam pomažu razlikovati **na koji način** je dijete završilo:

```
WIFEXITED(status) // istina ako je dijete završilo normalno (return / exit)
WIFSIGNALED(status) // istina ako je dijete prekinuto signalom
WTERMSIG(status) // broj signala koji je prekinuo dijete
```

Ovi makroi ispituju različite skupove bitova iste statusne riječi: WIFEXITED provjerava bit koji označava “uredan izlazak”, WIFSIGNALED ispituje bit “prekinut signalom”, a WTERMSIG izvlači broj signala iz dijela rezerviranog za signale (bitovi 0–6). Za bilo koji statusni broj točno jedan od WIFEXITED i WIFSIGNALED vratit će istinu — proces ne može istovremeno završiti i normalno i biti prekinut signalom. pokreni2 na temelju toga ispisuje različitu poruku:

```
if (WIFEXITED(s))
    printf("Normaln izlaz, izlazni status: %d\n", WEXITSTATUS(s));
else if (WIFSIGNALED(s))
    printf("Prekid signalom, signal: %d\n", WTERMSIG(s));
```

Povratna vrijednost 127, koju pokreni2 koristi u grani kad execvp() ne uspije, je UNIX konvencija za “naredba nije pronađena”.

Isprobajmo s različitim naredbama koje **uredno završavaju**:

```
$ ./pokreni2 ls
argumenti argumenti.c listenv1 listenv1.c ...
Normaln izlaz, izlazni status: 0

$ ./pokreni2 asdasdasd
execvp: No such file or directory
Normaln izlaz, izlazni status: 127

$ ./pokreni2 ls sdfsd sdf
ls: cannot access 'sdfsd sdf': No such file or directory
Normaln izlaz, izlazni status: 2
```

Zašto se izlazni status razlikuje među tri slučaja? U prvom slučaju ls je uspješno obavio svoj posao i exit-ao s 0 — to je UNIX konvencija “sve je prošlo dobro”. U drugom slučaju execvp u djetetu nije uspio (naredba asdasdasd ne postoji u PATH-u), pa se dijete vratilo na idući redak koda — perror i zatim return 127. U trećem slučaju ls se uspješno pokrenuo, ali je pri otvaranju datoteke sdfsd sdf utvrdio da ne postoji, ispisao poruku o grešci na standardni izlaz za greške i vratio 2 — što je specifična konvencija same naredbe ls za “ozbiljnija greška”. Sva tri scenarija su uredan izlazak; razlikuje se samo vrijednost koju je dijete vratilo.

- **ptr_err.c** — primjer koji namjerno izaziva grešku u rukovanju memorijom. Sav posao programa svodi se na nekoliko redaka koda:

```
int main() {
    int *val = (int*)99;
    *val = 3;
    return 0;
}
```

Pokazivač val postavlja se na potpuno proizvoljnu numeričku vrijednost (99) koja sigurno nije valjana adresa u memorijskom prostoru procesa, a zatim se kroz njega pokušava upisati. Jezgra to detektira i procesu šalje signal SIGSEGV (signal broj 11). Ako ptr_err pokrenemo korištenjem našeg pokreni2, dobit ćemo sljedeći rezultat:

```
$ ./pokreni2 ./ptr_err
```

Prekid signalom, signal: 11

Za razliku od svih prethodnih primjera, WIFEXITED ovdje vraća laž — dijete nije završilo normalno — pa se izvršava `else if (WIFSIGNALED(s))` grana, a `WTERMSIG(s)` vraća 11, što je broj signala SIGSEGV.

Ako `ptr_err` pokušate pokrenuti direktno iz ljuske, dobit ćete ispis `Segmentation fault` — ljuska na isti način kao i naš primjer sazna razlog prekida izvršavanja naredbe koju smo zadali, provjeri koji je signal uzrok prekida te, temeljem te informacije, ispisuje odgovarajuću poruku.

- **preusmjeri.c** — povezuje koncepte iz ovog poglavlja s mehanizmom preusmjeravanja ulaza/izlaza obrađenim u poglavlju o ulazno-izlaznim operacijama. Program prima dva ili više argumenata: prvi je ime datoteke u koju želimo preusmjeriti standardni izlaz, a ostatak je naredba koju želimo pokrenuti.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <fcntl.h>

int main(int argc, char **argv) {
    int fd;

    if (argc < 3) {
        printf("koristenje: %s <datoteka> <naredba> [arg1] [arg2] ...\n",
            argv[0]);
        return 0;
    }

    fd = creat(argv[1], (mode_t)0644);
    if (fd < 0) {
        perror("creat");
        return 1;
    }

    if (dup2(fd, STDOUT_FILENO) < 0) {
        perror("dup2");
        return 1;
    }
    if (fd != STDOUT_FILENO)
        close(fd);

    execvp(argv[2], &argv[2]);
    perror("execvp");

    return 1;
}
```

Redoslijed koraka je:

1. Pozivom `creat()` otvori se izlazna datoteka s pravima 0644.
2. Pozivom `dup2(fd, STDOUT_FILENO)` file deskriptor datoteke se duplicira na deskriptor 1 — upravo onaj koji proces koristi za standardni izlaz. Od tog trenutka svaki `write` ili `printf` kojemu je određeno `STDOUT_FILENO` efektivno piše u otvorenu datoteku.
3. Stari deskriptor `fd` više nije potreban pa se zatvara, uz uobičajenu zaštitnu provjeru (`fd != STDOUT_FILENO`) za rijedak slučaj kad je `creat()` slučajno vratio baš 1.
4. Pozivom `execvp()` program se zamjenjuje traženom naredbom. Ključno zapažanje: **otvoreni file deskriptori nasljeđuju se kroz `exec()`** — nova naredba naslijedi preusmjerenje na datoteku, iako ona sama nema pojma o tome.

Dobivamo funkcionalni ekvivalent ljuškinog operatora >:

```
$ ./preusmjeri izlaz.txt ls -la
$ cat izlaz.txt
ukupno 24
drwxr-xr-x 2 dkrst users 4096 Apr 23 15:10 .
...
```

Na istom principu radi i sama UNIX ljuška kad obrađuje naredba > datoteka: prije exec-a naredbe, ljuška otvori datoteku i pozivom dup2 podmetne je na deskriptor 1.

5.0.5 Ograničavanje resursa (setrlimit)

- **limitraj.c** i **potrosac.c** — ilustriraju sistemski poziv `setrlimit()`, kojim proces može postaviti ograničenje na različite resurse koje mu sustav dopušta koristiti. `limitraj` postavlja `RLIMIT_CPU` na 3 sekunde (i tvrdo i meko ograničenje), što znači da sljedeći proces smije potrošiti najviše 3 sekunde procesorskog vremena; nakon toga jezgra mu šalje signal koji ga prekida. Nakon postavljanja limita, `limitraj` pozivom `execvp()` pokreće naredbu zadanu argumentima. Kako se limiti resursa — kao i file deskriptori — **nasljeđuju kroz `exec()`**, pokrenuta naredba nasljeđuje i ograničenje. Ovo je osnovna ideja iza mehanizma ugrađene naredbe `ulimit` u ljušci, kao i alata poput `timeout` i različitih “sandbox” okruženja.

```
/* limitraj.c */
#include <stdio.h>
#include <unistd.h>
#include <sys/resource.h>

int main(int argc, char **argv) {
    struct rlimit l;
    if (argc < 2) {
        printf("koristenje: %s <naredba> [arg1] [arg2] ...\n", argv[0]);
        return 0;
    }

    l.rlim_cur = 3;
    l.rlim_max = 3;
    setrlimit(RLIMIT_CPU, &l);

    execvp(argv[1], &argv[1]);
    perror("execvp");

    return 1;
}
```

Program `potrosac.c` je pratilac koji služi samo za demonstraciju djelovanja `limitraj`-a — sastoji se od jedne jedine beskonačne petlje `while(1);`, bez ijednog sistemskog poziva ili I/O operacije.

```
/* potrosac.c */
int main() {
    while (1);
    return 0;
}
```

Kad se `potrosac` pokrene samostalno, trošio bi procesor neograničeno dugo. Pokrenut kroz `limitraj`, nasljeđuje njegov CPU limit:

```
$ ./limitraj ./potrosac
Killed
```

Ljuška nakon prekida procesa signalom ispisuje `Killed` (na Linux sustavima s

engleskom lokalizacijom; na nekim drugim sustavima ispis može biti CPU time limit exceeded ili izostati sasvim). Bitno je zapaziti koliko dugo program traje — oko **tri sekunde**, a ne više.

Procesorsko i stvarno vrijeme. Limit RLIMIT_CPU broji isključivo **procesorsko vrijeme** (*CPU time*) — ukupno vrijeme tijekom kojeg je proces zaista izvršavao instrukcije na nekom procesoru. To nije isto što i **stvarno vrijeme** (*wall-clock time* ili *real time*) koje mjeri koliko je proteklo vremena od pokretanja do kraja procesa, uključujući sve periode kad je proces bio pauziran od strane raspoređivača, kad je čekao na I/O, ili kad je sustav izvršavao druge procese. Ova dva vremena razlikuju se po tome da **procesorsko vrijeme nikad nije duže od stvarnog** — proces ne može utrošiti više procesorskih sekundi nego što je sekundi proteklo u stvarnosti. Najčešće je procesorsko vrijeme **kraće** od stvarnog: čim proces počne čekati na nešto (pritisak tipke, čitanje s diska, poruku preko mreže), stvarni sat nastavlja kucati, a procesorski stoji jer proces ne radi. U slučaju potrosac-a razlika između ta dva vremena je minimalna jer proces zaista cijelo vrijeme samo okreće procesor; kod realnih programa koji komuniciraju s korisnikom ili diskom razlika zna biti vrlo velika. Razlika je vidljiva i korisniku — UNIX naredba time ispisuje sva tri vremena (stvarno/*real*, korisničko procesorsko/*user*, sistemsko procesorsko/*sys*):

```
$ time ./limitraj ./potrosac
Killed

real    0m3.012s
user    0m3.000s
sys     0m0.001s
```

5.0.6 Osirotjeli procesi (engl. *orphan processes*)

- **noparent.c** — detaljniji primjer fork() mehanizma koji prikazuje što se dogodi kada **roditelj završi prije djeteta**. Program stvara stablo od tri procesa — PARENT 1, CHILD 1 i CHILD 2 — pri čemu CHILD 2 (unuk po odnosu prema PARENT 1) namjerno spava duže od svog direktnog roditelja (CHILD 1). Zbog toga CHILD 1 završi dok njegovo dijete još radi, a time CHILD 2 postaje **osirotjeli proces** (*orphan*). U takvoj situaciji jezgra automatski preuzima ulogu novog roditelja — tradicionalno je to proces init s PID-om 1.

```
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int pid;

    pid = fork();
    if (pid == 0) { /* CHILD 1 */
        pid = fork();
        if (pid == 0) { /* CHILD 2 */
            printf("CHILD 2:\t PID: %d\t PPID: %d\n",
                getpid(), getppid());
            sleep(3);
            printf("CHILD 2:\t PID: %d\t PPID: %d\n",
                getpid(), getppid());
        } else { /* CHILD 1 - PARENT 2 */
            sleep(1);
            printf("CHILD 1:\t PID: %d\t PPID: %d\n",
                getpid(), getppid());
            exit(0);
        }
    } else { /* PARENT 1 */
```

```

pid = wait(NULL);
printf("PARENT 1:\t PID: %d\t izlazi: %d\n",
      getpid(), pid);
printf("PARENT 1:\t PID: %d\t PPID: %d\n",
      getpid(), getppid());
sleep(5);
}

return 0;
}

```

Očekivani ispis s konkretnim PID-ovima (skraćeno; SH je PID ljuske iz koje je program pokrenut, npr. 14567; PARENT 1 dobiva PID 25016, CHILD 1 25017, CHILD 2 25018):

```

CHILD 2:  PID: 25018  PPID: 25017
CHILD 1:  PID: 25017  PPID: 25016
PARENT 1: PID: 25016  izlazi: 25017
PARENT 1: PID: 25016  PPID: 14567
CHILD 2:  PID: 25018  PPID: 1

```

U prvom ispisu CHILD 2 vidi svog pravog roditelja CHILD 1 (PPID = 25017); nakon tri sekunde spavanja, njegov drugi ispis prijavljuje **PPID = 1** — u međuvremenu je CHILD 1 završio, a jezgra je posvojenje prenijela na init.

Napomena o modernim sustavima. Na suvremenim Linux distribucijama koje koriste systemd i korisničke sesije (systemd --user), posljednji PPID često neće biti 1 nego PID korisničkog systemd procesa. Razlog je mehanizam **subreapera** uveden u Linux jezgri 3.4 (2012.): pozivom prctl(PR_SET_CHILD_SUBREAPER, 1) proces se može registrirati kao “zamjenski init” za sve svoje potomke. Kad sirotan nastane, jezgra traži najbližeg pretka-subreapera umjesto da ga automatski pošalje na PID 1. Na desktop Linuxu sa systemd-om upravo je systemd --user takav subreaper, pa se tamo vidi PPID jednak njegovom PID-u. Konkretni iznos može se provjeriti naredbom `ps -p <ppid> -o pid,ppid,comm`. Klasično pravilo (*sirotan* → *PID 1*) i dalje vrijedi u odsutnosti subreapera — primjerice pri pokretanju iz SSH sesije na minimalnom serveru.

5.1 Prevođenje

Direktorij dolazi s priloženim Makefile-om koji prati iste konvencije kao i Makefile datoteke u direktorijima P02-Osnove_programiranja/ i P03-Ulazno_izlazne_operacije/ (varijable CC, CFLAGS, LDFLAGS, TARGETS; implicitno pravilo .c.o; pravila default, all, clean). Detaljan opis strukture i korake gradnje Makefilea vidjeti u ../P02-Osnove_programiranja/README.md.

Tipična uporaba:

```

make          # gradi zadani cilj (novi)
make all      # gradi sve primjere
make pokreni2 # gradi samo zadani primjer
make clean    # briše izvršne i objektne datoteke

```

5.2 Bibliografija

[1] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2013.

[2] L. Budin, M. Golub, D. Jakobović, and L. Jelenković, *Operacijski sustavi*, 3. izd. Zagreb, Hrvatska: Element, 2013.

Poglavlje 6

Signali

U ovom poglavlju upoznat ćemo **signale** — UNIX-ov primarni mehanizam za asinkronu komunikaciju s procesom. Signal je kratka poruka koju jezgra šalje procesu kao obavijest da se dogodio neki događaj: korisnik je pritisnuo Ctrl+C, proces je pokušao pristupiti nedopuštenoj memorijskoj adresi, istekao je timer, neki drugi proces je eksplicitno zatražio prekid, i tako dalje. Iz perspektive procesa, signal može stići u **bilo kojem trenutku** između dvije strojne instrukcije — proces nema mogućnost predvidjeti kada će se to dogoditi.

Proces na primljeni signal može reagirati na nekoliko načina: prepustiti zadanu reakciju jezgri (što za većinu signala znači prekid procesa), eksplicitno ignorirati signal, ili registrirati vlastitu funkciju — **rukovatelj signala** (engl. *signal handler*) — koja će se izvršiti svaki put kad takav signal stigne. U ovom poglavlju kroz nekoliko primjera ilustriramo kako registriramo rukovatelj signala, kako se signali hvataju, kako se koriste za komunikaciju među procesima i o čemu je potrebno voditi računa pri pisanju rukovatelja. Signali su tema iznimno bogata specifičnim slučajevima i suptilnim detaljima — čitatelj koji se njima profesionalno bavi gotovo sigurno će se vraćati na *Advanced Programming in the UNIX Environment*, Stevens & Rago [1], koja signalima posvećuje cijelo opsežno poglavlje s pažljivom raspravom o sigurnosti rukovatelja, semantici višestrukih signala i razlikama među UNIX inačicama.

6.1 Najčešći signali

Signali su, gledano iz perspektive operacijskog sustava i programa, jednostavno **mali cjelobrojni identifikatori** — svaki signal ima svoj broj. Zbog čitljivosti i prenosivosti koda, u programima nikad ne baratamo izravno tim brojevima, nego koristimo **simboličke konstante** definirane u standardnoj zaglavnoj datoteci `<signal.h>` (npr. `SIGINT`, `SIGTERM`, `SIGKILL`...).

POSIX standard definira tridesetak signala, a u praksi se najčešće susrećemo sa skupinom njih desetak. Sljedeća tablica daje pregled onih s kojima ćemo raditi u ovom poglavlju i u tipičnim programima:

Signal	Broj	Predefinirana akcija	Značenje
SIGHUP	1	prekid procesa	terminal je zatvoren ili je sesija prekinuta; često se koristi i kao "ponovno učitaj konfiguraciju"
SIGINT	2	prekid procesa	korisnik je pritisnuo Ctrl+C u terminalu; programi ga često hvataju kako bi od korisnika zatražili potvrdu izlaska — "čisti" izlaz
SIGQUIT	3	prekid procesa + core file	korisnik je pritisnuo Ctrl+ — kao SIGINT, ali generira i core file
SIGILL	4	prekid procesa + core file	proces je pokušao izvršiti nevažeću strojnu instrukciju
SIGABRT	6	prekid procesa + core file	proces je sam sebe prekinuo pozivom abort() (npr. zbog assert greške)
SIGFPE	8	prekid procesa + core file	aritmetička greška (dijeljenje s nulom, prelijevanje)
SIGKILL	9	prekid procesa	bezuvjetni prekid — ne može se uhvatiti niti ignorirati ; jezgra ga koristi kao zadnju mjeru kad se proces ogлуši o SIGTERM
SIGSEGV	11	prekid procesa + core file	proces je pokušao pristupiti memorijskoj adresi kojoj nema pravo pristupa (<i>segmentation fault</i>)
SIGPIPE	13	prekid procesa	pokušaj pisanja u cijevovod (<i>pipe</i>) ili socket čiji je drugi kraj zatvoren
SIGALRM	14	prekid procesa	istekao je timer postavljen pozivom alarm()
SIGTERM	15	prekid procesa	pristojan zahtjev za prekid; programi ga često koriste za uredno spremanje stanja i "čisti" izlaz. Ako se program ogлуši o SIGTERM, jezgra ga može bezuvjetno prekinuti signalom SIGKILL
SIGUSR1	10	prekid procesa	korisnički signal br. 1 — bez unaprijed definirane semantike, slobodan za vlastite svrhe

Signal	Broj	Predefinirana akcija	Značenje
SIGUSR2	12	prekid procesa	korisnički signal br. 2 — bez unaprijed definirane semantike, slobodan za vlastite svrhe
SIGCHLD	17	ignorira se	dijete procesa je promijenilo stanje (završilo, zaustavljeno itd.)
SIGSTOP	19	zaustavi proces	bezuvjetno zaustavljanje — ne može se uhvatiti niti ignorirati
SIGCONT	18	nastavi proces	nastavi izvršavanje zaustavljenog procesa
SIGTSTP	20	zaustavi proces	korisnik je pritisnuo Ctrl+Z u terminalu — svojevrsna “pauza” za pokrenuti program (zaustavlja ga privremeno; nastavak slanjem SIGCONT)
SIGXCPU	24	prekid procesa + core file	premašen je CPU limit postavljen <code>setrlimit()</code> -om (vidi Limiti)
SIGXFSZ	25	prekid procesa + core file	premašen je file size limit postavljen <code>setrlimit()</code> -om (vidi Limiti)

Brojevi signala u tablici odgovaraju POSIX-u za osnovne signale i podudaraju se s vrijednostima na većini modernih UNIX sustava. Ipak, najsigurniji pristup je u kodu uvijek koristiti **simbolička imena** (SIGINT, SIGTERM...) umjesto numeričkih vrijednosti — time se izbjegavaju zabune i osigurava prenosivost koda na sustave gdje neki manje uobičajeni signali mogu imati drukčije brojeve. Brojeve navodimo samo radi reference (npr. uz ispis “Prekid signalom, signal: 11” iz `pokreni2-a`). Potpuni popis dostupan je u priručniku: `man 7 signal`.

Posebnu pažnju zaslužuju **SIGKILL** i **SIGSTOP** — to su jedina dva signala koja se ne mogu uhvatiti niti ignorirati. Razlog je praktičan: bez ova dva signala sustav ne bi imao apsolutni mehanizam za bezuvjetan prekid ili zaustavljanje “neposlušnog” procesa. Svaki drugi signal proces može uhvatiti i odlučiti kako reagirati — uključujući i SIGTERM, što neki programi iskorištavaju za uredno spremanje stanja prije izlaska.

Što je core file? Za neke signale predefinirana akcija nije samo prekid procesa, nego i stvaranje *core file*-a — datoteke koja sadrži snimku kompletnog memorijskog prostora procesa u trenutku kad je signal primljen (varijable na stogu, hrpi, registri procesora, otvoreni file deskriptori i drugi metapodatci). Datoteka se obično zove `core` ili `core.<pid>` i nastaje u trenutnom radnom direktoriju procesa. Ovo ponašanje vezano je uz signale koji uglavnom znače da smo u programu *nešto zabrljali* — SIGSEGV (pristup nevažećoj memoriji), SIGFPE (aritmetička greška), SIGILL (nevažeća instrukcija), SIGABRT (eksplicitan prekid pomoću `abort()`). Core file omogućuje *post-mortem* analizu: alatima poput gdb programer može učitati

core file zajedno s izvršnom datotekom, vidjeti gdje je proces bio u trenutku pada, koje su vrijednosti varijabli imale, kakav je bio stack trace itd. Naravno, ako za to imamo volje i znanja — u suprotnom core file se može jednostavno obrisati.

6.1.1 Hvatanje signala

Iako teoretska priča o hvatanju signala koji nas obavještavaju o nastupanju asinhronih događaja možda zvuči složeno, na sljedećim primjerima pokazat ćemo da se implementacija ovog naizgled složenog koncepta može realizirati u svega nekoliko linija koda.

- **potvrđi.c** — minimalan primjer hvatanja signala koji uvodi sve ključne koncepte rada s njima. Program registrira vlastiti rukovatelj za signal SIGINT (signal koji se procesu šalje kad korisnik u terminalu pritisne Ctrl+C); kad korisnik pritisne Ctrl+C prvi put, program ne završi, nego ispiše poruku da treba pritisnuti Ctrl+C još jednom ako se zaista želi izaći. Ovo je obrazac kakav vidamo u stvarnim alatima (npr. htop, vim) i u skriptama koje se ne smiju nehotice prekinuti.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int brojac = 0;

void uhvati(int signum) {
    brojac++;
}

int main() {
    signal(SIGINT, uhvati);

    while (brojac < 2) {
        pause();
        if (brojac == 1)
            printf("Pritisnite ponovo CTRL - C ukoliko zelite izaci\n");
    }

    return 0;
}
```

Za registraciju rukovatelja koristi se sistemski poziv `signal`:

```
#include <signal.h>

void (*signal(int signum, void (*handler)(int)))(int);
```

Na prvi pogled deklaracija djeluje zastrašujuće, ali zapravo je riječ o funkciji koja prima dva argumenta i vraća pokazivač:

- **signum** — broj signala koji želimo hvatati (npr. SIGINT)
- **handler** — rukovatelj signala; pokazivač na funkciju koja će biti pozvana kada signal `signum` stigne. Rukovatelj signala kao argument prima jednu cjelobrojnu vrijednost (`int`) koja označava broj signala koji je pozvao rukovatelj (isti rukovatelj može se pozivati za više različitih signala). Rukovatelj signala ima povratni tip `void` — nema povratnu vrijednost.
- **Povratna vrijednost** — pokazivač na *prethodno* registriranu funkciju, ili `SIG_ERR` u slučaju greške.

Ovdje se prvi put susrećemo s **pokazivačem na funkciju** kao argumentom drugoj funkciji. Ideja je jednostavna: kako svaka funkcija, baš kao i svaka varijabla, u izvršnoj datoteci ima svoju adresu u memoriji, tako i njezino

ime u kodu (bez zagrada) jednostavno označava upravo tu adresu. Drugim riječima, **ime funkcije je njezin pokazivač**. U našem programu funkcija uhvati definirana je tako da prima `int` i ne vraća ništa — što točno odgovara tipu argumenta handler — pa je dovoljno predati samo njezino ime: `signal(SIGINT, uhvati)`.

Mehanizam je sljedeći. Funkcija `signal(SIGINT, uhvati)` jezgri kaže: *“od ovog trenutka, kad god mom procesu stigne signal SIGINT, ne primjenjuj zadanu reakciju (prekid procesa) nego pozovi funkciju uhvati.”* Kad korisnik pritisne `Ctrl+C`, jezgra prekine trenutno izvršavanje glavnog programa točno tamo gdje je bilo, pozove `uhvati`, a kad se ona vrati, glavni program nastavlja izvršavanje od mjesta gdje je bio prekinut. Sav posao koji rukovatelj napravi — u ovom slučaju jednostavno povećanje varijable `brojac` — vidljiv je glavnom programu kroz globalnu varijablu, što je standardni način “razgovora” između `main`-a i rukovatelja.

Glavni program koristi sistemski poziv `pause()`, koji uspava proces sve dok ne stigne bilo koji signal. Kad signal stigne i njegov rukovatelj završi izvršavanje, `pause()` se vrati i petlja nastavlja: provjeri trenutnu vrijednost `brojac`-a, ako je on 1 ispiše uputu, a u sljedećem prolazu petlje opet uđe u `pause()` čekajući novi signal. Tek kad `brojac` dosegne 2, izlazi iz petlje i program uredno završava.

Pokrenimo program i isprobajmo:

```
$ ./potvrdi
^C
Pritisnite ponovo CTRL - C ukoliko zelite izaci
^C
Korisnik je potvrdio odlazak - kraj programa!
$
```

Da bismo bolje uočili koliki je doprinos rukovatelja, predlažemo i sljedeći eksperiment: zakomentirajte redak `signal(SIGINT, uhvati);`, ponovno prevedite program i pokrenite ga. Pritisak `Ctrl+C` sada će dovesti do trenutnog prekida programa — nećete vidjeti ni poruku iz petlje, ni završnu poruku. Bez registriranog rukovatelja primjenjuje se **zadana reakcija** jezgre na `SIGINT`, a ona za ovaj signal podrazumijeva prekid procesa.

Funkcionalno je program gotovo trivijalan, ali pokriva nekoliko važnih koncepata vrijednih da se odmah istaknu:

- **Rukovatelj je vrlo kratak** — samo inkrementira brojač i ne pokušava ništa složenije od toga (npr. nema poziva `printf`-a). Ovo nije slučajno: rukovatelj signala se izvršava u posebnom kontekstu — može prekinuti glavni program u doslovno bilo kojem trenutku, uključujući i sredinu poziva drugih funkcija. Preporuka je iz rukovatelja pozivati samo tzv. **async-signal-safe** funkcije — funkcije za koje POSIX standard jamči da ih je sigurno pozvati iz signal handlera. U taj skup ne spada i `printf` — njegovo korištenje u rukovatelju može u rijetkim slučajevima dovesti do iznenađujućih grešaka. Detaljnije ćemo o ovome u kasnijem primjeru, ali već sad uvodimo dobru praksu: rukovatelj postavlja zastavicu, glavni program reagira.
- **Komunikacija kroz globalnu varijablu** — rukovatelj i glavni program “razgovaraju” kroz `brojac`. Strogo gledano, takve dijeljene varijable trebale bi biti deklarirane s tipom `volatile sig_atomic_t` umjesto običnog `int`-a:
 - * Ključna riječ `volatile` govori prevoditelju da vrijednost varijable može biti promijenjena “iza leđa” glavnog programa (od strane rukovatelja), pa

optimizator ne smije njezinu vrijednost cache-irati u registru kroz iteracije petlje.

- * Tip `sig_atomic_t` jamči da se čitanje i pisanje varijable obavlja u jednoj nedjeljivoj operaciji — rukovatelj ne može uhvatiti glavni program “u sredini” upisa.

Na većini modernih arhitektura (uključujući x86) u praksi će raditi i obični `int`.

Napomena. U nekim povijesnim verzijama UNIX-a rukovatelj signala resetirao bi se svaki put kada bi signal bio primljen, pa ga je bilo potrebno ponovno registrirati. U modernim verzijama UNIX-a ovo ponašanje gotovo sigurno nećete susresti.

6.1.2 Vlastiti alarm — `SIGALRM` i `alarm()`

- **`alarm_clock.c`** — primjer u kojem proces zakaže slanje signala samome sebi. Izvor signala najčešće dolazi izvan samog procesa, npr. ukoliko korisnik pritisne `Ctrl+C`, ili eksplicitno zatražimo slanje signala pozivom `kill` iz ljuske. UNIX, međutim, omogućuje i da proces zatraži od jezgre da mu nakon određenog broja sekundi pošalje signal `SIGALRM`. Tu funkcionalnost pruža sistemski poziv `alarm()`:

```
#include <unistd.h>

unsigned int alarm(unsigned int seconds);
```

Ovaj poziv jezgri kaže: “za točno *seconds* sekundi pošalji mi `SIGALRM`.” Ako je već postojao prethodni alarm, on se zamjenjuje novim, a poziv vraća broj sekundi koje su preostale do isporuke prethodnog alarma. Pozivom `alarm(0)` aktivni alarm se otkazuje. Bitno je primijetiti da je `alarm` **jednokrat** — ako želimo periodičko “tikanje” svakih *N* sekundi, novi alarm moramo zakazati svaki put iznova.

Upravo to radi naš primjer:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int brojac = 0;

void alrm_handler(int signum) {
    brojac++;
    alarm(1);
}

int main() {
    signal(SIGALRM, alrm_handler);
    alarm(1);

    while (brojac < 5) {
        pause();
        printf("tik %d\n", brojac);
    }

    printf("kraj!\n");
    return 0;
}
```

Glavni program registrira rukovatelj za `SIGALRM`, postavlja prvi alarm na jednu sekundu i ulazi u petlju koja čeka pet “tikanja”. Rukovatelj je vrlo kratak — samo poveća brojac i odmah zakaže sljedeći alarm — pa se ovaj ciklus odvija jednom u sekundi. Ispis “tik *N*” obavlja sam glavni program nakon što se vrati iz `pause()`-a, sljedeći obrazac koji smo uveli kod prethodnog primjera (*rukovatelj postavlja*

zastavicu, glavni program reagira).

Pokrenimo program:

```
$ ./alarm_clock
tik 1
tik 2
tik 3
tik 4
tik 5
kraj!
```

Cijeli ispis traje točno pet sekundi.

Primijetite da smo rukovatelj imenovali `alrm_handler` umjesto naprosto uhvati, kao u prethodnom primjeru. Pri imenovanju funkcija dobra je praksa odabrati intuitivna imena koja odmah daju naslutiti što funkcija radi. U slučaju rukovatelja signala nije loše u ime ugraditi i ime samog signala — npr. `alrm_handler` za `SIGALRM`, ili `int_handler` za `SIGINT`, što bi u prethodnom primjeru bilo prikladnije ime od općenitog uhvati. Ovo je naravno samo sugestija autora; čitatelj ima punu slobodu imenovati svoje funkcije kako god mu odgovara.

- **stoperica.c** — primjer koji kombinira tehnike iz prethodna dva: koristi `SIGALRM` za odbrojavanje sekundi, a `SIGINT` (Ctrl+C) za zaustavljanje. Funkcionalno se ponaša kao jednostavna stoperica: nakon pokretanja svake sekunde ispiše “tik N”, a kad korisnik pritisne Ctrl+C ispiše ukupno proteklo vrijeme i uredno završi.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int brojac = 0;
int broji = 1;

void alrm_handler(int signum) {
    brojac++;
    alarm(1);
}

void int_handler(int signum) {
    broji = 0;
}

int main() {
    signal(SIGALRM, alrm_handler);
    signal(SIGINT, int_handler);

    printf("Stoperica pokrenuta -- pritisnite Ctrl+C za zaustavljanje.\n");
    alarm(1);

    while (broji) {
        pause();
        if (broji)
            printf("tik %d\n", brojac);
    }

    printf("Proteklo: %d sekundi\n", brojac);
    return 0;
}
```

Program ima dvije globalne varijable kroz koje glavni program i rukovatelj “razgovaraju”: `brojac` koji broji koliko je sekundi prošlo i `broji` koji upravlja izvršavanjem glavne petlje. Registrirana su dva rukovatelja s različitim ulogama: `alrm_handler` poveća brojač i ponovno zakaže alarm (točno kao u prethodnom primjeru), dok `int_handler` postavi `broji = 0` čime signalizira glavnoj petlji da treba završiti.

Glavna petlja `while (broji)` u svakom prolazu pasivno čeka signal pomoću `pause()`. Kad signal stigne i odgovarajući rukovatelj završi, `pause()` se vrati i petlja nastavlja. Slijedi mali ali važan detalj: prije ispisa “tik N” provjeravamo da je `broji` i dalje istinit. Razlog je u tome što se `pause()` vraća za **bilo koji** primljeni signal, uključujući i `SIGINT`. Bez te provjere, posljednji ispis bio bi suvišan “tik N” prije završne poruke “Proteklo: N sekundi”. Ovaj obrazac — *nakon pause(), ponovo provjeri stanje pa tek onda reagiraj* — tipičan je kad više signala utječe na istu petlju.

Pokrenimo stopericu i nakon nekoliko sekundi pritisnimo `Ctrl+C`:

```
$ ./stoperica
Stoperica pokrenuta -- pritisnite Ctrl+C za zaustavljanje.
tik 1
tik 2
tik 3
^CProteklo: 3 sekundi
```

Imenovanjem rukovatelja prema signalu na koji odgovaraju (`alarm_handler` i `int_handler`), kod postaje samodokumentirajuć — već iz naziva je jasno koji rukovatelj reagira na koji signal, što je posebno korisno kad u programu imamo više signala koje hvatamo.

- **stoperica2.c** — funkcionalno identičan prethodnom programu, ali strukturno drukčiji: koristi **jedan zajednički rukovatelj** za oba signala umjesto dva odvojena. Na ovom primjeru ilustrirano je korištenje argumenta `signum` koji rukovatelj prima — broj signala koji je upravo isporučen procesu.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int brojac = 0;
int broji = 1;

void rukovatelj(int signum) {
    switch (signum) {
        case SIGALRM:
            brojac++;
            alarm(1);
            break;
        case SIGINT:
            broji = 0;
            break;
    }
}

int main() {
    signal(SIGALRM, rukovatelj);
    signal(SIGINT, rukovatelj);

    printf("Stoperica pokrenuta -- pritisnite Ctrl+C za zaustavljanje.\n");
    alarm(1);

    while (broji) {
        pause();
        if (broji)
            printf("tik %d\n", brojac);
    }

    printf("Proteklo: %d sekundi\n", brojac);
    return 0;
}
```

Do sada smo argument rukovatelja, iako ga je deklaracija zahtijevala, jednostavno ignorirali. U ovom primjeru njegova svrha postaje jasna: kad jedna

funkcija obrađuje više različitih signala, jezgra joj kao argument predaje broj signala koji je upravo isporučen, pa funkcija na temelju toga može odlučiti što dalje. `switch` je u takvim slučajevima prirodniji od lanca `if/else if` jer se lakše proširuje dodavanjem novih `case` grana za nove signale.

Pokretanje i ispis su identični prethodnom primjeru:

```
$ ./stoperica2
Stoperica pokrenuta -- pritisnite Ctrl+C za zaustavljanje.
tik 1
tik 2
tik 3
^CProteklo: 3 sekundi
```

Koji je pristup bolji — razdvojeni rukovatelji kao u `stoperica.c` ili zajednički kao u `stoperica2.c`? Stvar je ukusa i konteksta. Razdvojeni rukovatelji su pregledniji kad je logika za svaki signal značajno različita i kad u svakom rukovatelju ima više od nekoliko redaka koda. Zajednički rukovatelj je prikladan kad signali dijele zajedničke resurse ili pomoćne varijable, ili kad očekujemo da će se broj obrađivanih signala s vremenom povećavati. U realnim programima često su zastupljena oba pristupa istovremeno: na primjer, jedan zajednički rukovatelj za sve signale koji označavaju zahtjev za prekidom (`SIGINT`, `SIGTERM`, `SIGHUP`) i poseban rukovatelj za vremenske signale.

6.1.3 Korištenje signala za komunikaciju između procesa

U dosadašnjim primjerima signali su dolazili iz dva izvora: izvana, od korisnika preko `Ctrl+C`, ili iznutra, kad ih je proces sam sebi zakazao pozivom `alarm()`. UNIX, međutim, dopušta i da jedan proces eksplicitno **pošalje signal drugom procesu** — što čini signale jednim od najjednostavnijih oblika međuprocenjske komunikacije (engl. *Inter-Process Communication*, IPC). Za to služi sistemski poziv `kill`:

```
#include <signal.h>
#include <sys/types.h>

int kill(pid_t pid, int sig);
```

Povratna vrijednost: 0 u slučaju uspjeha; -1 u slučaju greške (npr. ne postoji proces s navedenim PID-om, ili nemamo dovoljnu razinu ovlasti za slanje signala tom procesu).

Ime `kill` je povijesno — prvotno je sistemski poziv služio isključivo za prekid drugog procesa pomoću `SIGKILL` ili `SIGTERM`. Vremenom se njegova uloga proširila i danas se koristi za slanje **bilo kojeg** signala, ali ime je ostalo. Istog imena je i istoimena ugrađena naredba ljuske (`kill -<sig> <pid>`, npr. `kill -USR1 12345`), kojom korisnik može poslati proizvoljni signal nekom procesu iz terminala.

- **razgovor.c** — primjer u kojem dva procesa komuniciraju isključivo putem signala. Roditelj `fork`-a dijete, zatim mu u petlji svake sekunde šalje `SIGUSR1`. Dijete na svaki primljeni signal ispiše poruku i broji koliko ih je primilo. Nakon pet “dojava”, roditelj djetetu pošalje `SIGTERM` da uredno završi, pa pričekava njegov završetak pozivom `wait()`.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int radi = 1;
```

```

int prosao = 0;

void usr1_handler(int signum) {
    prosao++;
}

void term_handler(int signum) {
    radi = 0;
}

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        /* CHILD */
        signal(SIGUSR1, usr1_handler);
        signal(SIGTERM, term_handler);

        while (radi) {
            pause();
            if (radi)
                printf("[child] primio sam SIGUSR1, prolaz %d\n", prosao);
        }

        printf("[child] završavam, ukupno prolaza: %d\n", prosao);
        return 0;
    }

    /* PARENT */
    printf("[parent] pokrenuo dijete s PID-om %d\n", pid);

    for (int k = 0; k < 5; k++) {
        sleep(1);
        printf("[parent] šaljem SIGUSR1 djetetu\n");
        kill(pid, SIGUSR1);
    }

    sleep(1);
    printf("[parent] šaljem SIGTERM djetetu\n");
    kill(pid, SIGTERM);

    wait(NULL);
    printf("[parent] dijete je završilo, izlazim\n");
    return 0;
}

```

Struktura programa razdvaja se odmah nakon `fork()`-a u dvije grane. **Dijete** registrira dva rukovatelja: `usr1_handler` koji broji primljene `SIGUSR1` signale, i `term_handler` koji postavlja `radi = 0` čime signalizira glavnoj petlji da treba završiti. Petlja je standardna `while (radi) pause()` konstrukcija s provjerom `if (radi)` prije `printf`-a, kao u `stoperica.c`-u, da se izbjegne suvišan ispis nakon `SIGTERM`-a.

Bitno je primijetiti da se rukovatelji **registriraju samo u dječjoj grani**, a ne i u roditelju. Razlog je u tome što roditelj ove signale uopće ne prima — on je **pošiljatelj**, a ne primatelj. Roditelj eksplicitno šalje `SIGUSR1` i `SIGTERM` djetetu pozivom `kill(pid, ...)`, a sam ne treba reagirati na te signale. Da smo registraciju rukovatelja stavili **prije** `fork()`-a, oba bi je procesa naslijedila — pa bi i roditelj imao registrirane rukovatelje koji se nikad ne bi izvršili, što ne bi bila greška, ali je nepotrebno. Stavljanjem registracije unutar dječje grane jasno odvajamo uloge: **dijete reagira, roditelj šalje**.

Roditelj zna PID djeteta jer mu ga je `fork()` vratio, pa može pozivati `kill(pid, SIGUSR1)` da mu šalje signale u željenim trenucima. Po završetku radne petlje šalje `SIGTERM`, a zatim `wait(NULL)` čeka da dijete uredno završi (čime se ono prestaje voditi kao zombi proces u tablici procesa, kao što smo objasnili u prethodnom poglavlju).

Pokrenimo program:

```
$ ./razgovor
[parent] pokrenuo dijete s PID-om 12345
[parent] saljem SIGUSR1 djetetu
[child] primio sam SIGUSR1, prolaz 1
[parent] saljem SIGUSR1 djetetu
[child] primio sam SIGUSR1, prolaz 2
[parent] saljem SIGUSR1 djetetu
[child] primio sam SIGUSR1, prolaz 3
[parent] saljem SIGUSR1 djetetu
[child] primio sam SIGUSR1, prolaz 4
[parent] saljem SIGUSR1 djetetu
[child] primio sam SIGUSR1, prolaz 5
[parent] saljem SIGTERM djetetu
[child] završavam, ukupno prolaza: 5
[parent] dijete je završilo, izlazim
```

Iz ispisa je vidljiv pravilan redoslijed događaja: roditelj svake sekunde signalizira djetetu, dijete trenutno reagira, i tako pet puta zaredom; konačno `SIGTERM` zatvara petlju u djetetu, a roditelj nakon `wait()`-a izlazi.

Bitno je naglasiti da `kill(pid, sig)` samo “isporuči” signal djetetu — ne čeka da rukovatelj završi izvršavanje, a nema nikakve garancije da će signal biti obrađen prije nego roditelj nastavi sa svojim radom. Ovo je primjer **asinkronog** mehanizma: pošiljalac i primatelj nisu usklađeni vremenski. Za pravu sinkroniziranu komunikaciju (gdje pošiljalac čeka primateljev odgovor) postoje drugi UNIX IPC mehanizmi — cijevi, redovi poruka, dijeljena memorija — kojima ćemo se baviti u kasnijim poglavljima.

Značenje signala.

Većina UNIX signala ima predefinirano značenje. Međutim, kako programer može napisati vlastiti rukovatelj za većinu signala, time praktički može promijeniti njihovo predefinirano značenje — npr. iskoristiti `SIGSEGV` (kojim nas jezgra upozorava da smo s pokazivačem izašli izvan svog memorijskog prostora) za razmjenu poruka između roditelja i djeteta, kao u našem primjeru. Iako ovo nije nikakav problem implementirati, ideja je vrlo loša: što ako stvarno u programu napravimo grešku u rukovanju memorijom i jezgra nas pokuša na to upozoriti, a mi taj signal protumačimo kao poruku od drugog procesa?

Ideja da signalima pridijelimo posve drugačije značenje otprilike je jednako dobra kao da se studenti koji slušaju kolegij *Programiranje za UNIX* na FESB-u dogovore da crveno svjetlo na semaforu za njih znači “kreni”, a zeleno “stani”: dok su sami na cesti, sustav može funkcionirati, ali ako se pojavi bilo koji drugi vozač, posljedice su potencijalno katastrofalne.

Iz istog razloga treba poštivati predefinirana značenja signala, a za vlastite komunikacijske protokole između grupe procesa koristiti slobodne signale `SIGUSR1` i `SIGUSR2`, kao u našem primjeru.

6.2 Sistemski poziv sigaction

U svim dosadašnjim primjerima rukovatelje smo registrirali korištenjem funkcije `signal()`. Iako je `signal()` jednostavan i intuitivan, njegova povijest puna je nedosljednosti između UNIX implementacija. U starijim System V verzijama rukovatelj se nakon prve isporuke signala automatski poništavao (resetirao na zadanu reakciju), pa je sljedeća instanca istog signala — koja stigne prije nego rukovatelj stigne ponovno registrirati samog sebe — uzrokovala prekid procesa. Na BSD sustavima rukovatelj je ostajao registriran. Drugi izvor razlika bilo je ponašanje **prekinutih sistemskih poziva**: kad signal stigne procesu koji je u sporom sistemskom pozivu (`read`, `accept`, `pause`, `wait`...), BSD je sistemski poziv automatski nastavljao, dok je System V vraćao grešku s `errno = EINTR`. Treći problem je “atomarnost” registracije — bilo je moguće da signal stigne usred zamjene rukovatelja i pozove pogrešnu funkciju.

Da bi se ove razlike uklonile i ponašanje precizno definiralo, POSIX uvodi noviju funkciju `sigaction()`, koja u potpunosti zamjenjuje `signal()` i nudi dodatne mogućnosti: blokiranje drugih signala tijekom obrade rukovatelja, kontrolu nad nastavkom prekinutih sistemskih poziva, te alternativnu formu rukovatelja koja prima detaljne informacije o izvoru signala.

```
#include <signal.h>

int sigaction(int signum, const struct sigaction *act,
              struct sigaction *oldact);

struct sigaction {
    void (*sa_handler)(int);
    void (*sa_sigaction)(int, siginfo_t *, void *);
    sigset_t sa_mask;
    int sa_flags;
    void (*sa_restorer)(void); /* obsolete, ne dirati */
};
```

Povratna vrijednost: 0 u slučaju uspjeha; -1 u slučaju greške (npr. nevažeći broj signala, ili pokušaj postavljanja rukovatelja za SIGKILL odnosno SIGSTOP).

Argumenti:

- **signum** — broj signala koji se hvata (kao i kod `signal()`-a).
- **act** — pokazivač na strukturu koja opisuje novu akciju za taj signal.
- **oldact** — pokazivač na strukturu u koju će se upisati **prethodna** akcija; korisno ako želimo privremeno preuzeti signal pa kasnije vratiti staro ponašanje. Ako nas prethodna akcija ne zanima, kao treći argument možemo koristiti NULL pokazivač.

Najvažnija polja strukture `struct sigaction`:

- **sa_handler** — pokazivač na funkciju koja će biti pozvana kad signal stigne, isto kao drugi argument funkcije `signal()`. Umjesto pokazivača na funkciju, polju se mogu dodijeliti i posebne vrijednosti SIG_DFL (vрати zadanu reakciju jezgre) ili SIG_IGN (ignoriraj signal).
- **sa_mask** — skup signala koje treba blokirati **dok se rukovatelj izvršava**. Po POSIX defaultu, dok se izvršava rukovatelj za neki signal S, taj isti signal S automatski je blokiran — ako tijekom obrade S-a stigne nova instanca, ona čeka u redu (engl. *pending*) i isporučuje se tek nakon što rukovatelj završi. Polje `sa_mask` proširuje ovo blokiranje: tu možemo navesti **dodatne** signale koji će biti blokirani tijekom izvršavanja rukovatelja. Ovo je iznimno korisno kad više signala dijele isti rukovatelj ili manipuliraju istim podacima — postavljanjem cross-maske između

njih sprječavamo da jedan rukovatelj prekine drugog usred kritične sekcije.

- **sa_flags** — bit-maska zastavica koje fino podešavaju ponašanje. Najčešće korištene su:
 - SA_RESTART — automatski nastavi prekinute sistemske pozive (BSD ponašanje); bez ove zastavice, sistemski poziv prekinut signalom vraća grešku EINTR.
 - SA_NOCLDWAIT — kad se postavi za SIGCHLD, jezgra automatski uklanja zombije bez potrebe za wait()-om.
 - SA_SIGINFO — proširuje rukovatelj dodatnim informacijama o signalu (vidi opis polja sa_sigaction niže).

Polje sa_sigaction je alternativa polju sa_handler — koristi se uz zastavicu SA_SIGINFO i daje rukovatelju prošireni potpis void f(int signum, siginfo_t *info, void *context). Drugi argument info je struktura s detaljnim podacima o signalu (PID procesa pošiljatelja, UID njegovog vlasnika, razlog isporuke...). Treći argument context daje pristup CPU registrima u trenutku prekida (rijetko se koristi izravno). Dva polja, sa_handler i sa_sigaction, na nekim su sustavima implementirana kao union — pa je dobra praksa koristiti samo jedno od njih i nikad oba istovremeno. Polje sa_restorer je interni Linux mehanizam i ne smije se eksplicitno postavljati. Zato je dobra praksa cijelu strukturu prije korištenja inicijalizirati memset-om, čime osiguravamo da neiskorištena polja imaju nulte vrijednosti.

Atomarna zamjena rukovatelja. Bitno je razumjeti da sigaction() postavlja novu akciju atomski: nije moguće da signal stigne usred izmjene, pronade pola-staro-pola-novo stanje, i pozove pogrešan rukovatelj. To je važna garancija koju System V signal() nije pružao.

Postupak za registraciju rukovatelja sigaction()-om uvijek slijedi isti obrazac:

1. Deklarirati struct sigaction sa,
2. Nulirati strukturu pomoću memset(&sa, 0, sizeof(sa)),
3. Postaviti sa.sa_handler na pokazivač na rukovatelj,
4. Inicijalizirati masku pomoću sigemptyset(&sa.sa_mask) (ili dodati signale koje treba blokirati),
5. Postaviti sa.sa_flags na željenu kombinaciju zastavica (najčešće 0),
6. Pozvati sigaction(signum, &sa, NULL).

Za sav novi kod preporučuje se sigaction() umjesto signal(). U ostatku ovog poglavlja ipak smo se služili objema funkcijama: signal() u jednostavnijim primjerima gdje smo htjeli zadržati pregledan kod, a sigaction() ondje gdje su nam potrebne njegove dodatne mogućnosti.

- **potvrđi2.c** — funkcionalno identičan primjeru potvrdi.c s početka poglavlja, ali rukovatelj se registrira pomoću sigaction()-a:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <string.h>

int brojac = 0;

void int_handler(int signum) {
    brojac++;
}

int main() {
    struct sigaction sa;
```

```

memset(&sa, 0, sizeof(sa));
sa.sa_handler = int_handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;

sigaction(SIGINT, &sa, NULL);

while (brojac < 2) {
    pause();
    if (brojac == 1)
        printf("Pritisnite ponovo CTRL - C ukoliko zelite izaci\n");
}

printf("Korisnik je potvrdio izlazak - kraj programa!\n");
return 0;
}

```

Glavna petlja, rukovatelj i logika su nepromijenjeni u odnosu na `potvrdi.c` — razlika je samo u načinu registracije. Iako je ovo nešto više koda od jednostavnog `signal(SIGINT, int_handler)` poziva, zauzvrat dobivamo precizno definirano ponašanje koje vrijedi na svim POSIX sustavima.

Pokretanje i ispis su identični prethodnom primjeru:

```

$ ./potvrdi2
^C
Pritisnite ponovo CTRL - C ukoliko zelite izaci
^C
Korisnik je potvrdio izlazak - kraj programa!
$

```

6.3 Blokiranje i ignoriranje signala

Do sada smo signale uvijek “hvatali” — registrirali rukovatelja koji bi se pozvao kad signal stigne. UNIX, međutim, nudi i druge načine da odredimo što se događa kad signal stigne procesu. Dva najvažnija su **blokiranje** i **ignoriranje**, i između njih postoji važna razlika.

Blokiranje ne uklanja signal — samo odgađa njegovu isporuku. Svaki proces ima takozvanu **masku signala** (engl. *signal mask*): skup signala koji su trenutno blokirani. Maska signala je još jedan od podataka o procesu koje jezgra čuva u tablici procesa — uz sve drugo što smo dosad spominjali (PID, PPID, tablica otvorenih datoteka, izlazni status, limiti resursa). Postupno se vidi koliko različitih informacija jezgra mora držati za svaki proces, sve uredno upakirano u jedan slog tablice procesa. Kad signal stigne procesu, a taj signal je u njegovoj maski, jezgra ga pohrani u **red čekanja** (engl. *pending*). Tamo ostaje sve dok proces ne ukloni signal iz maske — tek tada se isporučuje, i tek tada se pokreće rukovatelj (ili zadana akcija). Blokiranje koristimo kad imamo dio koda koji ne smije biti prekinut signalom, ali ne želimo trajno izgubiti signal.

Ignoriranje je kvalitativno drugačije: signal se isporučuje, ali se odmah odbacuje. Proces ne saznaje da je signal stigao i nikad ne reagira na njega. Za ignoriranje koristimo specijalnu vrijednost `SIG_IGN` koju postavimo kao “rukovatelj” pomoću `sigaction()`-a (ili `signal()`-a). Ignoriranje koristimo kad nas određeni signal jednostavno ne zanima.

6.3.1 Sistemski poziv `sigprocmask`

Maska signala glavnog programa mijenja se sistemskim pozivom `sigprocmask`:

```
#include <signal.h>

int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

Povratna vrijednost: 0 u slučaju uspjeha; -1 u slučaju greške (npr. nevažeća vrijednost how).

Argumenti:

- **how** — što se radi s argumentom set:
 - SIG_BLOCK — dodaj signale iz set trenutnoj maski,
 - SIG_UNBLOCK — ukloni signale iz set iz trenutne maske,
 - SIG_SETMASK — postavi masku točno na set.
- **set** — skup signala kojim mijenjamo masku; ako je NULL, maska se ne mijenja (poziv samo dohvaća staru u oldset).
- **oldset** — pokazivač u koji se upisuje **prethodna** maska, korisno za kasnije vraćanje stare vrijednosti; ako nas to ne zanima, predaje se NULL.

Argumente tipa sigset_t koriste i drugi pozivi koje smo već vidjeli (polje sa_mask u struct sigaction). To je apstraktni skup signala kojim ne baratamo izravno, nego pomoću pomoćnih funkcija:

```
int sigemptyset(sigset_t *set);           /* prazan skup */
int sigfillset(sigset_t *set);           /* svi signali */
int sigaddset(sigset_t *set, int signum); /* dodaj signal u skup */
int sigdelset(sigset_t *set, int signum); /* makni signal iz skupa */
int sigismember(const sigset_t *set, int signum); /* je li u skupu? */
```

Funkcija sigemptyset inicijalizira set kao prazan skup, na koji potom novim signalima dodajemo pojedinačno pomoću sigaddset. Ovaj pristup je prikladan kada treba sastaviti masku s relativno malim brojem signala — npr. samo SIGINT u našem maska.c primjeru. Obrnuto, kada nam treba maska koja sadrži većinu signala, prirodnije je krenuti od potpunog skupa i ukloniti samo one koje ne želimo: sigfillset inicijalizira set kao skup koji sadrži sve signale, a sigdelset zatim uklanja pojedinačne signale po potrebi. Funkcija sigismember provjerava sadrži li set zadani signal — koristimo je npr. nakon poziva sigpending da bismo provjerili je li određeni signal u redu čekanja.

Postoji i poziv sigpending(sigset_t *set) koji u set upisuje signale koji su upravo **u redu čekanja** — stigli su procesu, ali su blokirani pa se još nisu isporučili. Korisno kad prije skidanja maske želimo provjeriti hoće li nešto biti isporučeno.

- **maska.c** — kratak primjer blokiranja SIGINT-a tijekom kratke “kritične sekcije”. Program registrira jednostavan rukovatelj koji ispiše poruku, blokira SIGINT, “radi” pet sekundi (sleep), pa skida masku.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <string.h>

void int_handler(int signum) {
    printf("SIGINT obraden\n");
}

int main() {
    struct sigaction sa;
    sigset_t blok, prethodna;

    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = int_handler;
```

```

sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
sigaction(SIGINT, &sa, NULL);

sigemptyset(&blok);
sigaddset(&blok, SIGINT);

sigprocmask(SIG_BLOCK, &blok, &prethodna);
sleep(5);
sigprocmask(SIG_SETMASK, &prethodna, NULL);

return 0;
}

```

Pokrenimo program i tijekom njegovog rada iz druge ljuske pošaljimo SIGINT (npr. `kill -INT <pid>`). Iako signal stiže odmah, proces neće reagirati dok ne završi `sleep` — rukovatelj se izvršava tek nakon poziva `sigprocmask(SIG_SETMASK, &prethodna, NULL)` koji vraća masku na prethodnu vrijednost (bez SIGINT-a). Iz ovoga je jasno da je signal sve to vrijeme čekao u redu.

Bitan detalj: koliko god SIGINT-a pošaljemo tijekom blokade, rukovatelj će se izvršiti **samo jednom**. Razlog je u tome što jezgra za “klasične” UNIX signale ne vodi brojač pojavljivanja, samo zastavicu “u redu čekanja je / nije”. Više instanci istog signala koje stignu tijekom blokade spojuju se u jednu jedinu isporuku. (POSIX-realtime signali, brojevi 32–64, imaju pravi red s brojačem, ali to je tema za posebnu raspravu.)

Pažljivi čitatelj će uočiti suptilan problem s gornjim kodom. Redoslijed operacija je: prvo se pozivom `sigaction()` registrira rukovatelj, a tek zatim pozivom `sigprocmask()` blokira SIGINT. Što se događa ako SIGINT stigne u kratkom vremenskom prozoru između ova dva poziva? Signal će se isporučiti — rukovatelj će raditi svoj posao prije nego što program uopće uđe u “kritičnu sekciju”. U našem benignom primjeru posljedica je samo blago netočan tajming, ali u stvarnom programu u kojem kritična sekcija mijenja zajedničke podatke, ovakav race condition može dovesti do korupcije podataka i neočekivanog ponašanja programa.

Možda biste pomislili da problem rješavamo postavljanjem SIGINT-a u polje `sa.sa_mask` strukture `struct sigaction`. Međutim, **sa_mask blokira signale samo dok se rukovatelj izvršava** — kad se rukovatelj vrati, blokada nestaje, pa to ne pomaže za zaštitu naše kritične sekcije.

Ostavljamo ovu situaciju kao **vježbu za čitatelja**: pokušajte preraditi `maska.c` tako da garantirano nijedan SIGINT ne može biti isporučen prije nego što program uđe u kritičnu sekciju, čak ni ako signal stigne u “neugodnom” trenutku. (Savjet: razmislite o redoslijedu poziva `sigaction()` i `sigprocmask()`.)

- **maska2.c** — funkcionalno drugačiji primjer, ali strukturom vrlo blizak prethodnom: umjesto da blokiramo SIGINT, ovdje ga **ignoriramo**. Rukovatelj nije potreban — kao “akciju” za signal postavljamo specijalnu vrijednost `SIG_IGN`:

```

#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <string.h>

int main() {
    struct sigaction sa;

```

```

memset(&sa, 0, sizeof(sa));
sa.sa_handler = SIG_IGN;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
sigaction(SIGINT, &sa, NULL);

sleep(5);
printf("kraj programa - SIGINT-i su tijekom rada bili ignorirani\n");

return 0;
}

```

Pokretanje izgleda slično `maska.c`-u — pet sekundi spavanja tijekom kojih program ne reagira na `Ctrl+C`, a na kraju ispiše završnu poruku koja nam jasno pokazuje da je program došao do kraja, tj. da `SIGINT` nije prekinuo izvršavanje. Kvalitativna razlika u odnosu na `maska.c` dolazi do izražaja kad usporedimo izlaze: `maska.c` će prije završne poruke iz `main`-a ispisati `SIGINT` obraden (ako smo poslali signal tijekom spavanja), dok `maska2.c` to neće ispisati nikad — signal je tiho odbačen u trenutku isporuke.

Razlika između blokiranja i ignoriranja, sažeto:

	Blokiranje (<code>sigprocmask</code>)	Ignoriranje (<code>SIG_IGN</code>)
Što se događa kada signal stigne	jezgra ga pohrani u red čekanja	jezgra ga odbaci
Kad se uvjet ukloni	signal se isporučuje, rukovatelj radi	ništa se ne događa, signal više ne postoji
Više instanci istog signala	spajaju se u jednu isporuku	sve odbačene
Tipičan use case	“ne sad, ali ne želim izgubiti signal”	“ovaj signal me zaista ne zanima”

Razlika postaje važna kad nas zanima da na signal ipak reagiramo, samo ne odmah. Klasičan primjer: tijekom kritične transakcije s bazom podataka, ne želimo prekid `Ctrl+C`-om, ali nakon završene transakcije želimo provjeriti je li korisnik tražio izlaz pa se uredno zatvoriti. Tu pomaže blokiranje; ignoriranje bi sve takve zahtjeve trajno izgubilo.

6.4 Pokupljanje djece — SIGCHLD

U poglavlju o procesima (P04) susreli smo se s pojmom **zombi procesa**: proces koji je završio izvršavanje, ali čiji zapis u tablici procesa jezgra još uvijek čuva, jer roditelj nije pozvao `wait()` da pokupi izlazni status. Tamo smo zaključili da je dobra programerska praksa za svako dijete obavezno pozvati `wait()`. Sada se postavlja pitanje: kako to napraviti elegantno ako roditelj u međuvremenu treba raditi nešto drugo, a ne samo blokirati u `wait()`-u?

Odgovor leži u signalu `SIGCHLD`. Svaki put kad proces dijete promijeni stanje (završi izvršavanje, bude zaustavljeno signalom, ili bude nastavljeno), jezgra šalje roditelju signal `SIGCHLD`. Po defaultu se ovaj signal ignorira — što je razlog zašto smo dosad mogli “zaboraviti” na njega. Međutim, ako registriramo rukovatelj za `SIGCHLD`, dobivamo elegantan obrazac u kojem roditelj pokuplja djecu **asinkrono**, kad god ona završe, dok glavni program nesmetano nastavlja sa svojim radom.

Ovo je jedan od najčešćih obrazaca u stvarnom UNIX programiranju — koriste ga UNIX ljuske, web serveri (npr. Apache, nginx za radne procese), baze podataka i

mnogi drugi sustavi koji upravljaju većim brojem podređenih procesa.

- **nozombie.c** — primjer u kojem roditelj forka tri djeteta s različitim trajanjem, a sam u glavnoj petlji broji sekunde. Pokupljanje djece obavlja se u SIGCHLD rukovatelju, asinkrono u odnosu na glavnu petlju.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

void chld_handler(int sigum) {
    int status;
    pid_t pid;

    pid = wait(&status);
    printf("[parent] dijete %d pokupljeno (status %d)\n",
           pid, WEXITSTATUS(status));
}

int main() {
    struct sigaction sa;
    int trajanje[] = {3, 1, 2};
    int i, sek;

    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = chld_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;
    sigaction(SIGCHLD, &sa, NULL);

    for (i = 0; i < 3; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            printf("[child %d] PID %d, spavam %d s\n",
                   i+1, getpid(), trajanje[i]);
            sleep(trajanje[i]);
            return i+1;
        }
    }

    for (sek = 1; sek <= 5; sek++) {
        sleep(1);
        printf("[parent] sekunda %d\n", sek);
    }

    return 0;
}
```

Roditelj registrira rukovatelj `chld_handler` za signal `SIGCHLD`, a zatim u petlji forka tri djeteta. Svako dijete spava drukčiji broj sekundi (3, 1 ili 2) i pri završetku vraća svoj redni broj kao izlazni status. Glavna petlja roditelja jednostavno broji sekunde — pet puta odspava sekundu i ispiše broj. Dok roditelj broji, djeca jedno po jedno završavaju; svaki put kad proces dijete završi, jezgra roditelju isporuči `SIGCHLD`, rukovatelj se izvrši i pozove `wait(&status)` da pokupi gotovo dijete.

Pokrenimo program:

```
$ ./nozombie
[child 2] PID 40, spavam 1 s
[child 3] PID 41, spavam 2 s
[child 1] PID 39, spavam 3 s
[parent] sekunda 1
[parent] dijete 40 pokupljeno (status 2)
[parent] sekunda 2
[parent] dijete 41 pokupljeno (status 3)
[parent] sekunda 3
[parent] dijete 39 pokupljeno (status 1)
```

```
[parent] sekunda 4
[parent] sekunda 5
```

Iz ispisa se vidi vremenski tijek: nakon prve sekunde završava dijete 2 (spavalo je 1 sekundu), pa rukovatelj odmah javlja da je pokupljeno. Nakon druge sekunde isto se dogodi za dijete 3, a nakon treće za dijete 1. Glavni program ovo ne primjećuje — uredno nastavlja brojati sekunde do pet.

Zašto SA_RESTART? Glavni program u svojoj petlji koristi `sleep(1)`. Kad `SIGCHLD` stigne tijekom `sleep-a`, jezgra prekida sistemski poziv i poziva rukovatelj. Po defaultu, kad se rukovatelj vrati, prekinuti sistemski poziv vraća grešku s `errno = EINTR`. Za `sleep` to znači: vraća se prije isteka tražene sekunde — brojač sekundi bio bi neispravan. Postavljanjem zastavice `SA_RESTART` u `sa_flags` jezgri kažemo: *“nakon obrade signala automatski nastavi prekinuti sistemski poziv”*. Tako naš `sleep` spava punu sekundu čak i ako tijekom toga stigne `SIGCHLD`.

Napomena o printf-u u rukovatelju. Pažljivi čitatelj zapazit će da naš `chld_handler` poziva `printf` — što je u suprotnosti s preporukom koju smo uveli na samom početku poglavlja: u rukovatelju treba pozivati samo `async-signal-safe` funkcije, a `printf` to nije. U ovom primjeru smo se ipak odlučili za `printf` radi jasnoće — svrha je pokazati u kojem se trenutku rukovatelj zaista izvrši i koje dijete pokuplja, što ispis čini ključnim za razumijevanje primjera. U produkcijskom kodu ovo nije prihvatljivo: rukovatelj bi tipično samo postavio zastavicu ili upisao podatke u red iz kojeg ih glavni program kasnije izvuče i ispiše. Imajte to na umu kad ovaj uzorak budete prilagođavali za vlastite programe.

Pokupljanje više djece odjednom. U ovom primjeru djeca završavaju jedno po jedno, s razmakom od jedne sekunde, pa svaka instanca `SIGCHLD-a` dovodi do pokupljanja točno jednog djeteta. Međutim, ako bi više djece završilo gotovo istovremeno, mogla bi se dogoditi situacija u kojoj je rukovatelj pozvan jednom, a u međuvremenu su dvije ili više djece spremne za pokupljanje. Razlog leži u tome što su signali “klasične” UNIX vrste — više instanci istog signala koje stignu blizu jedne drugoj spojuju se u jednu isporuku (kao što smo vidjeli kod blokiranja). U produkcijskom kodu rukovatelj zato obično poziva `waitpid(-1, &status, WNOHANG)` u petlji, sve dok funkcija ne vrati 0 ili -1, čime se garantira da su sva spremna djeca pokupljena. U ovom uvodnom primjeru nismo se bavili tom mogućnošću jer nam vremenski razmak između djece to ne nalaže.

Glavna pouka. Roditelj nije ni jednom eksplicitno pozvao `wait()` u svojoj glavnoj petlji — sva djeca su uredno pokupljena. Petlja roditelja bavi se isključivo svojim “korisnim” poslom (brojanjem sekundi), a sustav za pokupljanje radi neovisno, kroz signale. Time se izbjegava i potreba za stalnim provjerama “je li završilo neko dijete”, i opasnost da pokupljanje propustimo (što bi dovelo do gomilanja zombija).

6.5 Prevođenje

Direktorij dolazi s priloženim `Makefile-om` koji prati iste konvencije kao i `Makefile` datoteke u prethodnim poglavljima (varijable `CC`, `CFLAGS`, `LD_FLAGS`, `TARGETS`; implicitno pravilo `.c.o`; pravila `default`, `all`, `clean`). Detaljan opis strukture i korake gradnje `Makefilea` vidjeti u `../P02-0snove_programiranja/README.md`.

Tipična uporaba:

```
make          # gradi zadani cilj (potvrdi)
make all      # gradi sve primjere
make stoperica # gradi samo zadani primjer
make clean    # briše izvršne i objektne datoteke
```

6.6 Bibliografija

[1] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2013.

Poglavlje 7

Komunikacija između procesa

Procesi u UNIX sustavu žive u potpuno odvojenim adresnim prostorima — varijable jednog procesa su nevidljive drugom, čak i ako je jedan proces nastao od drugog pozivom `fork()`, ili ako imaju istog zajedničkog pretka — isti proces-roditelj stvorio ih je s dva poziva `fork()`. Ova izolacija je temeljna značajka modernog operacijskog sustava: ona štiti procese međusobno (jedan ne može srušiti ili neispravno izmijeniti podatke drugog), olakšava paralelno izvršavanje, i čini sustav robusnijim. Međutim, u praksi često želimo da procesi razmjenjuju podatke — bilo da prosljeđuju rezultate, koordiniraju rad, ili dijele zajedničke resurse. Skup mehanizama koji to omogućuju zajedničkim imenom zovemo **međuprocena komunikacija** (engl. *Inter-Process Communication*, IPC).

UNIX nudi cijelu lepezu IPC mehanizama, od najjednostavnijeg cjevovoda do složenih koncepata dijeljene memorije, ili mrežnih socketa koji omogućavaju ne samo komunikaciju između procesa na istom računalu, već i među procesima koji se izvršavaju na različitim krajevima svijeta. IPC je opsežno područje — sveobuhvatna skripta koja bi pokrila sve mehanizme i ilustrirala ih dovoljnim brojem primjera vjerojatno bi zahtijevala volumen barem jednak ovoj skripti u cjelini. Stoga ćemo se ovdje ograničiti na osnovne mehanizme, dovoljno detaljno obrađene da čitatelj stekne uvid u područje i razumijevanje osnovnih koncepata međuprocena komunikacije, a zainteresiranog čitatelja potaknemo da sam nastavi s istraživanjem onoga što ostaje izvan dometa ovog poglavlja — primjerice u temeljnom djelu *Advanced Programming in the UNIX Environment*, Stevens & Rago [1], koje cijelo poglavlje 15 posvećuje IPC mehanizmima i pruža najtemeljitiji obrazovni materijal na tu temu. Prije nego krenemo, vrijedi se kratko podsjetiti na još jedan oblik komunikacije koji smo već obradili.

7.1 Pregled mehanizama IPC-a

UNIX ima više IPC mehanizama, svaki s vlastitim svojstvima i tipičnom primjenom:

Mehanizam	Opseg	Tipična primjena
Signali	unutar sustava	obavješćavanje procesa o događajima (asinkroni "prekidi")
Anonimni cjevovodi (engl. <i>pipe</i>)	povezani procesi	jednosmjernan tok bajtova roditelj→dijete

Mehanizam	Opseg	Tipična primjena
Imenovani cjevovodi (FIFO)	unutar sustava	jednosmjernan tok bajtova između bilo koja dva procesa
POSIX redovi poruka (engl. <i>message queues</i>)	unutar sustava	strukturirana komunikacija s prioritetima
POSIX dijeljena memorija	unutar sustava	dijeljenje memorijskog prostora između procesa
POSIX semafori	unutar sustava	sinkronizacija pristupa zajedničkim resursima
System V IPC (poruke, semafori, dijeljena memorija)	unutar sustava	starija, ali raširena alternativa POSIX-u
Memory mapping (mmap)	unutar sustava	mapiranje datoteka i anonimne memorijske regije
Socketi	unutar i između sustava	komunikacija lokalna ili preko mreže

7.1.1 Signali kao oblik IPC-a

Signali, koje smo detaljno obradili u prethodnom poglavlju, ujedno su i najjednostavniji oblik međuprocenjske komunikacije. Jedan proces može poslati signal drugome (sistemske pozivom `kill()`), a primatelj na njega reagira ovisno o tome je li definirao vlastiti rukovatelj signalom. Signali su u osnovi vrlo jednostavan komunikacijski kanal — pošiljalac ne može uz signal poslati nikakve podatke (novije inačice signala kroz `sigqueue` mogu nositi i jednu cijelobrojnu vrijednost). Komunikacija koju signalima možemo ostvariti zapravo se svodi na jednostavne obavijesti: “*dogodilo se nešto*”. Usporedimo ovo s lampicom “*Check engine*” koju često nalazimo u modernim automobilima (a nažalost, često i svijetli): automobil nam signalizira da je nastupilo neko stanje na koje trebamo obratiti pažnju, ali nam ne daje nikakve dodatne informacije i na nama je da s tim postupimo kako najbolje znamo. Za pravu razmjenu podataka trebamo bogatije mehanizme koje obrađujemo u ovom poglavlju.

7.2 Anonimni cjevovodi

Najjednostavniji i najstariji UNIX IPC mehanizam je **anonimni cjevovod** (engl. *pipe*). Cjevovod je jednosmjerni komunikacijski kanal — niz bajtova koji jedan proces piše s jednog kraja, a drugi čita s drugog. Implementiran je kao međuspremnik (engl. *buffer*) u jezgri.

S programerske strane, cjevovod se predstavlja kao **par deskriptora datoteke**: jedan za čitanje, drugi za pisanje. Time se cjevovod uklapa u UNIX-ovu paradigmu “*sve je datoteka*” — čitamo i pišemo iste sistemske pozive (`read`, `write`) koje smo upoznali u poglavlju o ulazno/izlaznim operacijama.

Karakteristike anonimnih cjevovoda:

- **Jednosmjerni** — podaci teku od jednog procesa prema drugom, od pisača prema čitaču (engl. *writer to reader*); za dvosmjernu komunikaciju treba dva cjevovoda.
- **Anonimni** — nemaju ime u datotečnom sustavu; postoje samo dok ih neki proces drži otvorenima preko deskriptora.
- **Samo između povezanih procesa** — pošto su anonimni, jedini način da drugi proces dobije pristup deskriptoru je da ga **naslijedi** od roditelja kroz `fork()`. Cjevovod stoga obično povezuje roditelja i dijete (ili dvoje djece istog roditelja).

- **Ograničen kapacitet** — međuspremnik je konačan (na Linuxu obično 64 KB). Pisanje u puni cjevovod blokira pisaca sve dok čitač nešto ne pročita; čitanje iz praznog cjevovoda blokira čitača sve dok pisac nešto ne napiše.

7.2.1 Sistemski poziv pipe()

```
#include <unistd.h>

int pipe(int fd[3]);
```

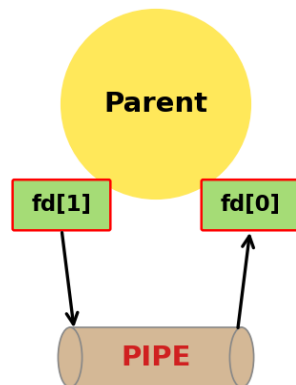
Povratna vrijednost: 0 u slučaju uspjeha, -1 u slučaju greške.

Argumenti:

- **fd** — polje od dva cijela broja u koje funkcija upisuje dva nova deskriptora datoteke: fd[0] (kraj za **čitanje**) i fd[1] (kraj za **pisanje**). Zgodno mnemoničko pomagalo: 0 se često asocira sa standardnim ulazom (čitanjem), 1 sa standardnim izlazom (pisanjem).

Nakon stvaranja cjevovoda, proces koji je pozvao pipe() drži oba kraja — i čitajući i pisajući deskriptor. U principu, ovaj proces sad može pisati u fd[1] i sam čitati to što je napisao iz fd[0], dakle, razgovarati sam sa sobom, kako je prikazano na slici 7.1:

Nakon poziva pipe()

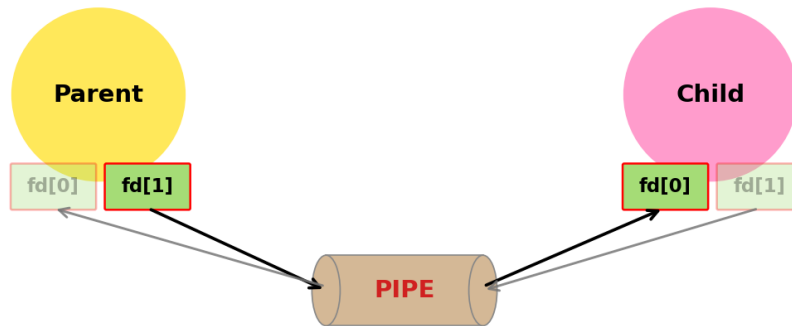


Slika 7.1: Proces drži oba kraja cjevovoda — može pisati u fd[1] i čitati iz fd[0], tj. razgovarati sam sa sobom.

Ljude koji govore sami sa sobom obično “čudno” gledamo, a ni kod procesa ovo nema previše smisla. Cjevovod postaje koristan tek kad ga **dijelimo s drugim procesom**, što se postiže pozivom fork() neposredno nakon pipe()-a.

Tipičan obrazac: pozovemo pipe() u roditelju, zatim fork(). Proces djeteta nasljeđuje sve otvorene deskriptore datoteka roditelja, pa tako oba procesa sada imaju sva četiri “kraja” cjevovoda (dva u roditelju, dva u djetetu — naslijeđeni). S obzirom da su cjevovodi zamišljeni za jednosmjernu komunikaciju, svaki proces trebao bi zatvoriti onaj kraj koji nema namjeru koristiti: npr. ako će informacija putovati od roditelja prema djetetu, tj. ako roditelj piše a dijete čita, roditelj zatvara fd[0], a dijete fd[1]. Ova situacija prikazana je na slici 7.2:

Nakon poziva `fork()` — roditelj piše, dijete čita



Slika 7.2: Roditelj zatvara `fd[0]` (svoj kraj za čitanje), dijete zatvara `fd[1]` (svoj kraj za pisanje). Zatvoreni krajevi cjevovoda prikazani su sivom bojom.

Time se uspostavlja jednosmjerni kanal: deskriptor `fd[2]` otvoren za pisanje ostaje otvoren samo u roditelju, dok deskriptor `fd[0]` otvoren za čitanje ostaje otvoren samo u djetetu. Komunikacija u suprotnom smjeru, od djeteta prema roditelju, nije više moguća (barem ne kroz ovaj cjevovod), jer su deskriptori koji bi omogućili komunikaciju u tom smjeru zatvoreni s obje strane. Ako trebamo komunikaciju u oba smjera, uobičajeno je rješenje stvoriti dva cjevovoda, svaki za svoj smjer.

7.2.2 Cjevovod između roditelja i djeteta

- **cjev.c** — minimalan primjer cjevovoda: roditelj šalje poruku djetetu kroz cjevovod.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int fd[3];
    pid_t pid;

    /* fd[0] - kraj za citanje
     * fd[2] - kraj za pisanje */
    if (pipe(fd) < 0) {
        perror("pipe");
        return 1;
    }

    pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        /* dijete - cita iz cjevovoda */
        char buf[128];
        close(fd[2]); /* ne treba nam pisanje */
        ssize_t n = read(fd[0], buf, sizeof(buf) - 1);
        if (n > 0) {
            buf[n] = '\0';
            printf("Dijete primilo: %s\n", buf);
        }
        close(fd[0]);
    }
}
```

```

    } else {
        /* roditelj - pise u cjevovod */
        const char *poruka = "Pozdrav iz roditelja!";
        close(fd[0]); /* ne treba nam citanje */
        write(fd[2], poruka, strlen(poruka));
        close(fd[2]);

        wait(NULL); /* cekaj da dijete zavrshi */
    }

    return 0;
}

```

Bitno je primijetiti redoslijed poziva: prvo se otvara cjevovod (u roditelju), a tek onda se radi `fork()`. Time se osigurava da oba procesa imaju iste deskriptore. Ukoliko bi prvo napravili `fork()`, a tek zatim `pipe()`, dijete ne bi imalo pristup cjevovodu. Još veća greška bila bi napraviti `fork()`, a zatim `pipe()` u oba procesa — ovim bi dobili dva potpuno odvojena cjevovoda, a svaki proces bi mogao jedino pričati sam sa sobom.

Primijetimo i da su lokalne varijable jasno razdvojene između grana: međuspremnik `buf` deklariran je samo unutar grane djeteta, a varijabla `poruka` (i tekst poruke) zapisana je samo u grani roditelja. Ovo je u primjeru namjerno napravljeno kako bi se dodatno naglasilo da dijete nije imalo mogućnost direktnog pristupa poruci definiranoj u procesu roditelju. Razmjena podataka između roditelja i djeteta moguća je isključivo kroz cjevovod — vrijednost se mora upisati u `fd[2]` na jednoj strani i pročitati iz `fd[0]` na drugoj.

U praksi smo varijablu mogli definirati samo jednom, na početku programa. Nakon poziva `fork()` adresni prostori roditelja i djeteta se odvajaju, pa smo u roditelju mogli koristiti istu varijablu za definiranje poruke, a u djetetu za primanje poruke kroz cjevovod — ili za bilo što drugo. Nakon poziva `fork()`, dvije naizgled “iste” varijable u dva procesa zapravo pokazuju na dvije odvojene adrese u odvojenim adresnim prostorima. Međutim, kao što smo već ranije rekli, u ovom primjeru namjerno koristimo različita imena varijabli kako bismo dodatno naglasili funkcionalnost primjera.

Pokretanje:

```

$ ./cijev
Dijete primilo: Pozdrav iz roditelja!

```

Važno je napomenuti da zatvaranje “nepotrebnog” kraja cjevovoda u svakom od procesa nije korak koji nam služi tek tome da bismo imali pregledniji kod — bez ovog koraka proces koji čita iz cjevovoda može ostati zauvijek blokiran u pozivu `read()`. Naime, nakon što proces koji u cjevovod piše zatvori `fd[2]`, ili završi s izvršavanjem (čime se zatvaraju i svi otvoreni deskriptori datoteka), idući poziv `read()` u čitatelju vratit će 0 (*End of file* — podsjetimo se sistemskog poziva `read()`). Međutim, ukoliko čitatelj sam drži otvorenim svoju kopiju kraja za pisanje (`fd[2]`), `read()` neće vratiti 0 jer je s gledišta jezgre drugi kraj cjevovoda i dalje otvoren — postoji još jedan deskriptor preko kojeg bi netko mogao pisati. Posljedica je vječno blokiranje čitatelja u `read()`-u, čak i nakon što je proces koji bi u cjevovod trebao pisati odavno gotov.

7.2.3 Preusmjeravanje i cjevovodi

U sljedećem primjeru iskoristit ćemo cjevovode da implementiramo još jednu uobičajenu funkciju ljuske: **preusmjeravanje standardnih ulaza i**

izlaza i ulančavanje procesa. Podsjetimo se prvog poglavlja (vidi sekciju o preusmjeravanju u P01) i operatora `|` kojim standardni izlaz jednog procesa možemo povezati izravno na standardni ulaz drugog. Tako, na primjer, ukoliko želimo prebrojati koliko u nekom direktoriju ima datoteka, možemo kombinirati dvije standardne UNIX naredbe: `ls` za pregled sadržaja direktorija i `wc -l` za brojanje redaka na standardnom ulazu (`wc` bez ikakve opcije ispisuje broj redaka, riječi i znakova; opcija `-l` ograničava ispis samo na broj redaka — pojedinosti nudi man `wc`):

```
$ ls | wc -l
14
```

Isti efekt možemo postići i sami, kombiniranjem dvaju procesa i jednog cjevovoda. Trik se sastoji od nekoliko koraka. Najprije glavni proces stvori cjevovod pozivom `pipe()`, nakon čega pozove `fork()` — sad imamo dva procesa koja imaju pristup krajevima cjevovoda za čitanje i pisanje. Kako nam je cilj da izlaz iz naredbe `ls` završi na ulazu naredbe `wc`, na standardni izlaz procesa koji će izvršiti `ls` dupliciramo `fd[2]` — kraj za pisanje cjevovoda koji smo upravo stvorili. Slično, u procesu koji će izvršiti `wc`, na standardni ulaz dupliciramo `fd[0]` — kraj cjevovoda za čitanje. Za dupliciranje deskriptora služi nam sistemski poziv `dup2()` koji smo već upoznali u poglavlju o ulazno/izlaznim operacijama. Tek nakon ovog preusmjeravanja, svaki proces pozove `exec` i postane `ls` odnosno `wc`. Time `ls`, koji ne zna ništa o našem cjevovodu, jednostavno piše svoje rezultate na svoj standardni izlaz — koji se “magično” završava u cjevovodu; a `wc -l`, isto neupućen u priču, čita s ulaza koji je zapravo drugi kraj istog cjevovoda.

- **prebroji.c** — implementacija `ls | wc -l`, korištenjem `pipe`, `fork`, `dup2` i `exec`.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int fd[3];
    pid_t pid;

    if (pipe(fd) < 0) {
        perror("pipe");
        return 1;
    }

    pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        /* preusmjeri standardni izlaz na fd[2] */
        dup2(fd[2], STDOUT_FILENO);
        if (fd[2] != STDOUT_FILENO)
            close(fd[2]);
        close(fd[0]);
        execlp("ls", "ls", (char *)NULL);
        perror("execlp ls");
        return 1;
    } else {
        /* preusmjeri standardni ulaz na fd[0] */
        dup2(fd[0], STDIN_FILENO);
        if (fd[0] != STDIN_FILENO)
            close(fd[0]);
        close(fd[2]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("execlp wc");
    }
}
```

```

    return 1;
}

```

Ključne stvari koje treba primijetiti:

- **dup2(fd[2], STDOUT_FILENO)** u djetetu znači: “neka deskriptor 1 (stdout) sad bude kopija onoga što pokazuje fd[2]” — efektivno, sve što ls napiše na svoj standardni izlaz zapravo ide u cjevovod.
- Jednako tome, **dup2(fd[0], STDIN_FILENO)** u roditelju preusmjerava wc-ov ulaz na čitajući kraj cjevovoda.
- **Oba procesa zatvaraju izvorne deskriptore cjevovoda** odmah nakon dup2-a. Razlog je upravo onaj koji smo opisali u prethodnoj sekciji: ako wc (roditelj) drži otvoren fd[2], kraj za pisanje na svojoj strani, on bi sam sebi spriječio kraj toka — read na cjevovodu ne bi nikad vratio 0, i wc bi zaglavio. Pažljivi čitatelj primijetiti će uvjete `if (fd[2] != STDOUT_FILENO)` i `if (fd[0] != STDIN_FILENO)` prije zatvaranja: oni štite od rubnog slučaja kad bi `pipe()` slučajno vratio `fd[0] == 0` ili `fd[2] == 1` (npr. ako je standardni ulaz/izlaz već ranije bio zatvoren). Tada bismo `close`-om upravo zatvorili kraj cjevovoda koji smo netom postavili kroz `dup2`, čime bi cijela konstrukcija propala.
- **Nije nužan eksplicitni wait** na dijete: nakon što roditelj pozove `exec` i postane wc, wc će prirodno blokirati u `read`-u dok se cjevovod ne zatvori s druge strane. To se događa kad dijete (ls) završi svoj posao i jezgra zatvori njegove deskriptore. Tek tada `read` u wc-u vrati 0 i wc ispiše broj redaka. Zanimljiva posljedica je da ls nakratko postaje **zombie proces** (jer ga njegov roditelj wc ne `wait`-a), no čim cijeli naš prebroji (koji je u međuvremenu postao wc) završi, sirotinjski ls zombie usvaja `init` proces (PID 1) koji ga uredno počisti pozivom `wait`. Tako smo izbjegli trajno zaglavljivanje zombie procesa, iako u kodu nigdje eksplicitno nismo čekali djecu.

Pokretanje:

```

$ ./prebroji
14
$ ls | wc -l
14

```

Rezultat je broj zapisa u trenutnom direktoriju — identičan onome što daje `ls | wc -l` u ljsuci.

Ovaj sažeti primjer pokazuje kako je u ljsuci implementirano ulančavanje procesa u blokove: za svaku komponentu cjevovoda pokrene zaseban proces, a između njih postavi cjevovod uz odgovarajuća preusmjeravanja. U pravoj ljsuci proces bi tekao ovako: ljuska prvo stvori cjevovod, a nakon toga pokrene dva nova procesa — u jednom `ls`, u drugom `wc -l`. U svakom od dva novostvorena procesa odradio bi se postupak koji smo upravo pokazali primjerom (preusmjeravanje deskriptora, zatim `exec`). Proces roditelj — sama ljuska — pak nastavio bi se izvršavati i pozvao `wait` za oba procesa djeteta, nakon čega bi čekao iduću naredbu korisnika. U našem slučaju nema iduće naredbe nakon `prebroji`, pa smo primjer pojednostavnili tako što je sam glavni proces postao jedna od komponenata cjevovoda (`wc -l`).

7.3 Imenovani cjevovodi (FIFO)

Anonimni cjevovodi imaju jedno ozbiljno ograničenje: budući da nemaju ime u datotečnom sustavu, mogu ih koristiti samo srodni procesi (preko fork-a). Što ako želimo da dva potpuno nezavisna procesa — pokrenuta od strane različitih korisnika, u različitim trenutcima — komuniciraju cjevovodom?

Odgovor su **imenovani cjevovodi** (engl. *named pipes*) ili **FIFO** (kratica od *First In, First Out* — prvi unesen, prvi pročitani, što opisuje sam način rada cjevovoda: bajt koji se prvi upiše s jednog kraja prvi će biti i pročitani s drugog kraja). Riječ je o cjevovodima koji imaju ime u datotečnom sustavu — vidljivi su naredbom `ls` (gdje su označeni slovom `p` u prvom stupcu prava pristupa, kao u `prw-r--r--` — `p` znači *pipe*), mogu se otvoriti kao i svaka druga datoteka — pozivom `open()`, i obrisati naredbom `rm` ili `unlink()`-om. S programerske strane, koriste se gotovo identično anonimnim cjevovodima: `read` i `write` rade isto, semantika blokiranja je ista, ograničenje smjera (jedan smjer po cjevovodu) je isto.

Ključna razlika u korištenju: FIFO se ne stvara `pipe()`-om, nego `mkfifo()` sistemskim pozivom (ili istoimenom naredbom `mkfifo` iz `ljske`). Postoji još jedna bitna razlika između ova dva poziva: `pipe()` u jednom koraku stvara cjevovod i vraća deskriptore datoteke za njegova oba kraja — jedan za čitanje, jedan za pisanje. `mkfifo()`, naprotiv, stvara samo datoteku tipa FIFO u datotečnom stablu, i ne vraća nikakve deskriptore. Tek kad tu datoteku potom otvorimo običnim pozivom `open()` (s `O_RDONLY` ili `O_WRONLY`), dobivamo deskriptor datoteke kojim možemo čitati ili pisati. Tip "FIFO" zapisan je u `st_mode` polju `i-noda` — sjetimo se makroa `S_ISFIFO` iz poglavlja o upravljanju datotekama, koji je upravo ono što provjerava taj bit u `i-nodu` i daje slovnu oznaku `p` koju vidimo u `ls -l` ispisu.

```
#include <sys/types.h>
#include <sys/stat.h>

int mkfifo(const char *pathname, mode_t mode);
```

Povratna vrijednost: 0 u slučaju uspjeha, -1 u slučaju greške.

Argumenti:

- **pathname** — putanja na kojoj će biti stvoren FIFO.
- **mode** — prava pristupa (kao za `open` ili `mkdir`); konačna prava se dobiju kombinacijom s `umask` procesa.

Jednom kad imamo datoteku tipa FIFO u datotečnom sustavu — bez obzira jesmo li je upravo napravili s `mkfifo`, ili je tu od ranije (možda ju je napravila sasvim druga aplikacija) — možemo je otvoriti kao bilo koju drugu datoteku, jednostavnim pozivom `open()`. Bitne karakteristike koje treba imati na umu kod FIFO-a:

- **Otvaranje blokira po defaultu:** `open()` na FIFO za čitanje blokira sve dok neki drugi proces ne otvori isti FIFO za pisanje (i obratno). Time se osigurava točka susreta (engl. *rendezvous point*) za procese.
- **Postoje na disku** sve dok ih netko ne obriše (`unlink` ili `rm`). Ovo je ponekad i poželjno — FIFO se može namjerno ostaviti u datotečnom sustavu kao trajna komunikacijska točka koju kasnije koriste razni programi. Ako pak FIFO više nije potreban, treba ga obrisati kako ne bismo gomilali nepotrebne datoteke u datotečnom sustavu.
- **Više čitača/pisača:** na isti FIFO može se istovremeno spojiti više procesa, ali tada poredak primanja nije zajamčen ako više pisača istovremeno piše.

7.3.1 Primjer: razmjena poruke kroz FIFO

Stvorit ćemo dva mala programa — jedan koji preuzima sa standardnog ulaza i šalje u FIFO, drugi koji čita iz FIFO-a i ispisuje na standardni izlaz — koji komuniciraju kroz FIFO. Kad ih pokrenemo u dvije zasebne ljuške, ovo demonstrira IPC između potpuno **nepovezanih** procesa.

- **fifoposalji.c** — stvara FIFO ako ne postoji, otvara ga za pisanje, te u petlji prosljeđuje sve što dolazi sa standardnog ulaza u FIFO sve dok read ne vrati 0 (npr. korisnik utipka Ctrl+D u terminalu).

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define FIFO_PATH "/tmp/moj_fifo"

int main(void) {
    int fd;
    char s;
    ssize_t n;

    /* stvori FIFO ako ne postoji */
    if (mkfifo(FIFO_PATH, 0666) < 0 && errno != EEXIST) {
        perror("mkfifo");
        return 1;
    }

    printf("Otvaram FIFO za pisanje (cekam citatelja)...\n");
    fd = open(FIFO_PATH, O_WRONLY);
    if (fd < 0) {
        perror("open");
        return 1;
    }

    /* citaj sa standardnog ulaza i prosljedjuj u FIFO znak po znak,
     * sve dok read ne vrati 0 (korisnik je utipkao Ctrl+D u terminalu) */
    while ((n = read(STDIN_FILENO, &s, 1)) > 0)
        write(fd, &s, 1);

    close(fd);
    return 0;
}
```

- **fifoprimi.c** — otvara isti FIFO za čitanje i u petlji prosljeđuje sve pročitano na standardni izlaz.

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define FIFO_PATH "/tmp/moj_fifo"

int main(void) {
    int fd;
    char s;
    ssize_t n;

    /* stvori FIFO ako jos ne postoji */
    if (mkfifo(FIFO_PATH, 0666) < 0 && errno != EEXIST) {
        perror("mkfifo");
        return 1;
    }
}
```

```

printf("Otvaram FIFO za citanje (cekam pisca)...\n");
fd = open(FIFO_PATH, O_RDONLY);
if (fd < 0) {
    perror("open");
    return 1;
}

/* citaj iz FIFO-a i ispisuj na standardni izlaz znak po znak,
 * dok read ne vrati 0 (kad pisac zatvori svoj kraj cjevovoda) */
while ((n = read(fd, &s, 1)) > 0)
    write(STDOUT_FILENO, &s, 1);

close(fd);
return 0;
}

```

Pokrenimo programe u dvije **zasebne** ljsuke. U prvoj pokrećemo čitatelja:

```

## ljsuka 1 (citatelj):
$ ./fifoprimi
Otvaram FIFO za citanje (cekam pisca)...

```

Program će “blokirati” na pozivu `open()` jer još nema pisca. Sad u drugoj ljsuci pokrećemo pisca i utipkavamo nekoliko redaka, završavajući s `Ctrl+D`:

```

## ljsuka 2 (pisac):
$ ./fifoposalji
Otvaram FIFO za pisanje (cekam citatelja)...
pozdrav iz druge ljsuke!
ovo je drugi redak.
^D
$

```

Tek kad je u drugoj ljsuci pokrenut `fifoposalji`, čitatelj u prvoj ljsuci se odblokira i počinje primati podatke. Svaki redak koji utipkamo u drugoj ljsuci ispisuje se u prvoj:

```

## ljsuka 1 (nastavak):
pozdrav iz druge ljsuke!
ovo je drugi redak.
$

```

Pokušajte sad pokrenuti programe obrnutim redoslijedom — prvo program koji piše, a tek nakon toga program koji čita iz FIFO-a. Slobodno utipkajte i nekoliko redaka u program-pisac prije nego što pokrenete čitača. Rezultat je uvijek isti: čim pokrenete program koji čita, sve što je upisano u FIFO (koji je u osnovi cjevovod, samo što ima ime u datotečnom stablu) pojaviti će se na njegovom drugom kraju.

7.3.1.1 Veličina FIFO datoteke

Provjerimo i datotečni sustav — FIFO je stvarno tu:

```

$ ls -l /tmp/moj_fifo
prw-rw-rw- 1 dkrst users 0 May  7 18:00 /tmp/moj_fifo

```

Veličina FIFO datoteke prikazana korištenjem naredbe `ls` je **uvijek nula**, čak i ako smo pokrenuli program koji piše u FIFO i upisali nekoliko redaka prije nego što smo pokrenuli čitača. Razlog je u tome što se podaci nikada ne zapisuju stvarno na disk — kad ih netko čita, prolaze kroz cjevovod u jezgri; kad još nitko ne čita, čuvaju se u međuspremniku u jezgri. Ovo je bitno naglasiti jer čini ključnu razliku u odnosu na korištenje obične datoteke kao komunikacijske točke između dva procesa: kod obične datoteke podaci se stvarno zapisuju i čitaju s diska, što je višestruko sporiji

proces od komunikacije kroz međuspremnik u jezgri. FIFO se na disku očituje samo kao ime i tip — sve “zanimljivo” događa se u memoriji.

7.3.1.2 Pristup FIFO-u iz drugog programa

Pošto je FIFO datoteka vidljiva i dostupna u datotečnom sustavu, možemo je otvoriti i s drugim programima. Pokušajmo našu datoteku `/tmp/moj_fifo` otvoriti s programom `cat` — standardnom UNIX naredbom koja čita datoteku i ispisuje njezin sadržaj na standardni izlaz:

```
## ljska 1 (citatelj):
$ cat /tmp/moj_fifo
```

U ovom slučaju, `cat` blokira u pokušaju čitanja iz FIFO-a, baš kao što je radio i naš `fifoprimi`. Pokrenimo sad u drugoj ljski našeg pisca i utipkajmo poruku:

```
## ljska 2 (pisac):
$ ./fifoposalji
Otvaram FIFO za pisanje (cekam citatelja)...
ne treba nam vlastiti program za citanje!
^D
$
```

Čim utipkamo `Ctrl+D`, `fifoposalji` zatvori svoj kraj cjevovoda, `cat` u prvoj ljski dobije `0` od `read-a` (kraj toka) i ispiše ono što je primio:

```
## ljska 1 (nastavak):
ne treba nam vlastiti program za citanje!
$
```

Na sličan način, naš program `fifoposalji` mogli bismo zamijeniti naredbom `cat` (uz preusmjeravanje izlaza, npr. `cat > /tmp/moj_fifo`) ili još jednostavnije, naredbom `echo` (`echo "poruka" > /tmp/moj_fifo`). FIFO datoteci možemo pristupiti bilo kojim alatom koji zna otvarati datoteke — od strane FIFO-a nema nikakve razlike između specijaliziranog programa kao što je naš `fifoposalji` i standardne naredbe poput `echo-a`. To je još jedna ilustracija osnovnog UNIX principa — *sve je datoteka*. Mehanizam IPC-a (FIFO) preko datotečnog sustava postaje dostupan svim postojećim alatima, bez potrebe da znaju išta posebno o cjevovodima. Naši programi `fifoposalji` i `fifoprimi` korisni su prvenstveno kao primjeri koji ilustriraju kako se s FIFO-om radi izravno kroz systemske pozive; u praksi bi se često koristili upravo `cat`, `echo`, ili neki drugi standardni alat.

Kad više ne trebamo FIFO, brišemo ga kao i bilo koju drugu datoteku:

```
$ rm /tmp/moj_fifo
```

7.4 Dijeljena memorija

Cjevovodi i FIFO-i prirodni su za slijedne tokove podataka — bajt po bajt, jedna strana piše, druga čita. Kad procesi trebaju zajedno raditi nad istim podacima — npr. pet procesa istovremeno ažurira jedan brojač, ili dva procesa dijele veliku tablicu — cjevovod nije idealan način razmjene podataka. Osim što svaka razmjena mora proći kroz međuspremnik u jezgri (uz odgovarajuće systemske pozive `read` i `write`), svaki proces mora dodatno držati i vlastitu kopiju podataka u svojoj lokalnoj memoriji. Sjetimo se: procesi međusobno ne vide međusobne adresne prostore, pa kad proces A želi obavijestiti proces B o promjeni vrijednosti, A mora poslati svoje lokalne promjene kroz cjevovod, a B ih mora procesirati i ugraditi u svoju lokalnu kopiju podataka. Ovaj postupak, osim što uključuje parsiranje i analizu promjena u

odnosu na lokalnu kopiju, zahtijeva i višestruke kopije podataka u memoriji. Za pet procesa koja koordiniraju nad istim brojačem, to bi značilo pet kopija u memoriji + neprestano usklađivanje porukama — neefikasno i sklono greškama.

Dijeljena memorija je drugačiji pristup: dva ili više procesa pristupaju istim podacima u memoriji, izravno i bez kopiranja, putem pokazivača koji predstavlja “prozor” na istu fizičku memorijsku regiju.

Pristupi dijeljenoj memoriji u UNIX-u dolaze u dvije inačice:

- **POSIX dijeljena memorija** — modernija implementacija, preporučena za nove programe. Identifikatori su putanje (počinju s /), pa imaju i vlasništvo i prava pristupa kao i obične datoteke.
- **System V dijeljena memorija** — stariji standard, ali još uvijek u širokoj upotrebi, osobito kod starijih programa. Identifikatori su cjelobrojni “ključevi”.

U osnovi je riječ o istoj stvari — oba pristupa omogućuju da više procesa pristupi istoj memorijskoj regiji izravno, bez kopiranja kroz jezgru. Razlikuju se uglavnom u API-ju i načinu imenovanja: koje funkcije pozivamo, kako objekte identificiramo i kako njima upravljamo. Konceptualno, ono što naučite za jednu implementaciju lako se prenosi na drugu.

POSIX dijeljena memorija u biti se sastoji od dva koraka: stvori se “objekt dijeljene memorije” (zapravo specijalna datoteka u memorijskom datotečnom sustavu, obično /dev/shm), i potom se taj objekt mapira u adresni prostor procesa pomoću `mmap()`. Tako svaki proces dobiva pokazivač kojim može čitati i pisati u dijeljeni dio.

Funkcije za rad s POSIX dijeljenom memorijom su:

```
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>

int shm_open(const char *name, int oflag, mode_t mode);
int shm_unlink(const char *name);

void *mmap(void *addr, size_t length, int prot,
           int flags, int fd, off_t offset);
int munmap(void *addr, size_t length);

int ftruncate(int fd, off_t length);
```

7.4.0.1 Funkcija `shm_open()`

Stvara novi (ili otvara postojeći) objekt dijeljene memorije. Ponaša se kao `open()` za obične datoteke, samo što “datoteka” živi u memorijskom datotečnom sustavu.

Povratna vrijednost: deskriptor datoteke u slučaju uspjeha, -1 u slučaju greške.

Argumenti:

- **name** — ime objekta (mora počinjati s /, npr. “/moj_brojac”).
- **oflag** — kombinacija zastavica kao kod `open()`: `O_CREAT`, `O_RDWR`, `O_EXCL`, ...
- **mode** — prava pristupa (kao za `open`).

7.4.0.2 Funkcija `ftruncate()`

Postavlja veličinu datoteke (ili shm objekta) na zadanu vrijednost. Novostvoren shm objekt ima veličinu 0, pa ga prije mapiranja moramo “razvući” na željenu veličinu pomoću `ftruncate`. Ovaj korak konceptualno je sličan onome što radimo i

kod dinamičke alokacije memorije: kad u C programu koristimo `malloc(n)`, jezgra (preko bibliotečne funkcije) rezervira `n` bajtova za naš proces i vraća pokazivač na taj prostor — bez te rezervacije nemamo gdje pisati. Slično, `shm_open` nam vraća rukovatelj (deskriptor datoteke) na novi objekt dijeljene memorije, ali sam objekt još uvijek nema rezervirano nikakvog stvarnog prostora. Tek `ftruncate(fd, n)` rezervira `n` bajtova za taj objekt, a nakon toga možemo pozvati `mmap` da nam jezgra preslika tih `n` bajtova u adresni prostor procesa i vrati pokazivač kojim do njih dolazimo.

Povratna vrijednost: 0 u slučaju uspjeha, -1 u slučaju greške.

7.4.0.3 Funkcija `mmap()`

Mapira datoteku (ili shm objekt) u adresni prostor procesa. Vraćeni pokazivač pokazuje na memorijski blok koji je *zapravo* sadržaj te datoteke — pisanje u njega odmah se reflektira na “datoteku” (a kod shm to je dijeljena memorija). Funkcija ne razlikuje shm objekt od bilo koje druge datoteke u datotečnom sustavu — `mmap` jednako tako može mapirati i regularnu datoteku na disku, čime sadržaj te datoteke postaje izravno dostupan kroz pokazivač. Toj primjeni `mmap`-a vraćamo se u zasebnoj sekciji kasnije u poglavlju.

Povratna vrijednost: pokazivač na mapirani blok, ili `MAP_FAILED` (vrijednost (`void *`) -1) u slučaju greške.

Argumenti:

- **addr** — preferirana adresa za mapiranje; gotovo uvijek se zadaje `NULL`, što znači “neka jezgra odluči gdje”.
- **length** — broj bajtova koji se mapira.
- **prot** — kombinacija dozvoljenih operacija: `PROT_READ`, `PROT_WRITE`, `PROT_EXEC`, `PROT_NONE`.
- **flags** — najvažnija je `MAP_SHARED` (promjene su vidljive drugim procesima i, ako je riječ o regularnoj datoteci, zapisuju se na disk) ili `MAP_PRIVATE` (proces dobiva privatnu kopiju, drugi je ne vide).
- **fd** — deskriptor datoteke (ili shm objekta) koja se mapira; za **anonimne mape** (mape koje nisu povezane ni s jednom datotekom — koriste se kao “čista” memorijska regija, npr. za dijeljenje memorije između roditelja i djeteta nakon `fork`-a) postavlja se -1 uz zastavicu `MAP_ANONYMOUS`.
- **offset** — pomak unutar datoteke od kojeg počinje mapiranje. Ne moramo dakle mapirati cijelu datoteku ni nužno od početka — kombinacijom `offset` i `length` mapiramo proizvoljan dio (raspon od `length` bajtova počevši od `offset`-a). To je posebno korisno za velike datoteke (npr. baze podataka od više GB) iz kojih nas zanima samo neki dio. Vrijednost `offset`-a mora biti višekratnik veličine stranice virtualne memorije (tipično 4 KB; točnu vrijednost za sustav daje `sysconf(_SC_PAGE_SIZE)`), jer cijela infrastruktura virtualne memorije radi u jedinicama stranica.

7.4.0.4 Funkcija `munmap()`

Oslobađa dodijeljeni raspon memorije i prekida vezu s datotekom ili anonimnim blokom. Pokazivač na blok više ne vrijedi.

Povratna vrijednost: 0 u slučaju uspjeha, -1 u slučaju greške.

7.4.0.5 Funkcija shm_unlink()

Briše imenovani shm objekt iz sustava (slično unlink-u za datoteke). Ako je objekt još uvijek mapiran u nekom procesu, on i dalje radi sve dok ga taj proces ne munmap-a; tek tada se stvarno oslobađa.

Povratna vrijednost: 0 u slučaju uspjeha, -1 u slučaju greške.

7.4.1 Primjer: dijeljenje memorije između nezavisnih procesa

Pokazat ćemo dva mala programa — jedan koji upisuje tekst u dijeljenu memoriju, i drugi koji ga iz nje čita — slično paru fifoposalji/fifoprimi. Ključna razlika prema FIFO-u: **podaci ostaju u memoriji i nakon što program završi**, sve dok ih netko ne ukloni pozivom shm_unlink ili dok se sustav ne ponovno pokrene. Tako bilo koji proces u međuvremenu može pročitati ono što je tamo upisano.

- **shm_pisi.c** — stvara (ili otvara) shm objekt /moja_memorija, mapira ga, i upisuje tekst zadan kao argument naredbenog retka.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>

#define SHM_NAME "/moja_memorija"
#define VELICINA 256

int main(int argc, char *argv[]) {
    int fd;
    char *podaci;
    const char *poruka;

    if (argc < 2)
        poruka = "Pozdrav iz zajedničke memorije!";
    else
        poruka = argv[2];

    /* stvori (ili otvori postojeći) objekt dijeljene memorije */
    fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    if (fd < 0) { perror("shm_open"); return 1; }

    /* postavi velicinu (rezerviraj prostor) */
    if (ftruncate(fd, VELICINA) < 0) { perror("ftruncate"); return 1; }

    /* mapiraj objekt u adresni prostor */
    podaci = mmap(NULL, VELICINA, PROT_READ | PROT_WRITE,
                  MAP_SHARED, fd, 0);
    if (podaci == MAP_FAILED) { perror("mmap"); return 1; }

    /* upisi poruku; kopiramo najviše VELICINA-1 bajtova kako bi
     * preostao prostor za završni nul-znak (ako je poruka duža,
     * visak se odbacuje, a nikad se ne piše izvan rezerviranog bloka) */
    strncpy(podaci, poruka, VELICINA - 1);
    podaci[VELICINA - 1] = '\0';

    printf("Upisano u %s: %s\n", SHM_NAME, podaci);

    munmap(podaci, VELICINA);
    close(fd);
    return 0;
}
```

- **shm_citaj.c** — otvara isti shm objekt samo za čitanje, mapira ga, ispisuje sadržaj.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>

#define SHM_NAME "/moja_memorija"
#define VELICINA 256

int main(void) {
    int fd;
    char *podaci;

    fd = shm_open(SHM_NAME, O_RDONLY, 0);
    if (fd < 0) { perror("shm_open"); return 1; }

    podaci = mmap(NULL, VELICINA, PROT_READ, MAP_SHARED, fd, 0);
    if (podaci == MAP_FAILED) { perror("mmap"); return 1; }

    printf("Procitano iz %s: %s\n", SHM_NAME, podaci);

    munmap(podaci, VELICINA);
    close(fd);
    return 0;
}

```

Pokrenimo programe — mogu se izvršavati u istoj ljusci ili u različitima, vremenski razmaknuti, sasvim svejedno:

```

$ ./shm_pisi "Pozdrav iz prvog procesa!"
Upisano u /moja_memorija: Pozdrav iz prvog procesa!

```

```

$ ./shm_citaj
Procitano iz /moja_memorija: Pozdrav iz prvog procesa!

```

```

$ ./shm_citaj
Procitano iz /moja_memorija: Pozdrav iz prvog procesa!

```

Drugi `shm_citaj` u istom primjeru ispisuje istu poruku iako je prvi `shm_citaj` već završio — tekst je i dalje u memoriji. Možemo ga prepisati ponovnim pozivom `shm_pisi-a`:

```

$ ./shm_pisi "Druga poruka"
Upisano u /moja_memorija: Druga poruka

```

```

$ ./shm_citaj
Procitano iz /moja_memorija: Druga poruka

```

`shm` objekt je trajan — vidi se i u datotečnom sustavu pod `/dev/shm`:

```

$ ls -l /dev/shm/
total 4
-rw-r--r-- 1 dkrt users 256 May  7 18:00 moja_memorija

```

Ako su u sustavu još i drugi procesi koji koriste POSIX dijeljenu memoriju, pojavit će se i njihovi `shm` objekti — `/dev/shm/` je zajednički direktorij za sve. Za razliku od FIFO-a, ovdje veličina nije nula — `shm` objekt rezervira pravi prostor (256 bajtova, kao što smo zatražili `fttruncate-om`), a sav sadržaj koji upisujemo zapravo se nalazi u tom prostoru. Iako `/dev/shm` izgleda kao obični dio datotečnog stabla, njegov sadržaj zapravo ne završi na disku — `tmpfs` je memorijski datotečni sustav koji jezgra drži u RAM-u. To znači da je pristup tim podacima jednako brz kao i pristup bilo kojoj drugoj memorijskoj lokaciji, bez latencije koja je neizbježna kod pisanja na fizičke diskove.

Kad `shm` objekt više ne trebamo, brišemo ga pozivom `shm_unlink`, ili u ljusci jednostavno:

```

$ rm /dev/shm/moja_memorija

```

Nakon brisanja, novi pokušaj čitanja više nije moguć — objekt je nestao iz sustava:

```
$ ./shm_citaj
shm_open: No such file or directory
```

shm_open u shm_citaj programu pozvan je s O_RDONLY (bez O_CREAT), pa kad nema postojećeg objekta s tim imenom, vraća grešku. Da smo programu prepustili da sam stvori objekt, dobili bismo prazan blok memorije bez ikakvog sadržaja.

Ovaj jednostavan par programa pokazuje tri ključne osobine POSIX dijeljene memorije:

- **Dijeljenje** — više nezavisnih procesa pristupa istom memorijskom prostoru, jer svi koriste isto ime (/moja_memorija) i mmap ih sve preslikava u istu fizičku regiju.
- **Perzistentnost** — shm objekt nadživljuje proces koji ga je stvorio. Sadržaj ostaje dostupan dok ga eksplicitno ne uklonimo, ili dok se sustav ne ponovno pokrene.
- **Imenovanost** — pristup ide kroz putanju u datotečnom sustavu, baš kao kod FIFO-a; ne treba fork ni nasljeđivanje deskriptora kao kod anonimnih cjevovoda.

Ono što naš primjer ne pokazuje, a što je u praksi vrlo bitno: kako se uskladiti kad više procesa istovremeno piše u istu memoriju. Bez dodatnog mehanizma može doći do situacija u kojima dva procesa istovremeno pišu u isti segment dijeljene memorije, pri čemu jedan proces može “pregaziti” podatke koje je upisao drugi i podatak postaje neispravan. Ovaj problem zovemo *race condition* (utrkivanje za resursom). U sljedećoj sekciji vidjet ćemo kako on nastaje i kako se može riješiti pomoću **semafora**.

7.5 Semafori: upravljanje pristupom dijeljenim resursima

Vratimo se na primjer iz uvoda: dva ili više procesa inkrementiraju brojač koji se nalazi u dijeljenoj memoriji. Zadatak je naizgled jednostavan — svaki put kada u bilo kojem od procesa nastupi određeno stanje, proces inkrementira brojač. Pri tom nas u ovom primjeru ne zanima koje je to stanje ili događaj koji bi proces trebao detektirati — taj dio odvija se u lokalnom memorijskom prostoru procesa i nema nikakvog utjecaja na varijablu u dijeljenoj memoriji. Jedino što proces mora napraviti u trenutku kada stanje detektira je jednostavna inkrementacija count++, pri čemu je varijabla count u dijeljenoj memoriji.

Prisjetimo se koncepta atomskih operacija: atomska operacija jest niz koraka koji se ili u potpunosti izvode, ili se ne izvodi niti jedan korak — nema mogućnosti da operacija bude prekinuta na pola. Možete li se sjetiti operacije jednostavnije od count++ i postoji li uopće šansa da bi ova naredba mogla biti prekinuta, tj. da se ne izvodi atomski?

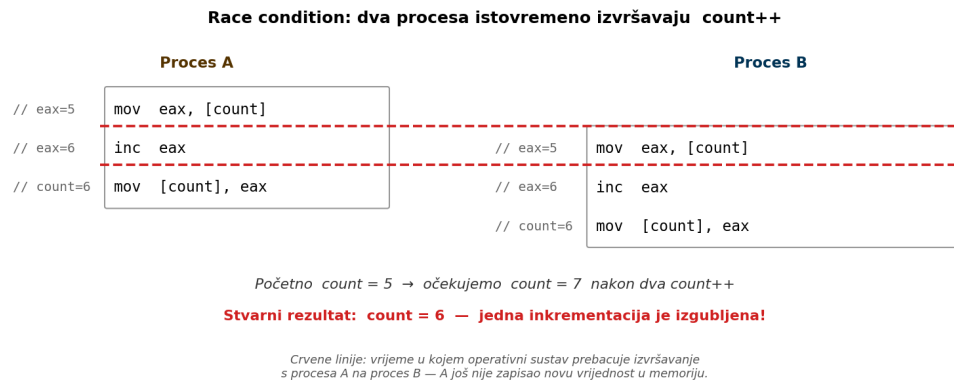
Zapravo, i te kako postoji, a direktna je posljedica arhitekture digitalnih računala kod kojih se sva aritmetika nad varijablama odvija u procesoru (CPU), a ne izravno u memoriji (RAM). Svaka aritmetička operacija nad bilo kojom varijablom u memoriji zapravo se odvija u tri zasebna koraka:

1. **Učitaj** trenutnu vrijednost iz memorije u registar procesora.
2. **Uvećaj** vrijednost u registru za 1.
3. **Pohrani** novu vrijednost iz registra natrag u memoriju.

Na strojnom jeziku (asembleru) x86 obitelji procesora, ako je `count` varijabla u memoriji a `eax` registar, izraz `count++` prevodi se otprilike u ove tri instrukcije:

```
mov  eax, [count]    ; korak 1: učitaj count iz memorije u registar eax
inc  eax             ; korak 2: povecaj eax za 1
mov  [count], eax    ; korak 3: pohrani vrijednost registra eax natrag u count
```

Operacijski sustav u svakom trenutku može prekinuti izvršavanje procesa — između bilo koja dva koraka — i pustiti drugi proces da nastavi. Ako oba procesa rade nad istim podatkom u dijeljenoj memoriji, lako se dogodi situacija prikazana na slici 7.3:



Slika 7.3: Stanje utrke (race condition) pri inkrementaciji dijeljenog brojača

Oba procesa su izvršila po jedno povećanje, ali konačna vrijednost u memoriji je 6, a ne 7 kako bismo očekivali. Jedna inkrementacija je **izgubljena**, jer su oba procesa pročitala istu staru vrijednost prije nego što je itko stigao zapisati novu. Dok god se procesi izvršavaju neometano (svaki dovrši sva tri koraka prije nego se prebaci na drugog), sve radi ispravno; problem se javlja samo kad ih se “pogode” preklopiti baš oko iste lokacije u strojnom kodu. To čini ovu vrstu pogreške posebno opasnom — javlja se neredovito, i pojavljuje se baš onda kad sustav ima najviše posla.

7.5.0.1 Demonstracija: dijeljeni brojač bez sinkronizacije

- **shm_brojac.c** — dva procesa (roditelj i dijete) dijele jedan cjelobrojni brojač u shm objektu i svaki ga povećava milijun puta. Očekivani rezultat na kraju je $2 \times 1\,000\,000 = 2\,000\,000$.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/wait.h>

#define SHM_NAME "/moj_brojac"
#define ITERACIJA 1000000

int main(void) {
    int fd;
    int *brojac;
    pid_t pid;

    fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    if (fd < 0) { perror("shm_open"); return 1; }
    if (ftruncate(fd, sizeof(int)) < 0) { perror("ftruncate"); return 1; }
```

```

brojac = mmap(NULL, sizeof(int), PROT_READ | PROT_WRITE,
              MAP_SHARED, fd, 0);
if (brojac == MAP_FAILED) { perror("mmap"); return 1; }

*brojac = 0;

pid = fork();
if (pid < 0) { perror("fork"); return 1; }

/* oba procesa povecavaju brojac istovremeno - bez sinkronizacije */
for (int i = 0; i < ITERACIJA; i++)
    (*brojac)++;

if (pid == 0) {
    /* dijete je gotovo - izlazi (inace bi i ono ispisalo rezultat) */
    munmap(brojac, sizeof(int));
    close(fd);
    return 0;
}

/* roditelj ceka dijete kako ne bi ostao zombie proces, pa
 * tek onda procita i ispise konacnu vrijednost brojaca */
wait(NULL);
printf("Konacna vrijednost: %d (ocekivano: %d)\n",
       *brojac, 2 * ITERACIJA);

munmap(brojac, sizeof(int));
close(fd);
shm_unlink(SHM_NAME);
return 0;
}

```

Pokrenimo nekoliko puta zaredom:

```

$ ./shm_brojac
Konacna vrijednost: 1141112 (ocekivano: 2000000)
$ ./shm_brojac
Konacna vrijednost: 1078418 (ocekivano: 2000000)
$ ./shm_brojac
Konacna vrijednost: 2000000 (ocekivano: 2000000)
$ ./shm_brojac
Konacna vrijednost: 1115554 (ocekivano: 2000000)
$ ./shm_brojac
Konacna vrijednost: 2000000 (ocekivano: 2000000)

```

Različiti pokušaji daju različite rezultate — ponekad točan, ponekad manje od očekivanog. Ovo je race condition u praksi.

Ako primijetite da na svom sustavu gotovo svaki put dobivate točnu konačnu vrijednost, možete doraditi program tako da pokrene više procesa djece (npr. tri, četiri ili više) koji bi svi paralelno s roditeljem inkrementirali isti brojač. Što više procesa istodobno radi nad istom varijablom, to je veća šansa da se njihove “load → increment → store” sekvence preklope, pa će neispravna konačna vrijednost biti znatno češća. Ostavljamo to čitatelju kao vježbu: izmijenite `shm_brojac.c` tako da umjesto jednog `fork`-a poziv stoji u petlji koja pokrene `N` djece, a roditelj na kraju pričekava sva. Probajte različite vrijednosti `N`-a i različite brojeve iteracija pa promatrajte koliko često rezultat odstupa od očekivanog.

7.5.1 Semafor kao sinkronizacijski mehanizam

Semafor je sinkronizacijska primitiva koja čuva cjelobrojnu vrijednost i podržava dvije atomske operacije:

- **wait** (povijesno ime: **P**) — smanji vrijednost semafora za 1; ako bi rezultat bio negativan, blokiraj dok netko drugi ne pozove **post**.

- **post** (povijesno ime: V) — povećaj vrijednost semafora za 1; ako su procesi blokirani u wait-u, jedan od njih se odblokira.

Imena P i V potječu iz originalnog rada “*Cooperating Sequential Processes*” iz 1965. godine, koji je objavljen 1968. i zabilježen u arhivi E. W. Dijkstra pod oznakom EWD 123 [2] — Dijkstra je bio Nizozemac i izabrao je slova prema nizozemskim glagolima *passeren* (“proći”) odnosno *probeer te verlagen* (“probaj smanjiti”) za P, te *verhogen* (“povisiti”) za V. Iako su P i V i danas često spominjani u literaturi (osobito akademskoj), u praksi se znatno češće koriste opisnija imena wait i post.

Inicijaliziramo li semafor na 1 i koristimo li ga kao zaštitu kritične sekcije — dijela koda koji ne smije izvršavati više procesa istovremeno — semafor djeluje kao **mutex** (engl. *mutual exclusion lock*). Prvi proces uđe u kritičnu sekciju (wait smanji vrijednost s 1 na 0); svi ostali procesi koji nakon toga pozovu wait blokiraju dok proces koji je semafor zaključao ne završi i ne pozove post (vrati vrijednost na 1, oslobađa jednog koji je čekao). Drugačije inicijalne vrijednosti omogućuju složenije obrasce sinkronizacije (npr. semafor inicijaliziran na N dopušta N procesa istodobno u kritičnoj sekciji).

POSIX standard nudi dvije inačice semafora — **imenovane** (identificirani putanjom kao i shm objekti, dostupni nepovezanim procesima) i **anonimne** (žive u memoriji, dostupni samo procesima koji ih dijele). Mi ćemo koristiti imenovane semafore jer su jednostavniji za upotrebu i pristupa im se gotovo identično kao shm objektima.

```
#include <semaphore.h>

sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value);
int sem_wait(sem_t *sem);
int sem_post(sem_t *sem);
int sem_close(sem_t *sem);
int sem_unlink(const char *name);
```

7.5.1.1 Funkcija sem_open()

Stvara ili otvara imenovani semafor.

Povratna vrijednost: pokazivač na sem_t u slučaju uspjeha, SEM_FAILED u slučaju greške.

Argumenti:

- **name** — ime semafora (kao kod shm: počinje s /).
- **oflag** — 0_CREAT, eventualno 0_EXCL.
- **mode** — prava pristupa (samo pri stvaranju).
- **value** — početna vrijednost (samo pri stvaranju). Za “mutex” obrazac koristi se 1.

7.5.1.2 Funkcije sem_wait() i sem_post()

sem_wait smanjuje vrijednost semafora za 1, blokira ako je već 0. sem_post povećava za 1, oslobađa eventualno blokirane procese.

Povratna vrijednost (obje): 0 u slučaju uspjeha, -1 u slučaju greške.

7.5.1.3 Funkcije `sem_close()` i `sem_unlink()`

`sem_close` zatvara *deskriptor* semafora u trenutnom procesu (semafor i dalje postoji u sustavu). `sem_unlink` briše imenovani semafor iz sustava (kao `shm_unlink` za `shm` objekte).

Povratna vrijednost (obje): 0 u slučaju uspjeha, -1 u slučaju greške.

7.5.1.4 Demonstracija: dijeljeni brojač sa semaforom

Sad ćemo isti `shm_brojac` proširiti tako da je svaka inkrementacija zaštićena binarnim semaforom — drugim riječima, samo jedan proces u jednom trenutku smije ulaziti u "load → increment → store" sekvencu.

- `shm_brojac_sem.c` — dijeljeni brojač sa sinkronizacijom.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <semaphore.h>

#define SHM_NAME "/moj_brojac"
#define SEM_NAME "/moj_sem"
#define ITERACIJA 1000000

int main(void) {
    int fd;
    int *brojac;
    sem_t *sem;
    pid_t pid;

    fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    if (fd < 0) { perror("shm_open"); return 1; }
    if (ftruncate(fd, sizeof(int)) < 0) { perror("ftruncate"); return 1; }
    brojac = mmap(NULL, sizeof(int), PROT_READ | PROT_WRITE,
                  MAP_SHARED, fd, 0);
    if (brojac == MAP_FAILED) { perror("mmap"); return 1; }
    *brojac = 0;

    /* binarni semafor inicijaliziran na 1 */
    sem = sem_open(SEM_NAME, O_CREAT, 0666, 1);
    if (sem == SEM_FAILED) { perror("sem_open"); return 1; }

    pid = fork();
    if (pid < 0) { perror("fork"); return 1; }

    /* oba procesa povecavaju brojac, pristup zasticen semaforom */
    for (int i = 0; i < ITERACIJA; i++) {
        sem_wait(sem);
        (*brojac)++;
        sem_post(sem);
    }

    if (pid == 0) {
        sem_close(sem);
        munmap(brojac, sizeof(int));
        close(fd);
        return 0;
    }

    wait(NULL);
    printf("Konačna vrijednost: %d (očekivano: %d)\n",
           *brojac, 2 * ITERACIJA);

    sem_close(sem);
    sem_unlink(SEM_NAME);
}
```

```
munmap(brojac, sizeof(int));
close(fd);
shm_unlink(SHM_NAME);
return 0;
}
```

Pokretanje:

```
$ ./shm_brojac_sem
Konacna vrijednost: 2000000 (ocekivano: 2000000)
$ ./shm_brojac_sem
Konacna vrijednost: 2000000 (ocekivano: 2000000)
$ ./shm_brojac_sem
Konacna vrijednost: 2000000 (ocekivano: 2000000)
```

Sada je rezultat **uvijek točan**, neovisno o tome kako se procesi međusobno prepleću — `sem_wait` osigurava da samo jedan proces u danom trenutku radi `(*brojac)++`. Cijena je manja brzina jer svaki ulazak u kritičnu sekciju zahtijeva zaključavanje semafora (`sem_wait`), a svaki izlazak njegovo otključavanje (`sem_post`) — a obje operacije imaju svoj trošak. Ipak, kod gdje su podaci točni je gotovo uvijek bolji od bržeg koji daje pogrešne rezultate.

Napomena: u poglavlju o `pthread`-ima vraćamo se na ovu problematiku, ali sinkronizaciju kritične sekcije rješavamo pomoću **mutex-a** — sinkronizacijske primitive koja je konceptualno identična binarnom semaforu, ali se nalazi u drugoj biblioteci i koristi se u kontekstu niti unutar istog procesa.

7.6 POSIX redovi poruka

Cjevovodi i FIFO-i prenose **niz neformatiranih bajtova** — ako primatelj ne zna unaprijed gdje jedna poruka završava i druga počinje, mora si sam izgraditi protokol (npr. zaglavlje s duljinom poruke). **Redovi poruka** rješavaju taj problem: prenose **diskretne, atomske poruke** s definiranim granicama. Dodatno nude i **prioritete** — poruka višeg prioriteta čita se prije poruke nižeg, neovisno o redoslijedu slanja.

POSIX redovi poruka identificiraju se imenom kao i `shm/sem` objekti.

```
#include <fcntl.h>
#include <sys/stat.h>
#include <mqueue.h>

mqd_t  mq_open(const char *name, int oflag, ...);
int     mq_send(mqd_t mqdes, const char *msg_ptr,
               size_t msg_len, unsigned int msg_prio);
ssize_t mq_receive(mqd_t mqdes, char *msg_ptr,
                  size_t msg_len, unsigned int *msg_prio);
int     mq_close(mqd_t mqdes);
int     mq_unlink(const char *name);
```

7.6.0.1 Funkcija `mq_open()`

Otvora (ili stvara) red poruka. Pri stvaranju treba dodatne argumente: `mode` (prava) i `attr` (atributi reda — najvažniji su `mq_maxmsg` koliko poruka može stati u red, i `mq_msgsize` najveća veličina jedne poruke u bajtovima).

Povratna vrijednost: u slučaju uspjeha vraća se *deskriptor* reda poruka tipa `mqd_t`; u slučaju greške vraća se vrijednost `(mqd_t)-1` — odnosno vrijednost `-1`

eksplicitno pretvorena u tip `mqd_t`. Razlog ovakvog zapisa je u tome što POSIX standard ne propisuje da `mqd_t` mora biti cjelobrojni tip (na nekim sustavima može biti pokazivač ili struktura), pa se "neispravna vrijednost" mora eksplicitno označiti pretvorbom (engl. *cast*).

7.6.0.2 Funkcija `mq_send()`

Stavlja poruku u red. Ako je red pun, blokira (osim ako je red otvoren u neblokirajući mod).

Povratna vrijednost: 0 u slučaju uspjeha, -1 u slučaju greške.

Argumenti:

- `mqdes` — deskriptor reda.
- `msg_ptr` — pokazivač na podatke poruke.
- `msg_len` — broj bajtova u poruci (mora biti $\leq$ `mq_msgsize`).
- `msg_prio` — prioritet (0 do `MQ_PRIO_MAX-1`; veći broj znači viši prioritet).

7.6.0.3 Funkcija `mq_receive()`

Uzima sljedeću poruku iz reda. Ako je red prazan, blokira (osim ako je red u neblokirajućem modu). **Uvijek se uzima poruka najvišeg prioriteta**; ako više poruka ima isti prioritet, uzima se najstarija.

Povratna vrijednost: broj pročitanih bajtova, ili -1 u slučaju greške.

Argumenti:

- `mqdes` — deskriptor reda.
- `msg_ptr` — međuspremnik za poruku.
- `msg_len` — veličina međuspremnika (mora biti $\geq$ `mq_msgsize` koja je upisana u attribute reda — inače se javlja greška).
- `msg_prio` — pokazivač u koji se upisuje prioritet primljene poruke (može biti NULL ako nas prioritet ne zanima).

7.6.1 Primjer: razmjena poruka kroz red

- `mq_posalji.c` — šalje poruku u red, s opcionalno zadanim prioritetom.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <mqueue.h>
#include <sys/stat.h>

#define MQ_NAME "/moj_mq"
#define MAX_MSG_SIZE 256

int main(int argc, char *argv[]) {
    mqd_t mq;
    const char *poruka;
    unsigned prioritet = 0;

    if (argc < 2) {
        poruka = "Pozdrav kroz red poruka!";
    } else {
        poruka = argv[2];
        if (argc >= 3) prioritet = (unsigned)atoi(argv[3]);
    }
}
```

```

struct mq_attr attr;
attr.mq_flags = 0;
attr.mq_maxmsg = 10;
attr.mq_msgsize = MAX_MSG_SIZE;
attr.mq_curmsgs = 0;

mq = mq_open(MQ_NAME, O_CREAT | O_WRONLY, 0666, &attr);
if (mq == (mqd_t)-1) {
    perror("mq_open");
    return 1;
}

if (mq_send(mq, poruka, strlen(poruka) + 1, prioritet) < 0) {
    perror("mq_send");
    mq_close(mq);
    return 1;
}

printf("Poslana poruka (prioritet=%u): %s\n", prioritet, poruka);
mq_close(mq);
return 0;
}

```

- **mq_primi.c** — čita sljedeću poruku iz reda i ispisuje je s prioritetom.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <mqueue.h>

#define MQ_NAME "/moj_mq"
#define MAX_MSG_SIZE 256

int main(void) {
    mqd_t mq;
    char buf[MAX_MSG_SIZE];
    unsigned prioritet;
    ssize_t n;

    mq = mq_open(MQ_NAME, O_RDONLY);
    if (mq == (mqd_t)-1) {
        perror("mq_open");
        return 1;
    }

    printf("Cekam poruku...\n");
    n = mq_receive(mq, buf, MAX_MSG_SIZE, &prioritet);
    if (n < 0) {
        perror("mq_receive");
        mq_close(mq);
        return 1;
    }

    printf("Primljeno (prioritet=%u): %s\n", prioritet, buf);

    mq_close(mq);
    mq_unlink(MQ_NAME);
    return 0;
}
/* ukloni red kad smo gotovi */

```

Demonstracija prioriteta (više poruka, primatelj ih čita po prioritetu):

```

## posalji više poruka razlicitih prioriteta:
$ ./mq_posalji "Niska vaznost" 1
Poslana poruka (prioritet=1): Niska vaznost
$ ./mq_posalji "Visoka vaznost" 9
Poslana poruka (prioritet=9): Visoka vaznost
$ ./mq_posalji "Srednja vaznost" 5
Poslana poruka (prioritet=5): Srednja vaznost

```

```
## primi ih jednu po jednu (najvazniji prvi):
$ ./mq_primi
Cekam poruku...
Primljeno (prioritet=9): Visoka vaznost

$ ./mq_primi
Cekam poruku...
Primljeno (prioritet=5): Srednja vaznost

$ ./mq_primi
Cekam poruku...
Primljeno (prioritet=1): Niska vaznost
```

Iako su poruke poslane redom 1-9-5, primljene su 9-5-1 — **u opadajućem redoslijedu prioriteta**, kao što se i očekuje.

7.6.1.1 Perzistentnost reda poruka i alati ljsuke

Kao i POSIX dijeljena memorija, i POSIX red poruka je **perzistentan** — nakon što proces koji je slao poruke završi, red i sve njegove poruke ostaju u jezgri sve dok ih netko ne pročita ili dok se red eksplicitno ne ukloni pozivom `mq_unlink` (ili dok se sustav ponovno ne pokrene). Možete to lako provjeriti tako da pošaljete nekoliko poruka, prekinete primatelja prije nego što je pročitao sve, i kasnije ga ponovno pokrenete — preostale poruke i dalje su dostupne.

Na Linuxu, POSIX redove poruka možemo pregledavati i kroz datotečni sustav. Jezgra im pridružuje virtualni `mqueue` datotečni sustav, koji je na većini distribucija već montiran pod `/dev/mqueue` (ako nije, montiramo ga sa `sudo mount -t mqueue none /dev/mqueue`). Standardni alati ljsuke rade nad ovim direktorijem kao i nad bilo kojim drugim datotečnim sustavom:

```
$ ls -l /dev/mqueue/
-rw-rw-rw- 1 dkrst users 80 May  8 14:00 moja_poruka
```

Naredba `cat` nad takvom “datotekom” ne ispisuje sadržaj poruka, već metapodatke o redu — koliko bajtova podataka red trenutno sadrži, je li netko registriran za asinkrono obavješćavanje o pristizanju novih poruka i slično:

```
$ cat /dev/mqueue/moja_poruka
QSIZE:42 NOTIFY:0 SIGNO:0 NOTIFY_PID:0
```

`QSIZE` je ukupan broj bajtova svih poruka koje se nalaze u redu. Stvarni sadržaj poruka iz ljsuke nije moguće pročitati — za to nam treba program koji koristi `mq_receive` (poput našeg `mq_primi`). Ovdje vidimo razliku u odnosu na FIFO datoteke, gdje smo sadržaj mogli pročitati običnim `cat`-om: kod redova poruka ne radi se o linearnom toku bajtova nego o strukturi poruka s prioritetima, a takav model ne uklapa se u jednostavnu apstrakciju datoteke kao niza bajtova.

Red poruka možemo i ukloniti iz ljsuke, što je ekvivalentno pozivu `mq_unlink`:

```
$ rm /dev/mqueue/moja_poruka
```

7.7 Mapiranje datoteka u memoriju

Funkciju `mmap()` upoznali smo ranije u kontekstu dijeljene memorije. Ona ima i drugu, jednako važnu primjenu: **mapiranje regularnih datoteka** u adresni prostor procesa. Umjesto da datoteku čitamo `read`-om u međuspremnik, dobivamo pokazivač na memorijski blok koji je datoteka — pristupamo bajtovima datoteke kao da su elementi polja.

Razmotrimo razliku:

```
/* klasicni pristup: read u medjusprennik */
char buf[1024];
int fd = open("podaci.bin", O_RDONLY);
read(fd, buf, sizeof(buf));
char prvi = buf[0];

/* mmap pristup: datoteka mapirana u memoriju */
int fd = open("podaci.bin", O_RDONLY);
struct stat st;
fstat(fd, &st);
char *podaci = mmap(NULL, st.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
char prvi = podaci[0];
```

Prednosti mapiranja:

- **Bez kopiranja** — jezgra “pretvara” stranice datoteke u stranice u memoriji procesa. Stranica (engl. *page*) je osnovna jedinica kojom jezgra upravlja memorijom: komad memorije fiksne veličine, tipično 4 KB. Možemo ju zamisliti kao memorijski analog bloku podataka na disku — datoteka je na disku podijeljena u blokove, a u memoriji se ti blokovi drže u stranicama jezginog *cachea* (page cache). Mapiranjem datoteke u memoriju procesa zapravo govorimo jezgri da stranice iz tog *cachea* treba učiniti dostupnima u virtualnom adresnom prostoru našeg procesa. Pri prvom pristupu nekoj stranici, jezgra je tek tada učita s diska (engl. *demand paging*) — pa ne plaćamo cijenu učitavanja onoga što ne pročitamo.
- **Slučajan pristup** — možemo skočiti u sredinu datoteke (`podaci[ogromni_offset]`) bez `lseek`-a.
- **Više procesa može mapirati istu datoteku** — i to na potpuno isti način kao što smo to radili kod dijeljene memorije: svaki proces vlastitim pozivom `mmap` dobiva pokazivač u svoj adresni prostor, ali svi ti pokazivači zapravo gledaju u istu fizičku regiju memorije (iste stranice u *cacheu* jezgre). Tako se velike datoteke (npr. baze podataka) mogu efikasno dijeliti između više procesa bez dupliciranja u memoriji. Sjetimo se naše opaske iz uvoda sekcije o dijeljenoj memoriji: jezgra ne pravi nikakvu razliku između `shm` objekta i obične datoteke — i jedno i drugo se preko `mmap`-a mapira u memoriju procesa po identičnom mehanizmu.

Nedostaci:

- **Velicina mora biti unaprijed poznata** — `mmap` zahtijeva da znamo koliko bajtova mapirati. Za dinamičke datoteke koje rastu (npr. log datoteke) ovo nije praktično.
- **Konačna velicina mapirane memorije** — ako korištenjem pokazivača pišemo izvan granica dodijeljenog memorijskog segmenta, dobijemo `SIGSEGV`.
- **Greške se manifestiraju kao SIGBUS/SIGSEGV** — signal `SIGSEGV` (*segmentation violation*) javlja se kad pristupimo memoriji koja nam ne pripada (npr. čitanje ili pisanje izvan mapirane regije), dok `SIGBUS` (*bus error*) signalizira greške pri pristupu *unutar* mapirane regije koje jezgra ne uspijeva poslužiti — najčešće zato što stranica više nije dostupna na disku (npr. datoteka je u međuvremenu skraćena ili obrisana, pa pokušaj učitavanja stranice koja je iza novog kraja datoteke završava ovim signalom). Uobičajena (default) akcija za oba signala je prekidanje procesa, što je teže za debugirati od greške koju vrati `read()`.

Za male datoteke ili kad treba upravljati streaming-om, klasični `read/write` ostaju primjereniji. Za velike datoteke kojima se pristupa nasumično, ili koje treba dijeliti između procesa, `mmap` je često znatno brži i elegantniji.

7.7.0.1 Analogija s cjevovodima

Sad kad smo upoznali oba lica mmap-a — i mapiranje shm objekta, i mapiranje regularne datoteke — vrijedi povući paralelu s mehanizmima koje smo ranije upoznali. UNIX je vrlo dosljedan u tome kako razdvaja **anonimne** i **imenovane** varijante istog koncepta:

	Bez imena u datotečnom sustavu	S imenom u datotečnom sustavu
Tok bajtova	anonimni cjevovod (pipe())	FIFO (mkfifo())
Memorijska regija	mmap s MAP_ANONYMOUS	shm_open + mmap (ili mmap na regularnu datoteku)

Anonimna varijanta uvijek se može dijeliti samo između srodnih procesa (dijete-roditelj, ili procesi koji imaju zajedničkog pretka), jer se rukovatelj resursom nasljeđuje kroz fork. Imenovana varijanta dostupna je svakom procesu koji zna ime u datotečnom sustavu — bez obzira jesu li procesi povezani. Isti obrazac, primijenjen u dva različita konteksta (tokovi bajtova naspram regija memorije).

7.8 System V IPC — kratko upoznavanje

Prije nego što je POSIX standardizirao IPC funkcije koje smo upravo upoznali, AT&T-jev System V UNIX je imao vlastiti skup mehanizama: System V poruke, semafore i dijeljenu memoriju. Iako su POSIX inačice danas preporučene za nove programe, System V API i dalje postoji na svim modernim UNIX sustavima i čest je u starijim kodovima — pa je dobro znati prepoznati ga.

System V mehanizmi imaju jedinstven obrazac:

1. **Identifikator** je cjelobrojni “ključ” (key_t), ne putanja. Dobiva se ili pozivom ftok() (koji od putanje + brojke generira ključ) ili specijalnom vrijednošću IPC_PRIVATE (znači: stvori novi red kojeg će dijeliti samo srodni procesi).
2. **Stvaranje/otvaranje** se radi *get funkcijom: msgget, semget, shmget. Vraćaju cjelobrojni “id”.
3. **Operacije** koriste taj id: msgsnd/msgrcv, semop, shmat/shmdt.
4. **Brisanje** se radi *ctl funkcijom s naredbom IPC_RMID: msgctl(id, IPC_RMID, NULL).

Važno je naglasiti: System V objekti **opstaju i nakon završetka procesa koji ih je stvorio** — dok ih netko eksplicitno ne obriše ili dok se sustav ponovno ne pokrene. Ako program zaboravi pozvati IPC_RMID, objekt ostaje u sustavu kao “smeće”. Sustav se može pregledati naredbom **ipcs**, a ručno čistiti naredbom **ipcrm**:

```
$ ipcs                # prikazi sve aktivne System V IPC objekte
$ ipcrm -q <msgid>    # obriši red poruka
$ ipcrm -m <shmid>    # obriši shared memory segment
$ ipcrm -s <semid>    # obriši semafor
```

7.8.1 Primjer: System V redovi poruka

Po uzoru na POSIX par mq_posalji/mq_primi, napraviti ćemo i ovdje dva mala programa: jedan koji šalje poruku u System V red, i drugi koji iz tog reda čita. Razlika u odnosu na POSIX je u načinu identificiranja reda — umjesto putanje sa / na početku, ovdje koristimo cjelobrojni **ključ** (key_t), koji je tipično generiran

funkcijom `ftok` iz neke putanje u datotečnom sustavu i jednog dodatnog `char` identifikatora. Dva procesa koja koriste isti par (putanja, ID) dobit će isti ključ, pa će pristupiti istom redu.

- **`sysv_msg_posalji.c`** — šalje jednu poruku u red. Tekst poruke preuzima iz argumenta naredbenog retka, ili koristi zadanu vrijednost ako argumenta nema.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>

#define KEY_PATH "/tmp"
#define KEY_ID 'K'
#define MAX_TEXT 128

struct moja_poruka {
    long mtype;
    char mtext[MAX_TEXT];
};

int main(int argc, char *argv[]) {
    key_t kljuc;
    int msqid;
    struct moja_poruka p;
    const char *poruka;

    if (argc < 2)
        poruka = "Pozdrav iz System V reda!";
    else
        poruka = argv[2];

    /* generiraj kljuc iz putanje i char ID-a -- isti par (putanja, id) u
     * razlicitim procesima daje isti kljuc, pa procesi nadju isti red */
    kljuc = ftok(KEY_PATH, KEY_ID);
    if (kljuc < 0) { perror("ftok"); return 1; }

    /* otvori ili stvori red poruka */
    msqid = msgget(kljuc, IPC_CREAT | 0666);
    if (msqid < 0) { perror("msgget"); return 1; }

    /* posalji poruku tipa 1 */
    p.mtype = 1;
    strncpy(p.mtext, poruka, MAX_TEXT - 1);
    p.mtext[MAX_TEXT - 1] = '\0';

    if (msgsnd(msqid, &p, strlen(p.mtext) + 1, 0) < 0) {
        perror("msgsnd");
        return 1;
    }

    printf("Poslano: %s\n", p.mtext);
    return 0;
}
```

Bitna razlika prema POSIX `mq_*` API-ju je već vidljiva u definiciji strukture poruke: System V poruka nije puki niz bajtova nego **struktura s `long mtype` poljem na početku**. To polje koristi se i za “kanale” — u istom redu mogu biti poruke različitih `mtype` vrijednosti, a `msgrcv` može filtrirati po njima (npr. “uzmi prvu poruku tipa 5”).

- **`sysv_msg_primi.c`** — čita jednu poruku iz istog reda i ispisuje ju.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
```

```

#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>

#define KEY_PATH "/tmp"
#define KEY_ID 'K'
#define MAX_TEXT 128

struct moja_poruka {
    long mtype;
    char mtext[MAX_TEXT];
};

int main(void) {
    key_t kljuc;
    int msqid;
    struct moja_poruka p;
    ssize_t n;

    kljuc = ftok(KEY_PATH, KEY_ID);
    if (kljuc < 0) { perror("ftok"); return 1; }

    /* otvori postojeći red (bez IPC_CREAT) */
    msqid = msgget(kljuc, 0666);
    if (msqid < 0) { perror("msgget"); return 1; }

    /* primi poruku bilo kojeg tipa (msgtyp=0) */
    n = msgrcv(msqid, &p, MAX_TEXT, 0, 0);
    if (n < 0) { perror("msgrcv"); return 1; }

    printf("Primljeno (tip=%ld): %s\n", p.mtype, p.mtext);
    return 0;
}

```

Pokrenimo `sysv_msg_posalji` nekoliko puta s različitim porukama:

```

$ ./sysv_msg_posalji "Prva poruka"
Poslano: Prva poruka
$ ./sysv_msg_posalji "Druga poruka"
Poslano: Druga poruka
$ ./sysv_msg_posalji "Trecu poruka"
Poslano: Trecu poruka

```

Iako su pošiljaatelji već završili s radom, sve tri poruke i dalje žive u sustavu — System V red poruka opstaje neovisno o procesima koji su ga koristili. Možemo se u to uvjeriti pregledom svih aktivnih System V redova naredbom `ipcs`:

```

$ ipcs -q

----- Message Queues -----
key      msqid      owner      perms      used-bytes   messages
0x4b000155 0          dkrst      666        38           3

```

Vidimo naš red — identificiran je generiranim ključem (0x4b000155), dodijeljen mu je `msqid` 0, sadrži ukupno 3 poruke od 38 bajtova. Sada pokrenimo primatelja, koji će pročitati jednu poruku:

```

$ ./sysv_msg_primi
Primljeno (tip=1): Prva poruka

```

Provjerimo stanje reda nakon čitanja:

```

$ ipcs -q

----- Message Queues -----
key      msqid      owner      perms      used-bytes   messages
0x4b000155 0          dkrst      666        26           2

```

Red sad sadrži 2 poruke (26 bajtova) — `msgrcv` je atomski uklonio prvu poruku iz reda. Pošto ni POSIX ni standardni System V API nemaju ekvivalent “peek”

operacije (čitanje bez uklanjanja), poruka koju jednom pročitamo nepovratno je uklonjena iz reda.

Za razliku od POSIX redova, System V red **neće** automatski nestati kad svi procesi zatvore svoje deskriptore — *ne postoji* `mq_unlink` ekvivalent koji se automatski poziva. Red moramo eksplicitno ukloniti, što u kodu radimo pozivom `msgctl(msqid, IPC_RMID, NULL)`, a iz ljske naredbom `ipcrm`:

```
$ ipcrm -q 0
$ ipcs -q
```

```
----- Message Queues -----
key          msqid      owner          perms        used-bytes   messages
```

Bez eksplicitnog brisanja, red ostaje u sustavu i nakon završetka svih procesa koji su ga koristili — u praksi često zarobljen i nakon više dana, sve dok administrator ručno ne obriše ili dok se sustav ne resetira. Ovo je čest izvor “smeća” u sustavima koji intenzivno koriste System V IPC.

System V semafori i dijeljena memorija imaju analogan API (`semget/semop/semctl` za semafore, `shmget/shmat/shmdt/shmctl` za dijeljenu memoriju). Konceptualno rade istu stvar kao POSIX inačice — samo s drugačijim načinom imenovanja i upravljanja.

Čitatelj koji želi dublje istražiti UNIX međuprocesnu komunikaciju — i POSIX i System V varijantu — može pogledati izvrsni *Beej's Guide to Unix IPC* [3], koji svakim mehanizmom prolazi s detaljnim primjerima i pristupačnim objašnjenjima. Vodič je besplatno dostupan online i predstavlja jednu od najboljih praktičnih referenci na ovu temu. Za teorijski temeljitiji i akademski pristup, isti materijal — uz dublje razmatranje detalja implementacije, prijenosnosti i specifičnih slučajeva — obrađen je u već citiranom djelu *Advanced Programming in the UNIX Environment*, Stevens & Rago [1].

7.9 Što smo zapravo radili

Vrijedi se na kraju ovog poglavlja na trenutak osvrnuti na sliku koja je nastala. Procesi u UNIX sustavu, ako žele surađivati, imaju na raspolaganju lepezu mehanizama — od najjednostavnijih (signali, cjevovodi) preko sofisticiranih (redovi poruka, dijeljena memorija) do mrežno-orijentiranih (socketi, koje obrađujemo u sljedećem poglavlju). Svaki mehanizam ima svoje mjesto:

- **Signali** za jednostavno obavješćavanje — kad nije važan sadržaj, samo da se nešto dogodilo.
- **Cjevovodi i FIFO** za jednosmjernan tok bajtova — klasika za “filterski” stil programa kakav vidimo svaki dan u UNIX ljsuci (`ls | grep | wc`).
- **Redovi poruka** kad nam treba strukturirana, atomska razmjena diskretnih poruka — eventualno s prioritetima.
- **Dijeljena memorija + semafori** za izrazito brzu razmjenu velikih količina podataka — uz oprez da svaki pristup zajedničkim podacima mora biti sinkroniziran.

Kao što su gotovo svi UNIX alati zapravo tanki sloj iznad sistemskih poziva, tako je i lepeza IPC mehanizama u biti zbirka pažljivo dizajniranih primitiva u jezgri. Razumijevanjem ovih primitiva razumijemo i kako rade veći sustavi koje koristimo svakodnevno: bazu podataka, web poslužitelje, procesne arhitekture

orkestracijskih alata. Iza svega stoji par desetaka sistemskih poziva, isti već desetljećima.

7.10 Prevođenje

Direktorij dolazi s priloženim Makefile-om koji prati iste konvencije kao i u ostalim poglavljima. Posebnost je u tome što **POSIX shm, semafori i redovi poruka traže linkanje s posebnim bibliotekama**:

- POSIX shared memory (shm_open itd.) — `-lrt`
- POSIX semafori (sem_open itd.) — `-lpthread`
- POSIX redovi poruka (mq_*) — `-lrt`

Ove dodatne biblioteke su u Makefile-u već navedene gdje su potrebne. System V IPC nema dodatnih ovisnosti.

```
make all          # gradi sve primjere
make cijev        # gradi pojedinačni primjer
make clean        # čisti generirane datoteke
```

7.11 Bibliografija

[1] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2013.

[2] E. W. Dijkstra, “Cooperating Sequential Processes”, in *Programming Languages: NATO Advanced Study Institute*, F. Genuys, Ed. London, U.K.: Academic Press, 1968, pp. 43–112. Rukopis: EWD 123, dostupno na: <https://www.cs.utexas.edu/users/EWD/transcriptions/EWD01xx/EWD123.html>. Pristupljeno: svibanj 2026.

[3] B. Hall, *Beej’s Guide to Unix IPC*. [Online]. Dostupno na: <https://beej.us/guide/bgipc/>. Pristupljeno: svibanj 2026.

Poglavlje 8

Višenitno programiranje

Stvaranje novog procesa sistemskim pozivom `fork` moćan je mehanizam koji nam daje mogućnost da u naše programe ugradimo paralelno izvršavanje. Međutim, stvaranje novog procesa ujedno je i skup mehanizam: podrazumijeva kopiranje cijelog adresnog prostora postojećeg procesa, dodjelu identifikatora i ažuriranje struktura u jezgri koje čuvaju informacije o aktivnim procesima. Pored toga, komunikacija među procesima zahtijeva eksplicitne mehanizme — cjevovode, signale, dijeljenu memoriju, redove poruka, semafore i tako dalje.

U ovom poglavlju upoznajemo **niti** (engl. *threads*) — lakšu jedinicu izvršavanja koja postoji *unutar* procesa. U literaturi na hrvatskom jeziku ponekad se umjesto naziva “nit” koristi i termin “dretva”; oba naziva označavaju isto. U engleskoj literaturi niti se često nazivaju i *lightweight processes* — “lagani procesi” — što dobro opisuje njihovu prirodu: imaju mnoge mogućnosti kao i procesi (paralelno izvršavanje, vlastiti tok upravljanja), ali bez troška kopiranja cijelog adresnog prostora i bez potrebe za posebnim IPC mehanizmima. Niti su odgovor na pitanje: kako iskoristiti više jezgri suvremenih procesora za ubrzanje računski zahtjevnih programa, a istovremeno omogućiti da dijelovi programa međusobno jednostavno dijele podatke?

8.1 Niti i procesi

Da bismo razumjeli odnos između niti i procesa, prisjetimo se što je proces. Proces je instanca programa u izvršavanju — apstrakcija koju operacijski sustav drži za svaku aktivnu izvršnu jedinicu. Svaki proces ima svoj jedinstveni identifikator (PID), vlastiti adresni prostor (memorija u kojoj su smješteni programski kod, statički podaci, hrpa i stog), vlastite deskriptore otvorenih datoteka, vlastitu tablicu rukovatelja signala, vlasnika i grupu, radni direktorij, varijable okruženja i niz drugih atributa. Svaki proces, kad ga jezgra raspoređuje na procesor, ima jedan tok izvršavanja — slijed instrukcija koji jezgra prati kroz brojač instrukcija (engl. *program counter*, PC) i sadržaj procesorskih registara.

Nit je upravo to — tok izvršavanja. Pri tom proces možemo promatrati kao “spremnik” resursa (adresni prostor, strojni kod programa, deskriptori otvorenih datoteka, ...), a niti su jedinice izvršavanja koje unutar tog spremnika žive. U svakom procesu imamo najmanje jednu nit — jedan tok izvršavanja, ali ih može biti i više, pri čemu sve niti dijele iste resurse i izvršavaju se unutar istog adresnog prostora.

Upravo ovo ključna je razlika između procesa i niti: procesi su međusobno izolirani, dok niti unutar istog procesa dijele zajedničke resurse.

Resurs	Vlasništvo
PID (identifikator procesa)	zajednički za sve niti procesa
Programski kod (text segment)	dijelev sve niti procesa
Globalne i statičke varijable	dijelev sve niti procesa
Hrpa (memorija dobivena malloc-om)	dijelev sve niti procesa
Otvoreni deskriptori datoteka	dijelev sve niti procesa
Trenutni radni direktorij, umask, vlasnik	dijelev sve niti procesa
Tablica rukovatelja signala	dijelev sve niti procesa
Stog (engl. <i>stack</i>)	vlastiti za svaku nit
Brojač instrukcija i procesorski registri	vlastiti za svaku nit
Identifikator niti (pthread_t)	jedinstven za svaku nit
Lokalna pohrana niti (TLS)	vlastita za svaku nit
Maska blokiranih signala	vlastita za svaku nit

Vlastiti stog svake niti je ključ za razumijevanje. Kad nit poziva funkciju, njezini argumenti, lokalne varijable i adresa povratka iz funkcije idu na njen stog — ne na neki “zajednički” stog procesa. Zato dvije niti mogu istovremeno biti unutar iste funkcije, svaka sa svojim privatnim lokalnim varijablama, bez ikakve interferencije. Globalne varijable, hrpa i ostale “zajedničke” strukture su mjesto gdje niti komuniciraju — i upravo zato im je potrebna sinkronizacija, ključna stavka za razumijevanje i upravljanje nitima u višenitnom procesu.

8.2 Raspoređivač (scheduler) i niti

Promislimo malo o načinu na koji raspoređivač upravlja procesima: kada na sustavu imamo više procesa nego resursa (procesorskih jezgri), raspoređivač upravlja procesima na način da ih izvršava u dijeljenom vremenu (engl. *time sharing*) — svaki proces dobiva na određeno vrijeme potrebne resurse, nakon kojeg ih prepušta drugim procesima dok čeka svoj red da nastavi izvršavanje. Međutim, što ako proces ima više tokova izvršavanja, tj. više niti: što je jedinica koju raspoređivač zapravo raspoređuje — proces ili nit?

Na suvremenom Linuxu (kao i na većini modernih UNIX sustava) odgovor je nit: raspoređivač vidi niti kao osnovne jedinice izvršavanja i svakoj niti dodjeljuje procesorsko vrijeme neovisno o drugima. Razmislimo o značenju ovog podatka: ukoliko unutar svog procesa koristimo više niti, vjerojatno ćemo dobiti više procesorskog vremena u odnosu na programe koji imaju samo jedan tok izvršavanja — samo jednu nit. Još važnije: ako raspoređivač svaku nit promatra nezavisno, dobra je šansa da će se različite niti unutar našeg procesa izvršavati nezavisno na različitim procesorskim jezgrama (gotovo sva moderna računala imaju više procesorskih jezgri). Ovo osigurava istinski paralelizam, ali otvara i mogućnost da dvije (ili više) niti koje se istovremeno izvršavaju pristupe istom podatku u zajedničkoj memoriji — što je idealan scenarij za stanje trke (race condition), koji smo već spominjali u ranijim poglavljima skripte.

Što to znači za pisanje programa? Dva su važna zaključka. Prvo, **niti se izvršavaju paralelno**, ne sekvencijalno — kad pokrenemo deset niti, njihov međusobni redoslijed izvršavanja je nepredvidiv i može se mijenjati od pokretanja do pokretanja. Drugo, **raspoređivač može prekinuti bilo koju nit u bilo**

kojem trenutku — i ne samo između “logičkih” instrukcija C koda nego doslovno između bilo koje dvije strojne instrukcije, što smo najbolje vidjeli na primjeru inkrementiranja cjelobrojne varijable (`i++`). To nas vodi u temu `race conditiona`, koju ćemo detaljno razraditi u nastavku.

Povijesna napomena. Današnji 1:1 model upravljanja nitima (svaka nit u korisničkom prostoru odgovara jednoj niti jezgre) nije oduvijek bio jedini pristup. Postojali su sustavi gdje su sve niti procesa dijelile jednu “kvotu” procesorskog vremena — tzv. *M:1* model ili *user-level threads*. Tu je raspoređivač na razini jezgre vidio samo proces, a raspoređivanje među nitima radila je biblioteka u korisničkom prostoru. Prednost takvog pristupa bila je u brzini stvaranja niti i prebacivanja konteksta među njima (sve se događalo u korisničkom prostoru, bez sistemskih poziva), ali uz veliki nedostatak: cijeli proces nije mogao iskoristiti više procesorskih jezgri jer je jezgri u stvari bio jedan tok izvršavanja. Postojali su i tzv. *hibridni M:N modeli* koji su pokušavali kombinirati prednosti oba pristupa. Današnji Linux koristi isključivo 1:1 model, čime je razvoj višenitnih programa znatno pojednostavljen — sve odluke o raspoređivanju donosi jezgra, a program ne mora razmišljati o tome u kakvom je odnosu njegova “logička” nit prema niti jezgre.

8.3 POSIX niti — pthreads

POSIX standard definira sučelje za rad s nitima koje se naziva **POSIX threads** ili kraće **pthreads**. Sve funkcije ovog API-ja imaju prefiks `pthread_` i deklarirane su u zaglavlju `<pthread.h>`. Implementacija pthreads-a na Linuxu naziva se **NPTL** (*Native POSIX Threads Library*) i dolazi kao dio biblioteke `libc`. Definitivna referenca za pthreads je *Programming with POSIX Threads*, Butenhof [1] — knjiga koja sustavno obrađuje svaki aspekt biblioteke, od osnova do graničnih slučajeva. Niti su također dobro pokrivene u *Advanced Programming in the UNIX Environment*, Stevens & Rago [2] (poglavlja 11 i 12), a šire koncepte niti kao osnovne apstrakcije operacijskog sustava (lightweight procesi, raspoređivanje niti, sinkronizacija) na hrvatskom jeziku obrađuje *Operacijski sustavi*, Budin, Golub, Jakobović & Jelenković [3].

Za prevođenje programa koji koriste pthreads potrebno je linkanje s pthread bibliotekom:

```
gcc program.c -lpthread -o program
```

Neke distribucije i neki kompajleri preferiraju oblik `-pthread` (s crticom, bez `l`), koji uz linkanje s bibliotekom postavlja i potrebne predprocesorske makroe (`_REENTRANT` ili `_POSIX_C_SOURCE`). Oba oblika funkcioniraju za naše primjere.

8.4 Stvaranje i terminiranje niti

Tri osnovne funkcije za rad s nitima su `pthread_create` (stvaranje niti), `pthread_exit` (eksplicitno terminiranje niti uz povratnu vrijednost) i `pthread_join` (čekanje da nit završi i dohvaćanje njene povratne vrijednosti). Sve tri deklarirane su u zaglavlju `<pthread.h>`. Sintaksu i način korištenja obradit ćemo u dvije cjeline: prvo stvaranje, a zatim par koji upravlja životnim ciklusom — terminiranje i prikupljanje rezultata.

8.4.1 Stvaranje niti — pthread_create

```
#include <pthread.h>

int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                  void *(*start_routine)(void *), void *arg);
```

Povratna vrijednost: 0 u slučaju uspjeha. U slučaju greške *ne postavlja se* errno nego se vraća sam kod greške (npr. EAGAIN ako je sustav iscrpio resurse za stvaranje nove niti). Ovo je razlika u odnosu na uobičajeni UNIX stil i izvor čestih grešaka — provjera s `perror(...)` neće funkcionirati izravno na povratnoj vrijednosti `pthread_create` funkcija; za ispis poruke o grešci treba koristiti `strerror(rezultat)`, gdje je rezultat povratna vrijednost funkcije `pthread_create`.

Argumenti:

- **thread** — pokazivač na varijablu tipa `pthread_t` u koju će se upisati identifikator nove niti.
- **attr** — atributi niti (veličina stoga, je li joinable ili detached, ...); ako predamo NULL, koriste se zadane vrijednosti.
- **start_routine** — pokazivač na funkciju koja će biti polazna točka nove niti; mora imati potpis `void (*)(void *)`, tj. prima jedan generički pokazivač kao argument i vraća jedan generički pokazivač kao rezultat.
- **arg** — pokazivač na argumente koji će biti predani polaznoj funkciji nove niti.

Nova nit počinje izvršavanje pozivom `start_routine(arg)`.

8.4.2 Terminiranje i prikupljanje rezultata niti — pthread_exit i pthread_join

Ove dvije funkcije čine logički par: jedna završava nit i ostavlja “iza sebe” povratnu vrijednost, druga čeka da nit završi i tu vrijednost pokupi.

```
#include <pthread.h>

void pthread_exit(void *retval);
int pthread_join(pthread_t thread, void **retval);
```

Nit može završiti na jedan od sljedećih načina:

- vraćanjem iz polazne funkcije (return s vrijednošću koja postaje povratna vrijednost niti);
- eksplicitnim pozivom `pthread_exit(retval)`;
- na zahtjev druge niti (`pthread_cancel`, vidi niže);
- ili završetkom cijelog procesa (npr. `exit` ili return iz main-a, što ubije sve niti, ili prekidom zbog signala).

`pthread_exit` završava pozivajuću nit i nikad se ne vraća, pa joj je tip `void`. Resursi pridruženi niti (struktura u jezgri, stog niti, ...) ostaju u sustavu dok ih netko ne pokupi pozivom `pthread_join`. Analogija s procesima je gotovo izravna: `pthread_exit` je za nit ono što je `exit` za proces, a `pthread_join` je za nit ono što je `wait` za proces (P05).

Argument za `pthread_exit`:

- **retval** — pokazivač koji postaje povratna vrijednost niti. Valja voditi računa da pokazivač koji prosljedimo funkciji `pthread_exit` ne pokazuje na varijablu na stogu! Svaka nit ima vlastiti stog, koji nestaje kada nit završi pa pokazivač na

bilo koju lokalnu varijablu nije validan. Ovo je uobičajena greška, a umjesto pokazivača na lokalnu varijablu, nit tipično vraća pokazivač na memoriju alociranu na hrpi (engl. *heap*), pozivom `malloc`.

Povratna vrijednost za `pthread_join`: 0 u slučaju uspjeha, kod greške inače (ista konvencija kao kod `pthread_create`).

Argumenti za `pthread_join`:

- **thread** — identifikator niti koju čekamo (vrijednost koju nam je `pthread_create` ranije upisao u `pthread_t`).
- **retval** — pokazivač na pokazivač u koji `pthread_join` upisuje povratnu vrijednost niti (onu koju je nit predala `pthread_exit`-u, ili koju je vratila kroz `return` iz polazne funkcije). Ako nas povratna vrijednost ne zanima, možemo predati `NULL`. Razlog zašto je argument *pokazivač na pokazivač* je taj što funkcija mora u našu varijablu upisati adresu koja je predana funkciji `pthread_exit`. Stoga moramo proslijediti pokazivač na pokazivač — tj. pokazivač na varijablu tipa pokazivač, u koju će nova adresa biti upisana.

`pthread_join` blokira pozivajuću nit dok zadana nit ne završi. Nakon uspješnog poziva, sustav oslobađa sve resurse vezane za tu nit.

8.4.3 Otkazivanje niti — `pthread_cancel`

Nit može poslati drugoj niti zahtjev za prekid izvršavanja pozivom `pthread_cancel`:

```
#include <pthread.h>

int pthread_cancel(pthread_t thread);
```

Povratna vrijednost: 0 u slučaju uspjeha; kod greške inače (ista konvencija kao kod `pthread_create`).

Bitno je naglasiti da `pthread_cancel` nije bezuvjetna naredba — ona šalje **zahtjev** za otkazivanje, a hoće li i kad ciljana nit zaista završiti ovisi o njenom stanju otkaznosti i o tome dolazi li do tzv. *cancellation pointa* (poziva neke od funkcija koje POSIX definira kao mjesta na kojima se zahtjev za otkazivanje obrađuje, npr. `sleep`, `read`, `pthread_cond_wait`). Po defaultu su niti otkazne i obrada zahtjeva događa se na prvom *cancellation pointu*. Otkazivanje niti je relativno složena tema o kojoj nećemo dublje govoriti — koristit ćemo je samo u jednom primjeru kasnije, gdje glavna nit prekida pozadinske radnike koji izvide beskonačnu petlju.

8.4.4 Primjer: `pozdrav1`

- **pozdrav1.c** — glavna nit (u kodu označena kao *prva nit*) stvara novu nit i ispisuje dvije svoje poruke prije izlaska iz `main`-a. Nova nit (*druga nit* u kodu) nakon kratke pauze ispisuje svoje dvije poruke.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>

void *pozdrav(void *arg) {
    sleep(1);
    printf("Pozdrav iz druge niti!\n");
    printf("Druga nit izlazi!\n");
    return NULL;
}
```

```
int main(void) {
    pthread_t nit;

    if (pthread_create(&nit, NULL, pozdrav, NULL) != 0) {
        perror("pthread_create");
        return 1;
    }

    printf("Pozdrav iz prve niti.\n");
    printf("Prva nit nit izlazi!\n");
    return 0;
}
```

Ispis:

```
$ ./pozdrav1
Pozdrav iz prve niti.
Prva nit nit izlazi!
```

Iako bismo očekivali da se nakon pokretanja programa “jave” obje niti — prvo prva, a zatim, nakon kratke pauze, i druga — drugi pozdrav se nikada ne dogodi. Razlog ne bi smio biti prekid izvršavanja prve niti: sve niti unutar jednog procesa su, kao što smo ranije naveli, ravnopravne i predstavljaju nezavisne tokove izvršavanja unutar istog spremnika, pa završetak jedne niti ne povlači i završetak ostalih. Međutim, sjetimo se na trenutak značenja poziva `return` iz funkcije `main`: dok u svim drugim funkcijama u programu `return` znači povratak u funkciju pozivatelja, poziv `return` iz funkcije `main` po C standardu ekvivalentan je pozivu `exit` — prekidu izvršavanja procesa (više detalja vidjeti u poglavlju P05 – Okruženje procesa, sekcija “Životni ciklus procesa”).

Isto značenje `return` ima i u našem primjeru: `return` iz funkcije `main` je `exit`, a `exit` terminira **cijeli proces**, uključujući sve preostale niti, neovisno o tome jesu li završile svoj posao ili nisu. U trenutku kad bi se druga nit probudila iz `sleep(1)`, proces više ne postoji.

Postoje različiti načini da ovo popravimo. U sljedeća dva primjera vidjet ćemo dva pristupa.

8.4.5 Primjer: pozdrav2

- **pozdrav2.c** — najjednostavnije rješenje: prva nit prije izlaska iz `main`-a pozove `pthread_join(nit, NULL)` i tako čeka da druga nit završi. Kod se od `pozdrav1.c` razlikuje samo u jednom dodanom retku.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>

void *pozdrav(void *arg) {
    sleep(1);
    printf("Pozdrav iz druge niti!\n");
    printf("Druga nit izlazi!\n");
    return NULL;
}

int main(void) {
    pthread_t nit;

    if (pthread_create(&nit, NULL, pozdrav, NULL) != 0) {
        perror("pthread_create");
        return 1;
    }
}
```

```

printf("Pozdrav iz prve niti.\n");
pthread_join(nit, NULL);      /* novo: cekaj drugu nit prije izlaska */
printf("Prva nit nit izlazi!\n");
return 0;
}

```

`pthread_join(nit, NULL)` blokira prvu nit dok druga ne završi. Tek kad druga ispiše obje svoje poruke i vrati se iz polazne funkcije, prva nit nastavlja s ispisom svoje druge poruke i poziva `return 0`. U tom trenutku `exit` terminira proces — ali sad je sve već odrađeno.

Ispis:

```

$ ./pozdrav2
Pozdrav iz prve niti.
Pozdrav iz druge niti!
Druga nit izlazi!
Prva nit nit izlazi!

```

Pažljivim pogledom na redoslijed vidimo da je prva nit zaista čekala drugu — između njezine prve i druge poruke ubacile su se obje poruke druge niti.

8.4.6 Primjer: pozdrav3

Drugi primjer (`pozdrav2`) riješio je problem preuranjenog terminiranja procesa. U ovom primjeru, prva nit ostala je “glavna” — ona stvara novu nit, čeka na njezin završetak i završava proces. Na sljedećem primjeru pokazat ćemo da su niti zaista ravnopravne — ne postoji glavna nit koja stvara i povezuje (*join*) druge niti. Jednom kada se stvori nova nit, ili više njih, sve niti unutar jednog procesa su ravnopravne i bilo koja nit može povezati (*join*) bilo koju drugu.

- **pozdrav3.c** — uloge `pthread_join`-a sad su zamijenjene. Prva nit nakon stvaranja druge ispisuje svoje dvije poruke i poziva `pthread_exit(NULL)`. Druga nit, nakon vlastite prve poruke, poziva `pthread_join(glavna, NULL)` na *prvoj* niti, i tek tada ispiše svoju zadnju poruku i završi.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>

pthread_t glavna;

void *pozdrav(void *arg) {
    sleep(1);
    printf("Pozdrav iz druge niti!\n");
    pthread_join(glavna, NULL);      /* druga nit ceka prvu */
    printf("Druga nit izlazi!\n");
    return NULL;
}

int main(void) {
    pthread_t nit;

    glavna = pthread_self();          /* spremi vlastiti ID u globalnu varijablu */
    if (pthread_create(&nit, NULL, pozdrav, NULL) != 0) {
        perror("pthread_create");
        return 1;
    }

    printf("Pozdrav iz prve niti.\n");
    printf("Prva nit nit izlazi!\n");
    pthread_exit(NULL);              /* terminira samo prvu nit, NE proces */
}

```

Dva su nova elementa u odnosu na prethodne primjere. Prvo, prva nit poziva `pthread_self()` da dohvati vlastiti identifikator i pohrani ga u globalnu varijablu `glavna`, vidljivu drugoj niti (sjetimo se da niti dijele cijeli adresni prostor procesa, uključujući globalne varijable). Funkcija `pthread_self` deklarirana je u `<pthread.h>`:

```
pthread_t pthread_self(void);
```

Funkcija vraća identifikator niti koja ju je pozvala.

Drugo, prva nit umjesto `return 0` poziva `pthread_exit(NULL)`. Time se gasi samo nit koja je pozvala `pthread_exit`, a proces ostaje živ jer u njemu još uvijek postoji druga nit. Druga nit nakon ispisa pozdrava povezuje prvu. Ovdje vidimo još jednu ključnu razliku u odnosu na procese, kod kojih postoji jasna hijerarhija roditelj-dijete, a dijete ne može prikupiti izlazni status roditelja. Kod niti hijerarhije nema — kao što druga nit u našem primjeru bez problema čeka i povezuje prvu, tako bilo koja nit u procesu može čekati bilo koju drugu. Nakon povratka iz `pthread_join`, druga nit ispiše zadnju poruku i završi.

Ispis:

```
$ ./pozdrav3
Pozdrav iz prve niti.
Prva nit nit izlazi!
Pozdrav iz druge niti!
Druga nit izlazi!
```

Redoslijed ispisa je zanimljiv: prva nit ispiše svoje obje poruke i pozove `pthread_exit`; druga nit se u međuvremenu probudi iz `sleep(1)`, ispiše svoju prvu poruku, čeka prvu kroz `pthread_join` (koji u tom trenutku odmah uspijeva jer je prva već završila), pa ispiše svoju drugu poruku.

Ovaj primjer pokazuje nekoliko važnih stvari odjednom. **Niti su zaista ravnopravne** — druga nit poziva `pthread_join` na *prvoj* niti, što bi u svijetu procesa bilo nezamislivo. **Prva nit (ona koja izvršava main) nije "posebna"** osim po tome što počinje izvršavati `main` i što `return` iz `main`-a terminira proces; ako umjesto `return` koristimo `pthread_exit`, ponaša se kao bilo koja druga nit. I konačno — **proces živi sve dok mu živi barem jedna nit**.

8.4.7 Primjer: kvadrat — prijenos podataka u nit i natrag

U prethodnim primjerima glavna nit i dalje je nešto "radila" — ispisivala pozdrav. Da bismo vidjeli kako se podaci stvarno predaju u nit i kako se rezultat vraća, napraviti ćemo program koji od nove niti traži da izračuna kvadrat broja zadanog kao argument naredbenog retka. Niti koja računa kvadrat broj predajemo kao četvrti argument funkcije `pthread_create`, a rezultat se vraća kao argument funkcije `pthread_exit`. Nit pozivatelj povratnu vrijednost prima kao drugi argument `pthread_join`.

- **kvadrat.c** — prvi pokušaj, s tipičnom početničkom pogreškom.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>

void *kvadrat(void *arg) {
    int broj = *(int*)arg;
    int r = broj*broj;
    pthread_exit((void*)&r);
}
```

```

}

int main(int argc, char **argv) {
    pthread_t nit;
    int broj;
    int *retval;

    if (argc < 2) {
        printf("koristenje: %s <broj>\n", argv[0]);
        return 0;
    }

    broj = atoi(argv[1]);
    if (pthread_create(&nit, NULL, kvadrat, &broj) != 0) {
        perror("pthread_create");
        return 1;
    }

    pthread_join(nit, (void**)&retval);
    printf("%d^2 = %d\n", broj, *retval);
    return 0;
}

```

Glavna nit pretvara argument naredbenog retka u cijeli broj i predaje funkciji `pthread_create` pokazivač na njega (varijabla `broj` je u glavnoj niti i ostaje živa do kraja `main`-a, pa je sigurno proslijediti njezinu adresu). U funkciji niti rezultat se izračuna u lokalnoj varijabli `r`, čija se adresa predaje `pthread_exit`-u. Glavna nit zatim u `retval` dobiva tu adresu i ispisuje vrijednost.

Pokrenimo program nekoliko puta:

```

$ ./kvadrat 7
7^2 = -44044288
$ ./kvadrat 7
7^2 = -469766144
$ ./kvadrat 7
7^2 = -1574965248

```

Rezultat je svaki put drugačiji i ni jednom točan. Razlog je upravo onaj koji smo spomenuli uz prototip `pthread_exit`-a: varijabla `r` živi na stogu niti koja računa kvadrat, a stog se oslobađa u trenutku kada nit završi. Kad glavna nit nakon povratka iz `pthread_join` dohvati `*retval`, čita iz područja memorije koje je nekad bilo stog te niti, ali sad sadrži nepoznat sadržaj.

Logično pitanje koje se može postaviti: zašto pristup kroz pokazivač u glavnoj niti ne uzrokuje grešku `SIGSEGV`? Sjetimo se da niti dijele isti adresni prostor procesa, a stog jedne niti nije “izvan” adresnog prostora — to je memorija unutar njega, alocirana za potrebe te niti pri njenom stvaranju. Kad nit završi, sustav tu memoriju vraća u svoj bazen slobodnih stranica, ali sa stajališta hardvera adresa i dalje pripada procesu i moguć je pristup. Operacijski sustav nema načina znati da ono što čitamo više “ne pripada” niti koja je davno završila — pristup je tehnički legalan, samo je sadržaj smeće.

Treba reći da povremeno možemo dobiti i `SIGSEGV` — primjerice ako sustav u međuvremenu vrati stranicu na kojoj je bio stog niti operacijskom sustavu, pa ona više nije mapirana u adresni prostor procesa. U ovom slučaju, pokušaj čitanja s adrese na koju pokazivač pokazuje izaziva segmentacijsku grešku, koja se manifestira na način da nam jezgra pošalje signal `SIGSEGV`.

Iako ovo zvuči kontraintuitivno, takav ishod je u nekom smislu sretniji: program javlja grešku i odmah znamo da nešto ne valja. Mnogo je opasnija upravo ova tiha varijanta koju vidimo u našem primjeru — program *naizgled radi*, prolazi

sve trivijalne provjere, ali rezultat je svaki put kriv. Greške ovog tipa mogu ostati neprimijećene jako dugo, sve dok se okolnosti ne poklope da ih netko slučajno otkrije.

Ovo je važno pravilo koje vrijedi pamtit: **nikad ne vraćajte iz niti pokazivač na lokalnu varijablu**. Isti princip vrijedi i za “obične” funkcije u C-u, ali se kod niti ova greška još teže otkriva jer ovisi o tome kad i kako sustav recklira memoriju oslobođenih stogova.

- **kvadrat2.c** — ispravljena verzija. Umjesto lokalne varijable na stogu, memoriju za rezultat alociramo na hrpi pozivom `malloc`-a. Hrpa je dio adresnog prostora procesa i memorija ondje ostaje validna sve dok je eksplicitno ne oslobodimo pozivom `free`.

```
void *kvadrat(void *arg) {
    int broj = *(int*)arg;
    int *r = (int*)malloc(sizeof(int));

    *r = broj*broj;
    pthread_exit((void*)r);
}
```

Sve ostalo u programu je identično `kvadrat.c`-u. Sad rezultat radi pouzdano:

```
$ ./kvadrat2 7
7^2 = 49
```

Mali nedostatak ovog rješenja: glavna nit nakon ispisa rezultata trebala bi pozvati `free(retval)` da oslobodi alociranu memoriju. U našem trivijalnom programu to nije problem jer i tako odmah završavamo, ali u dugotrajnim programima izostavljen `free` znači curenje memorije. Iako smo navikli da memoriju alociranu s `malloc`-om oslobađamo s `free` u funkciji u kojoj je memorija alocirana (jer je pokazivač u kojem je pohranjena memorijska adresa najčešće lokalna varijabla), u ovom slučaju to nije moguće jer je funkcija u kojoj je memorija alocirana završila s izvršavanjem završetkom niti. U ovom slučaju, nit koja je primila rezultat drži pokazivač na memorijsku adresu, pa je njena odgovornost da istu i oslobodi.

8.4.8 Primjer: argumenti — više niti s različitim argumentima

Često nam je potrebno stvoriti niz niti i svakoj predati različit argument (npr. njezin redni broj, ulazne podatke za obradu, ili index u nekom polju). U sljedećem primjeru stvorit ćemo pet niti i svakoj predati njezin redni broj kroz argument `pthread_create`-a. Cilj je da svaka nit ispiše broj 0, 1, 2, 3 i 4 — svaka svoj.

- **argumenti.c**

```
#define BROJ_NITI 5

void *radnik(void *arg) {
    int id = *(int *)arg;
    printf("Nit %d pocinje rad\n", id);
    sleep(1);
    printf("Nit %d zavrсила\n", id);
    return NULL;
}

int main(void) {
    pthread_t niti[BROJ_NITI];
    int podaci;
    int i;
```

```

for (i = 0; i < BROJ_NITI; i++) {
    podaci = i;
    if (pthread_create(&niti[i], NULL, radnik, &podaci) != 0) {
        perror("pthread_create");
        return 1;
    }
}

for (i = 0; i < BROJ_NITI; i++)
    pthread_join(niti[i], NULL);

printf("Sve niti su završile.\n");
return 0;
}

```

Logika je očita: u svakoj iteraciji petlje upišemo trenutnu vrijednost `i` u varijablu `podaci`, te niti predamo njezinu adresu kao argument. Pokrenimo program i pogledajmo što se događa:

```

$ ./argumenti
Nit 3 pocinje rad
Nit 3 pocinje rad
Nit 3 pocinje rad
Nit 3 pocinje rad
Nit 4 pocinje rad
Nit 3 završila
Nit 3 završila
Nit 3 završila
Nit 3 završila
Nit 4 završila
Sve niti su završile.

```

Umjesto očekivanih 0, 1, 2, 3, 4, vidimo da nekoliko niti misli da im je id jednak 3 (ili neki drugi broj, ovisno o pokretanju). Što je pošlo po krivu?

Sjetimo se da nit, kad je stvorimo, ne počinje *odmah* izvršavati svoju polaznu funkciju — između poziva `pthread_create` i početka izvršavanja funkcije `radnik` može proći određeno vrijeme dok raspoređivač ne odluči pokrenuti novostvorenu nit. U međuvremenu, glavna nit nastavlja izvršavati petlju i u svakoj iteraciji **prepisuje vrijednost varijable `podaci`**. Sjetimo se da nitima prenosimo adresu varijable (pokazivač), ne njezinu vrijednost — u našem slučaju sve niti su dobile pokazivač na istu varijablu (istu adresu u memoriji), koju “glavna” nit mijenja u petlji. Kada niti pokušaju pročitati vrijednost s adrese koju su dobile kao argument, vide trenutnu vrijednost — posljednju upisanu, ili onu koja se u tom trenutku tamo zatekla.

Ovo je tipična zamka u radu s nitima: argument koji predajemo niti mora “preživjeti” do trenutka kad ga nit pročita i ne smije se u međuvremenu mijenjati. Ukoliko niti predamo pokazivač na varijablu koja se nakon poziva `pthread_create` mijenja — gotovo je sigurno da će barem neke od niti dobiti pogrešnu vrijednost.

- **argumenti2.c** — ispravljena verzija. Umjesto jedne varijable koju u petlji prepisujemo, koristimo polje, gdje svaka nit dobiva pokazivač na svoj zasebni element. Tih pet elemenata polja zadržavaju svoje vrijednosti sve do kraja `main`-a, kad sve niti već odavno čitaju svoje argumente.

```

int main(void) {
    pthread_t niti[BROJ_NITI];
    int podaci[BROJ_NITI]; /* zasebna kopija ID-a za svaku nit */
    int i;

    for (i = 0; i < BROJ_NITI; i++) {
        podaci[i] = i;
        if (pthread_create(&niti[i], NULL, radnik, &podaci[i]) != 0) {

```

```

        perror("pthread_create");
        return 1;
    }
}

for (i = 0; i < BROJ_NITI; i++)
    pthread_join(niti[i], NULL);

printf("Sve niti su završile.\n");
return 0;
}

```

Funkcija radnik je identična kao u prethodnoj verziji. Razlika je samo u main-u: podaci je sad polje od BROJ_NITI elemenata, i i-toj niti predajemo &podaci[i] — pokazivač koji za nju ostaje stabilan jer nitko više ne dira upravo taj element polja.

```

$ ./argumenti2
Nit 0 pocinje rad
Nit 1 pocinje rad
Nit 2 pocinje rad
Nit 3 pocinje rad
Nit 4 pocinje rad
Nit 0 završila
Nit 4 završila
Nit 3 završila
Nit 2 završila
Nit 1 završila
Sve niti su završile.

```

Niti sad pravilno vide svoje brojeve. Usput primjećujemo i jednu zanimljivu osobinu: niti su završile u drugačijem redoslijedu nego što su počele. Ovo nije slučajno — **redoslijed izvršavanja niti nije unaprijed određen** i može se mijenjati od pokretanja do pokretanja, ovisno o tome kako se raspoređivač odluči ponašati u trenutku izvršavanja.

8.5 Joinable i detached niti

Po defaultu, niti su **joinable** — njihovi resursi (struktura koja čuva povratnu vrijednost niti, identifikator, ...) ostaju u sustavu sve dok ih netko ne pokupi pozivom pthread_join. Ovo je analogno zombi procesima iz P05: ako ne pokupimo nit, imamo curenje resursa.

Često, međutim, ne želimo čekati rezultat niti — pokrenemo neki pozadinski zadatak i prepustimo ga sustavu. Za to služe **detached niti**: nakon završetka, njihovi resursi se **automatski oslobađaju**, bez potrebe za eksplicitnim join-om. Detached nit se ne smije i ne može joinati — pokušaj je greška.

Nit možemo napraviti detached na dva načina:

1. **Pri stvaranju** — postavljanjem atributa PTHREAD_CREATE_DETACHED u pthread_attr_t strukturu koju predajemo pthread_create-u.
2. **Naknadno** — pozivom pthread_detach(nit) u bilo kojem trenutku nakon stvaranja.

Funkcije koje koristimo deklarirane su u <pthread.h>:

```

int pthread_attr_init(pthread_attr_t *attr);
int pthread_attr_setdetachstate(pthread_attr_t *attr, int detachstate);
int pthread_attr_destroy(pthread_attr_t *attr);
int pthread_detach(pthread_t thread);

```

`pthread_attr_init` inicijalizira strukturu atributa zadanim vrijednostima; `pthread_attr_setdetachstate` u toj strukturi postavlja stanje otkačenosti (`PTHREAD_CREATE_DETACHED` ili `PTHREAD_CREATE_JOINABLE`); `pthread_attr_destroy` oslobađa strukturu kad nam više nije potrebna. Sve tri vraćaju 0 u slučaju uspjeha, odnosno kod greške. `pthread_detach` je alternativni način — služi za odvajanje već stvorene niti, pa ga ne moramo koristiti ako smo nit stvorili kao detached već pri pozivu `pthread_create`.

- **nit_detached.c** — primjer detached niti kao “ispali i zaboravi” zadataka.

```
void *pozadinski_posao(void *arg) {
    int id = *(int *)arg;
    free(arg);
    printf("Detached nit %d: radim svoj posao...\n", id);
    sleep(2);
    printf("Detached nit %d: gotovo.\n", id);
    return NULL;
}

int main(void) {
    pthread_t    nit;
    pthread_attr_t atribut;

    pthread_attr_init(&atribut);
    pthread_attr_setdetachstate(&atribut, PTHREAD_CREATE_DETACHED);

    for (int i = 0; i < 3; i++) {
        int *id = malloc(sizeof(int));
        *id = i;
        pthread_create(&nit, &atribut, pozadinski_posao, id);
    }

    pthread_attr_destroy(&atribut);

    sleep(3);    /* daj nitima vremena da odrade posao */
    return 0;
}
```

Primijetite da ne pozivamo `pthread_join` ni za jednu nit — sustav će sam pospremiti njihove resurse kad završe. Moramo, međutim, dati nitima vremena da završe prije nego `main` izađe (čime se proces gasi i niti nasilno prekidaju). U pravim aplikacijama detached niti se često koriste u poslužiteljima: glavna nit prima dolazne zahtjeve i za svaki stvara detached nit koja obrađuje taj zahtjev, bez potrebe da glavna nit prati svakoga.

8.6 Race condition

Vratimo se na problem iz prethodnog poglavlja, u kojem dva ili više procesa inkrementiraju brojač. Kao što smo vidjeli, operacija inkrementiranja varijable `count++` nije atomska — na razini strojnog koda sastoji se od tri koraka: učitaj vrijednost iz memorije u registar, povećaj vrijednost registra za 1, prebaci vrijednost registra u memoriju. Ako između koraka raspoređivač prebaci kontrolu na drugu nit (ili drugi proces), vrijednost brojača može postati nekonzistentna, tj. može doći do gubitka inkrementacija.

U prethodnom poglavlju, dva su procesa pristupala varijabli u dijeljenoj memoriji, a kao mehanizam sinkronizacije koristili smo semafore. Kada imamo više niti koje se izvršavaju unutar istog procesa, situacija je slična, ali uz jednu bitnu razliku: niti dijele memorijski prostor automatski. Ovo se, naravno, ne odnosi na lokalne varijable funkcija koje se čuvaju na stogu (sjetimo se da svaka nit ima svoj stog),

ali sve globalne varijable i hrpa (engl. *heap*) zajedničke su i dostupne svim nitima unutar jednog procesa.

- **broji.c** — osam niti istovremeno inkrementira zajedničku globalnu varijablu `count`. Svaka nit u petlji povećava brojač sto tisuća puta, pa očekujemo konačnu vrijednost $8 \times 100000 = 800000$.

```
#include <stdlib.h>
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <math.h>

#define NTHREADS 8

pthread_t thr_counter[NTHREADS];
unsigned long count = 0;

void *counter(void *arg) {
    int *c = (int *)arg;
    double d;
    printf("c: %d\n", *c);
    for (int k = 0; k < *c; k++) {
        /* petlja koja simulira "neki posao" */
        for (int j = 0; j < 5000; j++)
            d = sqrt((double)j);

        count++;
    }
    pthread_exit(NULL);
}

int main() {
    int cnt = 100000, k;

    for (k = 0; k < NTHREADS; k++) {
        pthread_create(&thr_counter[k], NULL, counter, (void *)&cnt);
    }

    for (k = 0; k < NTHREADS; k++) {
        pthread_join(thr_counter[k], NULL);
    }

    printf("Ukupno (%d * %d) = %lu\n", NTHREADS, cnt, count);
    return 0;
}
```

Unutar glavne petlje, prije svake inkrementacije brojača, umetnuli smo malu petlju koja računa korijene brojeva od 0 do 4999. Ovaj račun ne mijenja `count` ni na koji način — postoji samo zato da nit provede malo vremena radeći “nešto” prije nego dođe do dijeljene varijable. To produžuje vrijeme provedeno u jednoj iteraciji, što povećava vjerojatnost da se niti međusobno ispreplete baš u trenutku kada smo “u sredini” sekvence strojnog koda `mov / inc / mov` (vidi prethodno poglavlje) koju kompajler generira za `count++`. Bez ovog usporavanja, sekvenca bi na modernim procesorima bila tako brza da bismo race teško uhvatili u demonstraciji.

Napomena o kompajlerskom upozorenju. Pri prevođenju ćemo dobiti upozorenje *“warning: variable ‘d’ set but not used”*. Razlog je očit: varijablu `d` u svakoj iteraciji unutarnje petlje samo postavljamo, nikad je ne čitamo niti ispisujemo. Za potrebe ovog primjera upozorenje slobodno zanemarite — `d` nam i ne treba, jer nas zanima samo to da unutarnja petlja oduzme niti malo vremena. Možemo ga ušutkati eksplicitnim “korištenjem” varijable (npr. `(void)d;` na kraju funkcije), ali u demonstracijskom kodu to nije neophodno.

Da bismo lakše uočili nedeterminizam, program pokrenemo nekoliko puta u nizu kroz for petlju ljske. Konstrukt `for i in 1 2 3; do ... ; done` izvršava sve između `do` i `done` jednom za svaku vrijednost iz liste `1 2 3` — dakle tri puta. Unutar petlje pokrećemo `./broji` i provlačimo izlaz kroz `grep` `Ukupno` da iz cijelog ispisa filtriramo samo zadnji red s rezultatom (jer nas u ovoj demonstraciji ne zanimaju početne poruke `c: 100000` koje svaka nit ispiše).

```
$ for i in 1 2 3; do ./broji | grep Ukupno; done
Ukupno (8 * 100000) = 743521
Ukupno (8 * 100000) = 698104
Ukupno (8 * 100000) = 776892
```

Konačna vrijednost `count`-a nije `800000` i nije ista u svakom pokretanju. Niti su se preplitale “na različitim mjestima”, pa je broj izgubljenih inkrementacija svaki put drugačiji. Ovisno o broju jezgri vašeg računala i opterećenju sustava, rezultat može biti puno bliži očekivanom (ako je preplitanje rijetko) ili puno dalji od njega — ali samo iznimno ćemo dobiti točno `800000`. Ovaj nedeterminizam upravo je ono što `race condition` čini opasnim: program može raditi savršeno tisuću puta, a tisuću prvi put dati pogrešan rezultat.

8.7 Mutex

Rješenje `race conditiona` kod niti je **mutex** (engl. *mutual exclusion lock*). Konceptualno je identičan binarnom semaforu — sinkronizacijska primitiva koja osigurava da samo jedna nit u danom trenutku može izvršavati zaštićeni dio koda (kritičnu sekciju). Razlika je u tome što je mutex optimiziran za niti unutar istog procesa — implementacija je u korisničkom prostoru kad nema sporova (samo atomski `test-and-set` u memoriji), pa je znatno brža od POSIX semafora.

Funkcije za rad s mutexima deklarirane su u `<pthread.h>`:

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;    /* statička inicijalizacija */

int pthread_mutex_init(pthread_mutex_t *mutex,
                       const pthread_mutexattr_t *attr);
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

Statički alocirani mutex inicijalizira se makro vrijednošću `PTHREAD_MUTEX_INITIALIZER`. Mutex alocirani dinamički (na hrpi) treba inicijalizirati pozivom `pthread_mutex_init` i kasnije osloboditi pozivom `pthread_mutex_destroy`. Za potrebe primjera u ovoj skripti zadržat ćemo se isključivo na statičkoj inicijalizaciji, koja je jednostavnija i pokriva sve naše scenarije. Za one radoznale, koji se žele detaljnije upoznati s mutexima, kao i dublje u razumijevanje višenitnosti i programiranja korištenjem POSIX niti, upućujemo na dodatnu literaturu [1], [2].

Operacije nad mutexom:

- **pthread_mutex_lock** — pokušava zaključati mutex. Ako je već zaključan od strane druge niti, blokira pozivajuću nit dok mutex ne postane slobodan.
- **pthread_mutex_unlock** — otpušta mutex. Smije ga otpustiti samo nit koja ga drži (ovisno o tipu mutex-a — postoji više varijanti, a kod nekih ovo ponašanje nije strogo definirano, ali za naše potrebe zadani tip je dovoljan).

Sve četiri funkcije vraćaju `0` u slučaju uspjeha, odnosno kod greške (ista konvencija kao kod `pthread_create`).

8.7.1 Primjer: broji2

U ovom primjeru pristup varijabli `count` ograničen je korištenjem mutexa. Kada neka od niti “zaključa” mutex, bilo koja druga nit koja pozove `pthread_mutex_lock` blokirat će sve dok nit koja drži mutex isti ne otključa pozivom `pthread_mutex_unlock`. Kada se to dogodi, bilo koja od niti koje su čekale postaje kandidat za nastavak — implementacija pthreads-a sama bira koju će probuditi, a POSIX standard ne propisuje konkretan redoslijed. Drugim riječima, ne smijemo unaprijed pretpostavljati kojoj će niti od onih koje čekaju mutex biti dodijeljen.

```
pthread_mutex_t count_lock = PTHREAD_MUTEX_INITIALIZER;

void *counter(void *arg) {
    int *c = (int *)arg;
    double d;
    printf("c: %d\n", *c);

    for (int k = 0; k < *c; k++) {
        /* petlja koja simulira "neki posao" */
        for (int j = 0; j < 5000; j++)
            d = sqrt((double)j);

        pthread_mutex_lock(&count_lock);
        count++;
        pthread_mutex_unlock(&count_lock);
    }

    pthread_exit(NULL);
}
```

Rezultat:

```
$ for i in 1 2 3; do ./broji2 | grep Ukupno; done
Ukupno (8 * 100000) = 800000
Ukupno (8 * 100000) = 800000
Ukupno (8 * 100000) = 800000
```

Uvijek točno 800000. Cijena mutexa je pad performansi (svaki ulazak/izlazak iz kritične sekcije ima trošak), ali u zamjenu dobivamo točan i (još važnije) predvidiv rezultat.

Gdje postaviti mutex?

Naš `broji2.c` mutex postavlja samo oko `count++` — `sqrt` petlja je *izvan* kritične sekcije. Promotrimo i alternativnu varijantu, u kojoj bismo `pthread_mutex_lock` postavili na sami početak vanjske petlje, a `pthread_mutex_unlock` na njen kraj:

```
for (int k = 0; k < *c; k++) {
    pthread_mutex_lock(&count_lock);
    for (int j = 0; j < 5000; j++)
        d = sqrt((double)j);
    count++;
    pthread_mutex_unlock(&count_lock);
}
```

Ovaj kod je također ispravan — brojač je i dalje zaštićen, krajnji rezultat je 800000. Ali ovdje smo unutar kritične sekcije zatvorili i račun korijena, koji uopće ne dira zajedničku varijablu `count`. Posljedica je da dok jedna nit izvodi svojih 5000 `sqrt` operacija, sve ostale niti bespotrebno čekaju. Račun korijena niti su mogli raditi savršeno paralelno bez ikakve interferencije; ovakvim zaključavanjem ih nepotrebno serijaliziramo i izgubimo veliki dio prednosti višenitnog programa.

Pravilo dobre prakse: mutex držimo zaključanim što kraće moguće. Zaključavamo ga neposredno prije pristupa dijeljenom resursu, otključavamo neposredno nakon. Sve što ne mijenja dijeljene podatke ostaje izvan kritične sekcije, kako bi druge niti mogle raditi svoj posao paralelno.

8.7.2 Deadlock

Mutexi rješavaju jedan problem ali otvaraju drugi — **deadlock** (zaglavljenje, uzajamna blokada). Da bismo razumjeli kako nastaje, zamislimo program s dva mutexa, M1 i M2, koji štite dva različita resursa (npr. dvije strukture podataka). Nit A u nekom dijelu koda treba pristup objema strukturama, pa zaključava prvo M1, a zatim M2. U drugom dijelu koda, nit B također treba obje strukture — ali iz nekog razloga (možda je drugi programer pisao tu funkciju, ili je redoslijed dolazio iz drugačijeg konteksta) zaključava ih u obrnutom redoslijedu: prvo M2, pa M1.

```
Nit A:                Nit B:
pthread_mutex_lock(&M1);  pthread_mutex_lock(&M2);
pthread_mutex_lock(&M2);  pthread_mutex_lock(&M1);
...                    ...
pthread_mutex_unlock(&M2); pthread_mutex_unlock(&M1);
pthread_mutex_unlock(&M1); pthread_mutex_unlock(&M2);
```

Sve dok se A i B ne izvršavaju istovremeno, sve je u redu. Problem nastaje ukoliko obje niti dođu u kritični dio koda u isto vrijeme:

1. Nit A zaključa M1.
2. Raspoređivač prebaci kontrolu na nit B (ili ona ionako radi paralelno na drugoj jezgri).
3. Nit B zaključa M2.
4. Nit B pokušava zaključati M1 — blokira jer ga drži A.
5. Nit A pokušava zaključati M2 — blokira jer ga drži B.

Obje niti sad zauvijek čekaju jedna drugu. Nijedna ne može otpustiti svoj mutex jer ne može dovršiti svoj posao, a posao ne može dovršiti jer čeka onaj drugi mutex. Proces je živ ali se nikad više neće dogoditi ništa — to je deadlock.

Prethodni primjer, na kojem smo ilustrirali korištenje mutexa, je krajnje jednostavan i očigledan: dijeljenoj varijabli se pristupa na samo jednom mjestu u jednoj funkciji, pa se možda čini da ne možete napraviti ovako banalnu grešku. Međutim, u složenim višenitnim programima dijeljenim varijablama često pristupamo iz različitih dijelova koda, koji ponekad čak ne moraju biti funkcionalno povezani — bar ne na način koji je “na prvu” jasan. Autor iz vlastitog iskustva može posvjedočiti da je deadlock puno lakše napraviti nego što izgleda na ovakvom uvodnom primjeru. Što program postaje veći, što više mutexa ima i što su raspršeniji po kodu, to su prilike za nepažljivi obrnuti redoslijed zaključavanja češće.

Klasična ilustracija ovog problema je *problem pet filozofa* (engl. *dining philosophers*), koji je 1971. formulirao E. W. Dijkstra [4]: pet filozofa sjedi za okruglim stolom, između svaka dva filozofa nalazi se jedan štapić, a filozof treba *oba* štapića (lijevi i desni) da bi mogao jesti. Ako svi istovremeno uzmu lijevi štapić i čekaju desni, nijedan nikad neće početi jesti.

Najjednostavniji način izbjegavanja deadlocka u praksi je konzistentan redoslijed zaključavanja: ako se svi dijelovi koda koji trebaju oba mutexa dogovore da uvijek zaključavaju M1 prije M2 (recimo, prema adresama u memoriji, ili abecednom redu

imena), deadlock je nemoguć. U većim programima ovo zahtijeva pažljiv dizajn — što su mutexi raspršeniji po kodu, to je teže jamčiti konzistentnost. Postoje i drugi pristupi (npr. `pthread_mutex_trylock` koji ne blokira nego odmah javlja neuspjeh, pa nit može otpustiti ono što već drži i pokušati ponovno), ali rasprava o njima nadilazi opseg ovog uvoda.

8.8 Kondicijske varijable

Mutex rješava problem isključivog pristupa, ali postoji i druga klasa problema — **čekanje na uvjet**. Razmotrimo klasični problem **proizvođač-potrošač** (engl. *producer-consumer*): jedna nit proizvodi podatke i zapisuje ih u ograničeni cirkularni međuspremnik, druga ih iz međuspremnika čita i obrađuje. Razmotrimo dvije situacije:

- Što kad je međuspremnik **prazan**? Potrošač mora čekati, uvjet da može nastaviti je da druga nit doda najmanje jedan podatak u međuspremnik.
- Što kad je međuspremnik **pun**? Proizvođač mora čekati, uvjet da može nastaviti je da druga nit preuzme najmanje jedan podatak iz međuspremnika (oslobodi mjesto u međuspremniku).

Naivno rješenje bi bilo aktivno čekanje (engl. *busy wait*) u petlji — “*provjeravaj uvjet svaki put, dok ne bude istinit*”. Neiskusni programeri često će posegnuti za aktivnim čekanjem, ali ovaj pristup iznimno iscrpljuje resurse procesora i troši procesorsko vrijeme. Puno bolje rješenje bilo bi ukoliko nit koja ne može nastaviti dok se određeni uvjet ne ispunji jednostavno “zaspi”, sve dok joj netko drugi ne signalizira promjenu stanja.

Takav mehanizam osiguravaju nam **kondicijske varijable** (engl. *condition variable*). Nit može pozivom `pthread_cond_wait` zaspati čekajući signal, a druga nit pozivom `pthread_cond_signal` (ili `pthread_cond_broadcast`) može probuditi jednu (ili sve) niti koje čekaju.

```
#include <pthread.h>

pthread_cond_t cv = PTHREAD_COND_INITIALIZER;    /* statička inicijalizacija */

int pthread_cond_init(pthread_cond_t *cond, const pthread_condattr_t *attr);
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_broadcast(pthread_cond_t *cond);
int pthread_cond_destroy(pthread_cond_t *cond);
```

Sve funkcije vraćaju 0 u slučaju uspjeha, odnosno kod greške (ista konvencija kao kod `pthread_create`).

Standardni obrazac korištenja kondicijske varijable je:

```
pthread_mutex_lock(&mutex);
while (!uvjet)                                /* UVIJEK while, ne if! */
    pthread_cond_wait(&cv, &mutex);           /* atomski: otpusti mutex + cekaj */
/* sad smo budni, mutex smo zaključali mi, uvjet je istinit */
... napravi posao ...
pthread_mutex_unlock(&mutex);
```

Pogledajmo kako ovaj obrazac funkcionira. Prije nego nit uopće provjeri uvjet, mora zaključati mutex koji štiti varijable o kojima uvjet ovisi — u našem primjeru to su brojač podataka u međuspremniku `buff_items` i sam međuspremnik. Jednom kada nit drži mutex, može pročitati stanje uvjeta, s obzirom da ne postoji

moгуćnost da bilo koja druga nit stanje promijeni za vrijeme ove provjere. Ako uvjet *nije* zadovoljen, nit poziva `pthread_cond_wait`, koja je središnja funkcija ovog mehanizma.

`pthread_cond_wait` prima dva argumenta — kondicijsku varijablu i **mutex koji pozivajuća nit drži zaključan**. Funkcija atomski otpušta mutex i stavlja nit u stanje spavanja (čekanja). U ovom trenutku, mutex je slobodan i može ga uzeti bilo koja druga nit koja to zatraži funkcijom `pthread_mutex_lock`. Kada druga nit dobije mutex, mijenja stanje štićenih varijabli, nakon čega funkcijom `pthread_cond_signal` šalje signal niti koja na uvjet čeka te otključava mutex.

Nit probuđena signalom atomski zaključava mutex te radi ponovnu provjeru uvjeta (u petlji `while`). Ukoliko je uvjet zadovoljen (tj. uvjet u `while` vrati `false`), mutex ostaje zaključan, a nit na siguran način pristupa štićenim varijablama. U slučaju da uvjet i dalje nije zadovoljen, ponavlja se raniji scenarij: u atomskoj operaciji nit oslobađa mutex i ide u stanje spavanja do ponovnog primanja signala.

Obratite pažnju da se uvjet provjerava u petlji `while`. Pažljiv čitatelj mogao bi zaključiti da se umjesto `while` može koristiti i `if`: ukoliko uvjet nije zadovoljen, nit oslobađa mutex i ide u stanje spavanja, a kada dobije signal da je uvjet zadovoljen (iz druge niti), nastavlja s obradom. Međutim, može se dogoditi da, bez obzira na promjenu stanja varijabli od strane druge niti i slanje signala za buđenje, uvjet i dalje ne bude zadovoljen (npr. u složenom okruženju s više od dvije niti). Pored ovog, postoji i mogućnost tzv. lažnih buđenja (engl. *spurious wakeups*) — sustav ponekad može probuditi uspavanu nit i bez signala za buđenje, uslijed raznih prekida, signala procesu i drugih implementacijskih detalja jezgre. Drugim riječima, povratak iz `pthread_cond_wait` ne garantira da je uvjet zadovoljen. Jedino što znamo je da je “nešto” probudilo uspavanu nit koja je čekala na uvjet, ali prije nastavka nužno moramo ponovo provjeriti je li uvjet zadovoljen. Upravo zato koristimo `while`: postoji mogućnost da ćemo nekoliko puta biti probuđeni prije nego što uvjet na koji čekamo bude stvarno zadovoljen.

Terminologija: Kada govorimo o “signalima” koje niti šalju jedna drugoj kroz `pthread_cond_signal`, *ne govorimo o klasičnim UNIX signalima* iz prethodnog poglavlja (`SIGINT`, `SIGTERM`, `SIGCHLD` i tako dalje). Iako je terminologija slična i može odvesti čitatelja u krivom smjeru, ovo su dva potpuno različita mehanizma. UNIX signali su asinkrone obavijesti procesu, koje obrađuje rukovatelj signala (engl. *signal handler*) ili sustav zadanim ponašanjem. Signali kondicijskih varijabli su sinkroni mehanizam unutar `pthread`s biblioteke, isključivo za buđenje niti koje su pozvale `pthread_cond_wait` na istoj kondicijskoj varijabli. Niti ne komuniciraju kroz UNIX signale (osim u graničnim slučajevima, vidi sljedeću sekciju), nego kroz `pthread`s sinkronizacijske primitive — `pthread_cond_signal` je jedna od njih.

Atomske operacije: Ključ ispravnog rada cijelog mehanizma jest atomičnost otpuštanja mutexa i ulaska u stanje spavanja u `pthread_cond_wait`-u. Da te dvije operacije nisu atomske, nit bi nakon otpuštanja mutexa, a prije nego što stigne zaspati, mogla biti prekinuta — i upravo u tom trenutku druga nit bi mogla promijeniti stanje i poslati signal koji bi naša nit propustila, jer još uvijek ne spava. Atomičnost garantira da do takvog “izgubljenog signala” ne može doći. Isti princip vrijedi i u obrnutom smjeru, pri buđenju niti: kad signal stigne, `pthread_cond_wait` atomski budi nit i zaključava mutex za nju,

kao jednu nedjeljivu operaciju. Da nije tako, druga nit bi između buđenja i zaključavanja mogla “ugrabit” mutex i opet promijeniti stanje varijabli prije nego što naša probuđena nit dođe na red — što bi nas dovelo u situaciju da nakon povratka iz `cond_wait`-a stanje više nije ono koje smo očekivali.

- **nit_cond.c** — proizvođač-potrošač s ograničenim cirkularnim međuspremnikom.

Ovdje koristimo dvije kondicijske varijable: jednu za “ima mjesta u međuspremniku” (proizvođač čeka na nju kad je pun) i jednu za “ima robe u međuspremniku” (potrošač čeka na nju kad je prazan).

```
#define VEL_BUFFERA 4
#define BROJ_STAVKI 30

static int          buffer[VEL_BUFFERA];
static int          upis_idx = 0, cit_idx = 0, buff_items = 0;

static pthread_mutex_t mutex      = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t ima_mjesta = PTHREAD_COND_INITIALIZER;
static pthread_cond_t ima_robe   = PTHREAD_COND_INITIALIZER;

/* nasumicna pauza izmedju 10 i 200 milisekundi */
static void slucajna_pauza(void) {
    usleep((rand() % 191 + 10) * 1000);
}

void *proizvodjac(void *arg) {
    for (int i = 0; i < BROJ_STAVKI; i++) {
        pthread_mutex_lock(&mutex);
        while (buff_items == VEL_BUFFERA)
            pthread_cond_wait(&ima_mjesta, &mutex);

        buffer[upis_idx] = i;
        upis_idx = (upis_idx + 1) % VEL_BUFFERA;
        buff_items++;

        pthread_cond_signal(&ima_robe);
        pthread_mutex_unlock(&mutex);
        slucajna_pauza();
    }
    return NULL;
}

void *potrosac(void *arg) {
    for (int i = 0; i < BROJ_STAVKI; i++) {
        pthread_mutex_lock(&mutex);
        while (buff_items == 0)
            pthread_cond_wait(&ima_robe, &mutex);

        int v = buffer[cit_idx];
        cit_idx = (cit_idx + 1) % VEL_BUFFERA;
        buff_items--;

        pthread_cond_signal(&ima_mjesta);
        pthread_mutex_unlock(&mutex);
        slucajna_pauza();
    }
    return NULL;
}
```

Proizvođač i potrošač između iteracija “rade” nasumično dugo (10–200 ms), pa ne možemo unaprijed znati hoće li se međuspremnik brže puniti ili prazniti — to je upravo razlog zašto trebamo *dvije* kondicijske varijable. U jednom trenutku proizvođač može biti brži pa se međuspremnik puni dok ne dosegne maksimum (4 stavke), nakon čega proizvođač blokira na `ima_mjesta`. U drugom trenutku, potrošač može biti brži pa međuspremnik ostaje prazan, a potrošač blokira na

ima_robe. Kondicijske varijable osiguravaju da nijedna nit ne troši procesorsko vrijeme u aktivnom čekanju i da niti uvijek ispravno reagiraju na promjene stanja međuspremnika.

Primjer ispisa (prvih nekoliko redaka):

```
$ ./nit_cond
Proizvodjac: stavio 0 (buff_items 1)
    Potrosac: uzeo 0 (buff_items 0)
Proizvodjac: stavio 1 (buff_items 1)
    Potrosac: uzeo 1 (buff_items 0)
Proizvodjac: stavio 2 (buff_items 1)
    Potrosac: uzeo 2 (buff_items 0)
Proizvodjac: stavio 3 (buff_items 1)
Proizvodjac: stavio 4 (buff_items 2)
    Potrosac: uzeo 3 (buff_items 1)
...
```

Svako pokretanje dat će drugačiji raspored, ali međuspremnik nikad neće prijeći iznos VEL_BUFFERA ni pasti ispod nule — invarijanta koju garantira kombinacija mutexa i dviju kondicijskih varijabli.

8.8.1 Više proizvođača i potrošača

U prethodnom primjeru obradili smo situaciju u kojoj postoji samo jedan proizvođač i jedan potrošač. U stvarnim produkcijskim sustavima često imamo situaciju u kojoj više proizvođača istovremeno dodaje podatke u međuspremnik i više potrošača koji ih iz spremnika istovremeno preuzimaju. Iako ova situacija na prvu može izgledati kaotična i složena za obraditi, princip je isti, sve su ostalo nijanse.

- **nit_cond2.c** — proširenje primjera nit_cond.c na **3 proizvođača i 3 potrošača**. Svaki proizvođač proizvodi po 10 stavki, dok istovremeno svaki potrošač preuzima 10 stavki, pa kroz međuspremnik prolazi ukupno 30 stavki, podjednako podijeljenih među nitima.

Da bismo razlikovali što je tko proizveo, svaki proizvođač generira stavke u obliku $id * 100 + i$, gdje je id njegov redni broj (0, 1 ili 2), a i redni broj stavke u njegovom proizvodnom ciklusu. Proizvođač 0 dakle generira 0, 1, 2, ... 9, proizvođač 1 generira 100, 101, ... 109, proizvođač 2 generira 200, 201, ... 209. Funkcija polazne niti proizvodjac i potrosac sad primaju id kao argument (po istom obrascu kao u argumenti2.c — polje s ID-jevima čiji elementi ostaju stabilni).

```
#define VEL_BUFFERA    4
#define N_PROIZ        3    /* broj niti proizvodjaca */
#define N_POTR         3    /* broj niti potrosaca */
#define STAVKI_PO_NITI 10    /* svaka nit proizvede/potrosi ovoliko */

static int             buffer[VEL_BUFFERA];
static int             upis_idx = 0, cit_idx = 0, buff_items = 0;

static pthread_mutex_t mutex      = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t  ima_mjesta = PTHREAD_COND_INITIALIZER;
static pthread_cond_t  ima_robe  = PTHREAD_COND_INITIALIZER;

void *proizvodjac(void *arg) {
    int id = *(int *)arg;
    for (int i = 0; i < STAVKI_PO_NITI; i++) {
        int stavka = id * 100 + i;    /* Razlikujemo tko je sto proizveo */

        pthread_mutex_lock(&mutex);
        while (buff_items == VEL_BUFFERA)
            pthread_cond_wait(&ima_mjesta, &mutex);
```

```

    buffer[upis_idx] = stavka;
    upis_idx = (upis_idx + 1) % VEL_BUFFERA;
    buff_items++;

    pthread_cond_broadcast(&ima_robe);    /* Probudimo sve potrosace */
    pthread_mutex_unlock(&mutex);
    slucajna_pauza();
}
return NULL;
}

void *potrosac(void *arg) {
    int id = *(int *)arg;
    for (int i = 0; i < STAVKI_PO_NITI; i++) {
        pthread_mutex_lock(&mutex);
        while (buff_items == 0)
            pthread_cond_wait(&ima_robe, &mutex);

        int v = buffer[cit_idx];
        cit_idx = (cit_idx + 1) % VEL_BUFFERA;
        buff_items--;

        pthread_cond_broadcast(&ima_mjesta); /* Probudimo sve proizvojdjace */
        pthread_mutex_unlock(&mutex);
        slucajna_pauza();
    }
    return NULL;
}

```

Dvije bitne razlike u odnosu na `nit_cond.c`. Prvo, umjesto `pthread_cond_signal` koristimo **`pthread_cond_broadcast`**:

```
pthread_cond_broadcast(&ima_robe);    /* Probudimo sve potrosace */
```

Razlog je suptilan. S jednim potrošačem, znamo da signalom budimo upravo njega. S više potrošača, `pthread_cond_signal` budi *bilo kojeg* — implementacija sama bira. Ovaj pristup nije nužno pogrešan i radit će sasvim dobro u jednostavnijim slučajevima. Međutim, u složenijim scenarijima različite niti mogu čekati različite uvjete pa korištenje `pthread_cond_signal` umjesto `pthread_cond_broadcast` može dovesti do situacije u kojoj se budi “kriva” nit — ona koja još uvijek ne može nastaviti jer uvjet na koji ona čeka nije zadovoljen. Ova situacija potencijalno vodi u deadlock, jer nit koja bi mogla nastaviti nije dobila signal i ostaje u stanju spavanja. Nasuprot ovome, `pthread_cond_broadcast` budi sve niti koje čekaju, pa svaka od njih iznova provjerava uvjet, a s izvršavanjem može nastaviti samo ona za koju je uvjet zadovoljen. Ostale jednostavno opet zaspu.

Sada je potpuno jasno zašto moramo koristiti petlju `while` oko `pthread_cond_wait`: kada npr. proizvođač pošalje `pthread_cond_broadcast` i probudi sve potrošače, samo jedan od njih će uspjeti uzeti podatak prije nego buffer opet postane prazan. Ostale probuđene niti, kad konačno dobiju mutex, vidjet će da je `buff_items == 0` i vratiti se u stanje spavanja — do idućeg buđenja.

Ispis programa:

```

$ ./nit_cond2
Proizvodjac 0: stavio 0 (buff_items 1)
Proizvodjac 1: stavio 100 (buff_items 2)
Proizvodjac 2: stavio 200 (buff_items 3)
    Potrosac 0: uzeo 0 (buff_items 2)
    Potrosac 1: uzeo 100 (buff_items 1)
    Potrosac 2: uzeo 200 (buff_items 0)
Proizvodjac 1: stavio 101 (buff_items 1)

```

```

        Potrosac 0: uzeo 101 (buff_items 0)
Proizvodjac 0: stavio 1 (buff_items 1)
        Potrosac 1: uzeo 1 (buff_items 0)
...

```

Po stavkama vidimo tko je što proizveo i tko je što uzeo. Redosljed javljanja proizvođača i potrošača je svaki put drugačiji, ali ključne stavke ostaju nepromijenjene:

- vrijednost brojača stavki u međuspremniku uvijek je u dopuštenim granicama: $0 \leq \text{buff_items} \leq \text{VEL_BUFFERA}$;
- ukupan broj proizvedenih stavki jednak je ukupnom broju potrošenih i iznosi 30;
- nijedna stavka se ne gubi niti se duplicira.

8.9 Signali i niti

UNIX signali, o kojima smo pisali u poglavlju P06, i niti su dvije neovisno razvijene apstrakcije UNIX-a, a njihova interakcija dolazi s nizom specifičnosti koje treba poznavati i o njima pažljivo voditi računa. Najvažnije pravilo je:

Kad signal dolazi procesu, jezgra ga može dostaviti **bilo kojoj niti tog procesa koja taj signal trenutno ne blokira**. Sve dok ne specificiramo drugačije, ne možemo predvidjeti kojoj će niti signal stići.

Tablica rukovatelja signala je **dijeljena** među svim nitima procesa — ne postoji “moj rukovatelj SIGINT-a u niti A” različit od “rukovatelja SIGINT-a u niti B”. Ali **maska blokiranih signala je privatna za svaku nit**. To znači da svaka nit može neovisno odrediti koje signale želi primati, a koje blokirati.

Funkcija `pthread_sigmask` je za nit ekvivalent funkcije `sigprocmask` za proces:

```
int pthread_sigmask(int how, const sigset_t *set, sigset_t *oldset);
```

Povratna vrijednost: 0 u slučaju uspjeha; kod greške inače (ista konvencija kao kod `pthread_create`).

Argumenti su isti kao kod `sigprocmask`: `how` je `SIG_BLOCK`, `SIG_UNBLOCK` ili `SIG_SETMASK`; `set` je maska signala; `oldset` je izlazni argument u koji funkcija pohrani staru masku.

Važno je naglasiti da u višenitnim programima obavezno treba koristiti `pthread_sigmask` umjesto `sigprocmask` — druga varijanta ima nedefinirano ponašanje u višenitnom okruženju.

8.9.1 Standardni obrazac: jedna nit obrađuje signale

U većini višenitnih programa najbolji pristup je centralizirati obradu signala u **jednoj niti**, dok sve ostale niti blokiraju signale. Tako izbjegavamo nepredvidivost koja nit primi signal. Obrazac:

1. Glavna (izvorna) nit prije stvaranja novih niti blokira sve signale koje želi obrađivati.
2. Stvaranje dodatnih niti — nove niti naslijeđuju masku blokiranih signala od izvorne pa i one blokiraju iste signale.

3. Jedna od niti (glavna ili dedikirana nit za obradu signala) sinkrono čeka signale pozivom `sigwait`, i obrađuje ih.

Podsjetimo se: termin “glavna nit” u osnovi znači “izvorna” ili “prva pokrenuta” nit — ona koja izvršava `main`. Sve niti unutar procesa su međusobno ravnopravne, kao što smo pokazali na primjeru `pozdrav3.c`.

`sigwait` funkcionira potpuno drugačije od rukovatelja signala — umjesto registracije funkcije za obradu signala i asinkronog poziva kada signal stigne, `sigwait` sinkrono (blokirajući) čeka da neki od signala iz maske stigne te vraća njegov broj. Ako je u maski više signala, funkcija blokira sve dok ne stigne *bilo koji* od njih. Time se izbjegava kompleksnost pisanja *async-signal-safe* koda u rukovatelju (problem je detaljnije obrađen u poglavlju o signalima).

- **`nit_signal.c`** — primjer višenitnog programa koji u dedikiranoj niti očekuje `SIGINT` (korisnik je stisnuo `Ctrl+C` na tipkovnici).

```
#define BROJ_NITI 3

void *radnik(void *arg) {
    int id = *(int *)arg;
    while (1) {
        printf("Radnik %d radi...\n", id);
        sleep(1);
    }
    return NULL;
}

int main(void) {
    pthread_t niti[BROJ_NITI];
    int podaci[BROJ_NITI];
    sigset_t maska;
    int primljeni_signal;

    /* blokiraj SIGINT u glavnoj niti -- naslijedit ce ga sve pomocne */
    sigemptyset(&maska);
    sigaddset(&maska, SIGINT);
    pthread_sigmask(SIG_BLOCK, &maska, NULL);

    for (int i = 0; i < BROJ_NITI; i++) {
        podaci[i] = i;
        pthread_create(&niti[i], NULL, radnik, &podaci[i]);
    }

    printf("Pritisni Ctrl+C za izlaz...\n");

    /* sinkrono cekaj signal iz maske */
    sigwait(&maska, &primljeni_signal);
    printf("\nGlavna nit primila SIGINT, gasim radnike.\n");

    for (int i = 0; i < BROJ_NITI; i++)
        pthread_cancel(niti[i]);
    for (int i = 0; i < BROJ_NITI; i++)
        pthread_join(niti[i], NULL);

    return 0;
}
```

Niti radnici jednom u sekundi ispisuju poruku. Kad korisnik pritisne `Ctrl+C`, signal `SIGINT` dolazi procesu — ali sve niti ga blokiraju osim glavne koja na njega čeka kroz `sigwait`. `sigwait` se vraća, a glavna nit radnicima šalje zahtjev za prekidom (engl. *cancellation request*) korištenjem funkcije `pthread_cancel`, pokupi ih `pthread_join`-om i izlazi.

Vrijedi se na trenutak osvrnuti na to kako pozivi `pthread_cancel` zaista uspijevaju zaustaviti niti koje su u beskonačnoj `while (1)` petlji. U sekciji o otkazivanju

niti spomenuli smo da `pthread_cancel` šalje *zahtjev* za prekid, koji se obrađuje tek kad nit dođe do **točke otkazivanja** (engl. *cancellation point*) — POSIX-om definirane funkcije pri čijem pozivu sustav provjerava postoji li čekajući zahtjev za otkazivanjem. U našoj radničkoj niti upravo `sleep(1)` igra tu ulogu: `sleep` je jedna od standardno definiranih točaka otkazivanja, pa nit pri ulasku u `sleep` provjeri zahtjev, vidi da je `pthread_cancel` pozvan i sama se terminira. Bez ijednog *cancellation pointa* u petlji, nit bi ignorirala sve zahtjeve za otkazivanjem i nastavila zauvijek raditi — što je važan detalj kojeg programer mora biti svjestan kad piše dugotrajne radničke petlje.

8.9.2 Slanje signala specifičnoj niti

U dosadašnjim primjerima fokusirali smo se na obradu signala koji dolaze procesu izvana (npr. `SIGINT` od korisnika). Ponekad, međutim, želimo iz jednog dijela vlastitog programa eksplicitno signalizirati nešto određenoj niti tog istog procesa — na primjer, glavna nit može poslati prekidni signal jednoj specifičnoj radničkoj niti dok ostale nastavljaju rad. Za to nije dovoljan običan `kill` koji signal šalje cijelom procesu (a jezgra ga zatim dostavlja nekoj od niti, po svom izboru), nego nam treba način da signal ciljano usmjerimo na pojedinu nit.

Za slanje signala procesu koristi se `kill(pid, sig)`. Za slanje signala specifičnoj niti unutar procesa postoji `pthread_kill`:

```
int pthread_kill(pthread_t thread, int sig);
```

Povratna vrijednost: 0 u slučaju uspjeha; kod greške inače (ista konvencija kao kod `pthread_create`).

Argumenti:

- **thread** — identifikator niti kojoj se signal šalje (vrijednost koju nam je vratio `pthread_create`).
- **sig** — broj signala koji se šalje. Vrijednost 0 ne šalje nikakav signal, ali se i dalje obavlja provjera postoji li nit s navedenim identifikatorom — koristan trik za provjeru “živi” li određena nit.

Ovo se rijetko koristi u praksi — kad imamo višenitni program, signali su uglavnom alat za vanjsku komunikaciju (od korisnika ili drugih procesa), pa nas obično ne zanima kojoj će točno niti stići. `pthread_kill` je tu kad nam stvarno treba precizna kontrola.

8.10 Thread-safe i reentrant funkcije

Prilikom pisanja višenitnih programa potrebno je voditi računa o još jednom važnom detalju. Mnoge funkcije iz standardne C biblioteke i POSIX-a nisu izvorno dizajnirane s nitima na umu — pretpostavljaju da unutar procesa postoji samo jedan tok izvršavanja. Kad ih pozovemo iz više niti istovremeno, mogu se dogoditi neočekivani problemi.

Funkcija je **thread-safe** ako se može sigurno pozivati iz više niti istovremeno. POSIX.1 definira skupinu thread-safe funkcija koje se mogu pozivati iz više niti istovremeno; većina standardnih funkcija ima ovo svojstvo, uz manju listu eksplicitno izuzetih. Pored toga, moderne implementacije C biblioteke pažljivo

su dorađene da budu thread-safe — npr. malloc interno koristi mutex-e kako bi osigurao ispravnost. Evo nekoliko primjera sigurnih i nesigurnih funkcija:

- **strtok** — tokenizira string, koristi internu statičku varijablu za pamćenje pozicije. Dvije niti koje istovremeno parsiraju različite stringove međusobno utječu jedna na drugu — jedan od čestih uzroka pogrešaka (engl. *bugs*) u višenitnim programima. Sigurna alternativa je **strtok_r** (sufiks *_r* od *reentrant*).
- **localtime**, **gmtime**, **asctime** — vraćaju pokazivač na internu statičku strukturu, koja se prepisuje pri sljedećem pozivu. Sigurne alternative su **localtime_r**, **gmtime_r**, **asctime_r**.
- **errno** — u starijim implementacijama **errno** je obična globalna varijabla, što ga čini nesigurnim za korištenje u višenitnom kodu. Moderne implementacije koriste *thread-local* **errno**, tako da svaka nit ima vlastiti **errno** pa ga u svom višenitnom kodu možemo koristiti bez brige.

Vrijedi razjasniti razliku između termina **thread-safe** i **reentrant**, jer se često brkaju. *Thread-safe* funkcija je ona koja se može sigurno pozivati iz više niti istovremeno — često se implementira korištenjem mutex-a kojim funkcija interno štiti svoje dijeljeno stanje, pa pozivi iz različitih niti čekaju jedan na drugi. *Reentrant* funkcija je stroži pojam: ona se može sigurno pozvati ponovo i dok prethodni poziv te iste funkcije još nije završio — npr. iz rukovatelja signala koji je prekinuo izvršavanje funkcije, ili rekurzivno. Reentrant funkcija ne smije imati nikakvog internog (statičkog) stanja niti koristiti mutex-e (jer bi zaglavila samu sebe ako se pozove dok već drži mutex). Sve reentrant funkcije su thread-safe, ali ne i obrnuto — funkcija koja koristi interni mutex može biti thread-safe ali nije reentrant, jer bi ponovni poziv iste funkcije dok prethodni još nije završen (npr. iz dvije niti, ili iz rukovatelja signala) mogao stvoriti deadlock.

Kad pišete višenitni program, **provjerite man stranice funkcija koje koristite** — sekcija "ATTRIBUTES" navodi je li funkcija thread-safe, a ako nije, obično ukazuje na *_r* varijantu ili alternativu.

8.11 Što smo zapravo radili

Niti su, uz signale i IPC, jedan od stupova suvremenog UNIX programiranja. Razumijevanjem niti otvaramo vrata cijelom svijetu paralelnih i konkurentnih programa — od jednostavnih web poslužitelja koji obrađuju više zahtjeva istovremeno, preko numeričkih programa koji iskorištavaju sve jezgre suvremenog procesora, do složenih distribuiranih sustava.

Najvažnije lekcije ovog poglavlja:

- Niti dijele adresni prostor procesa — to je istovremeno njihova najveća snaga (jednostavna komunikacija) i najveća opasnost (race condition).
- Pthreads sučelje je relativno jednostavno — nekoliko desetaka funkcija (u ovoj skripti obradili smo samo najznačajnije), ali bogato semantikom.
- Pisanje višenitnog koda zahtijeva eksplicitnu sinkronizaciju pristupa dijeljenim varijablama. Mutex-i i kondicijske varijable glavni su alati koji pokrivaju veliku većinu potreba za sinkronizacijom.
- Razumijevanje klasičnih problema (proizvođač-potrošač, deadlock, ...) ključno je za razumijevanje višenitnog programiranja.

Završimo s dvije misli koje je vrijedi ponijeti iz ovog poglavlja, neovisno o specifičnostima pthreads sučelja:

Kada razmišljamo o nitima i projektiramo višenitne arhitekture, važno je razmišljati o podacima, ne o kodu: pitanje “*kako ova nit radi?*” manje je važno od pitanja “*koje podatke ova nit dira i tko ih još dira?*”. Niti koje rade nad disjunktivnim skupovima podataka mogu raditi paralelno bez ijednog mutexa; niti koje dijele podatke moraju biti pažljivo sinkronizirane bez obzira na to koliko im je kod sličan ili različit. Pravilna identifikacija dijeljenih podataka i njihove razgranatosti kroz program prvi je korak svake suvislo dizajnirane višenitne arhitekture.

Sinkronizacijski problemi rješavaju se olovkom i papirom, ne na računalu: tek kada smo dobro promislili i razradili sve moguće scenarije pristupa dijeljenim podacima, te osmislili efikasan mehanizam sinkronizacije između niti i zaštite dijeljenih podataka, pristupamo kodiranju. Kodiranje je zadnji korak, a “pravi posao” događa se prije, na papiru. Pokušaj otklanjanja deadlocka *ad hoc*, mijenjajući kod dok program nekako ne proradi, gotovo uvijek završi programom koji *uglavnom radi* — sve dok se okolnosti ne poklope. Murphyeve zakon, jedan od važnijih inženjerskih postulata uopće, uči nas da će se ovo dogoditi možda i godinama kasnije, u produkciji, upravo u onom trenutku kada greška u našem kodu može prouzročiti nuklearnu katastrofu, treći svjetski rat, pad aviona, ili neku sličnu benignu posljedicu.

U neke od tema nismo dublje ulazili, ili smo ih samo usputno spomenuli: otkazivanje niti (engl. *cancellation*) i pridružene točke otkazivanja koje smo dotaknuli u primjeru `nit_signal.c`, lokalnu pohranu niti (TLS — `pthread_key_create`), specijalizirane oblike zaključavanja kao što su spin lockovi i read-write lockovi (`pthread_rwlock_*`), barijere (`pthread_barrier_*`), te razne attribute niti (prioriteti, *policy* raspoređivanja, veličina stoga). Svi ovi mehanizmi dio su pthreads biblioteke, ali se u tipičnim programima koriste nešto rjeđe i nisu ključni za razumijevanje višenitnosti. Radoznali čitatelj ili programer kojem ovi mehanizmi zatrebaju može ih pronaći u dodatnoj literaturi [1], [2].

8.12 Prevođenje

Direktorij dolazi s priloženim Makefile-om koji prati iste konvencije kao i Makefile datoteke u prethodnim poglavljima. Posebnost je u tome da pthreads zahtijevaju linkanje s pthread bibliotekom (`-lpthread`), što je u Makefile-u već navedeno za svaki primjer.

```
make all          # gradi sve primjere
make nit_pozdrav  # gradi pojedinačni primjer
make clean       # čisti generirane datoteke
```

8.13 Bibliografija

[1] D. R. Butenhof, *Programming with POSIX Threads*. Boston, MA, USA: Addison-Wesley Professional, 1997.

[2] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2013.

[3] L. Budin, M. Golub, D. Jakobović, and L. Jelenković, *Operacijski sustavi*, 3. izd. Zagreb, Hrvatska: Element, 2013.

- [4] E. W. Dijkstra, "Hierarchical ordering of sequential processes," *Acta Informatica*, vol. 1, no. 2, pp. 115-138, 1971.

Poglavlje 9

Socketi

U svim primjerima do sada, procesi i niti međusobno su komunicirali unutar jednog računala — putem datoteka, cjevovoda, zajedničke memorije, redova poruka, signala ili semafora.

Velika većina današnjih programa, međutim, mora komunicirati i preko mreže — web preglednik s web serverom, baza podataka s aplikacijom, mikroservisi međusobno. Sučelje kojim se to radi u UNIX-u zove se **socket**, a definirano je 1983. godine u sklopu sustava 4.2BSD UNIX [2], kao dio integracije TCP/IP protokola u jezgru. Tim Billa Joya na Sveučilištu California, Berkeley osmislio ga je kao prirodno proširenje UNIX filozofije “*sve je datoteka*” — socket je u biti deskriptor datoteke nad kojim radimo `read` i `write`, samo što ga stvaramo posebnim sistemskim pozivom, a s druge strane veze se, u ovom slučaju, nalazi drugi proces na udaljenom računalu. Sučelje je gotovo bez izmjena preuzeto u POSIX standard, a danas je dio svakog modernog operacijskog sustava: Linux, *BSD, macOS, Solaris, čak i Windows (gdje je Winsock API gotovo identičan kopiji BSD socketa). Drugim riječima, sve što ovdje učimo o UNIX socketima vrijedi praktički univerzalno. Po tome socketi nisu posebnost — UNIX, započet u Bell Labs-u 1969., izvor je iznenađujuće velikog broja koncepata koje danas smatramo univerzalnim u računarstvu (npr. hijerarhijski datotečni sustav), od kojih su neki stari više od pola stoljeća!

U ovom poglavlju ćemo obraditi dvije domene socketeta:

- **UNIX domain sockets** (AF_UNIX) — komunikacija između procesa na istom računalu, gdje “adresu” čini putanja u datotečnom sustavu.
- **Network sockets** (AF_INET) — TCP/IP komunikacija preko mreže (ili lokalno preko 127.0.0.1), gdje adresu čine IP adresa i port.

Cilj nije iscrpno pokriti mrežno programiranje. Mrežno programiranje tema je dovoljno široka i opsežna za cijelu knjigu i daleko premašuje opseg ove skripte. U ovom poglavlju dati ćemo okvir za razumijevanje koncepta socketeta i poticaj čitatelju da nastavi učiti i istraživati mehanizme mrežne komunikacije, koji su postali temelj modernog društva koje praktički počiva na ideji potpune povezanosti.

9.1 Socket kao deskriptor

Prije nego dublje zaronimo u svijet adresa, portova i paketa, podsjetimo se još jednom da socket nije ništa drugo nego deskriptor datoteke — što je zapravo i

logično ako znamo da je na UNIX-u sve datoteka. Funkcija `socket()` vraća `int` — isti tip podatka koji dobijemo kada s `open()` otvorimo datoteku, ili s `pipe()` stvorimo cjevovod. Nad njim možemo zvati `read()`, `write()` i `close()` baš kao na običnoj datoteci. Sve što smo do sada naučili o deskriptorima vrijedi i ovdje.

Razlika je samo u tome *kako* deskriptor stvaramo i kako ga povezujemo s drugom stranom komunikacije. Umjesto `open(putanja, ...)`, koristimo niz funkcija:

```
int socket(int domena, int tip, int protokol);
int bind(int fd, const struct sockaddr *addr, socklen_t len);
int listen(int fd, int backlog);
int accept(int fd, struct sockaddr *addr, socklen_t *len);
int connect(int fd, const struct sockaddr *addr, socklen_t len);
int close(int fd);
```

Odgodimo na kratko detaljan opis argumenata i povratnih vrijednosti funkcija i promotrimo tipičan tijek razgovora između dvije strane:

SERVER	KLIJENT
-----	-----
<code>socket()</code>	
<code>bind()</code>	
<code>listen()</code>	
<code>accept()</code> --- blokira ---	<code>socket()</code>
<-- <code>connect()</code> ----	<code>connect()</code>
<code>read()</code> <-- <code>write()</code> -----	<code>write()</code>
<code>write()</code> --- <code>read()</code> ----->	<code>read()</code>
<code>close()</code>	<code>close()</code>

Server “otvara dućan” pozivima `socket/bind/listen/accept`, dok klijent samo poziva `socket/connect` i spaja se na otvorenu konekciju na kojoj server čeka klijente. Nakon što se veza jednom uspostavi, obje strane imaju deskriptor datoteke putem kojeg mogu “razgovarati” `read` i `write` funkcijama koje već dobro poznajemo.

Pogledajmo sad detaljnije svaku od ovih funkcija.

socket — stvara novi socket. Argumenti su:

- **domena** — `AF_UNIX` za lokalne sockete ili `AF_INET` za TCP/IP nad IPv4 (postoji i `AF_INET6` za IPv6).
- **tip** — najčešće `SOCK_STREAM` (pouzdana, tokom-orijentirana komunikacija, kao TCP) ili `SOCK_DGRAM` (paketi, kao UDP). Obje varijante obrađujemo u ovom poglavlju.
- **protokol** — gotovo uvijek 0, što znači “izaberi zadani protokol za ovu kombinaciju domene i tipa”.

Povratna vrijednost je deskriptor datoteke (`int >= 0`) za novostvoreni socket, ili -1 uz postavljen `errno` u slučaju greške.

bind — vezuje socket za konkretnu adresu. Koristi ga server da kaže “Ja sam taj koji sluša na ovoj adresi”. Argumenti su:

- **fd** — deskriptor socket-a kojeg vežemo (dobiven prethodnim pozivom `socket-a`).
- **addr** — pokazivač na strukturu adrese. Konkretni tip strukture ovisi o domeni (`struct sockaddr_un` za `AF_UNIX`, `struct sockaddr_in` za `AF_INET`), a prilikom samog poziva tip se uvijek postavlja (cast operator) na `struct sockaddr *`.
- **len** — veličina strukture adrese u bajtovima, najčešće `sizeof(adresa)`.

Povratna vrijednost je 0 u slučaju uspjeha, -1 u slučaju greške.

listen — označava socket kao “slušajući”, odnosno spreman primiti dolazne veze. Argumenti su:

- **fd** — deskriptor socketa (server-side).
- **backlog** — koliko klijenata sustav smije držati u redu čekanja prije nego što ih server prihvati pozivom `accept`. Tipično 5 do 128.

Povratna vrijednost je 0 u slučaju uspjeha, -1 u slučaju greške.

accept — prihvaća sljedećeg klijenta iz reda čekanja. **Blokira** pozivajuću nit (ili proces) dok klijent ne stigne. Argumenti su:

- **fd** — deskriptor slušajućeg socketa.
- **addr** — izlazni argument, pokazivač na strukturu u koju funkcija upiše adresu klijenta koji se spojio (ili NULL ako nas to ne zanima).
- **len** — ulazno-izlazni argument: na ulazu mu predajemo veličinu strukture `addr`, na izlazu funkcija u njega upiše stvarnu veličinu zapisane adrese.

Važno: povratna vrijednost funkcije `accept` je novi deskriptor datoteke — onaj koji će server koristiti za “razgovor” s klijentom. Deskriptor `fd`, koji je argument funkcije, ostaje i dalje otvoren i možemo ga ponovo koristiti u sljedećem pozivu `accept` za prihvaćanje novog klijenta. Ovaj detalj je ključan za razumijevanje logike rada servera koji “osluškuje” dolazne pozive na socketu, osobito ukoliko server opslužuje više klijenata.

connect — koristi ga klijent da se spoji na server. Argumenti su isti kao kod `bind`-a: `fd` socketa, `addr` adresa servera, `len` njena veličina. Povratna vrijednost: 0 u slučaju uspjeha (veza uspostavljena), -1 u slučaju greške (npr. ukoliko s druge strane ne postoji server koji na toj adresi očekuje dolazne veze).

close — zatvara socket. Ako je u tijeku razgovor, druga strana će dobiti EOF (read vraća 0) i znat će da je veza prekinuta.

send i recv — socketima namijenjene inačice write i read

Iako na povezanom socketu možemo komunicirati `read`-om i `write`-om kao nad bilo kojim deskriptorom, POSIX nudi i dvije funkcije osmišljene specifično za sockete:

```
#include <sys/socket.h>

ssize_t send(int fd, const void *buf, size_t len, int flags);
ssize_t recv(int fd, void *buf, size_t len, int flags);
```

Prva tri argumenta i povratna vrijednost (broj prenesenih bajtova, ili -1 uz postavljen `errno`) istovjetni su onima kod `write`-a i `read`-a. Jedina je razlika **četvrti argument flags**, kojeg `read/write` nemaju — njime upravljamo ponašanjem koje ima smisla samo nad socketom. Kada je `flags` jednak 0, pozivi su potpuno ekvivalentni: `send(fd, buf, len, 0)` radi isto što i `write(fd, buf, len)`, a `recv(fd, buf, len, 0)` isto što i `read(fd, buf, len)`. Upravo zato primjeri u ovom poglavlju, radi jednostavnosti, koriste poznate `read/write`.

Smisao funkcija `send/recv` leži dakle u zastavicama (`flags`), koje nad mrežnom vezom omogućuju operacije kakve obična datoteka nema, primjerice:

- **MSG_PEEK** — “proviri” u dolazne podatke bez njihovog uklanjanja iz reda, pa ih sljedeći `recv` vraća ponovo.
- **MSG_WAITALL** — blokiraj dok ne stigne točno `len` bajtova (ublažava problem *short read*-a opisan na kraju poglavlja).

- **MSG_DONTWAIT** — izvedi samo ovaj poziv neblokirajuće, bez trajne promjene postavki socketa.
- **MSG_NOSIGNAL** — pri pisanju u prekinutu vezu vrati grešku EPIPE umjesto slanja signala SIGPIPE (koji bi inače mogao prekinuti proces).
- **MSG_OOB** — slanje, odnosno primanje *out-of-band* podataka (hitnih podataka izvan uobičajenog toka).

Ukratko: za jednostavne programe, kao što su primjeri u ovoj skripti, read/write su sasvim dovoljni, a za send/recv posežemo kada nam zatreba neka od mogućnosti koje pruža flags.

Ilustrirajmo korištenje opisanih funkcija na konkretnim primjerima.

9.2 UNIX domain socketi

UNIX domain socketi (engl. *UNIX domain sockets*, kraće UDS) služe za komunikaciju između procesa **na istom računalu**. Adresa socketa je putanja u datotečnom sustavu — kad server pozove bind(), na disku se stvara posebna datoteka, preko koje klijenti mogu pronaći server. Tip datoteke je *socket*, jedan od sedam tipova datoteka o kojima smo već pisali u ranijim poglavljima. Tipičan izlist datoteke socket s `ls -l` izgleda ovako:

```
$ ls -l /tmp/uds_primjer
srwxr-xr-x 1 user user 0 May 20 08:54 /tmp/uds_primjer
```

Prvi znak ispisa, slovo s, govori nam da se radi o datoteci tipa socket. Veličina je 0 jer se u socketu podaci zapravo nikada stvarno ne pohranjuju — datoteka služi samo kao “imenovani” završetak komunikacije, a sav promet ide kroz interne strukture jezgre. Naredba `file` također će prepoznati i ispisati tip datoteke:

```
$ file /tmp/uds_primjer
/tmp/uds_primjer: socket
```

UDS su brži i jednostavniji od TCP/IP socketa za lokalnu komunikaciju jer ne idu kroz mrežni stog jezgre — sva razmjena ide kroz interne strukture jezgre. Mnogi sistemski servisi (X11, Docker, PostgreSQL) ih koriste za lokalnu komunikaciju klijenata s lokalnim serverom.

Adresa UDS-a definirana je strukturom `struct sockaddr_un` iz `<sys/un.h>`:

```
struct sockaddr_un {
    sa_family_t sun_family; /* uvijek AF_UNIX */
    char        sun_path[108]; /* putanja, npr. "/tmp/moj_socket" */
};
```

Zašto baš 108 znakova? Veličina od 108 znakova povijesno je naslijeđe iz 4.2BSD-a (1983.) i zadržana je radi binarne kompatibilnosti. POSIX standard ne propisuje točnu veličinu — samo da mora biti barem 92 znaka — pa različite implementacije variraju (Linux/macOS/FreeBSD koriste 108, neki UNIX sustavi minimalnih 92). U praksi je tipična putanja dovoljno kratka da ovo nije ograničenje, ali u nekim modernim okolnostima (npr. duboke putanje u Dockerima ili Kubernetesima) ovo zna biti izvor problema.

9.2.1 Primjer: uds_server i uds_klijent

Jednostavan klijent/server primjer korištenja UNIX domain socketa.

- **uds_server.c** — slušatelj na /tmp/uds_primjer. Prima jednu poruku od svakog klijenta, ispiše ju i vrati klijentu (echo), pa prekine vezu. Iznimka: kad primi "KRAJ", umjesto echoa odgovori "U REDU -- IZLAZIM!" i izlazi.

```
#define PUTANJA "/tmp/uds_primjer"
#define BACKLOG 5

int main(void) {
    int                fd_server, fd_klijent;
    struct sockaddr_un adresa;
    char               buffer[256];
    ssize_t            n;
    int                kraj = 0;
    const char         *poruka_kraja = "U REDU -- IZLAZIM!";

    setbuf(stdout, NULL);

    fd_server = socket(AF_UNIX, SOCK_STREAM, 0);

    /* Brisemo datoteku (socket) ako vec postoji */
    unlink(PUTANJA);

    memset(&adresa, 0, sizeof(adresa));
    adresa.sun_family = AF_UNIX;
    strncpy(adresa.sun_path, PUTANJA, sizeof(adresa.sun_path) - 1);

    bind(fd_server, (struct sockaddr *)&adresa, sizeof(adresa));
    listen(fd_server, BACKLOG);

    printf("Server slusa na %s\n", PUTANJA);

    while (!kraj) {
        fd_klijent = accept(fd_server, NULL, NULL);
        n = read(fd_klijent, buffer, sizeof(buffer) - 1);
        if (n > 0) {
            buffer[n] = '\0';
            printf("Primljeno: %s\n", buffer);

            if (strcmp(buffer, "KRAJ", 4) == 0) {
                write(fd_klijent, poruka_kraja, strlen(poruka_kraja));
                kraj = 1;
            } else {
                write(fd_klijent, buffer, n); /* echo natrag */
            }
        }
        close(fd_klijent);
    }

    printf("Server izlazi.\n");
    close(fd_server);
    unlink(PUTANJA);
    return 0;
}
```

Promotrimo redoslijed poziva. `socket()` stvara socket, ali on još nije ni za što vezan — to je samo “neaktivan” deskriptor. `bind()` mu daje adresu (/tmp/uds_primjer) i u datotečnom sustavu stvara datoteku tog tipa. `listen()` označava socket kao “slušajući” i postavlja red duljine `BACKLOG` za klijente koji čekaju da budu prihvaćeni. Tek `accept()` vraća *novi* deskriptor za konkretnu vezu s jednim klijentom — i blokira dok takav klijent ne stigne.

Bitno je razumjeti razliku između `fd_server` i `fd_klijent`. `fd_server` je *slušajući* socket — koristimo ga samo za `accept`-anje novih veza, ne za razgovor. `fd_klijent` je *konektirani* socket s konkretnim klijentom, kroz njega ide `read/write` razgovor. Server može tako simultano držati otvorene mnoge `fd_klijent`-e ako tako organizira opslugu.

Petlja se prekida kada `strcmp(buffer, "KRAJ", 4) == 0` — dakle kad nam stigne

poruka koja počinje znakovima KRAJ. Nakon izlaska iz petlje, zatvaramo slušajući socket i unlink-om brišemo socket datoteku, ostavljajući datotečni sustav čistim.

- **uds_klijent.c** — spaja se, šalje poruku zadanu kao argument, čita odgovor servera i ispisuje ga.

```
int main(int argc, char *argv[]) {
    int fd;
    struct sockaddr_un adresa;
    char buffer[256];
    ssize_t n;

    if (argc != 2) {
        fprintf(stderr, "Koristenje: %s \"poruka\" (KRAJ za prekid izvršavanja servera)\n",
        ↪ argv[0]);
        exit(EXIT_FAILURE);
    }

    fd = socket(AF_UNIX, SOCK_STREAM, 0);

    memset(&adresa, 0, sizeof(adresa));
    adresa.sun_family = AF_UNIX;
    strncpy(adresa.sun_path, PUTANJA, sizeof(adresa.sun_path) - 1);

    connect(fd, (struct sockaddr *)&adresa, sizeof(adresa));
    write(fd, argv[1], strlen(argv[1]));

    /* Procitaj odgovor servera */
    n = read(fd, buffer, sizeof(buffer) - 1);
    if (n > 0) {
        buffer[n] = '\0';
        printf("Odgovor: %s\n", buffer);
    }

    close(fd);
    return 0;
}
```

Klijent je kraći jer ne treba bind ni listen — ne čeka da mu se netko spoji, nego se sam spaja. `connect()` traži postojeću adresu i, ako uspije, ostavlja socket spreman za razgovor.

Pokretanje (u dva terminala):

Terminal A:	Terminal B:
\$./uds_server	\$./uds_klijent "Pozdrav!"
Server sluša na /tmp/uds_primjer	Odgovor: Pozdrav!
Primljeno: Pozdrav!	\$./uds_klijent "Kako si?"
Primljeno: Kako si?	Odgovor: Kako si?
Primljeno: KRAJ	\$./uds_klijent "KRAJ"
Server izlazi.	Odgovor: U REDU -- IZLAZIM!

Na primjeru se otvara nekoliko zanimljivih pitanja oko životnog vijeka socket datoteke. Razmotrimo ih sad detaljnije.

Može li socket datoteka u datotečnom sustavu postojati prije nego što naš server pozove `bind()`? Odgovor je da može — sistemski poziv `mknod` može stvoriti praznu datoteku tipa socket s odgovarajućim modom (`S_IFSOCK`). Takva datoteka, međutim, **nije povezana ni s jednim procesom**, što ima dvije važne posljedice. Prvo, nitko ne sluša na njoj, pa pokušaj klijenta da se na nju spoji neće uspjeti. Drugo — što je važnije za naš primjer — naš server **ne može jednostavno “preuzeti” tu postojeću datoteku za slušanje**. Ako server pokuša pozvati `bind()` na adresu na kojoj već postoji bilo kakva datoteka (uključujući i tu praznu socket datoteku), `bind()` vraća grešku `EADDRINUSE` (“Address already in use”). Jezgra ne razlikuje “živu” socket datoteku stvorenu prethodnim `bind`-om od one stvorene `mknod`-om — za nju je svaka postojeća datoteka prepreka.

Upravo iz ovog razloga dobra je programerska praksa da na kraju korištenja, nakon što nam socket više nije potreban, server obriše socket koji je koristio. Ovo radimo i u našem primjeru: u posljednjim redcima server poziva `close(fd_server)` te nakon toga `unlink(PUTANJA)`. Međutim, postoji mogućnost i da server prekine izvršavanje i bez “urednog” izlaska, npr. u slučaju prekida signalom, ili ukoliko teta čistačica zatreba slobodnu utičnicu za usisač pa naš server (računalo) jednostavno isključi iz struje. U svim tim slučajevima `unlink` se nikad ne pozove i socket datoteka ostaje u datotečnom sustavu, kao “siročić”. Sljedeće pokretanje servera tada ne bi moglo proći `bind()`. Upravo zato u našem kodu na **početku** servera, prije `bind`-a, pozivamo `unlink(PUTANJA)` — kao osiguranje od bilo kakvog zaostatka iz prethodnih pokretanja, neovisno o tome jesu li završila uredno ili ne.

Možemo li, umjesto našeg `uds_klijent-a`, koristiti neki drugi alat da se spojimo na socket i pošaljemo serveru poruku? Pitanje nije samo akademsko: u stvarnim sustavima često je potrebno projektirati i implementirati serversku aplikaciju koja će opsluživati različite klijente, koje mogu razviti drugi programeri prema specifikaciji serverskog API-a (engl. *Application Programming Interface*) — sučelja putem kojeg definiramo kako komunicirati s našim serverom. S našim serverom možemo komunicirati putem bilo kojeg alata koji zna otvoriti socket i poslati u njega poruku. U sljedećem primjeru koristimo `nc` (engl. *netcat*) sa zastavicom `-U` koja `nc-u` govori da ciljna adresa nije TCP host/port nego UDS putanja:

```
$ echo "Pozdrav iz netcata!" | nc -U /tmp/uds_primjer
$ echo "KRAJ" | nc -U /tmp/uds_primjer
```

Server će obraditi obje poruke kao da su stigle iz našeg `uds_klijent-a`, i nakon druge će izaći. Slično tome se može koristiti i moćniji alat `socat`.

Pokušajte pokrenuti server te mu nakon toga iz različitih terminala pokušajte slati poruke iz našeg klijenta, iz `nc-a`, eventualno i drugih alata — i sami isprobajte kako se ponaša, koje su mu granice, što se događa ako ga dva klijenta gađaju u isto vrijeme. Ovo je odličan način da se konkretno uvjerite u sve što smo dosad opisali i da napravite vlastite zaključke o ponašanju UDS-a.

9.3 Mrežni socketi (Network domain Sockets) - TCP/IP

Mrežna domena (`AF_INET`) koristi se za komunikaciju preko TCP/IP-a — kako između računala, tako i unutar istog računala kroz tzv. *loopback* adresu `127.0.0.1` (virtualno mrežno sučelje koje paket “vraća” istom računalu, ne ide na fizičku mrežu, pa ga koristimo za lokalnu klijent-server komunikaciju, često upravo pri razvoju i testiranju programa). Adresa se sada sastoji od dvije komponente: **IP adrese** (identificira računalo u mreži) i **porta** (broj između 0 i 65535 koji identificira virtualnu pristupnu točku, tj. konkretnu aplikaciju koja tu točku koristi).

Adresa je definirana strukturom `struct sockaddr_in` iz `<netinet/in.h>`:

```
struct sockaddr_in {
    sa_family_t    sin_family; /* AF_INET */
    in_port_t      sin_port;   /* port (network byte order!) */
    struct in_addr sin_addr;    /* IP adresa (network byte order!) */
};
```

9.3.1 Network byte order

Različita računala mogu interno predstavljati višebajtnu cijelu brojeve na različite načine — neki kao *little-endian* (najmanje značajan bajt prvi), drugi kao *big-endian* (najznačajniji bajt prvi). Danas u svjetskom računarstvu dominira *little-endian* (svi Intel i AMD procesori, većina ARM-a u uobičajenoj konfiguraciji), dok je *big-endian* zadržan u manjini sustava i u nekim mrežnim protokolima. Da bi komunikacija među računalima različite arhitekture radila, TCP/IP propisuje da se brojevi u mrežnim zaglavljima uvijek šalju u *network byte order*-u (što je *big-endian*) — pod tim nazivom misli se upravo na poredak koji se koristi *na mreži*, dogovoren standardom, neovisno o arhitekturi pošiljatelja i primatelja.

Za pretvaranje između lokalnog i mrežnog poretka koriste se funkcije iz `<arpa/inet.h>`:

```
uint16_t htons(uint16_t hostshort);
uint32_t htonl(uint32_t hostlong);
uint16_t ntohs(uint16_t netshort);
uint32_t ntohl(uint32_t netlong);
```

- `htons(x)` (engl. *host to network short*) — pretvara 16-bitni broj iz lokalnog u mrežni poredak.
- `htonl(x)` (engl. *host to network long*) — pretvara 32-bitni broj.
- `ntohs(x)` i `ntohl(x)` — obrnuti smjer.

Sve četiri funkcije rade isključivo aritmetiku nad samim brojem — vraćaju pretvorenu vrijednost — i ne mogu signalizirati grešku.

Za port (16 bita) koristimo `htons`, za IP adresu (32 bita kod IPv4) `htonl`. Ako ovo zaboravimo, na *little-endian* računalu (kakvi su praktički svi danas) socket će raditi “obrnuto” — bind na port 9000 zapravo će obuhvatiti potpuno drugačiji broj.

Ove funkcije uputno je koristiti **uvijek**, čak i kad znamo da radimo kod na sustavu na kojem je lokalni i mrežni poredak značajnih znamenki isti, a `htons/htonl` pretvaranje ne radi ništa (na *big-endian* sustavu, gdje su lokalni i mrežni poredak isti, `htons/htonl` su zapravo no-op i broj prolazi neizmijenjen). Pozivom ovih funkcija u svakom slučaju jamčimo da se adresa uvijek pretvori na ispravan način, neovisno o tome koji byte order koristi naš sustav, čime osiguravamo prenosivost koda na proizvoljnu arhitekturu.

Prije nego napišemo primjer, uvedimo još dvije pomoćne funkcije koje će nam trebati. Za pretvaranje IP adrese iz teksta u binarni oblik koristi se `inet_pton`, a za obrnuti smjer `inet_ntop`:

```
int inet_pton(int af, const char *src, void *dst);
const char *inet_ntop(int af, const void *src, char *dst, socklen_t size);
```

- `af` je adresna familija — kod nas uvijek `AF_INET` za IPv4.
- `src` je izvor (string kod `inet_pton`, binarna struktura `in_addr` kod `inet_ntop`).
- `dst` je odredište (obrnuto od `src`).
- `size` (samo kod `inet_ntop`) je veličina odredišnog buffera. Za IPv4 dovoljan je `INET_ADDRSTRLEN` (16 bajtova).

Funkcija `inet_pton` vraća 1 u slučaju uspjeha, 0 ako predani string nije valjana adresa za zadanu familiju, ili -1 u slučaju druge greške (npr. nepoznata familija). `inet_ntop` vraća pokazivač na rezultatni string (`dst`) u slučaju uspjeha, ili `NULL` u slučaju greške.

Treća pomoćna funkcija je `setsockopt`, kojom postavljamo razne opcije nad socketom:

```
int setsockopt(int fd, int level, int optname, const void *optval, socklen_t optlen);
```

`setsockopt` je vrlo bogata funkcija — pokriva desetine različitih opcija organiziranih u razine (`SOL_SOCKET` za općenite, `IPPROTO_TCP` za TCP-specifične, `IPPROTO_IP` za IP-specifične, ...), od kojih svaka mijenja različito ponašanje socketa (`timeout`, veličine buffera, `multicast`, `keepalive`, ...). U ovoj skripti zadržavamo se isključivo na osnovnoj upotrebi s opcijom `SO_REUSEADDR` na razini `SOL_SOCKET`, koja serveru dopušta odmah ponovno vezanje na port nakon zatvaranja — bez nje bi sustav držao port zauzetim još otprilike minutu (zbog TCP-ovog stanja `TIME_WAIT`), pa bismo prilikom restarta dobivali "Address already in use" grešku. Ovo se lako provjeri: izbacite iz koda ovaj `setsockopt` poziv, prevedite ponovno, pokrenite server, zaustavite ga (`Ctrl+C` ili `KRAJ`), i odmah ga pokušajte pokrenuti ponovo — drugi pokušaj će propasti dok ne istekne `TIME_WAIT` period. Zainteresiranom čitatelju za pregled svih dostupnih opcija upućujemo na man stranicu `socket(7)` i, za sustavni pregled, na Stevens UNP [1].

9.3.2 Primjer: `tcp_server` i `tcp_klijent`

Strukturno identičan UDS primjeru, samo koristi mrežnu domenu umjesto UNIX domain socketa.

- **`tcp_server.c`** — sluša na portu 9000. Prima jednu poruku od svakog klijenta, ispiše ju i vrati klijentu (`echo`), pa prekine vezu. Iznimka: kad primi "KRAJ", umjesto `echoa` odgovori "U REDU -- IZLAZIM!" i izlazi.

```
#define PORT    9000
#define BACKLOG 5

int main(void) {
    int          fd_server, fd_klijent;
    struct sockaddr_in adresa;
    char         buffer[256];
    ssize_t      n;
    int          kraj = 0;
    const char   *poruka_kraja = "U REDU -- IZLAZIM!";

    setbuf(stdout, NULL);

    /* Stvori socket: AF_INET = IPv4, SOCK_STREAM = TCP */
    fd_server = socket(AF_INET, SOCK_STREAM, 0);

    /* Dopusti ponovno korištenje porta odmah nakon zatvaranja */
    int opt = 1;
    setsockopt(fd_server, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    /* Pripremi adresu: slusaj na svim mreznim sucjeljima, port 9000 */
    memset(&adresa, 0, sizeof(adresa));
    adresa.sin_family      = AF_INET;
    adresa.sin_addr.s_addr = htonl(INADDR_ANY);
    adresa.sin_port        = htons(PORT);

    bind(fd_server, (struct sockaddr *)&adresa, sizeof(adresa));
    listen(fd_server, BACKLOG);

    printf("Server slusa na portu %d\n", PORT);

    while (!kraj) {
        fd_klijent = accept(fd_server, NULL, NULL);
        n = read(fd_klijent, buffer, sizeof(buffer) - 1);
        if (n > 0) {
            buffer[n] = '\0';

```

```

    printf("Primljeno: %s\n", buffer);

    if (strncmp(buffer, "KRAJ", 4) == 0) {
        write(fd_klijent, poruka_kraja, strlen(poruka_kraja));
        kraj = 1;
    } else {
        write(fd_klijent, buffer, n);    /* echo natrag */
    }
}
close(fd_klijent);
}

printf("Server izlazi.\n");
close(fd_server);
return 0;
}

```

Logika programa identična je kao u UDS serveru. Razlikuju se samo dvije stvari:

- **Tip strukture adrese:** struct sockaddr_in umjesto struct sockaddr_un. Polja su drugačija — par (IP, port) umjesto putanje.
- **INADDR_ANY** u sin_addr.s_addr znači "slušaj na svim mrežnim sučeljima ovog računala". To uključuje i loopback (127.0.0.1) i sva fizička sučelja na kojima naše računalo ima IP adresu. Ako naše računalo, primjerice, ima vanjsku IP adresu 192.168.1.42, klijenta možemo pokrenuti na bilo kojem drugom računalu u istoj mreži i spojiti se na taj IP — server će prihvatiti vezu. Ako bismo umjesto INADDR_ANY stavili konkretnu IP adresu (npr. 127.0.0.1), server bi slušao samo na loopback-u i bio dostupan isključivo procesima na istom računalu.

I nema više unlink-a — kod TCP-a nema socket datoteke u datotečnom sustavu o kojoj treba brinuti, port se automatski oslobađa kad proces izađe.

- **tcp_klijent.c** — spaja se, šalje poruku zadanu kao argument, čita odgovor servera i ispisuje ga. Drugi opcionalni argument je IP servera; ako je izostavljen, koristi se 127.0.0.1.

```

int main(int argc, char *argv[]) {
    int fd;
    struct sockaddr_in adresa;
    const char *poruka = argv[1];
    const char *ip = (argc == 3) ? argv[2] : "127.0.0.1";
    char buffer[256];
    ssize_t n;

    if (argc < 2 || argc > 3) {
        fprintf(stderr,
            "Koristenje: %s \"poruka\" [ip] (KRAJ za prekid izvršavanja servera)\n",
            argv[0]);
        exit(EXIT_FAILURE);
    }

    fd = socket(AF_INET, SOCK_STREAM, 0);

    memset(&adresa, 0, sizeof(adresa));
    adresa.sin_family = AF_INET;
    adresa.sin_port = htons(PORT);

    /* inet_pton: pretvori IP iz teksta u binarni oblik */
    inet_pton(AF_INET, ip, &adresa.sin_addr);

    connect(fd, (struct sockaddr *)&adresa, sizeof(adresa));
    write(fd, poruka, strlen(poruka));

    /* Procitaj odgovor servera */
    n = read(fd, buffer, sizeof(buffer) - 1);
}

```

```

    if (n > 0) {
        buffer[n] = '\0';
        printf("Odgovor: %s\n", buffer);
    }

    close(fd);
    return 0;
}

```

Pokretanje (u dva terminala):

Terminal A: <pre> \$./tcp_server Server sluša na portu 9000 Primljeno: Pozdrav! Primljeno: Kako si? Primljeno: KRAJ Server izlazi. </pre>	Terminal B: <pre> \$./tcp_klijent "Pozdrav!" Odgovor: Pozdrav! \$./tcp_klijent "Kako si?" Odgovor: Kako si? \$./tcp_klijent "KRAJ" Odgovor: U REDU -- IZLAZIM! </pre>
---	---

Ako pokrenemo klijent na drugom računalu u istoj mreži, moramo ga pozvati s IP adresom našeg servera kao drugim argumentom, npr:

```
$ ./tcp_klijent "Pozdrav!" 192.168.1.42
```

9.4 Više klijenata istovremeno

Naš `tcp_server` ima jednu manu: dok poslužuje jednog klijenta, svi ostali čekaju. Ako operacija dugo traje (npr. server treba provesti neki složen proračun, ili jednostavno čeka na podatke klijenta duže od očekivanog), ostali klijenti su blokirani. U produkciji to gotovo nikad nije prihvatljivo — moramo opslužiti više klijenata paralelno. Postoji više načina na koje ovaj problem možemo riješiti.

9.4.1 Posluživanje klijenta u novom procesu

Najjednostavniji pristup, koji se prirodno nadovezuje na sve što znamo iz poglavlja o okruženju procesa: čim `accept` vrati novi deskriptor, glavni proces pozove `fork`. Dijete preuzme razgovor s klijentom, roditelj odmah pozove sljedeći `accept`.

Ovo je dosta napredan primjer — kombinira `fork`, signale, runtime stanje koje signali mijenjaju, i "čisti" izlaz (engl. *clean exit*) umjesto trenutnog prekida. Sve te tehnike pokriveni su pojedinačno u prethodnim poglavljima skripte, pa pretpostavljamo da je čitatelj do sada s njima upoznat. Konkretno:

- Razgovor s klijentom u djetetu je **echo petlja** — dijete u petlji čita poruke i svaku vraća natrag, što znači da isti klijent može poslati više poruka jednu za drugom kroz istu vezu (a ne samo jednu, kao u prethodnim primjerima). Petlja se prekida kada klijent zatvori vezu ili pošalje "KRAJ". U prvom slučaju samo proces-dijete koji je opsluživao klijenta prekida izvršavanje, dok roditelj i ostala djeca nastavljaju normalno raditi. U drugom slučaju, kada klijent pošalje poruku "KRAJ", dijete odgovara s "U REDU -- IZLAZIM!" i pošalje signal `SIGTERM` roditelju kako bi se cijeli server uredno ugasio.
- Roditelj hvata **`SIGTERM`** (od djeteta) i **`SIGINT`** (Ctrl+C korisnika) istim handlerom, koji postavlja zastavu zaustavi. Kada glavna `accept` petlja vidi da je zastava postavljena, uredno izlazi iz petlje i ispiše `Server izlazi..` Time umjesto naglog prekida (kakav bismo dobili bez handlera za `SIGINT`) imamo **clean exit** — server uredno zatvori slušajući socket i izađe, a budući da je `SO_REUSEADDR` postavljen, port je odmah ponovo dostupan za sljedeće pokretanje.

- **SIGCHLD** se hvata kao i prije, kako se ne bi gomilali zombi procesi nakon završetka djece (kao u poglavlju o signalima).

Pogledajmo sad izvorni kod.

tcp_server_fork.c je varijanta TCP servera s fork-om za svakog novog klijenta, s echo komunikacijom, podrškom za "KRAJ" i clean exit na Ctrl+C.

Cijela razlika u odnosu na **tcp_server.c** je u glavnoj petlji i u funkciji **posluzi_klijenta**:

```
while (!zaustavi) {
    fd_klijent = accept(fd_server, NULL, NULL);
    if (fd_klijent < 0) {
        if (errno == EINTR) continue; /* prekinut signalom */
        perror("accept");
        continue;
    }

    pid_t pid = fork();
    if (pid == 0) {
        /* dijete: ne treba mu slusajuci socket */
        close(fd_server);
        posluzi_klijenta(fd_klijent);
        close(fd_klijent);
        exit(EXIT_SUCCESS);
    }
    /* roditelj: ne treba mu klijentov socket */
    close(fd_klijent);
}
```

Funkcija **posluzi_klijenta** čita poruku klijenta i vraća je natrag (echo), dok klijent ne zatvori vezu ili pošalje "KRAJ":

```
static void posluzi_klijenta(int fd_klijent) {
    char    buffer[256];
    ssize_t  n;
    const char *poruka_kraja = "U REDU -- IZLAZIM!";

    while ((n = read(fd_klijent, buffer, sizeof(buffer) - 1)) > 0) {
        buffer[n] = '\0';
        printf("[PID %d] Primljeno: %s\n", (int)getpid(), buffer);

        if (strncmp(buffer, "KRAJ", 4) == 0) {
            write(fd_klijent, poruka_kraja, strlen(poruka_kraja));
            kill(getppid(), SIGTERM);
            break;
        } else {
            write(fd_klijent, buffer, n); /* echo natrag */
        }
    }
}
```

Ovaj obrazac koristi dvije važne osobine UNIX-a koje smo već upoznali: nakon fork-a, **dijete naslijedi sve otvorene deskriptore** roditelja (kao što smo vidjeli u poglavlju o okruženju procesa), pa tako i **fd_klijent**. I roditelj i dijete inicijalno imaju otvoren **fd_klijent**, zbog čega oboje moraju pozvati **close** — dijete kad završi razgovor, roditelj odmah jer mu nije potreban. Ako roditelj zaboravi zatvoriti svoj primjerak, deskriptor će ostati otvoren u procesu i klijent neće prepoznati da je veza zatvorena.

Postavljanje handlera za signale podsjeća na ono iz prethodnog poglavlja o signalima. Za **SIGCHLD** koristimo **SA_RESTART**, jer ne želimo da **SIGCHLD** (koji stiže svaki put kad neko dijete završi) prekida našu **accept** petlju:

```
static void rukovatelj_sigchld(int sig) {
    (void)sig;
    while (waitpid(-1, NULL, WNOHANG) > 0)
        ;
}

sa.sa_handler = rukovatelj_sigchld;
sigemptyset(&sa.sa_mask);
sa.sa_flags = SA_RESTART;
sigaction(SIGCHLD, &sa, NULL);
```

Za SIGTERM (koji nam dijete šalje pri primitku “KRAJ”) i SIGINT (Ctrl+C) koristimo **isti** handler — oba znače “zaustavi se” — i **bez** SA_RESTART-a, jer baš želimo da accept vrati grešku EINTR:

```
static volatile sig_atomic_t zaustavi = 0;

static void rukovatelj_zaustavi(int sig) {
    (void)sig;
    zaustavi = 1;
}

sa.sa_handler = rukovatelj_zaustavi;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
sigaction(SIGTERM, &sa, NULL);
sigaction(SIGINT, &sa, NULL);
```

Kad signal stigne, handler postavi `zaustavi = 1`. `accept` se vrati s greškom EINTR, u petlji to prepoznamo i nastavimo na sljedeću iteraciju, ali `while (!zaustavi)` provjera u sljedećem prolazu vidi da treba izaći. Tako uredno zatvorimo slušajući socket i izađemo iz `main`-a, što je standardna definicija *clean exit*-a.

Test s nekoliko klijenata paralelno:

Terminal A:	Terminal B:
\$./tcp_server_fork	\$./tcp_klijent "Klijent A"
Server sluša na portu 9000 ...	Odgovor: Klijent A
[PID 28491] Primitljeno: Klijent A	\$./tcp_klijent "Klijent B"
[PID 28522] Primitljeno: Klijent B	Odgovor: Klijent B
[PID 28537] Primitljeno: KRAJ	\$./tcp_klijent "KRAJ"
Server izlazi.	Odgovor: U REDU -- IZLAZIM!

Probajte ovaj server pokrenuti, spojiti se s nekoliko klijenata, i pritisnuti **Ctrl+C** u terminalu servera — uvjerit ćete se da server uredno izlazi (ispisuje “Server izlazi.”) umjesto da bude nasilno prekinut. Bez SIGINT handlera, Ctrl+C bi proces prekinuo izvana, bez ikakvog clean-up koda.

Da bismo ispitali echo petlju s više poruka kroz istu vezu, naš `tcp_klijent` nije dovoljan — on po dizajnu šalje jednu poruku, čita jedan odgovor i izlazi. Za interaktivni test možemo koristiti `nc` (netcat), koji slijed redaka sa standardnog ulaza šalje serveru jedan po jedan, a sve što primi od servera ispiše na standardni izlaz. U novom terminalu pokrenemo:

```
$ nc 127.0.0.1 9000
Prva poruka
Prva poruka                (echo natrag od servera)
Druga poruka
Druga poruka                (echo natrag od servera)
Trecaaa
Trecaaa                    (echo natrag od servera)
^D                          (Ctrl+D zatvara vezu)
```

Sve dok ne stisnemo Ctrl+D (oznaka kraja unosa na standardnom ulazu), `nc` šalje serveru svaki novi redak, a server ga vraća natrag — što izvrsno demonstrira

da dijete u svom read/write loop-u zaista može opslužiti više razmjena s istim klijentom.

Predlažemo čitatelju da kao vježbu dorade `tcp_klijent` (a po istom uzoru i `uds_klijent`) tako da ne šalje samo jednu poruku, nego u petlji čita redak sa standardnog ulaza, šalje ga serveru, čita i ispisuje odgovor — i to ponavlja sve dok korisnik ne stisne Ctrl+D, odnosno dok standardni ulaz ne signalizira EOF. Tako bi naš klijent funkcionirao analogno nc-u, ali s vlastitim ispisom oblika "Odgovor: ...".

Svaki klijent dobiva svoj proces, paralelno se opslužuju, ne čekaju jedan drugog.

9.4.2 Alternativni pristupi

Stvaranje novog procesa za svakog klijenta pozivom `fork` nije jedini način kojim možemo ostvariti istovremeno posluživanje više klijenata. Spomenimo ukratko alternativne obrasce, koje nećemo razrađivati kroz primjer:

- **Niti:** umjesto `fork`-a, nit koja osluškuje dolazne pozive na otvorenom socketu (glavna nit — kolokvijalno rečeno) za svakog klijenta stvori novu nit kroz `pthread_create`. Niti su lakše od procesa (manje memorije, brže stvaranje), ali zahtijevaju pažljivu sinkronizaciju ako dijele bilo koju varijablu ili neki drugi resurs.
- **Thread pool:** pri pokretanju servera stvori se fiksni broj niti koje iz reda preuzimaju novodošle klijente. Izbjegava se trošak stalnog stvaranja niti za svakog klijenta, ali se zadržava paralelnost.
- **Multipleksiranje I/O** (`select`, `poll`, `epoll`): jedan proces (bez `fork`-a, bez niti) pomoću sistemskih poziva `select`, `poll` ili `epoll` istovremeno prati više deskriptora i radi samo s onima na kojima ima podataka. Ovaj pristup omogućuje vrlo velik broj istovremenih veza (deseci tisuća) s minimalnim resursima, ali je programski složeniji jer cijeli server radi u jednoj petlji koja žonglira između svih veza. Tema multipleksiranja izlazi izvan okvira ove skripte i neće biti obrađena ni u tekstu ni kroz primjere — zainteresiranog čitatelja upućujemo na Stevens UNP [1].

Izbor pristupa ovisi o očekivanom opterećenju, vrsti rada koji server radi za klijenta (CPU vs I/O), i složenosti koju smo spremni unijeti u kod.

9.5 UDP socketi

Do sada smo radili isključivo s TCP-om (tipom `SOCK_STREAM`), koji je pouzdan, tokom-orientiran protokol — uspostavljamo vezu, šaljemo niz bajtova koji će na drugu stranu doći redom i bez gubitaka, na kraju zatvaramo vezu. To je idealno za većinu klijent-server scenarija, ali ne za sve.

Internet protokolarni stog nudi i alternativu: **UDP** (engl. *User Datagram Protocol*). UDP ne uspostavlja vezu, ne jamči redoslijed, ne jamči isporuku, ne brine za retransmisiju izgubljenih paketa — jednostavno pošalje paket (datagram) prema odredištu i vrati se aplikaciji. Aplikacija je ta koja, ako joj treba pouzdanost, mora sama implementirati potvrde, retransmisije i redoslijed. U zamjenu, UDP je **brži i jednostavniji**, s manje latencije, i koristi se tamo gdje je gubitak ponekog paketa

prihvatljiv: real-time prijenos audio/video signala (VoIP, video konferencije), online igre, DNS upiti, mrežno otkrivanje servisa, NTP (sinkronizacija sata) i sl.

U socket sučelju UDP koristi tip `SOCK_DGRAM` (umjesto `SOCK_STREAM`), a komunikacija se odvija kroz dvije funkcije:

```
ssize_t sendto (int fd, const void *buf, size_t len, int flags,
               const struct sockaddr *dest_addr, socklen_t addrlen);
ssize_t recvfrom(int fd, void *buf, size_t len, int flags,
                struct sockaddr *src_addr, socklen_t *addrlen);
```

Bitne razlike u odnosu na TCP:

- **Nema listen-a, nema accept-a, nema connect-a.** Server samo pozove socket i bind, i odmah je spreman primiti pakete od bilo koga.
- Svaki paket je samostalna jedinica. `sendto` u svakom pozivu eksplicitno navodi odredišnu adresu (jer nema "uspostavljene veze" prema fiksnom partneru).
- `recvfrom` uz primljeni paket vrati i adresu pošiljatelja u izlaznom argumentu, što serveru omogućuje da zna kome odgovoriti.

9.5.1 Primjer: udp_server i udp_klijent

UDP echo server po istom obrascu kao naši dosadašnji TCP primjeri: ispiše svaku poruku i vrati je natrag, a na "KRAJ" odgovori "U REDU -- IZLAZIM!" i prekida izvršavanje. Koristi port 9001 (kako se ne bi sukobio s TCP serverom na portu 9000).

- **udp_server.c:**

```
#define PORT 9001

int main(void) {
    int fd;
    struct sockaddr_in adresa, adresa_klijenta;
    socklen_t len;
    char buffer[256];
    ssize_t n;
    int kraj = 0;
    const char *poruka_kraja = "U REDU -- IZLAZIM!";

    setbuf(stdout, NULL);

    fd = socket(AF_INET, SOCK_DGRAM, 0); /* SOCK_DGRAM = UDP */

    memset(&adresa, 0, sizeof(adresa));
    adresa.sin_family = AF_INET;
    adresa.sin_addr.s_addr = htonl(INADDR_ANY);
    adresa.sin_port = htons(PORT);

    bind(fd, (struct sockaddr *)&adresa, sizeof(adresa));

    /* Bez listen-a i accept-a -- UDP nema vezu */
    printf("UDP server sluša na portu %d\n", PORT);

    while (!kraj) {
        len = sizeof(adresa_klijenta);
        n = recvfrom(fd, buffer, sizeof(buffer) - 1, 0,
                    (struct sockaddr *)&adresa_klijenta, &len);
        if (n > 0) {
            buffer[n] = '\0';
            printf("Primljeno: %s\n", buffer);

            if (strncmp(buffer, "KRAJ", 4) == 0) {
                sendto(fd, poruka_kraja, strlen(poruka_kraja), 0,
                      (struct sockaddr *)&adresa_klijenta, len);
                kraj = 1;
            }
        }
    }
}
```

```

    } else {
        sendto(fd, buffer, n, 0,
              (struct sockaddr *)&adresa_klijenta, len); /* echo */
    }
}

printf("Server izlazi.\n");
close(fd);
return 0;
}

```

- **udp_klijent.c** je strukturno jednostavan: pripremi adresu, jedan sendto, jedan recvfrom, gotov.

```

int main(int argc, char *argv[]) {
    int fd;
    struct sockaddr_in adresa;
    socklen_t len;
    const char *poruka = argv[1];
    const char *ip = (argc == 3) ? argv[2] : "127.0.0.1";
    char buffer[256];
    ssize_t n;

    fd = socket(AF_INET, SOCK_DGRAM, 0);

    memset(&adresa, 0, sizeof(adresa));
    adresa.sin_family = AF_INET;
    adresa.sin_port = htons(PORT);
    inet_pton(AF_INET, ip, &adresa.sin_addr);

    sendto(fd, poruka, strlen(poruka), 0,
          (struct sockaddr *)&adresa, sizeof(adresa));

    len = sizeof(adresa);
    n = recvfrom(fd, buffer, sizeof(buffer) - 1, 0,
                 (struct sockaddr *)&adresa, &len);
    if (n > 0) {
        buffer[n] = '\0';
        printf("Odgovor: %s\n", buffer);
    }

    close(fd);
    return 0;
}

```

Pokretanje izgleda potpuno isto kao kod TCP varijante:

Terminal A:	Terminal B:
\$./udp_server	\$./udp_klijent "Pozdrav!"
UDP server sluša na portu 9001	Odgovor: Pozdrav!
Primljeno: Pozdrav!	\$./udp_klijent "KRAJ"
Primljeno: KRAJ	Odgovor: U REDU -- IZLAZIM!
Server izlazi.	

Iz ovog jednostavnog primjera ne vidi se prava priroda UDP-a — pouzdana isporuka, gubici paketa, nepredvidiv redoslijed — sve to dolazi do izražaja tek u stvarnim mrežnim uvjetima i složenijim aplikacijama. Ali bitne **programerske** razlike (SOCK_DGRAM, sendto/recvfrom, odsutnost veze) primjer prikazuje izravno.

9.6 Prevođenje

```
$ make all
```

Pokrenite svaki server u jednom terminalu, klijente u drugima.

Za UNIX domain primjer, datoteka /tmp/uds_primjer ostat će na disku ako server

nasilno prekinemo (Ctrl+C). Sljedeće pokretanje servera ju automatski uklanja (unlink), ali možete je i ručno obrisati: `rm -f /tmp/uds_primjer`.

Za TCP primjere, koristimo port 9000. Ako vam port nije slobodan (zauzeo ga je drugi proces), promijenite konstantu `PORT` u izvornom kodu i prevedite ponovno.

9.7 Što smo zapravo radili

- **Socket** je generalizacija deskriptora datoteke za komunikaciju između procesa, lokalno ili preko mreže. Sve što znamo o deskriptorima iz poglavlja o ulazno-izlaznim operacijama vrijedi i ovdje.
- **Domena** određuje “namespace” adresa. `AF_UNIX` koristi putanje u datotečnom sustavu, `AF_INET` koristi par (IP, port).
- **Server** prolazi kroz socket → `bind` → `listen` → `accept`. Slušajući socket (`fd_server`) različit je od konektiranog socketa (`fd_klijent`); jedan prima nove veze, drugi vodi razgovor.
- **Klijent** prolazi kroz socket → `connect`, pa razgovor.
- Nakon uspostave veze, obje strane razgovaraju kroz `read/write` kao na obične datoteke.
- **Mrežni byte order** je big-endian; koristimo `htons/htonl` za pretvaranje, inače računala različite arhitekture ne mogu razgovarati.
- Za **više klijenata istovremeno** najjednostavniji obrazac je `fork` po klijentu — ali postoje i alternative (niti, `select/poll/epoll`, thread poolovi) koje treba znati kad performanse postanu kritične.
- **TCP** (`SOCK_STREAM`) i **UDP** (`SOCK_DGRAM`) su dvije glavne varijante mrežnih socketa. TCP je pouzdan i orijentiran na vezu; UDP šalje samostalne pakete bez veze, brže ali bez jamstava isporuke.

Mrežno programiranje je obimno područje. Spomenimo nekoliko važnih tema koje nismo razrađivali:

- **IPv6** (`AF_INET6`) kao zamjenu za IPv4.
- **Sigurna komunikacija** kroz TLS/SSL (najpoznatija implementacija je OpenSSL).
- **Pravilno čitanje cijele poruke** — `read` može vratiti manje bajtova nego što smo tražili (engl. *short read*); robusan kod radi u petlji dok ne pročita željeni iznos ili dok ne dobije EOF. Naši primjeri pretpostavljaju da jedan `read` vraća cijelu poruku, što je u praksi često, ali ne i garantirano.
- **Sastavljanje protokola** — kako definirati strukturu poruka iznad obične tokom-orijentirane veze (gdje su granice poruke, kako se kodiraju tipovi podataka, ...).

Sve ovo i mnogo više obrađeno je u referencama navedenim niže.

9.8 Bibliografija

[1] W. R. Stevens, B. Fenner, and A. M. Rudoff, *UNIX Network Programming, Volume 1: The Sockets Networking API*, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2003.

[2] S. J. Leffler, R. S. Fabry, and W. N. Joy, “A 4.2BSD Interprocess Communication Primer,” Tech. Rep. UCB/CSD-83-145, EECS Department, University of California, Berkeley, July 1983.

[3] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2013.

[4] B. Hall, *Beej's Guide to Network Programming*. Dostupno online: <https://beej.us/guide/bgnet/>. Pristupljeno: svibanj 2026.